

petreminder Application with Library
0.0.1

Generated by Doxygen 1.15.0

1 PetReminder Application	1
1.1 Overview	1
1.2 Features	1
1.3 Architecture	1
1.4 How to Run	2
1.5 Documentation	2
1.6 Team Members	2
1.7 License	2
2 Namespace Index	3
2.1 Package List	3
3 Hierarchical Index	5
3.1 Class Hierarchy	5
4 Class Index	7
4.1 Class List	7
5 File Index	11
5.1 File List	11
6 Namespace Documentation	15
6.1 Package com.mehdi.petreminder.gui	15
6.1.1 Detailed Description	15
6.2 Package com.mehdi.petreminder.gui.util	15
6.2.1 Detailed Description	15
6.3 Package com.mehdi.petreminder.model	16
6.3.1 Detailed Description	16
6.4 Package com.mehdi.petreminder.repository	16
6.4.1 Detailed Description	17
6.5 Package com.mehdi.petreminder.service	17
6.5.1 Detailed Description	17
7 Class Documentation	19
7.1 com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable > Class Template Reference	19
7.1.1 Detailed Description	20
7.1.2 Constructor & Destructor Documentation	20
7.1.2.1 BinaryRepository()	20
7.1.3 Member Function Documentation	21
7.1.3.1 close()	21
7.1.3.2 count()	21
7.1.3.3 delete()	21
7.1.3.4 deleteAll()	22
7.1.3.5 findAll()	22

7.1.3.6 findById()	23
7.1.3.7 getFilePath()	24
7.1.3.8 save()	24
7.1.3.9 update()	25
7.2 com.mehdi.petreminder.model.Bird Class Reference	25
7.2.1 Detailed Description	27
7.2.2 Constructor & Destructor Documentation	27
7.2.2.1 Bird() [1/3]	27
7.2.2.2 Bird() [2/3]	28
7.2.2.3 Bird() [3/3]	28
7.2.3 Member Function Documentation	29
7.2.3.1 getBirdType()	29
7.2.3.2 getIconName()	29
7.2.3.3 isCanTalk()	29
7.2.3.4 makeSound()	30
7.2.3.5 setBirdType()	30
7.2.3.6 setCanTalk()	30
7.2.3.7 toString()	31
7.3 com.mehdi.petreminder.model.Cat Class Reference	31
7.3.1 Detailed Description	33
7.3.2 Constructor & Destructor Documentation	33
7.3.2.1 Cat() [1/3]	33
7.3.2.2 Cat() [2/3]	34
7.3.2.3 Cat() [3/3]	34
7.3.3 Member Function Documentation	35
7.3.3.1 getBreed()	35
7.3.3.2 getIconName()	35
7.3.3.3 isIndoor()	35
7.3.3.4 makeSound()	36
7.3.3.5 setBreed()	36
7.3.3.6 setIndoor()	36
7.3.3.7 toString()	37
7.4 com.mehdi.petreminder.gui.panels.DashboardPanel Class Reference	37
7.4.1 Detailed Description	38
7.4.2 Constructor & Destructor Documentation	38
7.4.2.1 DashboardPanel()	38
7.4.3 Member Function Documentation	39
7.4.3.1 refresh()	39
7.5 com.mehdi.petreminder.model.Dog Class Reference	39
7.5.1 Detailed Description	41
7.5.2 Constructor & Destructor Documentation	41
7.5.2.1 Dog() [1/3]	41

7.5.2.2 Dog() [2/3]	42
7.5.2.3 Dog() [3/3]	42
7.5.3 Member Function Documentation	43
7.5.3.1 getBreed()	43
7.5.3.2 getIconName()	43
7.5.3.3 isTrained()	43
7.5.3.4 makeSound()	44
7.5.3.5 setBreed()	44
7.5.3.6 setTrained()	44
7.5.3.7 toString()	45
7.6 com.mehdi.petreminder.observer.EventManager Class Reference	45
7.6.1 Detailed Description	46
7.6.2 Member Function Documentation	46
7.6.2.1 clearAll()	46
7.6.2.2 getInstance()	46
7.6.2.3 getObserverCount()	47
7.6.2.4 notify()	47
7.6.2.5 resetInstance()	48
7.6.2.6 subscribe()	48
7.6.2.7 unsubscribe()	48
7.6.2.8 unsubscribeAll()	49
7.7 com.mehdi.petreminder.observer.EventType Enum Reference	49
7.7.1 Detailed Description	50
7.7.2 Constructor & Destructor Documentation	50
7.7.2.1 EventType()	50
7.7.3 Member Function Documentation	50
7.7.3.1 getDescription()	50
7.7.3.2 toString()	50
7.7.4 Member Data Documentation	51
7.7.4.1 MEDICAL_RECORD_ADDED	51
7.7.4.2 MEDICAL_RECORD_DELETED	51
7.7.4.3 PET_ADDED	51
7.7.4.4 PET_DELETED	51
7.7.4.5 PET_UPDATED	51
7.7.4.6 REMINDER_ADDED	52
7.7.4.7 REMINDER_COMPLETED	52
7.7.4.8 REMINDER_DELETED	52
7.7.4.9 STORAGE_CHANGED	52
7.8 com.mehdi.petreminder.model.ExerciseReminder Class Reference	52
7.8.1 Detailed Description	55
7.8.2 Constructor & Destructor Documentation	55
7.8.2.1 ExerciseReminder() [1/3]	55

7.8.2.2 ExerciseReminder() [2/3]	55
7.8.2.3 ExerciseReminder() [3/3]	56
7.8.3 Member Function Documentation	56
7.8.3.1 getDistanceKm()	56
7.8.3.2 getDurationMinutes()	57
7.8.3.3 getExerciseType()	57
7.8.3.4 getReminderType()	57
7.8.3.5 getSummary()	57
7.8.3.6 setDistanceKm()	58
7.8.3.7 setDurationMinutes()	58
7.8.3.8 setExerciseType()	58
7.8.3.9 toString()	59
7.8.3.10 validate()	59
7.9 com.mehdi.petreminder.model.FeedingReminder Class Reference	59
7.9.1 Detailed Description	62
7.9.2 Constructor & Destructor Documentation	62
7.9.2.1 FeedingReminder() [1/3]	62
7.9.2.2 FeedingReminder() [2/3]	62
7.9.2.3 FeedingReminder() [3/3]	63
7.9.3 Member Function Documentation	64
7.9.3.1 getFoodType()	64
7.9.3.2 getPortionGrams()	64
7.9.3.3 getReminderType()	64
7.9.3.4 getSummary()	65
7.9.3.5 isIncludesWaterChange()	65
7.9.3.6 setFoodType()	65
7.9.3.7 setIncludesWaterChange()	66
7.9.3.8 setPortionGrams()	66
7.9.3.9 toString()	66
7.9.3.10 validate()	67
7.10 com.mehdi.petreminder.annotation.Generated Interface Reference	67
7.10.1 Detailed Description	67
7.11 com.mehdi.petreminder.Generated Interface Reference	67
7.11.1 Detailed Description	67
7.12 com.mehdi.petreminder.model.GroomingReminder Class Reference	68
7.12.1 Detailed Description	70
7.12.2 Constructor & Destructor Documentation	70
7.12.2.1 GroomingReminder() [1/3]	70
7.12.2.2 GroomingReminder() [2/3]	71
7.12.2.3 GroomingReminder() [3/3]	71
7.12.3 Member Function Documentation	72
7.12.3.1 getGroomerName()	72

7.12.3.2 getGroomingType()	72
7.12.3.3 getReminderType()	73
7.12.3.4 getSummary()	73
7.12.3.5 isProfessional()	73
7.12.3.6 setGroomerName()	74
7.12.3.7 setGroomingType()	74
7.12.3.8 setProfessional()	74
7.12.3.9 toString()	75
7.12.3.10 validate()	75
7.13 com.mehdi.petreminder.gui.util.GuiConstants Class Reference	75
7.13.1 Detailed Description	76
7.13.2 Member Data Documentation	77
7.13.2.1 BG_COLOR	77
7.13.2.2 BODY_FONT	77
7.13.2.3 BORDER_COLOR	77
7.13.2.4 CARD_BG	77
7.13.2.5 DANGER	78
7.13.2.6 GAP	78
7.13.2.7 GAP_LARGE	78
7.13.2.8 NAV_ACTIVE	78
7.13.2.9 NAV_COLOR	78
7.13.2.10 NAV_FONT	79
7.13.2.11 NAV_HOVER	79
7.13.2.12 NAV_WIDTH	79
7.13.2.13 PRIMARY	79
7.13.2.14 SECTION_FONT	79
7.13.2.15 SMALL_FONT	80
7.13.2.16 SUCCESS	80
7.13.2.17 TEXT_PRIMARY	80
7.13.2.18 TEXT_SECONDARY	80
7.13.2.19 TITLE_FONT	80
7.13.2.20 WARNING	81
7.14 com.mehdi.petreminder.repository.IRepository< T > Interface Template Reference	81
7.14.1 Detailed Description	82
7.14.2 Member Function Documentation	82
7.14.2.1 close()	82
7.14.2.2 count()	82
7.14.2.3 delete()	83
7.14.2.4 deleteAll()	83
7.14.2.5 findAll()	83
7.14.2.6 findById()	84
7.14.2.7 save()	84

7.14.2.8 update()	85
7.15 com.mehdi.petreminder.gui.MainFrame Class Reference	85
7.15.1 Detailed Description	86
7.15.2 Constructor & Destructor Documentation	87
7.15.2.1 MainFrame()	87
7.15.3 Member Function Documentation	87
7.15.3.1 launch()	87
7.15.3.2 navigateTo()	87
7.15.3.3 onStorageChanged()	88
7.15.4 Member Data Documentation	88
7.15.4.1 CARD_DASHBOARD	88
7.15.4.2 CARD_MEDICAL	88
7.15.4.3 CARD_PETS	88
7.15.4.4 CARD_REMINDERS	89
7.15.4.5 CARD_SETTINGS	89
7.15.4.6 CARD_VET	89
7.16 com.mehdi.petreminder.gui.panels.MedicalPanel Class Reference	89
7.16.1 Detailed Description	90
7.16.2 Constructor & Destructor Documentation	90
7.16.2.1 MedicalPanel()	90
7.16.3 Member Function Documentation	91
7.16.3.1 refresh()	91
7.17 com.mehdi.petreminder.model.MedicalRecord Class Reference	91
7.17.1 Detailed Description	93
7.17.2 Constructor & Destructor Documentation	93
7.17.2.1 MedicalRecord() [1/2]	93
7.17.2.2 MedicalRecord() [2/2]	93
7.17.3 Member Function Documentation	94
7.17.3.1 equals()	94
7.17.3.2 getCost()	94
7.17.3.3 getDiagnosis()	95
7.17.3.4 getFormattedDate()	95
7.17.3.5 getId()	95
7.17.3.6 getNextCheckDate()	96
7.17.3.7 getNotes()	96
7.17.3.8 getPetId()	96
7.17.3.9 getPetName()	96
7.17.3.10 getRecordDate()	97
7.17.3.11 getRecordType()	97
7.17.3.12 getTreatment()	97
7.17.3.13 getVaccineName()	97
7.17.3.14 getVeterinarianName()	98

7.17.3.15 hashCode()	98
7.17.3.16 setCost()	98
7.17.3.17 setDiagnosis()	99
7.17.3.18 setId()	99
7.17.3.19 setNextCheckDate()	99
7.17.3.20 setNotes()	100
7.17.3.21 setPetId()	100
7.17.3.22 setPetName()	100
7.17.3.23 setRecordDate()	101
7.17.3.24 setRecordType()	101
7.17.3.25 setTreatment()	101
7.17.3.26 setVaccineName()	102
7.17.3.27 setVeterinarianName()	102
7.17.3.28 toString()	102
7.18 com.mehdi.petreminder.repository.MedicalRecordMySqlRepository Class Reference	103
7.18.1 Detailed Description	104
7.18.2 Constructor & Destructor Documentation	105
7.18.2.1 MedicalRecordMySqlRepository() [1/2]	105
7.18.2.2 MedicalRecordMySqlRepository() [2/2]	105
7.18.3 Member Function Documentation	105
7.18.3.1 getCreateTableSql()	105
7.18.3.2 getInsertSql()	106
7.18.3.3 getTableName()	106
7.18.3.4 getUpdateSql()	106
7.18.3.5 mapRow()	106
7.18.3.6 setInsertParams()	107
7.18.3.7 setUpdateParams()	107
7.19 com.mehdi.petreminder.service.MedicalRecordService Class Reference	107
7.19.1 Detailed Description	108
7.19.2 Constructor & Destructor Documentation	108
7.19.2.1 MedicalRecordService() [1/2]	108
7.19.2.2 MedicalRecordService() [2/2]	109
7.19.3 Member Function Documentation	109
7.19.3.1 addRecord()	109
7.19.3.2 close()	109
7.19.3.3 deleteAllRecords()	110
7.19.3.4 deleteRecord()	110
7.19.3.5 getAllRecords()	110
7.19.3.6 getRecordById()	111
7.19.3.7 getRecordCount()	111
7.19.3.8 getRecordsByPetId()	111
7.19.3.9 updateRecord()	112

7.20 com.mehdi.petreminder.repository.MedicalRecordSqliteRepository Class Reference	112
7.20.1 Detailed Description	114
7.20.2 Constructor & Destructor Documentation	114
7.20.2.1 MedicalRecordSqliteRepository() [1/2]	114
7.20.2.2 MedicalRecordSqliteRepository() [2/2]	115
7.20.3 Member Function Documentation	115
7.20.3.1 findByPetId()	115
7.20.3.2 getCreateTableSql()	116
7.20.3.3 getInsertSql()	116
7.20.3.4 getTableName()	116
7.20.3.5 getUpdateSql()	116
7.20.3.6 mapRow()	117
7.20.3.7 setInsertParams()	117
7.20.3.8 setUpdateParams()	118
7.21 com.mehdi.petreminder.model.MedicationReminder Class Reference	118
7.21.1 Detailed Description	120
7.21.2 Constructor & Destructor Documentation	121
7.21.2.1 MedicationReminder() [1/3]	121
7.21.2.2 MedicationReminder() [2/3]	121
7.21.2.3 MedicationReminder() [3/3]	122
7.21.3 Member Function Documentation	122
7.21.3.1 getDosage()	122
7.21.3.2 getDosageUnit()	123
7.21.3.3 getMedicationName()	123
7.21.3.4 getReminderType()	123
7.21.3.5 getSummary()	124
7.21.3.6 getVeterinarianName()	124
7.21.3.7 isPrescribed()	124
7.21.3.8 setDosage()	125
7.21.3.9 setDosageUnit()	125
7.21.3.10 setMedicationName()	125
7.21.3.11 setPrescribed()	126
7.21.3.12 setVeterinarianName()	126
7.21.3.13 toString()	126
7.21.3.14 validate()	127
7.22 com.mehdi.petreminder.repository.MySqlRepository< T > Class Template Reference	127
7.22.1 Detailed Description	128
7.22.2 Constructor & Destructor Documentation	129
7.22.2.1 MySqlRepository() [1/2]	129
7.22.2.2 MySqlRepository() [2/2]	129
7.22.3 Member Function Documentation	130
7.22.3.1 close()	130

7.22.3.2 count()	130
7.22.3.3 createTableIfNotExists()	130
7.22.3.4 delete()	131
7.22.3.5 deleteAll()	131
7.22.3.6 findAll()	132
7.22.3.7 findById()	132
7.22.3.8 getConnection()	133
7.22.3.9 getCreateTableSql()	133
7.22.3.10 getInsertSql()	133
7.22.3.11 getTableName()	133
7.22.3.12 getUpdateSql()	134
7.22.3.13 mapRow()	134
7.22.3.14 save()	134
7.22.3.15 setEntityId()	135
7.22.3.16 setInsertParams()	136
7.22.3.17 setUpdateParams()	136
7.22.3.18 update()	136
7.22.4 Member Data Documentation	137
7.22.4.1 connection	137
7.22.4.2 logger	137
7.23 com.mehdi.petreminder.gui.panels.NavPanel Class Reference	138
7.23.1 Detailed Description	138
7.23.2 Constructor & Destructor Documentation	139
7.23.2.1 NavPanel()	139
7.23.3 Member Function Documentation	139
7.23.3.1 setActiveButton()	139
7.24 com.mehdi.petreminder.model.Pet Class Reference	140
7.24.1 Detailed Description	141
7.24.2 Constructor & Destructor Documentation	141
7.24.2.1 Pet() [1/2]	141
7.24.2.2 Pet() [2/2]	142
7.24.3 Member Function Documentation	142
7.24.3.1 equals()	142
7.24.3.2 getAge()	143
7.24.3.3 getAgeString()	143
7.24.3.4 getBirthDate()	144
7.24.3.5 getGender()	144
7.24.3.6 getIconName()	144
7.24.3.7 getId()	144
7.24.3.8 getName()	145
7.24.3.9 getNotes()	145
7.24.3.10 getOwnerId()	145

7.24.3.11	getSpecies()	145
7.24.3.12	getWeight()	146
7.24.3.13	hashCode()	146
7.24.3.14	makeSound()	146
7.24.3.15	setBirthDate()	147
7.24.3.16	setGender()	147
7.24.3.17	setId()	147
7.24.3.18	setName()	148
7.24.3.19	setNotes()	148
7.24.3.20	setOwnerId()	148
7.24.3.21	setSpecies()	149
7.24.3.22	setWeight()	149
7.24.3.23	toString()	149
7.25	com.mehdi.petreminder.gui.dialogs.PetDialog Class Reference	150
7.25.1	Detailed Description	150
7.25.2	Constructor & Destructor Documentation	151
7.25.2.1	PetDialog()	151
7.25.3	Member Function Documentation	151
7.25.3.1	getPet()	151
7.25.3.2	isConfirmed()	151
7.26	com.mehdi.petreminder.repository.PetMySqlRepository Class Reference	152
7.26.1	Detailed Description	153
7.26.2	Constructor & Destructor Documentation	154
7.26.2.1	PetMySqlRepository() [1/2]	154
7.26.2.2	PetMySqlRepository() [2/2]	154
7.26.3	Member Function Documentation	154
7.26.3.1	findByOwnerId()	154
7.26.3.2	getCreateTableSql()	155
7.26.3.3	getInsertSql()	155
7.26.3.4	getTableName()	155
7.26.3.5	getUpdateSql()	155
7.26.3.6	mapRow()	156
7.26.3.7	setInsertParams()	156
7.26.3.8	setUpdateParams()	157
7.27	com.mehdi.petreminder.petreminder Class Reference	157
7.27.1	Detailed Description	157
7.27.2	Constructor & Destructor Documentation	158
7.27.2.1	petreminder()	158
7.27.3	Member Function Documentation	158
7.27.3.1	getCurrentTimeFormatted()	158
7.27.3.2	isNullOrEmpty()	158
7.28	com.mehdi.petreminder.petreminderApp Class Reference	159

7.28.1 Detailed Description	159
7.28.2 Member Function Documentation	160
7.28.2.1 getAppName()	160
7.28.2.2 getVersion()	160
7.28.2.3 main()	160
7.28.2.4 parseStorageArg()	161
7.28.2.5 startGui()	161
7.28.3 Member Data Documentation	161
7.28.3.1 APP_NAME	161
7.28.3.2 APP_VERSION	162
7.29 com.mehdi.petreminder.observer.PetReminderObserver Interface Reference	162
7.29.1 Detailed Description	162
7.29.2 Member Function Documentation	163
7.29.2.1 onEvent()	163
7.30 com.mehdi.petreminder.service.PetService Class Reference	163
7.30.1 Detailed Description	164
7.30.2 Constructor & Destructor Documentation	164
7.30.2.1 PetService() [1/2]	164
7.30.2.2 PetService() [2/2]	164
7.30.3 Member Function Documentation	165
7.30.3.1 addPet()	165
7.30.3.2 close()	165
7.30.3.3 deleteAllPets()	165
7.30.3.4 deletePet()	166
7.30.3.5 filterBySpecies()	166
7.30.3.6 getAllPets()	167
7.30.3.7 getPetById()	167
7.30.3.8 getPetCount()	167
7.30.3.9 searchByName()	168
7.30.3.10 updatePet()	168
7.31 com.mehdi.petreminder.gui.panels.PetsPanel Class Reference	169
7.31.1 Detailed Description	169
7.31.2 Constructor & Destructor Documentation	170
7.31.2.1 PetsPanel()	170
7.31.3 Member Function Documentation	170
7.31.3.1 refresh()	170
7.32 com.mehdi.petreminder.repository.PetSqliteRepository Class Reference	171
7.32.1 Detailed Description	172
7.32.2 Constructor & Destructor Documentation	173
7.32.2.1 PetSqliteRepository() [1/2]	173
7.32.2.2 PetSqliteRepository() [2/2]	173
7.32.3 Member Function Documentation	173

7.32.3.1 findByOwnerId()	173
7.32.3.2 getCreateTableSql()	174
7.32.3.3 getInsertSql()	174
7.32.3.4 getTableName()	174
7.32.3.5 getUpdateSql()	174
7.32.3.6 mapRow()	175
7.32.3.7 setInsertParams()	175
7.32.3.8 setUpdateParams()	176
7.33 com.mehdi.petreminder.model.Reminder Class Reference	176
7.33.1 Detailed Description	178
7.33.2 Constructor & Destructor Documentation	178
7.33.2.1 Reminder() [1/2]	178
7.33.2.2 Reminder() [2/2]	178
7.33.3 Member Function Documentation	179
7.33.3.1 equals()	179
7.33.3.2 getDescription()	179
7.33.3.3 getFormattedTime()	180
7.33.3.4 getId()	180
7.33.3.5 getPetId()	180
7.33.3.6 getPetName()	181
7.33.3.7 getPriority()	181
7.33.3.8 getRecurrencePattern()	181
7.33.3.9 getReminderType()	182
7.33.3.10 getScheduledTime()	182
7.33.3.11 getSummary()	182
7.33.3.12 hashCode()	183
7.33.3.13 isCompleted()	183
7.33.3.14 isOverdue()	183
7.33.3.15 isRecurring()	183
7.33.3.16 setCompleted()	184
7.33.3.17 setDescription()	184
7.33.3.18 setId()	184
7.33.3.19 setPetId()	185
7.33.3.20 setPetName()	185
7.33.3.21 setPriority()	185
7.33.3.22 setRecurrencePattern()	186
7.33.3.23 setRecurring()	186
7.33.3.24 setScheduledTime()	186
7.33.3.25 toString()	187
7.33.3.26 validate()	187
7.33.4 Member Data Documentation	187
7.33.4.1 FORMATTER	187

7.34 com.mehdi.petreminder.gui.dialogs.ReminderDialog Class Reference	188
7.34.1 Detailed Description	188
7.34.2 Constructor & Destructor Documentation	189
7.34.2.1 ReminderDialog()	189
7.34.3 Member Function Documentation	189
7.34.3.1 getReminder()	189
7.34.3.2 isConfirmed()	189
7.35 com.mehdi.petreminder.repository.ReminderMySqlRepository Class Reference	190
7.35.1 Detailed Description	191
7.35.2 Constructor & Destructor Documentation	192
7.35.2.1 ReminderMySqlRepository() [1/2]	192
7.35.2.2 ReminderMySqlRepository() [2/2]	192
7.35.3 Member Function Documentation	192
7.35.3.1 getCreateTableSql()	192
7.35.3.2 getInsertSql()	193
7.35.3.3 getTableName()	193
7.35.3.4 getUpdateSql()	193
7.35.3.5 mapRow()	194
7.35.3.6 setInsertParams()	194
7.35.3.7 setUpdateParams()	195
7.36 com.mehdi.petreminder.service.ReminderService Class Reference	195
7.36.1 Detailed Description	196
7.36.2 Constructor & Destructor Documentation	197
7.36.2.1 ReminderService() [1/2]	197
7.36.2.2 ReminderService() [2/2]	197
7.36.3 Member Function Documentation	197
7.36.3.1 addReminder()	197
7.36.3.2 close()	198
7.36.3.3 deleteAllReminders()	198
7.36.3.4 deleteReminder()	198
7.36.3.5 getAllReminders()	199
7.36.3.6 getOverdueReminders()	199
7.36.3.7 getPendingReminders()	199
7.36.3.8 getReminderById()	200
7.36.3.9 getReminderCount()	200
7.36.3.10 getRemindersByPetId()	200
7.36.3.11 markCompleted()	201
7.36.3.12 updateReminder()	201
7.37 com.mehdi.petreminder.gui.panels.RemindersPanel Class Reference	202
7.37.1 Detailed Description	202
7.37.2 Constructor & Destructor Documentation	202
7.37.2.1 RemindersPanel()	202

7.37.3 Member Function Documentation	203
7.37.3.1 refresh()	203
7.38 com.mehdi.petreminder.repository.ReminderSqliteRepository Class Reference	203
7.38.1 Detailed Description	205
7.38.2 Constructor & Destructor Documentation	205
7.38.2.1 ReminderSqliteRepository() [1/2]	205
7.38.2.2 ReminderSqliteRepository() [2/2]	206
7.38.3 Member Function Documentation	206
7.38.3.1 findByPetId()	206
7.38.3.2 getCreateTableSql()	207
7.38.3.3 getInsertSql()	207
7.38.3.4 getTableName()	207
7.38.3.5 getUpdateSql()	208
7.38.3.6 mapRow()	208
7.38.3.7 setInsertParams()	209
7.38.3.8 setUpdateParams()	210
7.39 com.mehdi.petreminder.repository.RepositoryException Class Reference	210
7.39.1 Detailed Description	210
7.39.2 Constructor & Destructor Documentation	211
7.39.2.1 RepositoryException() [1/2]	211
7.39.2.2 RepositoryException() [2/2]	211
7.40 com.mehdi.petreminder.repository.RepositoryFactory Class Reference	211
7.40.1 Detailed Description	212
7.40.2 Member Function Documentation	212
7.40.2.1 createMedicalRecordRepository()	212
7.40.2.2 createPetRepository()	213
7.40.2.3 createReminderRepository()	213
7.40.2.4 createUserRepository()	214
7.40.2.5 getActiveType()	214
7.41 com.mehdi.petreminder.service.ServiceException Class Reference	214
7.41.1 Detailed Description	215
7.41.2 Constructor & Destructor Documentation	215
7.41.2.1 ServiceException() [1/2]	215
7.41.2.2 ServiceException() [2/2]	216
7.42 com.mehdi.petreminder.gui.panels.SettingsPanel Class Reference	216
7.42.1 Detailed Description	216
7.42.2 Constructor & Destructor Documentation	217
7.42.2.1 SettingsPanel()	217
7.42.3 Member Function Documentation	217
7.42.3.1 refresh()	217
7.43 com.mehdi.petreminder.repository.SqliteRepository< T > Class Template Reference	217
7.43.1 Detailed Description	219

7.43.2 Constructor & Destructor Documentation	219
7.43.2.1 SqliteRepository() [1/2]	219
7.43.2.2 SqliteRepository() [2/2]	220
7.43.3 Member Function Documentation	220
7.43.3.1 close()	220
7.43.3.2 count()	220
7.43.3.3 createTableIfNotExists()	221
7.43.3.4 delete()	221
7.43.3.5 deleteAll()	222
7.43.3.6 findAll()	222
7.43.3.7 findById()	223
7.43.3.8 getConnection()	223
7.43.3.9 getCreateTableSql()	224
7.43.3.10 getInsertSql()	224
7.43.3.11 getTableName()	224
7.43.3.12 getUpdateSql()	224
7.43.3.13 mapRow()	225
7.43.3.14 save()	225
7.43.3.15 setEntityId()	226
7.43.3.16 setInsertParams()	226
7.43.3.17 setUpdateParams()	227
7.43.3.18 update()	227
7.43.4 Member Data Documentation	228
7.43.4.1 connection	228
7.43.4.2 logger	228
7.44 com.mehdi.petreminder.config.StorageConfig Class Reference	228
7.44.1 Detailed Description	229
7.44.2 Member Function Documentation	230
7.44.2.1 getActiveBackend()	230
7.44.2.2 getBinaryDirectory()	230
7.44.2.3 getMysqlPassword()	230
7.44.2.4 getMysqlUrl()	231
7.44.2.5 getMysqlUsername()	231
7.44.2.6 getSqliteFilePath()	231
7.44.2.7 loadFromFile()	232
7.44.2.8 reset()	232
7.44.2.9 saveToFile()	232
7.44.2.10 setActiveBackend()	233
7.44.2.11 setMysqlUrl()	233
7.45 com.mehdi.petreminder.config.StorageType Enum Reference	234
7.45.1 Detailed Description	234
7.45.2 Constructor & Destructor Documentation	234

7.45.2.1 StorageType()	234
7.45.3 Member Function Documentation	235
7.45.3.1 getDisplayName()	235
7.45.3.2 toString()	235
7.45.4 Member Data Documentation	235
7.45.4.1 BINARY	235
7.45.4.2 MYSQL	235
7.45.4.3 SQLITE	236
7.46 com.mehdi.petreminder.model.User Class Reference	236
7.46.1 Detailed Description	237
7.46.2 Constructor & Destructor Documentation	237
7.46.2.1 User() [1/2]	237
7.46.2.2 User() [2/2]	238
7.46.3 Member Function Documentation	238
7.46.3.1 equals()	238
7.46.3.2 getCreatedAt()	239
7.46.3.3 getEmail()	239
7.46.3.4 getFullName()	239
7.46.3.5 getId()	239
7.46.3.6 getLastLogin()	240
7.46.3.7 getPasswordHash()	240
7.46.3.8 getUsername()	240
7.46.3.9 hashCode()	240
7.46.3.10 isActive()	241
7.46.3.11 setActive()	241
7.46.3.12 setCreatedAt()	241
7.46.3.13 setEmail()	242
7.46.3.14 setFullName()	242
7.46.3.15 setId()	242
7.46.3.16 setLastLogin()	243
7.46.3.17 setPasswordHash()	243
7.46.3.18 setUsername()	243
7.46.3.19 toString()	244
7.47 com.mehdi.petreminder.repository.UserMySqlRepository Class Reference	244
7.47.1 Detailed Description	246
7.47.2 Constructor & Destructor Documentation	246
7.47.2.1 UserMySqlRepository() [1/2]	246
7.47.2.2 UserMySqlRepository() [2/2]	246
7.47.3 Member Function Documentation	247
7.47.3.1 getCreateTableSql()	247
7.47.3.2 getInsertSql()	247
7.47.3.3 getTableName()	247

7.47.3.4 getUpdateSql()	247
7.47.3.5 mapRow()	248
7.47.3.6 setInsertParams()	248
7.47.3.7 setUpdateParams()	248
7.48 com.mehdi.petreminder.repository.UserSqliteRepository Class Reference	249
7.48.1 Detailed Description	250
7.48.2 Constructor & Destructor Documentation	251
7.48.2.1 UserSqliteRepository() [1/2]	251
7.48.2.2 UserSqliteRepository() [2/2]	251
7.48.3 Member Function Documentation	251
7.48.3.1 findByUsername()	251
7.48.3.2 getCreateTableSql()	252
7.48.3.3 getInsertSql()	252
7.48.3.4 getTableName()	252
7.48.3.5 getUpdateSql()	252
7.48.3.6 mapRow()	253
7.48.3.7 setInsertParams()	253
7.48.3.8 setUpdateParams()	253
7.49 com.mehdi.petreminder.model.VetAppointment Class Reference	254
7.49.1 Detailed Description	256
7.49.2 Constructor & Destructor Documentation	256
7.49.2.1 VetAppointment() [1/3]	256
7.49.2.2 VetAppointment() [2/3]	257
7.49.2.3 VetAppointment() [3/3]	257
7.49.3 Member Function Documentation	258
7.49.3.1 getClinicName()	258
7.49.3.2 getClinicPhone()	259
7.49.3.3 getEstimatedCost()	259
7.49.3.4 getReason()	259
7.49.3.5 getReminderType()	259
7.49.3.6 getSummary()	260
7.49.3.7 getVeterinarianName()	260
7.49.3.8 isConfirmed()	260
7.49.3.9 setClinicName()	261
7.49.3.10 setClinicPhone()	261
7.49.3.11 setConfirmed()	261
7.49.3.12 setEstimatedCost()	262
7.49.3.13 setReason()	262
7.49.3.14 setVeterinarianName()	262
7.49.3.15 toString()	263
7.49.3.16 validate()	263
7.50 com.mehdi.petreminder.gui.panels.VetPanel Class Reference	263

7.50.1 Detailed Description	264
7.50.2 Constructor & Destructor Documentation	264
7.50.2.1 VetPanel()	264
7.50.3 Member Function Documentation	265
7.50.3.1 refresh()	265
8 File Documentation	267
8.1 Generated.java	267
8.2 petreminder-app/src/main/java/com/mehdi/petreminder/Generated.java File Reference	267
8.2.1 Detailed Description	267
8.3 Generated.java	268
8.4 petreminder-app/src/main/java/com/mehdi/petreminder/config/StorageConfig.java File Reference	268
8.4.1 Detailed Description	268
8.5 StorageConfig.java	268
8.6 petreminder-app/src/main/java/com/mehdi/petreminder/config/StorageType.java File Reference	270
8.6.1 Detailed Description	270
8.7 StorageType.java	270
8.8 petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/PetDialog.java File Reference	271
8.8.1 Detailed Description	271
8.9 PetDialog.java	271
8.10 petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/ReminderDialog.java File Reference	274
8.10.1 Detailed Description	274
8.11 ReminderDialog.java	274
8.12 petreminder-app/src/main/java/com/mehdi/petreminder/gui/MainFrame.java File Reference	277
8.12.1 Detailed Description	278
8.13 MainFrame.java	278
8.14 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/DashboardPanel.java File Reference	280
8.14.1 Detailed Description	280
8.15 DashboardPanel.java	281
8.16 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/MedicalPanel.java File Reference	282
8.16.1 Detailed Description	283
8.17 MedicalPanel.java	283
8.18 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/NavPanel.java File Reference	285
8.18.1 Detailed Description	285
8.19 NavPanel.java	285
8.20 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/PetsPanel.java File Reference	287
8.20.1 Detailed Description	287
8.21 PetsPanel.java	287
8.22 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/RemindersPanel.java File Reference	290
8.22.1 Detailed Description	291

8.23 RemindersPanel.java	291
8.24 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/SettingsPanel.java File Reference	293
8.24.1 Detailed Description	293
8.25 SettingsPanel.java	294
8.26 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/VetPanel.java File Reference . .	295
8.26.1 Detailed Description	296
8.27 VetPanel.java	296
8.28 petreminder-app/src/main/java/com/mehdi/petreminder/gui/util/GuiConstants.java File Reference . .	298
8.28.1 Detailed Description	298
8.29 GuiConstants.java	299
8.30 petreminder-app/src/main/java/com/mehdi/petreminder/model/Bird.java File Reference	299
8.30.1 Detailed Description	300
8.31 Bird.java	300
8.32 petreminder-app/src/main/java/com/mehdi/petreminder/model/Cat.java File Reference	300
8.32.1 Detailed Description	301
8.33 Cat.java	301
8.34 petreminder-app/src/main/java/com/mehdi/petreminder/model/Dog.java File Reference	302
8.34.1 Detailed Description	302
8.35 Dog.java	302
8.36 petreminder-app/src/main/java/com/mehdi/petreminder/model/ExerciseReminder.java File Reference	303
8.36.1 Detailed Description	303
8.37 ExerciseReminder.java	303
8.38 petreminder-app/src/main/java/com/mehdi/petreminder/model/FeedingReminder.java File Reference	304
8.38.1 Detailed Description	305
8.39 FeedingReminder.java	305
8.40 petreminder-app/src/main/java/com/mehdi/petreminder/model/GroomingReminder.java File Reference	306
8.40.1 Detailed Description	306
8.41 GroomingReminder.java	306
8.42 petreminder-app/src/main/java/com/mehdi/petreminder/model/MedicalRecord.java File Reference .	307
8.42.1 Detailed Description	308
8.43 MedicalRecord.java	308
8.44 petreminder-app/src/main/java/com/mehdi/petreminder/model/MedicationReminder.java File Refer- ence	309
8.44.1 Detailed Description	310
8.45 MedicationReminder.java	310
8.46 petreminder-app/src/main/java/com/mehdi/petreminder/model/Pet.java File Reference	311
8.46.1 Detailed Description	312
8.47 Pet.java	312
8.48 petreminder-app/src/main/java/com/mehdi/petreminder/model/Reminder.java File Reference	313
8.48.1 Detailed Description	314
8.49 Reminder.java	314
8.50 petreminder-app/src/main/java/com/mehdi/petreminder/model/User.java File Reference	315

8.50.1 Detailed Description	316
8.51 User.java	316
8.52 petreminder-app/src/main/java/com/mehdi/petreminder/model/VetAppointment.java File Reference	317
8.52.1 Detailed Description	317
8.53 VetAppointment.java	318
8.54 petreminder-app/src/main/java/com/mehdi/petreminder/observer/EventManager.java File Reference	319
8.54.1 Detailed Description	319
8.55 EventManager.java	319
8.56 petreminder-app/src/main/java/com/mehdi/petreminder/observer/EventType.java File Reference	321
8.56.1 Detailed Description	321
8.57 EventType.java	321
8.58 petreminder-app/src/main/java/com/mehdi/petreminder/observer/PetReminderObserver.java File Reference	321
8.58.1 Detailed Description	322
8.59 PetReminderObserver.java	322
8.60 petreminder-app/src/main/java/com/mehdi/petreminder/petreminder.java File Reference	322
8.60.1 Detailed Description	322
8.61 petreminder.java	323
8.62 petreminder-app/src/main/java/com/mehdi/petreminder/petreminderApp.java File Reference	323
8.62.1 Detailed Description	323
8.63 petreminderApp.java	324
8.64 petreminder-app/src/main/java/com/mehdi/petreminder/repository/BinaryRepository.java File Reference	325
8.64.1 Detailed Description	325
8.65 BinaryRepository.java	325
8.66 petreminder-app/src/main/java/com/mehdi/petreminder/repository/IRepository.java File Reference	327
8.66.1 Detailed Description	328
8.67 IRepository.java	328
8.68 petreminder-app/src/main/java/com/mehdi/petreminder/repository/MedicalRecordMySqlRepository.java File Reference	328
8.68.1 Detailed Description	329
8.69 MedicalRecordMySqlRepository.java	329
8.70 petreminder-app/src/main/java/com/mehdi/petreminder/repository/MedicalRecordSqliteRepository.java File Reference	330
8.70.1 Detailed Description	331
8.71 MedicalRecordSqliteRepository.java	331
8.72 petreminder-app/src/main/java/com/mehdi/petreminder/repository/MySqlRepository.java File Reference	332
8.72.1 Detailed Description	333
8.73 MySqlRepository.java	333
8.74 petreminder-app/src/main/java/com/mehdi/petreminder/repository/PetMySqlRepository.java File Reference	335
8.74.1 Detailed Description	335
8.75 PetMySqlRepository.java	336

8.76	petreminder-app/src/main/java/com/mehdi/petreminder/repository/PetSqliteRepository.java File Reference	337
8.76.1	Detailed Description	338
8.77	PetSqliteRepository.java	338
8.78	petreminder-app/src/main/java/com/mehdi/petreminder/repository/ReminderMySqlRepository.java File Reference	340
8.78.1	Detailed Description	340
8.79	ReminderMySqlRepository.java	340
8.80	petreminder-app/src/main/java/com/mehdi/petreminder/repository/ReminderSqliteRepository.java File Reference	343
8.80.1	Detailed Description	343
8.81	ReminderSqliteRepository.java	343
8.82	petreminder-app/src/main/java/com/mehdi/petreminder/repository/RepositoryException.java File Reference	346
8.82.1	Detailed Description	346
8.83	RepositoryException.java	346
8.84	petreminder-app/src/main/java/com/mehdi/petreminder/repository/RepositoryFactory.java File Reference	347
8.84.1	Detailed Description	347
8.85	RepositoryFactory.java	347
8.86	petreminder-app/src/main/java/com/mehdi/petreminder/repository/SqliteRepository.java File Reference	348
8.86.1	Detailed Description	349
8.87	SqliteRepository.java	349
8.88	petreminder-app/src/main/java/com/mehdi/petreminder/repository/UserMySqlRepository.java File Reference	351
8.88.1	Detailed Description	352
8.89	UserMySqlRepository.java	352
8.90	petreminder-app/src/main/java/com/mehdi/petreminder/repository/UserSqliteRepository.java File Reference	353
8.90.1	Detailed Description	353
8.91	UserSqliteRepository.java	353
8.92	petreminder-app/src/main/java/com/mehdi/petreminder/service/MedicalRecordService.java File Reference	354
8.92.1	Detailed Description	355
8.93	MedicalRecordService.java	355
8.94	petreminder-app/src/main/java/com/mehdi/petreminder/service/PetService.java File Reference	356
8.94.1	Detailed Description	357
8.95	PetService.java	357
8.96	petreminder-app/src/main/java/com/mehdi/petreminder/service/ReminderService.java File Reference	358
8.96.1	Detailed Description	359
8.97	ReminderService.java	359
8.98	petreminder-app/src/main/java/com/mehdi/petreminder/service/ServiceException.java File Reference	361
8.98.1	Detailed Description	361
8.99	ServiceException.java	361

Chapter 1

PetReminder Application

1.1 Overview

PetReminder is a comprehensive Java-based desktop application designed to help pet owners manage their pets' care routines, medical records, and veterinary appointments. Built using modern Java technologies including Swing (with FlatLaf), JDBC (SQLite/MySQL), and JUnit 5, it ensures robust performance, excellent test coverage, and a user-friendly interface.

1.2 Features

- **Pet Management:** Add, view, edit, and delete pet profiles (Cats, Dogs, Birds, etc.).
- **Reminders:** Schedule feeding, medication, exercise, and grooming routines.
- **Medical Records:** Keep track of vaccinations, surgeries, and treatments.
- **Vet Appointments:** Log and schedule upcoming visits to the veterinarian.
- **Data Persistence:** Supports SQLite for local storage and MySQL for distributed usage.

1.3 Architecture

- **Language:** Java 11+
- **UI Framework:** Java Swing with FlatLaf Light Theme and MigLayout
- **Database:** SQLite (default), MySQL, H2 (Testing)
- **Testing:** JUnit 5, Mockito
- **Build Tool:** Apache Maven

1.4 How to Run

Use the provided batch scripts in the repository root:

1. `4-install-required-apps.bat` - Install required local dependencies such as Doxygen, Lcov, ReportGenerator.
2. `7-build-app.bat` - Compiles the project, runs tests, and generates the fat JAR along with documentation (Doxygen) and coverage reports.
3. `8-run-app.bat` - Launches the PetReminder desktop application standalone.
4. `9-run-webpage.bat` - Starts a local web server (Maven Site) to view this documentation and Doxygen HTML outputs (port 9000).

1.5 Documentation

The source code is thoroughly documented using Javadoc. To view the Doxygen documentation, build the project using `7-build-app.bat` and run `9-run-webpage.bat`. Then navigate to <http://localhost:9000/doxygen/html/index.html>.

1.6 Team Members

Developed by students at Recep Tayyip Erdogan University (RTEU):

- Ibrahim Demirci
- Muhammed Mehdi Karagulle
- Zumre Uykun

1.7 License

This project is licensed under the **GNU Affero General Public License v3.0 (AGPL-3.0)**. See the [LICENSE](#) file in the root directory for more details.

Chapter 2

Namespace Index

2.1 Package List

Here are the packages with brief descriptions (if available):

com.mehdi.petreminder.gui	15
com.mehdi.petreminder.gui.util	15
com.mehdi.petreminder.model	16
com.mehdi.petreminder.repository	16
com.mehdi.petreminder.service	17

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

com.mehdi.petreminder.observer.EventManager	45
com.mehdi.petreminder.observer.EventType	49
com.mehdi.petreminder.annotation.Generated	67
com.mehdi.petreminder.Generated	67
com.mehdi.petreminder.gui.util.GuiConstants	75
com.mehdi.petreminder.repository.IRepository< T >	81
com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >	19
com.mehdi.petreminder.repository.MySqlRepository< T >	127
com.mehdi.petreminder.repository.SqliteRepository< T >	217
JDialog	
com.mehdi.petreminder.gui.dialogs.PetDialog	150
com.mehdi.petreminder.gui.dialogs.ReminderDialog	188
JFrame	
com.mehdi.petreminder.gui.MainFrame	85
JPanel	
com.mehdi.petreminder.gui.panels.DashboardPanel	37
com.mehdi.petreminder.gui.panels.MedicalPanel	89
com.mehdi.petreminder.gui.panels.NavPanel	138
com.mehdi.petreminder.gui.panels.PetsPanel	169
com.mehdi.petreminder.gui.panels.RemindersPanel	202
com.mehdi.petreminder.gui.panels.SettingsPanel	216
com.mehdi.petreminder.gui.panels.VetPanel	263
com.mehdi.petreminder.service.MedicalRecordService	107
com.mehdi.petreminder.repository.MySqlRepository< MedicalRecord >	127
com.mehdi.petreminder.repository.MedicalRecordMySqlRepository	103
com.mehdi.petreminder.repository.MySqlRepository< Pet >	127
com.mehdi.petreminder.repository.PetMySqlRepository	152
com.mehdi.petreminder.repository.MySqlRepository< Reminder >	127
com.mehdi.petreminder.repository.ReminderMySqlRepository	190
com.mehdi.petreminder.repository.MySqlRepository< User >	127
com.mehdi.petreminder.repository.UserMySqlRepository	244
com.mehdi.petreminder.petreminder	157
com.mehdi.petreminder.petreminderApp	159
com.mehdi.petreminder.observer.PetReminderObserver	162

com.mehdi.petreminder.service.PetService	163
com.mehdi.petreminder.service.ReminderService	195
com.mehdi.petreminder.repository.RepositoryFactory	211
RuntimeException	
com.mehdi.petreminder.repository.RepositoryException	210
com.mehdi.petreminder.service.ServiceException	214
Serializable	
com.mehdi.petreminder.model.MedicalRecord	91
com.mehdi.petreminder.model.Pet	140
com.mehdi.petreminder.model.Bird	25
com.mehdi.petreminder.model.Cat	31
com.mehdi.petreminder.model.Dog	39
com.mehdi.petreminder.model.Reminder	176
com.mehdi.petreminder.model.ExerciseReminder	52
com.mehdi.petreminder.model.FeedingReminder	59
com.mehdi.petreminder.model.GroomingReminder	68
com.mehdi.petreminder.model.MedicationReminder	118
com.mehdi.petreminder.model.VetAppointment	254
com.mehdi.petreminder.model.User	236
com.mehdi.petreminder.repository.SqliteRepository< MedicalRecord >	217
com.mehdi.petreminder.repository.MedicalRecordSqliteRepository	112
com.mehdi.petreminder.repository.SqliteRepository< Pet >	217
com.mehdi.petreminder.repository.PetSqliteRepository	171
com.mehdi.petreminder.repository.SqliteRepository< Reminder >	217
com.mehdi.petreminder.repository.ReminderSqliteRepository	203
com.mehdi.petreminder.repository.SqliteRepository< User >	217
com.mehdi.petreminder.repository.UserSqliteRepository	249
com.mehdi.petreminder.config.StorageConfig	228
com.mehdi.petreminder.config.StorageType	234

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >	19
Generic binary (Object serialization) repository implementasyonu	
com.mehdi.petreminder.model.Bird	25
Kus evcil hayvan sinifi	
com.mehdi.petreminder.model.Cat	31
Kedi evcil hayvan sinifi	
com.mehdi.petreminder.gui.panels.DashboardPanel	37
Ana kontrol paneli — özet kartlar ve bekleyen hatırlatıcılar	
com.mehdi.petreminder.model.Dog	39
Kopek evcil hayvan sinifi	
com.mehdi.petreminder.observer.EventManager	45
Observer Pattern Subject — olay yönetici	
com.mehdi.petreminder.observer.EventType	49
com.mehdi.petreminder.model.ExerciseReminder	52
Evcil hayvan egzersiz hatırlatıcısı	
com.mehdi.petreminder.model.FeedingReminder	59
Evcil hayvan besleme hatırlatıcısı	
com.mehdi.petreminder.annotation.Generated	67
com.mehdi.petreminder.Generated	67
Marks methods/constructors to be excluded from JaCoCo coverage	
com.mehdi.petreminder.model.GroomingReminder	68
Evcil hayvan tıraş ve bakım hatırlatıcısı	
com.mehdi.petreminder.gui.util.GuiConstants	75
GUI bileşenlerinde kullanılan sabitler	
com.mehdi.petreminder.repository.IRepository< T >	81
Generic CRUD repository arayüzü	
com.mehdi.petreminder.gui.MainFrame	85
Uygulamanın ana Swing penceresi	
com.mehdi.petreminder.gui.panels.MedicalPanel	89
Sağlık kayıtları CRUD paneli	
com.mehdi.petreminder.model.MedicalRecord	91
Evcil hayvanın sağlık kaydı	
com.mehdi.petreminder.repository.MedicalRecordMySqlRepository	103
MySQL üzerinde MedicalRecord CRUD	
com.mehdi.petreminder.service.MedicalRecordService	107
MedicalRecord CRUD ve iş kuralları	

com.mehdi.petreminder.repository.MedicalRecordSqliteRepository	
SQLite üzerinde MedicalRecord CRUD	112
com.mehdi.petreminder.model.MedicationReminder	
Evcil hayvan ilaç hatırlatıcısı	118
com.mehdi.petreminder.repository.MySqlRepository< T >	
MySQL tabanlı abstract repository temel sınıfı	127
com.mehdi.petreminder.gui.panels.NavPanel	
Sol navigasyon çubuğu — menü butonları	138
com.mehdi.petreminder.model.Pet	
Evcil hayvanın temel soyut sınıfı	140
com.mehdi.petreminder.gui.dialogs.PetDialog	
Pet ekleme ve düzenleme modal dialog'u	150
com.mehdi.petreminder.repository.PetMySqlRepository	
MySQL üzerinde Pet CRUD işlemleri	152
com.mehdi.petreminder.petreminder	
Pet Care Reminder System yardımcı/facade sınıfı	157
com.mehdi.petreminder.petreminderApp	
Pet Care Reminder System ana sınıfı	159
com.mehdi.petreminder.observer.PetReminderObserver	
Gözlemci arayuzu — durum değişikliklerini dinler	162
com.mehdi.petreminder.service.PetService	
Pet CRUD iş mantığı katmanı	163
com.mehdi.petreminder.gui.panels.PetsPanel	
Evcil hayvanlar listesi ve CRUD işlemleri	169
com.mehdi.petreminder.repository.PetSqliteRepository	
SQLite üzerinde Pet CRUD işlemleri	171
com.mehdi.petreminder.model.Reminder	
Evcil hayvan bakım hatırlatıcısının soyut temel sınıfı	176
com.mehdi.petreminder.gui.dialogs.ReminderDialog	
Reminder ekleme ve düzenleme modal dialog'u	188
com.mehdi.petreminder.repository.ReminderMySqlRepository	
MySQL üzerinde Reminder CRUD işlemleri	190
com.mehdi.petreminder.service.ReminderService	
Reminder CRUD ve iş kuralları	195
com.mehdi.petreminder.gui.panels.RemindersPanel	
Hatırlatıcılar CRUD paneli	202
com.mehdi.petreminder.repository.ReminderSqliteRepository	
SQLite üzerinde Reminder (tüm alt türler) CRUD işlemleri	203
com.mehdi.petreminder.repository.RepositoryException	
Repository işlemlerinde fırlatılan unchecked istisna	210
com.mehdi.petreminder.repository.RepositoryFactory	
Aktif storage backend'e göre repository üretir	211
com.mehdi.petreminder.service.ServiceException	
Servis katmanında fırlatılan unchecked istisna	214
com.mehdi.petreminder.gui.panels.SettingsPanel	
Storage backend ve uygulama ayarları	216
com.mehdi.petreminder.repository.SqliteRepository< T >	
SQLite tabanlı abstract repository temel sınıfı	217
com.mehdi.petreminder.config.StorageConfig	
Aktif storage backend'i yönetir	228
com.mehdi.petreminder.config.StorageType	234
com.mehdi.petreminder.model.User	
Uygulamayı kullanan evcil hayvan sahibi	236
com.mehdi.petreminder.repository.UserMySqlRepository	
MySQL üzerinde User CRUD işlemleri	244
com.mehdi.petreminder.repository.UserSqliteRepository	
SQLite üzerinde User CRUD işlemleri	249

com.mehdi.petreminder.model.VetAppointment	
Veteriner randevusu — Reminder 'dan türetilmiş somut sınıf	254
com.mehdi.petreminder.gui.panels.VetPanel	
Veteriner randevuları CRUD	263

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

petreminder-app/src/main/java/com/mehdi/petreminder/Generated.java	
JaCoCo coverage exclusion annotation	267
petreminder-app/src/main/java/com/mehdi/petreminder/petreminder.java	
Pet Care Reminder System kütüphane yardımcı sınıfı	322
petreminder-app/src/main/java/com/mehdi/petreminder/petreminderApp.java	
Pet Care Reminder System ana uygulama giriş noktası	323
petreminder-app/src/main/java/com/mehdi/petreminder/annotation/Generated.java	267
petreminder-app/src/main/java/com/mehdi/petreminder/config/StorageConfig.java	
Storage backend konfigürasyonu — runtime switch destekler	268
petreminder-app/src/main/java/com/mehdi/petreminder/config/StorageType.java	
Storage backend türleri	270
petreminder-app/src/main/java/com/mehdi/petreminder/gui/MainFrame.java	
Ana Swing penceresi — tüm panelleri barındırır	277
petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/PetDialog.java	
Pet ekleme / düzenleme diyalogu	271
petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/ReminderDialog.java	
Hatırlatıcı ekleme / düzenleme diyalogu	274
petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/DashboardPanel.java	
Dashboard — özet istatistikler ve son hatırlatıcılar	280
petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/MedicalPanel.java	
Sağlık kayıtları yönetim paneli	282
petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/NavPanel.java	
Sol navigasyon paneli	285
petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/PetsPanel.java	
Evciil hayvanlar yönetim paneli — tam CRUD	287
petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/RemindersPanel.java	
Hatırlatıcılar yönetim paneli	290
petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/SettingsPanel.java	
Ayarlar paneli — runtime storage backend değiştirme	293
petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/VetPanel.java	
Veteriner randevuları paneli	295
petreminder-app/src/main/java/com/mehdi/petreminder/gui/util/GuiConstants.java	
GUI için renk, font ve boyut sabitleri	298
petreminder-app/src/main/java/com/mehdi/petreminder/model/Bird.java	
Kus turunu temsil eden somut sınıf	299

petreminder-app/src/main/java/com/mehdi/petreminder/model/Cat.java	
Kedi turunu temsil eden somut sınıf	300
petreminder-app/src/main/java/com/mehdi/petreminder/model/Dog.java	
Kopek turunu temsil eden somut sınıf	302
petreminder-app/src/main/java/com/mehdi/petreminder/model/ExerciseReminder.java	
Egzersiz hatırlatıcısı sınıfı	303
petreminder-app/src/main/java/com/mehdi/petreminder/model/FeedingReminder.java	
Besleme hatırlatıcısı sınıfı	304
petreminder-app/src/main/java/com/mehdi/petreminder/model/GroomingReminder.java	
Tıraş/bakım hatırlatıcısı sınıfı	306
petreminder-app/src/main/java/com/mehdi/petreminder/model/MedicalRecord.java	
Evcil hayvan sağlık kaydı modeli	307
petreminder-app/src/main/java/com/mehdi/petreminder/model/MedicationReminder.java	
İlaç hatırlatıcısı sınıfı	309
petreminder-app/src/main/java/com/mehdi/petreminder/model/Pet.java	
Tüm evcil hayvan türleri için soyut temel sınıf	311
petreminder-app/src/main/java/com/mehdi/petreminder/model/Reminder.java	
Tüm hatırlatıcı türleri için soyut temel sınıf	313
petreminder-app/src/main/java/com/mehdi/petreminder/model/User.java	
Kullanıcı modeli sınıfı	315
petreminder-app/src/main/java/com/mehdi/petreminder/model/VetAppointment.java	
Veteriner randevu hatırlatıcısı sınıfı	317
petreminder-app/src/main/java/com/mehdi/petreminder/observer/EventManager.java	
Observer Pattern — Subject (yayinci) sınıfı	319
petreminder-app/src/main/java/com/mehdi/petreminder/observer/EventType.java	
Observer Pattern olay türleri	321
petreminder-app/src/main/java/com/mehdi/petreminder/observer/PetReminderObserver.java	
Observer Design Pattern — gözlemci arayuzu	321
petreminder-app/src/main/java/com/mehdi/petreminder/repository/BinaryRepository.java	
Binary file I/O repository — ObjectOutputStream / .bin dosyası	325
petreminder-app/src/main/java/com/mehdi/petreminder/repository/IRepository.java	
Generic repository interface — PDF LO.6 zorunluluğu	327
petreminder-app/src/main/java/com/mehdi/petreminder/repository/MedicalRecordMySqlRepository.java	
MedicalRecord için MySQL repository	328
petreminder-app/src/main/java/com/mehdi/petreminder/repository/MedicalRecordSqliteRepository.java	
MedicalRecord için SQLite repository	330
petreminder-app/src/main/java/com/mehdi/petreminder/repository/MySqlRepository.java	
MySQL JDBC repository — mysql-connector-j + Docker Compose	332
petreminder-app/src/main/java/com/mehdi/petreminder/repository/PetMySqlRepository.java	
Pet varlığı için MySQL repository implementasyonu	335
petreminder-app/src/main/java/com/mehdi/petreminder/repository/PetSqliteRepository.java	
Pet varlığı için SQLite repository implementasyonu	337
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ReminderMySqlRepository.java	
Reminder varlığı için MySQL repository	340
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ReminderSqliteRepository.java	
Reminder varlığı için SQLite repository	343
petreminder-app/src/main/java/com/mehdi/petreminder/repository/RepositoryException.java	
Repository katmanı için özel istisna sınıfı	346
petreminder-app/src/main/java/com/mehdi/petreminder/repository/RepositoryFactory.java	
Repository nesnesi üretici — Factory Pattern	347
petreminder-app/src/main/java/com/mehdi/petreminder/repository/SqliteRepository.java	
SQLite JDBC repository — org.xerial:sqlite-jdbc	348
petreminder-app/src/main/java/com/mehdi/petreminder/repository/UserMySqlRepository.java	
User varlığı için MySQL repository	351
petreminder-app/src/main/java/com/mehdi/petreminder/repository/UserSqliteRepository.java	
User varlığı için SQLite repository	353
petreminder-app/src/main/java/com/mehdi/petreminder/service/MedicalRecordService.java	
Sağlık kayıtları servis katmanı	354

petreminder-app/src/main/java/com/mehdi/petreminder/service/ PetService.java	
Pet iş mantığı servis katmanı	356
petreminder-app/src/main/java/com/mehdi/petreminder/service/ ReminderService.java	
Reminder iş mantığı servis katmanı	358
petreminder-app/src/main/java/com/mehdi/petreminder/service/ ServiceException.java	
Servis katmanı özel istisna sınıfı	361

Chapter 6

Namespace Documentation

6.1 Package com.mehdi.petreminder.gui

Packages

- package [util](#)

Classes

- class [MainFrame](#)
Uygulamanın ana Swing penceresi.

6.1.1 Detailed Description

Member documentation.

6.2 Package com.mehdi.petreminder.gui.util

Classes

- class [GuiConstants](#)
GUI bileşenlerinde kullanılan sabitler.

6.2.1 Detailed Description

Member documentation.

6.3 Package com.mehdi.petreminder.model

Classes

- class [Bird](#)
Kus evcil hayvan sınıfı.
- class [Cat](#)
Kedi evcil hayvan sınıfı.
- class [Dog](#)
Kopek evcil hayvan sınıfı.
- class [ExerciseReminder](#)
Evcil hayvan egzersiz hatırlatıcısı.
- class [FeedingReminder](#)
Evcil hayvan besleme hatırlatıcısı.
- class [GroomingReminder](#)
Evcil hayvan tıraş ve bakım hatırlatıcısı.
- class [MedicalRecord](#)
Evcil hayvanın sağlık kaydı.
- class [MedicationReminder](#)
Evcil hayvan ilaç hatırlatıcısı.
- class [Pet](#)
Evcil hayvanın temel soyut sınıfı.
- class [Reminder](#)
Evcil hayvan bakım hatırlatıcısının soyut temel sınıfı.
- class [User](#)
Uygulamayı kullanan evcil hayvan sahibi.
- class [VetAppointment](#)
Veteriner randevusu — [Reminder](#)'dan türetilmiş somut sınıf.

6.3.1 Detailed Description

Member documentation.

6.4 Package com.mehdi.petreminder.repository

Classes

- class [BinaryRepository](#)
Generic binary (Object serialization) repository implementasyonu.
- interface [IRepository](#)
Generic CRUD repository arayüzü.
- class [MedicalRecordMySqlRepository](#)
MySQL üzerinde [MedicalRecord](#) CRUD.
- class [MedicalRecordSqliteRepository](#)
SQLite üzerinde [MedicalRecord](#) CRUD.
- class [MySqlRepository](#)
MySQL tabanlı abstract repository temel sınıfı.
- class [PetMySqlRepository](#)

- MySQL üzerinde [Pet](#) CRUD işlemleri.*
- class [PetSqliteRepository](#)
SQLite üzerinde [Pet](#) CRUD işlemleri.
- class [ReminderMySqlRepository](#)
MySQL üzerinde [Reminder](#) CRUD işlemleri.
- class [ReminderSqliteRepository](#)
SQLite üzerinde [Reminder](#) (tüm alt türler) CRUD işlemleri.
- class [RepositoryException](#)
Repository işlemlerinde fırlatılan unchecked istisna.
- class [RepositoryFactory](#)
Aktif storage backend'e göre repository üretir.
- class [SqliteRepository](#)
SQLite tabanlı abstract repository temel sınıfı.
- class [UserMySqlRepository](#)
MySQL üzerinde [User](#) CRUD işlemleri.
- class [UserSqliteRepository](#)
SQLite üzerinde [User](#) CRUD işlemleri.

6.4.1 Detailed Description

Member documentation.

6.5 Package com.mehdi.petreminder.service

Classes

- class [MedicalRecordService](#)
[MedicalRecord](#) CRUD ve iş kuralları.
- class [PetService](#)
[Pet](#) CRUD iş mantığı katmanı.
- class [ReminderService](#)
[Reminder](#) CRUD ve iş kuralları.
- class [ServiceException](#)
Servis katmanında fırlatılan unchecked istisna.

6.5.1 Detailed Description

Member documentation.

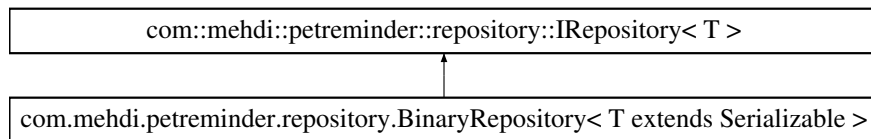
Chapter 7

Class Documentation

7.1 com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable > Class Template Reference

Generic binary (Object serialization) repository implementasyonu.

Inheritance diagram for com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >:



Public Member Functions

- [BinaryRepository](#) (String directoryPath, String entityName)

BinaryRepository yapıcısı.

- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- String [getFilePath](#) ()

Dosya yolunu döndürür (test için).

7.1.1 Detailed Description

Generic binary (Object serialization) repository implementasyonu.

OOP Generics: T extends Serializable. Her varlık türü için ayrı .bin dosyası kullanılır. Thread-safe değil — tek kullanıcı varsayılır.

Parameters

<T>	Serializable varlık türü
-----	--------------------------

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 31 of file [BinaryRepository.java](#).

7.1.2 Constructor & Destructor Documentation

7.1.2.1 BinaryRepository()

```
com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.BinaryRepository (
    String directoryPath,
    String entityName)
```

[BinaryRepository](#) yapıcısı.

Parameters

<i>directoryPath</i>	Dizin yolu (örn. data/binary)
<i>entityName</i>	Varlık adı (örn. pets, reminders)

Definition at line 48 of file [BinaryRepository.java](#).

```
00048                                     {
00049         File dir = new File(directoryPath);
00050         if (!dir.exists()) {
00051             dir.mkdirs();
00052         }
00053         this.filePath = directoryPath + File.separator + entityName + ".bin";
00054         initNextId();
00055     }
```

7.1.3 Member Function Documentation

7.1.3.1 close()

```
void com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.close ()
```

Repository bağlantısını/kaynaklarını kapatır.

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 228 of file [BinaryRepository.java](#).

```
00228         {
00229             // Binary repository için kaynak kapatma gerekmez
00230             logger.debug("BinaryRepository kapatıldı.");
00231         }
```

7.1.3.2 count()

```
int com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.count ()
```

Varlık sayısını döndürür.

Returns

Toplam kayıt sayısı

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 220 of file [BinaryRepository.java](#).

```
00220         {
00221             return findAll().size();
00222         }
```

References [findAll\(\)](#).

7.1.3.3 delete()

```
boolean com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.delete (
    int id)
```

ID'ye göre varlık siler.

Parameters

<i>id</i>	Silinecek varlığın ID'si
-----------	--------------------------

Returns

Silme başarılıysa true

Exceptions

RepositoryException	Silme başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 196 of file [BinaryRepository.java](#).

```

00196         {
00197             List<T> all = findAll();
00198             boolean removed = all.removeIf(e -> getEntityId(e) == id);
00199             if (removed) {
00200                 writeAll(all);
00201                 logger.debug("Silindi: id={}", id);
00202             }
00203             return removed;
00204         }

```

References [findAll\(\)](#).

7.1.3.4 deleteAll()

void [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#).deleteAll ()

Tüm varlıkları siler.

Exceptions

RepositoryException	Silme başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 210 of file [BinaryRepository.java](#).

```

00210         {
00211             writeAll(new ArrayList<>());
00212             nextId = 1;
00213             logger.debug("Tümü silindi: {}", filePath);
00214         }

```

7.1.3.5 findAll()

List< T > [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#).findAll
()

Tüm varlıkları listeler.

Returns

Varlık listesi (boş olabilir)

Exceptions

RepositoryException	Okuma başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 148 of file [BinaryRepository.java](#).

```

00148         {
00149             File file = new File(filePath);
00150             if (!file.exists() || file.length() == 0) {
00151                 return new ArrayList<>();
00152             }
00153             try (FileInputStream fis = new FileInputStream(file);
00154                  ObjectInputStream ois = new ObjectInputStream(fis)) {
00155                 Object obj = ois.readObject();
00156                 if (obj instanceof List) {
00157                     return (List<T>) obj;
00158                 }
00159             } catch (EOFException e) {
00160                 return new ArrayList<>();
00161             } catch (Exception e) {
00162                 logger.error("Binary okuma hatası: {}", e.getMessage());
00163                 throw new RepositoryException("Binary okuma hatası", e);
00164             }
00165             return new ArrayList<>();
00166         }

```

References [findAll\(\)](#).

Referenced by [count\(\)](#), [delete\(\)](#), [findAll\(\)](#), [findById\(\)](#), [save\(\)](#), and [update\(\)](#).

7.1.3.6 findById()

Optional< T > [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.findById](#)(
 ById (

int id)

ID'ye göre varlık getirir.

Parameters

<i>id</i>	Varlık ID'si
-----------	--------------

Returns

Varlık (bulunamazsa empty Optional)

Exceptions

RepositoryException	Okuma başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 136 of file [BinaryRepository.java](#).

```

00136         {
00137             return findAll().stream()
00138                 .filter(e -> getEntityId(e) == id)
00139                 .findFirst();
00140         }

```

References [findAll\(\)](#).

7.1.3.7 `getFilePath()`

String `com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.getFilePath()`

Dosya yolunu döndürür (test için).

Returns

Dosya yolu

Definition at line 251 of file [BinaryRepository.java](#).

```
00251         {
00252             return filePath;
00253         }
```

7.1.3.8 `save()`

int `com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.save (T entity)`

Yeni bir varlık kaydeder.

Parameters

<i>entity</i>	Kaydedilecek varlık
---------------	---------------------

Returns

Oluşturulan varlığın ID'si

Exceptions

<i>RepositoryException</i>	Kayıt başarısız olursa
--	------------------------

Implements `com.mehdi.petreminder.repository.IRepository< T >`.

Definition at line 119 of file [BinaryRepository.java](#).

```
00119         {
00120             if (entity == null) {
00121                 throw new RepositoryException("Entity null olamaz");
00122             }
00123             List<T> all = findAll();
00124             int id = nextId++;
00125             setEntityId(entity, id);
00126             all.add(entity);
00127             writeAll(all);
00128             logger.debug("Kaydedildi: id={}, tip={}", id, entity.getClass().getSimpleName());
00129             return id;
00130         }
```

References [findAll\(\)](#).

7.1.3.9 update()

```
boolean com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.update (
    T entity)
```

Mevcut varlığı günceller.

Parameters

<i>entity</i>	Güncellenecek varlık
---------------	----------------------

Returns

Güncelleme başarılıysa true

Exceptions

RepositoryException	Güncelleme başarısız olursa
-------------------------------------	-----------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 173 of file [BinaryRepository.java](#).

```
00173         {
00174             if (entity == null) return false;
00175             int id = getEntityId(entity);
00176             List<T> all = findAll();
00177             boolean found = false;
00178             for (int i = 0; i < all.size(); i++) {
00179                 if (getEntityId(all.get(i)) == id) {
00180                     all.set(i, entity);
00181                     found = true;
00182                     break;
00183                 }
00184             }
00185             if (found) {
00186                 writeAll(all);
00187                 logger.debug("Güncellendi: id={}", id);
00188             }
00189             return found;
00190         }
```

References [findAll\(\)](#).

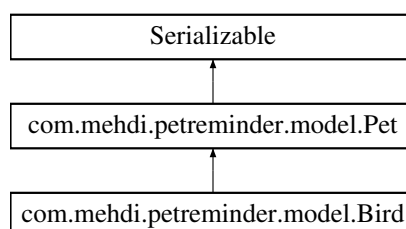
The documentation for this class was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/repository/[BinaryRepository.java](#)

7.2 com.mehdi.petreminder.model.Bird Class Reference

Kus evcil hayvan sınıfı.

Inheritance diagram for com.mehdi.petreminder.model.Bird:



Public Member Functions

- [Bird](#) ()
Varsayilan yapici.
- [Bird](#) (int id, String name, LocalDate birthDate, int ownerId)
Temel bilgilerle yapici.
- [Bird](#) (int id, String name, LocalDate birthDate, int ownerId, String gender, double weight, String notes, String birdType, boolean canTalk)
Tam parametreli yapici.
- String [getBirdType](#) ()
Kus turu getter.
- void [setBirdType](#) (String birdType)
Kus turu setter.
- boolean [isCanTalk](#) ()
Konusma getter.
- void [setCanTalk](#) (boolean canTalk)
Konusma setter.
- String [makeSound](#) ()
Kusa ozgu ses.
- String [getIconName](#) ()
GUI ikon adi.
- String [toString](#) ()
String temsili.

Public Member Functions inherited from [com.mehdi.petreminder.model.Pet](#)

- int [getId](#) ()
ID getter.
- void [setId](#) (int id)
ID setter.
- String [getName](#) ()
Isim getter.
- void [setName](#) (String name)
Isim setter.
- String [getSpecies](#) ()
Tur getter.
- void [setSpecies](#) (String species)
Tur setter.
- LocalDate [getBirthDate](#) ()
Dogum tarihi getter.
- void [setBirthDate](#) (LocalDate birthDate)
Dogum tarihi setter.
- int [getOwnerId](#) ()
Sahip ID getter.
- void [setOwnerId](#) (int ownerId)
Sahip ID setter.
- String [getGender](#) ()
Cinsiyet getter.
- void [setGender](#) (String gender)
Cinsiyet setter.

- double [getWeight](#) ()
Agirlik getter.
- void [setWeight](#) (double weight)
Agirlik setter.
- String [getNotes](#) ()
Notlar getter.
- void [setNotes](#) (String notes)
Notlar setter.
- int [getAge](#) ()
Yas hesaplar.
- String [getAgeString](#) ()
Yas bilgisini formatlı string olarak dondurur.
- boolean [equals](#) (Object obj)
İki [Pet](#) nesnesinin eşitliğini karşılaştırır.
- int [hashCode](#) ()
Hash kodu hesaplar.

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Pet](#)

- [Pet](#) ()
Varsayılan yapıcı metod.
- [Pet](#) (int id, String name, String species, LocalDate birthDate, int ownerId, String gender, double weight, String notes)
Tam parametrelili yapıcı metod.

7.2.1 Detailed Description

Kus evcil hayvan sınıfı.

OOP Inheritance: [Pet](#) sınıfından miras alır. OOP Polymorphism: [makeSound\(\)](#) ve [getIconName\(\)](#) override edilir.

Author

Muhammed Mehdi Karagulle, Ibrahim Demirci, Zumre Uykun

Version

1.0

Definition at line 20 of file [Bird.java](#).

7.2.2 Constructor & Destructor Documentation

7.2.2.1 Bird() [1/3]

```
com.mehdi.petreminder.model.Bird.Bird ()
```

Varsayılan yapıcı.

Definition at line 32 of file [Bird.java](#).

```
00032 { super(); }
```

7.2.2.2 Bird() [2/3]

```
com.mehdi.petreminder.model.Bird.Bird (  
    int id,  
    String name,  
    LocalDate birthDate,  
    int ownerId)
```

Temel bilgilerle yapici.

Parameters

<i>id</i>	Kimlik
<i>name</i>	Isim
<i>birthDate</i>	Dogum tarihi
<i>ownerId</i>	Sahip ID

Definition at line 41 of file [Bird.java](#).

```
00041  
00042         super(id, name, "Kus", birthDate, ownerId, "Erkek", 0.0, "");  
00043         this.birdType = "Muhabbet Kusu";  
00044         this.canTalk = false;  
00045     }
```

7.2.2.3 Bird() [3/3]

```
com.mehdi.petreminder.model.Bird.Bird (  
    int id,  
    String name,  
    LocalDate birthDate,  
    int ownerId,  
    String gender,  
    double weight,  
    String notes,  
    String birdType,  
    boolean canTalk)
```

Tam parametrelili yapici.

Parameters

<i>id</i>	Kimlik
<i>name</i>	Isim
<i>birthDate</i>	Dogum tarihi
<i>ownerId</i>	Sahip ID
<i>gender</i>	Cinsiyet
<i>weight</i>	Agirlik
<i>notes</i>	Notlar

<i>birdType</i>	Kus turu
<i>canTalk</i>	Konusabiliyor mu

Definition at line 59 of file [Bird.java](#).

```
00061      {
00062          super(id, name, "Kus", birthDate, ownerId, gender, weight, notes);
00063          this.birdType = birdType;
00064          this.canTalk = canTalk;
00065      }
```

7.2.3 Member Function Documentation

7.2.3.1 getBirdType()

`String com.mehdi.petreminder.model.Bird.getBirdType ()`

Kus turu getter.

Returns

Kus turu

Definition at line 68 of file [Bird.java](#).

```
00068 { return birdType; }
```

7.2.3.2 getIconName()

`String com.mehdi.petreminder.model.Bird.getIconName ()`

GUI ikon adi.

Returns

"bird"

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 93 of file [Bird.java](#).

```
00093 { return "bird"; }
```

7.2.3.3 isCanTalk()

`boolean com.mehdi.petreminder.model.Bird.isCanTalk ()`

Konusma getter.

Returns

Konusabiliyor mu

Definition at line 74 of file [Bird.java](#).

```
00074 { return canTalk; }
```

7.2.3.4 makeSound()

```
String com.mehdi.petreminder.model.Bird.makeSound ()
```

Kusa ozgu ses.

Returns

"Cik Cik!" veya "Merhaba!" (konusabiliyorsa)

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 84 of file [Bird.java](#).

```
00084         {  
00085         return canTalk ? "Merhaba!" : "Cik Cik!";  
00086     }
```

7.2.3.5 setBirdType()

```
void com.mehdi.petreminder.model.Bird.setBirdType (  
    String birdType)
```

Kus turu setter.

Parameters

<i>birdType</i>	Kus turu
-----------------	----------

Definition at line 71 of file [Bird.java](#).

```
00071 { this.birdType = birdType; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRe](#)

7.2.3.6 setCanTalk()

```
void com.mehdi.petreminder.model.Bird.setCanTalk (  
    boolean canTalk)
```

Konusma setter.

Parameters

<i>canTalk</i>	Konusabiliyor mu
----------------	------------------

Definition at line 77 of file [Bird.java](#).

```
00077 { this.canTalk = canTalk; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRe](#)

7.2.3.7 toString()

`String com.mehdi.petreminder.model.Bird.toString ()`

String temsili.

Returns

Kus bilgilerini iceren string

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 100 of file [Bird.java](#).

```
00100      {
00101          return String.format("Bird{id=%d, name='%s', type='%s', age=%s, canTalk=%b}",
00102                          getId(), getName(), birdType, getAgeString(), canTalk);
00103      }
```

References [com.mehdi.petreminder.model.Pet.getAgeString\(\)](#), [com.mehdi.petreminder.model.Pet.getId\(\)](#), and [com.mehdi.petreminder.model.Pet.getName\(\)](#).

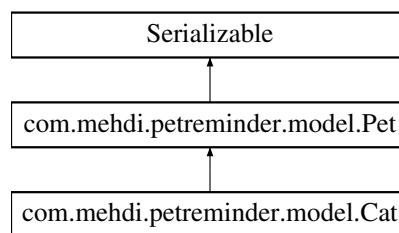
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/Bird.java](#)

7.3 com.mehdi.petreminder.model.Cat Class Reference

Kedi evcil hayvan sinifi.

Inheritance diagram for com.mehdi.petreminder.model.Cat:



Public Member Functions

- [Cat \(\)](#)
Varsayilan yapici.
- [Cat \(int id, String name, LocalDate birthDate, int ownerId\)](#)
Temel bilgilerle yapici.
- [Cat \(int id, String name, LocalDate birthDate, int ownerId, String gender, double weight, String notes, String breed, boolean indoor\)](#)
Tam parametreli yapici.
- String [getBreed \(\)](#)
lrk getter.
- void [setBreed \(String breed\)](#)

- lrk setter.*
- boolean `isIndoor` ()
- lc mekan getter.*
- void `setIndoor` (boolean indoor)
- lc mekan setter.*
- String `makeSound` ()
- Kediye ozgu ses.*
- String `getIconName` ()
- GUI ikon adi.*
- String `toString` ()
- String temsili.*

Public Member Functions inherited from `com.mehdi.petreminder.model.Pet`

- int `getId` ()
- ID getter.*
- void `setId` (int id)
- ID setter.*
- String `getName` ()
- Isim getter.*
- void `setName` (String name)
- Isim setter.*
- String `getSpecies` ()
- Tur getter.*
- void `setSpecies` (String species)
- Tur setter.*
- LocalDate `getBirthDate` ()
- Dogum tarihi getter.*
- void `setBirthDate` (LocalDate birthDate)
- Dogum tarihi setter.*
- int `getOwnerId` ()
- Sahip ID getter.*
- void `setOwnerId` (int ownerId)
- Sahip ID setter.*
- String `getGender` ()
- Cinsiyet getter.*
- void `setGender` (String gender)
- Cinsiyet setter.*
- double `getWeight` ()
- Agirlik getter.*
- void `setWeight` (double weight)
- Agirlik setter.*
- String `getNotes` ()
- Notlar getter.*
- void `setNotes` (String notes)
- Notlar setter.*
- int `getAge` ()
- Yas hesaplar.*
- String `getAgeString` ()
- Yas bilgisini formatlı string olarak dondurur.*
- boolean `equals` (Object obj)
- İki Pet nesnesinin eşitliğini karşılaştırır.*
- int `hashCode` ()
- Hash kodu hesaplar.*

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Pet](#)

- [Pet](#) ()
Varsayilan yapici metod.
- [Pet](#) (int id, String name, String species, LocalDate birthDate, int ownerId, String gender, double weight, String notes)
Tam parametrelili yapici metod.

7.3.1 Detailed Description

Kedi evcil hayvan sinifi.

OOP Inheritance: [Pet](#) sinifindan miras alir. OOP Polymorphism: [makeSound\(\)](#) ve [getIconName\(\)](#) override edilir.

Author

Muhammed Mehdi Karagulle, Ibrahim Demirci, Zumre Uykun

Version

1.0

Definition at line 17 of file [Cat.java](#).

7.3.2 Constructor & Destructor Documentation

7.3.2.1 [Cat\(\)](#) [1/3]

```
com.mehdi.petreminder.model.Cat.Cat ()
```

Varsayilan yapici.

Definition at line 29 of file [Cat.java](#).

```
00029 { super(); }
```

7.3.2.2 Cat() [2/3]

```
com.mehdi.petreminder.model.Cat.Cat (
    int id,
    String name,
    LocalDate birthDate,
    int ownerId)
```

Temel bilgilerle yapici.

Parameters

<i>id</i>	Kimlik
<i>name</i>	Isim
<i>birthDate</i>	Dogum tarihi
<i>ownerId</i>	Sahip ID

Definition at line 38 of file [Cat.java](#).

```
00038
00039         super(id, name, "Kedi", birthDate, ownerId, "Disi", 0.0, "");
00040         this.breed = "Karisik";
00041         this.indoor = true;
00042     }
```

7.3.2.3 Cat() [3/3]

```
com.mehdi.petreminder.model.Cat.Cat (
    int id,
    String name,
    LocalDate birthDate,
    int ownerId,
    String gender,
    double weight,
    String notes,
    String breed,
    boolean indoor)
```

Tam parametrelili yapici.

Parameters

<i>id</i>	Kimlik
<i>name</i>	Isim
<i>birthDate</i>	Dogum tarihi
<i>ownerId</i>	Sahip ID
<i>gender</i>	Cinsiyet
<i>weight</i>	Agirlik
<i>notes</i>	Notlar
<i>breed</i>	Irak

<i>indoor</i>	lc mekan mi
---------------	-------------

Definition at line 56 of file [Cat.java](#).

```
00058      {
00059          super(id, name, "Kedi", birthDate, ownerId, gender, weight, notes);
00060          this.breed = breed;
00061          this.indoor = indoor;
00062      }
```

7.3.3 Member Function Documentation

7.3.3.1 getBreed()

String com.mehdi.petreminder.model.Cat.getBreed ()

Irk getter.

Returns

Kedi irki

Definition at line 65 of file [Cat.java](#).

```
00065 { return breed; }
```

7.3.3.2 getIconName()

String com.mehdi.petreminder.model.Cat.getIconName ()

GUI ikon adi.

Returns

"cat"

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 88 of file [Cat.java](#).

```
00088 { return "cat"; }
```

7.3.3.3 isIndoor()

boolean com.mehdi.petreminder.model.Cat.isIndoor ()

lc mekan getter.

Returns

lc mekan mi

Definition at line 71 of file [Cat.java](#).

```
00071 { return indoor; }
```

7.3.3.4 makeSound()

```
String com.mehdi.petreminder.model.Cat.makeSound ()
```

Kediye ozgu ses.

Returns

"Miyav!"

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 81 of file [Cat.java](#).

```
00081 { return "Miyav!"; }
```

7.3.3.5 setBreed()

```
void com.mehdi.petreminder.model.Cat.setBreed (  
    String breed)
```

Irk setter.

Parameters

<i>breed</i>	Kedi irki
--------------	-----------

Definition at line 68 of file [Cat.java](#).

```
00068 { this.breed = breed; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRe](#)

7.3.3.6 setIndoor()

```
void com.mehdi.petreminder.model.Cat.setIndoor (  
    boolean indoor)
```

Ic mekan setter.

Parameters

<i>indoor</i>	Ic mekan mi
---------------	-------------

Definition at line 74 of file [Cat.java](#).

```
00074 { this.indoor = indoor; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRe](#)

7.3.3.7 toString()

String com.mehdi.petreminder.model.Cat.toString ()

String temsili.

Returns

Kedi bilgilerini iceren string

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 95 of file [Cat.java](#).

```

00095         {
00096             return String.format("Cat{id=%d, name='%s', breed='%s', age=%s, indoor=%b}",
00097                                   getId(), getName(), breed, getAgeString(), indoor);
00098         }

```

References [com.mehdi.petreminder.model.Pet.getAgeString\(\)](#), [com.mehdi.petreminder.model.Pet.getId\(\)](#), and [com.mehdi.petreminder.model.Pet.getName\(\)](#).

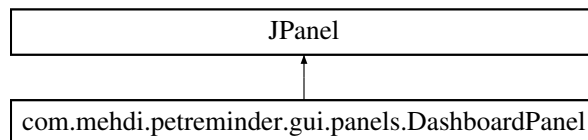
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/Cat.java](#)

7.4 com.mehdi.petreminder.gui.panels.DashboardPanel Class Reference

Ana kontrol paneli — özet kartlar ve bekleyen hatırlatıcılar.

Inheritance diagram for com.mehdi.petreminder.gui.panels.DashboardPanel:



Public Member Functions

- [DashboardPanel](#) ([MainFrame](#) mainFrame)
DashboardPanel yapıcısı.
- void [refresh](#) ()
Verileri yeniler.

7.4.1 Detailed Description

Ana kontrol paneli — özet kartlar ve bekleyen hatırlatıcılar.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 27 of file [DashboardPanel.java](#).

7.4.2 Constructor & Destructor Documentation

7.4.2.1 DashboardPanel()

```
com.mehdi.petreminder.gui.panels.DashboardPanel.DashboardPanel (  
    MainFrame mainFrame)
```

[DashboardPanel](#) yapıcısı.

Parameters

<i>mainFrame</i>	Ana pencere
------------------	-------------

Definition at line 52 of file [DashboardPanel.java](#).

```
00052                                     {  
00053     this.mainFrame = mainFrame;  
00054     petService      = new PetService();  
00055     reminderService = new ReminderService();  
00056     medicalService  = new MedicalRecordService();  
00057     setBackground(GuiConstants.BG_COLOR);  
00058     setLayout(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));  
00059     setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,  
00060         GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));  
00061     buildUI();  
00062 }
```

References [com.mehdi.petreminder.gui.util.GuiConstants.BG_COLOR](#), [com.mehdi.petreminder.gui.util.GuiConstants.GAP](#), and [com.mehdi.petreminder.gui.util.GuiConstants.GAP_LARGE](#).

7.4.3 Member Function Documentation

7.4.3.1 refresh()

```
void com.mehdi.petreminder.gui.panels.DashboardPanel.refresh ()
```

Verileri yeniler.

Definition at line 168 of file [DashboardPanel.java](#).

```
00168             {
00169         try {
00170             lblPetCount.setText(String.valueOf(petService.getPetCount()));
00171             lblReminderCount.setText(String.valueOf(reminderService.getReminderCount()));
00172             List<Reminder> overdue = reminderService.getOverdueReminders();
00173             lblOverdueCount.setText(String.valueOf(overdue.size()));
00174             lblMedicalCount.setText(String.valueOf(medicalService.getRecordCount()));
00175
00176             pendingModel.clear();
00177             List<Reminder> pending = reminderService.getPendingReminders();
00178             for (Reminder r : pending) {
00179                 String item = String.format("[%s] %s -- %s %s",
00180                     r.getPriority(),
00181                     r.getPetName(),
00182                     r.getReminderType(),
00183                     r.getFormattedTime());
00184                 if (r.isOverdue()) item = " " + item;
00185                 pendingModel.addElement(item);
00186             }
00187         } catch (Exception e) {
00188             // Veri yoksa sessizce devam et
00189         }
00190     }
```

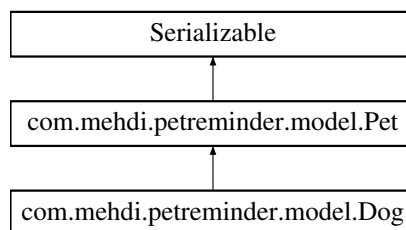
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/DashboardPanel.java](#)

7.5 com.mehdi.petreminder.model.Dog Class Reference

Kopek evcil hayvan sinifi.

Inheritance diagram for com.mehdi.petreminder.model.Dog:



Public Member Functions

- [Dog](#) ()
Varsayilan yapici.
- [Dog](#) (int id, String name, LocalDate birthDate, int ownerId)
Temel bilgilerle yapici.
- [Dog](#) (int id, String name, LocalDate birthDate, int ownerId, String gender, double weight, String notes, String breed, boolean trained)
Tam parametreli yapici.
- String [getBreed](#) ()
Irak getter.
- void [setBreed](#) (String breed)
Irak setter.
- boolean [isTrained](#) ()
Egitim durumu getter.
- void [setTrained](#) (boolean trained)
Egitim durumu setter.
- String [makeSound](#) ()
Kopege ozgu ses.
- String [getIconName](#) ()
GUI ikon adi.
- String [toString](#) ()
String temsili.

Public Member Functions inherited from [com.mehdi.petreminder.model.Pet](#)

- int [getId](#) ()
ID getter.
- void [setId](#) (int id)
ID setter.
- String [getName](#) ()
Isim getter.
- void [setName](#) (String name)
Isim setter.
- String [getSpecies](#) ()
Tur getter.
- void [setSpecies](#) (String species)
Tur setter.
- LocalDate [getBirthDate](#) ()
Dogum tarihi getter.
- void [setBirthDate](#) (LocalDate birthDate)
Dogum tarihi setter.
- int [getOwnerId](#) ()
Sahip ID getter.
- void [setOwnerId](#) (int ownerId)
Sahip ID setter.
- String [getGender](#) ()
Cinsiyet getter.
- void [setGender](#) (String gender)
Cinsiyet setter.

- double [getWeight](#) ()
Agirlik getter.
- void [setWeight](#) (double weight)
Agirlik setter.
- String [getNotes](#) ()
Notlar getter.
- void [setNotes](#) (String notes)
Notlar setter.
- int [getAge](#) ()
Yas hesaplar.
- String [getAgeString](#) ()
Yas bilgisini formatlı string olarak dondurur.
- boolean [equals](#) (Object obj)
İki [Pet](#) nesnesinin eşitliğini karşılaştırır.
- int [hashCode](#) ()
Hash kodu hesaplar.

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Pet](#)

- [Pet](#) ()
Varsayılan yapıcı metod.
- [Pet](#) (int id, String name, String species, LocalDate birthDate, int ownerId, String gender, double weight, String notes)
Tam parametrelili yapıcı metod.

7.5.1 Detailed Description

Kopek evcil hayvan sınıfı.

OOP Inheritance: [Pet](#) sınıfından miras alır. OOP Polymorphism: [makeSound\(\)](#) ve [getIconName\(\)](#) override edilir.

Author

Muhammed Mehdi Karagulle, Ibrahim Demirci, Zumre Uykun

Version

1.0

Definition at line 17 of file [Dog.java](#).

7.5.2 Constructor & Destructor Documentation

7.5.2.1 [Dog\(\)](#) [1/3]

```
com.mehdi.petreminder.model.Dog.Dog ()
```

Varsayılan yapıcı.

Definition at line 29 of file [Dog.java](#).

```
00029 { super(); }
```

7.5.2.2 Dog() [2/3]

```
com.mehdi.petreminder.model.Dog.Dog (
    int id,
    String name,
    LocalDate birthDate,
    int ownerId)
```

Temel bilgilerle yapici.

Parameters

<i>id</i>	Kimlik
<i>name</i>	Isim
<i>birthDate</i>	Dogum tarihi
<i>ownerId</i>	Sahip ID

Definition at line 38 of file [Dog.java](#).

```
00038
00039         super(id, name, "Kopek", birthDate, ownerId, "Erkek", 0.0, "");
00040         this.breed = "Karisik";
00041         this.trained = false;
00042     }
```

7.5.2.3 Dog() [3/3]

```
com.mehdi.petreminder.model.Dog.Dog (
    int id,
    String name,
    LocalDate birthDate,
    int ownerId,
    String gender,
    double weight,
    String notes,
    String breed,
    boolean trained)
```

Tam parametrelili yapici.

Parameters

<i>id</i>	Kimlik
<i>name</i>	Isim
<i>birthDate</i>	Dogum tarihi
<i>ownerId</i>	Sahip ID
<i>gender</i>	Cinsiyet
<i>weight</i>	Agirlik
<i>notes</i>	Notlar

<i>breed</i>	Irk
<i>trained</i>	Egitimli mi

Definition at line 56 of file [Dog.java](#).

```
00058         {
00059             super(id, name, "Kopek", birthDate, ownerId, gender, weight, notes);
00060             this.breed = breed;
00061             this.trained = trained;
00062         }
```

7.5.3 Member Function Documentation

7.5.3.1 getBreed()

String com.mehdi.petreminder.model.Dog.getBreed ()

Irk getter.

Returns

Kopek irki

Definition at line 65 of file [Dog.java](#).

```
00065 { return breed; }
```

7.5.3.2 getIconName()

String com.mehdi.petreminder.model.Dog.getIconName ()

GUI ikon adi.

Returns

"dog"

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 88 of file [Dog.java](#).

```
00088 { return "dog"; }
```

7.5.3.3 isTrained()

boolean com.mehdi.petreminder.model.Dog.isTrained ()

Egitim durumu getter.

Returns

Egitimli mi

Definition at line 71 of file [Dog.java](#).

```
00071 { return trained; }
```

7.5.3.4 makeSound()

```
String com.mehdi.petreminder.model.Dog.makeSound ()
```

Kopege ozgu ses.

Returns

"Hav! Hav!"

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 81 of file [Dog.java](#).

```
00081 { return "Hav! Hav!"; }
```

7.5.3.5 setBreed()

```
void com.mehdi.petreminder.model.Dog.setBreed (  
    String breed)
```

Irk setter.

Parameters

<i>breed</i>	Kopek irki
--------------	------------

Definition at line 68 of file [Dog.java](#).

```
00068 { this.breed = breed; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRepository.mapRow\(\)](#).

7.5.3.6 setTrained()

```
void com.mehdi.petreminder.model.Dog.setTrained (  
    boolean trained)
```

Egitim durumu setter.

Parameters

<i>trained</i>	Egitimli mi
----------------	-------------

Definition at line 74 of file [Dog.java](#).

```
00074 { this.trained = trained; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRepository.mapRow\(\)](#).

7.5.3.7 toString()

```
String com.mehdi.petreminder.model.Dog.toString ()
```

String temsili.

Returns

Kopek bilgilerini iceren string

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 95 of file [Dog.java](#).

```
00095         {
00096             return String.format("Dog{id=%d, name='%s', breed='%s', age=%s, trained=%b}",
00097                                   getId(), getName(), breed, getAgeString(), trained);
00098         }
```

References [com.mehdi.petreminder.model.Pet.getAgeString\(\)](#), [com.mehdi.petreminder.model.Pet.getId\(\)](#), and [com.mehdi.petreminder.model.Pet.getName\(\)](#).

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/Dog.java](#)

7.6 com.mehdi.petreminder.observer.EventManager Class Reference

Observer Pattern Subject — olay yoneticisi.

Public Member Functions

- void [subscribe](#) ([EventType](#) eventType, [PetReminderObserver](#) observer)
Belirli bir olay turune observer kaydeder.
- void [unsubscribe](#) ([EventType](#) eventType, [PetReminderObserver](#) observer)
Observer'i belirli bir olay turundan cikarir.
- void [unsubscribeAll](#) ([PetReminderObserver](#) observer)
Tum olay turlerinden observer'i cikarir.
- void [notify](#) ([EventType](#) eventType, Object data)
Belirli bir olay turu icin tum observer'lara bildirim gonderir.
- int [getObserverCount](#) ([EventType](#) eventType)
Belirli olay turune kayitli observer sayisini dondurur.
- void [clearAll](#) ()
Tum observer'lari temizler (test icin).

Static Public Member Functions

- static synchronized [EventManager](#) [getInstance](#) ()
Singleton instance dondurur.
- static synchronized void [resetInstance](#) ()
Singleton instance'i sifirlar (test icin).

7.6.1 Detailed Description

Observer Pattern Subject — olay yoneticisi.

PDF zorunlulugu: Observer Design Pattern. Bu sinif "Subject" rolundedir. Observer'lari kayıt altına alır ve bir olay gerceklestiginde tüm kayıtlı observer'lara bildirim gönderir. Thread-safe: CopyOnWriteArrayList kullanılır (GUI thread güvenli). Singleton: Uygulama genelinde tek instance.

Author

Muhammed Mehdi Karagulle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 28 of file [EventManager.java](#).

7.6.2 Member Function Documentation

7.6.2.1 clearAll()

```
void com.mehdi.petreminder.observer.EventManager.clearAll ()
```

Tüm observer'lari temizler (test için).

Definition at line 144 of file [EventManager.java](#).

```
00144      {
00145          for (EventType type : EventType.values()) {
00146              listeners.get(type).clear();
00147          }
00148          logger.debug("Tüm observer'lar temizlendi.");
00149      }
```

7.6.2.2 getInstance()

```
synchronized EventManager com.mehdi.petreminder.observer.EventManager.getInstance () [static]
```

Singleton instance döndürür.

Returns

[EventManager](#) tek instance

Definition at line 60 of file [EventManager.java](#).

```
00060      {
00061          if (instance == null) {
00062              instance = new EventManager();
00063          }
00064          return instance;
00065      }
```

Referenced by [com.mehdi.petreminder.service.PetService.addPet\(\)](#), [com.mehdi.petreminder.service.MedicalRecordService.addRecord\(\)](#), [com.mehdi.petreminder.service.ReminderService.addReminder\(\)](#), [com.mehdi.petreminder.service.PetService.deletePet\(\)](#), [com.mehdi.petreminder.service.MedicalRecordService.deleteRecord\(\)](#), [com.mehdi.petreminder.service.ReminderService.deleteReminder\(\)](#), [com.mehdi.petreminder.service.ReminderService.markCompleted\(\)](#), and [com.mehdi.petreminder.service.PetService.updatePet\(\)](#).

7.6.2.3 getObserverCount()

```
int com.mehdi.petreminder.observer.EventManager.getObserverCount (
    EventType eventType)
```

Belirli olay turune kayitli observer sayisini dondurur.

Parameters

<i>eventType</i>	Olay turu
------------------	-----------

Returns

Observer sayisi

Definition at line 136 of file [EventManager.java](#).

```
00136                                     {
00137         if (eventType == null) return 0;
00138         return listeners.get(eventType).size();
00139     }
```

7.6.2.4 notify()

```
void com.mehdi.petreminder.observer.EventManager.notify (
    EventType eventType,
    Object data)
```

Belirli bir olay turu icin tum observer'lara bildirim gonderir.

Parameters

<i>eventType</i>	Gerceklesen olay turu
<i>data</i>	Olayla ilgili veri (null olabilir)

Definition at line 116 of file [EventManager.java](#).

```
00116                                     {
00117         if (eventType == null) return;
00118         List<PetReminderObserver> list = listeners.get(eventType);
00119         logger.info("Olay tetiklendi: {} ({} observer)",
00120             eventType, list.size());
00121         for (PetReminderObserver observer : list) {
00122             try {
00123                 observer.onEvent(eventType, data);
00124             } catch (Exception e) {
00125                 logger.error("Observer bildirim hatasi: {} - {}",
00126                     observer.getClass().getSimpleName(), e.getMessage());
00127             }
00128         }
00129     }
```

Referenced by [com.mehdi.petreminder.service.PetService.addPet\(\)](#), [com.mehdi.petreminder.service.MedicalRecordService.addRecord\(\)](#), [com.mehdi.petreminder.service.ReminderService.addReminder\(\)](#), [com.mehdi.petreminder.service.PetService.deletePet\(\)](#), [com.mehdi.petreminder.service.MedicalRecordService.deleteRecord\(\)](#), [com.mehdi.petreminder.service.ReminderService.deleteReminder\(\)](#), [com.mehdi.petreminder.service.ReminderService.markCompleted\(\)](#), and [com.mehdi.petreminder.service.PetService.updatePet\(\)](#).

7.6.2.5 resetInstance()

```
synchronized void com.mehdi.petreminder.observer.EventManager.resetInstance () [static]
```

Singleton instance'i sıfırlar (test için).

Definition at line 154 of file [EventManager.java](#).

```
00154                                     {
00155         instance = null;
00156     }
```

7.6.2.6 subscribe()

```
void com.mehdi.petreminder.observer.EventManager.subscribe (
    EventType eventType,
    PetReminderObserver observer)
```

Belirli bir olay turune observer kaydeder.

Parameters

<i>eventType</i>	Dinlenecek olay turu
<i>observer</i>	Kayıt edilecek observer

Definition at line 72 of file [EventManager.java](#).

```
00072                                     {
00073         if (eventType == null || observer == null) {
00074             logger.warn("subscribe: null parametre, atlanıyor.");
00075             return;
00076         }
00077         List<PetReminderObserver> list = listeners.get(eventType);
00078         if (!list.contains(observer)) {
00079             list.add(observer);
00080             logger.debug("Observer kaydedildi: {} -> {}",
00081                 eventType, observer.getClass().getSimpleName());
00082         }
00083     }
```

7.6.2.7 unsubscribe()

```
void com.mehdi.petreminder.observer.EventManager.unsubscribe (
    EventType eventType,
    PetReminderObserver observer)
```

Observer'i belirli bir olay turundan cikarir.

Parameters

<i>eventType</i>	Olay turu
<i>observer</i>	Cikarılacak observer

Definition at line 90 of file [EventManager.java](#).

```
00090                                     {
00091         if (eventType == null || observer == null) return;
00092         List<PetReminderObserver> list = listeners.get(eventType);
00093         list.remove(observer);
00094         logger.debug("Observer cikarildi: {} -> {}",
00095             eventType, observer.getClass().getSimpleName());
00096     }
```


7.6.2.8 unsubscribeAll()

```
void com.mehdi.petreminder.observer.EventManager.unsubscribeAll (
    PetReminderObserver observer)
```

Tum olay turlerinden observer'i cikarir.

Parameters

<i>observer</i>	Cikarilacak observer
-----------------	----------------------

Definition at line 102 of file [EventManager.java](#).

```
00102                                     {
00103         if (observer == null) return;
00104         for (EventType type : EventType.values()) {
00105             listeners.get(type).remove(observer);
00106         }
00107         logger.debug("Observer tum olaylardan cikarildi: {}",
00108             observer.getClass().getSimpleName());
00109     }
```

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/observer/EventManager.java](#)

7.7 com.mehdi.petreminder.observer.EventType Enum Reference

Public Member Functions

- [EventType](#) (String description)
Yapici.
- String [getDescription](#) ()
Aciklamayi dondurur.
- String [toString](#) ()
toString override.

Public Attributes

- [STORAGE_CHANGED](#) =("Storage Backend Degisti")
Storage backend degisti (BINARY, SQLite, MySQL).
- [PET_ADDED](#) =("Yeni Pet Eklendi")
Yeni pet eklendi.
- [PET_DELETED](#) =("Pet Silindi")
Pet silindi.
- [PET_UPDATED](#) =("Pet Guncellendi")
Pet guncellendi.
- [REMINDER_ADDED](#) =("Yeni Hatirlatici Eklendi")
Yeni hatirlatici eklendi.
- [REMINDER_COMPLETED](#) =("Hatirlatici Tamamlandi")
Hatirlatici tamamlandi.
- [REMINDER_DELETED](#) =("Hatirlatici Silindi")
Hatirlatici silindi.
- [MEDICAL_RECORD_ADDED](#) =("Saglik Kaydi Eklendi")
Saglik kaydi eklendi.
- [MEDICAL_RECORD_DELETED](#) =("Saglik Kaydi Silindi")
Saglik kaydi silindi.

7.7.1 Detailed Description

Enum documentation.

Definition at line 21 of file [EventType.java](#).

7.7.2 Constructor & Destructor Documentation

7.7.2.1 EventType()

```
com.mehdi.petreminder.observer.EventType.EventType (
    String description)
```

Yapici.

Parameters

<i>description</i>	Olay aciklamasi
--------------------	-----------------

Definition at line 57 of file [EventType.java](#).

```
00057         {
00058         this.description = description;
00059     }
```

7.7.3 Member Function Documentation

7.7.3.1 getDescription()

```
String com.mehdi.petreminder.observer.EventType.getDescription ()
```

Aciklamayi dondurur.

Returns

Olay aciklamasi

Definition at line 65 of file [EventType.java](#).

```
00065         {
00066         return description;
00067     }
```

7.7.3.2 toString()

```
String com.mehdi.petreminder.observer.EventType.toString ()
```

toString override.

Returns

Olay aciklamasi

Definition at line 74 of file [EventType.java](#).

```
00074         {
00075         return description;
00076     }
```

7.7.4 Member Data Documentation

7.7.4.1 MEDICAL_RECORD_ADDED

```
com.mehdi.petreminder.observer.EventType.MEDICAL_RECORD_ADDED = ("Saglik Kaydi Eklendi")
```

Saglik kaydi eklendi.

Definition at line 45 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.MedicalRecordService.addRecord\(\)](#).

7.7.4.2 MEDICAL_RECORD_DELETED

```
com.mehdi.petreminder.observer.EventType.MEDICAL_RECORD_DELETED = ("Saglik Kaydi Silindi")
```

Saglik kaydi silindi.

Definition at line 48 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.MedicalRecordService.deleteRecord\(\)](#).

7.7.4.3 PET_ADDED

```
com.mehdi.petreminder.observer.EventType.PET_ADDED = ("Yeni Pet Eklendi")
```

Yeni pet eklendi.

Definition at line 27 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.PetService.addPet\(\)](#).

7.7.4.4 PET_DELETED

```
com.mehdi.petreminder.observer.EventType.PET_DELETED = ("Pet Silindi")
```

Pet silindi.

Definition at line 30 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.PetService.deletePet\(\)](#).

7.7.4.5 PET_UPDATED

```
com.mehdi.petreminder.observer.EventType.PET_UPDATED = ("Pet Guncellendi")
```

Pet guncellendi.

Definition at line 33 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.PetService.updatePet\(\)](#).

7.7.4.6 REMINDER_ADDED

```
com.mehdi.petreminder.observer.EventType.REMINDER_ADDED = ("Yeni Hatirlatici Eklendi")
```

Yeni hatirlatici eklendi.

Definition at line 36 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.ReminderService.addReminder\(\)](#).

7.7.4.7 REMINDER_COMPLETED

```
com.mehdi.petreminder.observer.EventType.REMINDER_COMPLETED = ("Hatirlatici Tamamlandi")
```

Hatirlatici tamamlandi.

Definition at line 39 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.ReminderService.markCompleted\(\)](#).

7.7.4.8 REMINDER_DELETED

```
com.mehdi.petreminder.observer.EventType.REMINDER_DELETED = ("Hatirlatici Silindi")
```

Hatirlatici silindi.

Definition at line 42 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.ReminderService.deleteReminder\(\)](#).

7.7.4.9 STORAGE_CHANGED

```
com.mehdi.petreminder.observer.EventType.STORAGE_CHANGED = ("Storage Backend Degisti")
```

Storage backend degisti (BINARY, SQLite, MySQL).

Definition at line 24 of file [EventType.java](#).

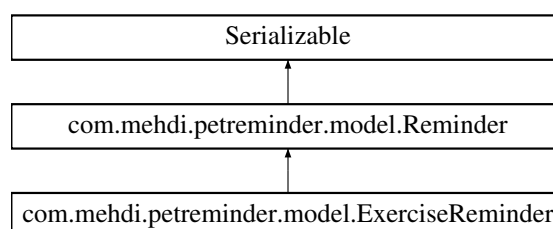
The documentation for this enum was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/observer/EventType.java](#)

7.8 com.mehdi.petreminder.model.ExerciseReminder Class Reference

Evcil hayvan egzersiz hatirlatici.

Inheritance diagram for `com.mehdi.petreminder.model.ExerciseReminder`:



Public Member Functions

- [ExerciseReminder](#) ()
Varsayılan yapıcı.
- [ExerciseReminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String exerciseType, int durationMinutes)
Temel parametrelili yapıcı.
- [ExerciseReminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern, String exerciseType, int durationMinutes, double distanceKm)
Tam parametrelili yapıcı.
- String [getExerciseType](#) ()
Egzersiz türü getter.
- void [setExerciseType](#) (String exerciseType)
Egzersiz türü setter.
- int [getDurationMinutes](#) ()
Süre getter.
- void [setDurationMinutes](#) (int durationMinutes)
Süre setter.
- double [getDistanceKm](#) ()
Mesafe getter.
- void [setDistanceKm](#) (double distanceKm)
Mesafe setter.
- String [getReminderType](#) ()
Hatırlatıcı türü.
- boolean [validate](#) ()
Egzersiz hatırlatıcısını doğrular.
- String [getSummary](#) ()
Özet bilgi.
- String [toString](#) ()
String temsili.

Public Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- int [getId](#) ()
ID getter.
- void [setId](#) (int id)
ID setter.
- int [getPetId](#) ()
Pet ID getter.
- void [setPetId](#) (int petId)
Pet ID setter.
- String [getPetName](#) ()
Pet adı getter.
- void [setPetName](#) (String petName)
Pet adı setter.
- LocalDateTime [getScheduledTime](#) ()
Planlanan zaman getter.
- void [setScheduledTime](#) (LocalDateTime scheduledTime)
Planlanan zaman setter.
- boolean [isCompleted](#) ()

- Tamamlandı mı getter.*
- void [setCompleted](#) (boolean completed)
- Tamamlandı setter.*
- String [getDescription](#) ()
- Açıklama getter.*
- void [setDescription](#) (String description)
- Açıklama setter.*
- String [getPriority](#) ()
- Öncelik getter.*
- void [setPriority](#) (String priority)
- Öncelik setter.*
- boolean [isRecurring](#) ()
- Tekrarlıyor mu getter.*
- void [setRecurring](#) (boolean recurring)
- Tekrarlama setter.*
- String [getRecurrencePattern](#) ()
- Tekrar deseni getter.*
- void [setRecurrencePattern](#) (String recurrencePattern)
- Tekrar deseni setter.*
- String [getFormattedTime](#) ()
- Planlanan zamanı biçimli string olarak döndürür.*
- boolean [isOverdue](#) ()
- Hatırlatıcının geçmiş mi olduğunu kontrol eder.*
- boolean [equals](#) (Object obj)
- Eşitlik kontrolü — ID bazlı.*
- int [hashCode](#) ()
- Hash kodu.*

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- [Reminder](#) ()
- Varsayılan yapıcı — Serialization için.*
- [Reminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern)
- Tam parametrelili yapıcı.*

Static Protected Attributes inherited from [com.mehdi.petreminder.model.Reminder](#)

- static final DateFormatter [FORMATTER](#)
- Tarih saat formatı.*

7.8.1 Detailed Description

Evcil hayvan egzersiz hatırlatıcısı.

OOP Inheritance: [Reminder](#)'dan miras alır.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 16 of file [ExerciseReminder.java](#).

7.8.2 Constructor & Destructor Documentation

7.8.2.1 ExerciseReminder() [1/3]

com.mehdi.petreminder.model.ExerciseReminder.ExerciseReminder ()

Varsayılan yapıcı.

Definition at line 31 of file [ExerciseReminder.java](#).

```
00031 { super(); }
```

7.8.2.2 ExerciseReminder() [2/3]

```
com.mehdi.petreminder.model.ExerciseReminder.ExerciseReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String exerciseType,
    int durationMinutes)
```

Temel parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>exerciseType</i>	Egzersiz türü
<i>durationMinutes</i>	Süre (dk)

Definition at line 42 of file [ExerciseReminder.java](#).

```
00044
00045     super(id, petId, petName, scheduledTime,
00046           "Egzersiz: " + exerciseType, "ORTA", false, "");
00047     this.exerciseType = exerciseType;
00048     this.durationMinutes = durationMinutes;
00049     this.distanceKm = 0.0;
00050 }
```

7.8.2.3 ExerciseReminder() [3/3]

```
com.mehdi.petreminder.model.ExerciseReminder.ExerciseReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String description,
    String priority,
    boolean recurring,
    String recurrencePattern,
    String exerciseType,
    int durationMinutes,
    double distanceKm)
```

Tam parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>description</i>	Açıklama
<i>priority</i>	Öncelik
<i>recurring</i>	Tekrarlıyor mu
<i>recurrencePattern</i>	Tekrar deseni
<i>exerciseType</i>	Egzersiz türü
<i>durationMinutes</i>	Süre
<i>distanceKm</i>	Mesafe (km)

Definition at line 66 of file [ExerciseReminder.java](#).

```
00070
00071     super(id, petId, petName, scheduledTime, description,
00072           priority, recurring, recurrencePattern);
00073     this.exerciseType = exerciseType;
00074     this.durationMinutes = durationMinutes;
00075     this.distanceKm = distanceKm;
00076 }
```

7.8.3 Member Function Documentation

7.8.3.1 getDistanceKm()

```
double com.mehdi.petreminder.model.ExerciseReminder.getDistanceKm ()
```

Mesafe getter.

Returns

Mesafe (km)

Definition at line 93 of file [ExerciseReminder.java](#).

```
00093 { return distanceKm; }
```


7.8.3.2 getDurationMinutes()

```
int com.mehdi.petreminder.model.ExerciseReminder.getDurationMinutes ()
```

Süre getter.

Returns

Süre (dk)

Definition at line 85 of file [ExerciseReminder.java](#).

```
00085 { return durationMinutes; }
```

7.8.3.3 getExerciseType()

```
String com.mehdi.petreminder.model.ExerciseReminder.getExerciseType ()
```

Egzersiz türü getter.

Returns

Egzersiz türü

Definition at line 79 of file [ExerciseReminder.java](#).

```
00079 { return exerciseType; }
```

7.8.3.4 getReminderType()

```
String com.mehdi.petreminder.model.ExerciseReminder.getReminderType ()
```

Hatırlatıcı türü.

Returns

"Egzersiz"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 100 of file [ExerciseReminder.java](#).

```
00100 { return "Egzersiz"; }
```

7.8.3.5 getSummary()

```
String com.mehdi.petreminder.model.ExerciseReminder.getSummary ()
```

Özet bilgi.

Returns

"Egzersiz: Yürüyüş 30dk"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 117 of file [ExerciseReminder.java](#).

```
00117 {
00118     String s = String.format("Egzersiz: %s %d dk", exerciseType, durationMinutes);
00119     if (distanceKm > 0) s += String.format(" (%.1f km)", distanceKm);
00120     return s;
00121 }
```

7.8.3.6 setDistanceKm()

```
void com.mehdi.petreminder.model.ExerciseReminder.setDistanceKm (
    double distanceKm)
```

Mesafe setter.

Parameters

<i>distanceKm</i>	Mesafe (km)
-------------------	-------------

Definition at line 96 of file [ExerciseReminder.java](#).

```
00096 { this.distanceKm = distanceKm; }
```

7.8.3.7 setDurationMinutes()

```
void com.mehdi.petreminder.model.ExerciseReminder.setDurationMinutes (
    int durationMinutes)
```

Süre setter.

Parameters

<i>durationMinutes</i>	Süre (dk)
------------------------	-----------

Definition at line 88 of file [ExerciseReminder.java](#).

```
00088                                     {
00089     this.durationMinutes = durationMinutes;
00090 }
```

7.8.3.8 setExerciseType()

```
void com.mehdi.petreminder.model.ExerciseReminder.setExerciseType (
    String exerciseType)
```

Egzersiz türü setter.

Parameters

<i>exerciseType</i>	Egzersiz türü
---------------------	---------------

Definition at line 82 of file [ExerciseReminder.java](#).

```
00082 { this.exerciseType = exerciseType; }
```

7.8.3.9 toString()

String com.mehdi.petreminder.model.ExerciseReminder.toString ()

String temsili.

Returns

Egzersiz hatırlatıcısı bilgisi

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 125 of file [ExerciseReminder.java](#).

```
00125         {
00126             return String.format("ExerciseReminder{id=%d, pet='%s', type='%s', %ddk}",
00127                                 getId(), getPetName(), exerciseType, durationMinutes);
00128         }
```

References [com.mehdi.petreminder.model.Reminder.getId\(\)](#), and [com.mehdi.petreminder.model.Reminder.getPetName\(\)](#).

7.8.3.10 validate()

boolean com.mehdi.petreminder.model.ExerciseReminder.validate ()

Egzersiz hatırlatıcısını doğrular.

Returns

Egzersiz türü ve süre geçerliyse true

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 107 of file [ExerciseReminder.java](#).

```
00107         {
00108             return exerciseType != null && !exerciseType.trim().isEmpty()
00109                    && durationMinutes > 0;
00110         }
```

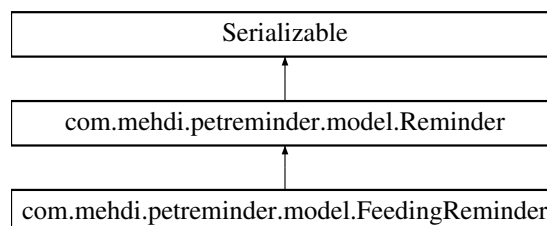
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/ExerciseReminder.java](#)

7.9 com.mehdi.petreminder.model.FeedingReminder Class Reference

Evcil hayvan besleme hatırlatıcısı.

Inheritance diagram for com.mehdi.petreminder.model.FeedingReminder:



Public Member Functions

- [FeedingReminder](#) ()
Varsayılan yapıcı.
- [FeedingReminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String foodType, double portionGrams)
Temel parametrelili yapıcı.
- [FeedingReminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern, String foodType, double portionGrams, boolean includesWaterChange)
Tam parametrelili yapıcı.
- String [getFoodType](#) ()
Besin türü getter.
- void [setFoodType](#) (String foodType)
Besin türü setter.
- double [getPortionGrams](#) ()
Porsiyon getter.
- void [setPortionGrams](#) (double portionGrams)
Porsiyon setter.
- boolean [isIncludesWaterChange](#) ()
Su değişimi getter.
- void [setIncludesWaterChange](#) (boolean includesWaterChange)
Su değişimi setter.
- String [getReminderType](#) ()
Hatırlatıcı türü.
- boolean [validate](#) ()
Besleme hatırlatıcısını doğrular.
- String [getSummary](#) ()
Özet bilgi.
- String [toString](#) ()
String temsili.

Public Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- int [getId](#) ()
ID getter.
- void [setId](#) (int id)
ID setter.
- int [getPetId](#) ()
Pet ID getter.
- void [setPetId](#) (int petId)
Pet ID setter.
- String [getPetName](#) ()
Pet adı getter.
- void [setPetName](#) (String petName)
Pet adı setter.
- LocalDateTime [getScheduledTime](#) ()
Planlanan zaman getter.
- void [setScheduledTime](#) (LocalDateTime scheduledTime)
Planlanan zaman setter.
- boolean [isCompleted](#) ()

- Tamamlandı mı getter.*
- void [setCompleted](#) (boolean completed)
- Tamamlandı setter.*
- String [getDescription](#) ()
- Açıklama getter.*
- void [setDescription](#) (String description)
- Açıklama setter.*
- String [getPriority](#) ()
- Öncelik getter.*
- void [setPriority](#) (String priority)
- Öncelik setter.*
- boolean [isRecurring](#) ()
- Tekrarlıyor mu getter.*
- void [setRecurring](#) (boolean recurring)
- Tekrarlama setter.*
- String [getRecurrencePattern](#) ()
- Tekrar deseni getter.*
- void [setRecurrencePattern](#) (String recurrencePattern)
- Tekrar deseni setter.*
- String [getFormattedTime](#) ()
- Planlanan zamanı biçimli string olarak döndürür.*
- boolean [isOverdue](#) ()
- Hatırlatıcının geçmiş mi olduğunu kontrol eder.*
- boolean [equals](#) (Object obj)
- Eşitlik kontrolü — ID bazlı.*
- int [hashCode](#) ()
- Hash kodu.*

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- [Reminder](#) ()
- Varsayılan yapıcı — Serialization için.*
- [Reminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern)
- Tam parametrelili yapıcı.*

Static Protected Attributes inherited from [com.mehdi.petreminder.model.Reminder](#)

- static final DateFormatter [FORMATTER](#)
- Tarih saat formatı.*

7.9.1 Detailed Description

Evcil hayvan besleme hatırlatıcısı.

OOP Inheritance: [Reminder](#)'dan miras alır. OOP Polymorphism: [getReminderType\(\)](#), [validate\(\)](#), [getSummary\(\)](#) override.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 17 of file [FeedingReminder.java](#).

7.9.2 Constructor & Destructor Documentation

7.9.2.1 FeedingReminder() [1/3]

```
com.mehdi.petreminder.model.FeedingReminder.FeedingReminder ()
```

Varsayılan yapıcı.

Definition at line 32 of file [FeedingReminder.java](#).

```
00032 { super(); }
```

7.9.2.2 FeedingReminder() [2/3]

```
com.mehdi.petreminder.model.FeedingReminder.FeedingReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String foodType,
    double portionGrams)
```

Temel parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>foodType</i>	Besin türü

<i>portionGrams</i>	Porsiyon (gram)
---------------------	-----------------

Definition at line 43 of file [FeedingReminder.java](#).

```

00045
00046         super(id, petId, petName, scheduledTime,
00047             "Besleme: " + foodType, "ORTA", false, "");
00048         this.foodType = foodType;
00049         this.portionGrams = portionGrams;
00050         this.includesWaterChange = false;
00051     }

```

7.9.2.3 FeedingReminder() [3/3]

```

com.mehdi.petreminder.model.FeedingReminder.FeedingReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String description,
    String priority,
    boolean recurring,
    String recurrencePattern,
    String foodType,
    double portionGrams,
    boolean includesWaterChange)

```

Tam parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>description</i>	Açıklama
<i>priority</i>	Öncelik
<i>recurring</i>	Tekrarlıyor mu
<i>recurrencePattern</i>	Tekrar deseni
<i>foodType</i>	Besin türü
<i>portionGrams</i>	Porsiyon (gram)
<i>includesWaterChange</i>	Su değişimi dahil mi

Definition at line 67 of file [FeedingReminder.java](#).

```

00071
00072         super(id, petId, petName, scheduledTime, description,
00073             priority, recurring, recurrencePattern);
00074         this.foodType = foodType;
00075         this.portionGrams = portionGrams;
00076         this.includesWaterChange = includesWaterChange;
00077     }

```

7.9.3 Member Function Documentation

7.9.3.1 getFoodType()

```
String com.mehdi.petreminder.model.FeedingReminder.getFoodType ()
```

Besin türü getter.

Returns

Besin türü

Definition at line 80 of file [FeedingReminder.java](#).

```
00080 { return foodType; }
```

7.9.3.2 getPortionGrams()

```
double com.mehdi.petreminder.model.FeedingReminder.getPortionGrams ()
```

Porsiyon getter.

Returns

Porsiyon (gram)

Definition at line 86 of file [FeedingReminder.java](#).

```
00086 { return portionGrams; }
```

7.9.3.3 getReminderType()

```
String com.mehdi.petreminder.model.FeedingReminder.getReminderType ()
```

Hatırlatıcı türü.

Returns

"Besleme"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 104 of file [FeedingReminder.java](#).

```
00104 { return "Besleme"; }
```


7.9.3.4 getSummary()

```
String com.mehdi.petreminder.model.FeedingReminder.getSummary ()
```

Özet bilgi.

Returns

"Besleme: Kuru Mama (150g)"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 120 of file [FeedingReminder.java](#).

```
00120         {
00121             return String.format("Besleme: %s (%.0fg)%s",
00122                 foodType, portionGrams,
00123                 includesWaterChange ? " + Su değişimi" : "");
00124         }
```

7.9.3.5 isIncludesWaterChange()

```
boolean com.mehdi.petreminder.model.FeedingReminder.isIncludesWaterChange ()
```

Su değişimi getter.

Returns

Su değişimi dahil mi

Definition at line 92 of file [FeedingReminder.java](#).

```
00092 { return includesWaterChange; }
```

7.9.3.6 setFoodType()

```
void com.mehdi.petreminder.model.FeedingReminder.setFoodType (
    String foodType)
```

Besin türü setter.

Parameters

<i>foodType</i>	Besin türü
-----------------	------------

Definition at line 83 of file [FeedingReminder.java](#).

```
00083 { this.foodType = foodType; }
```

7.9.3.7 setIncludesWaterChange()

```
void com.mehdi.petreminder.model.FeedingReminder.setIncludesWaterChange (
    boolean includesWaterChange)
```

Su değişimi setter.

Parameters

<i>includesWaterChange</i>	Su değişimi
----------------------------	-------------

Definition at line 95 of file [FeedingReminder.java](#).

```
00095                                     {
00096         this.includesWaterChange = includesWaterChange;
00097     }
```

7.9.3.8 setPortionGrams()

```
void com.mehdi.petreminder.model.FeedingReminder.setPortionGrams (
    double portionGrams)
```

Porsiyon setter.

Parameters

<i>portionGrams</i>	Porsiyon (gram)
---------------------	-----------------

Definition at line 89 of file [FeedingReminder.java](#).

```
00089 { this.portionGrams = portionGrams; }
```

7.9.3.9 toString()

```
String com.mehdi.petreminder.model.FeedingReminder.toString ()
```

String temsili.

Returns

Besleme hatırlatıcısı bilgisi

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 131 of file [FeedingReminder.java](#).

```
00131     {
00132         return String.format("FeedingReminder{id=%d, pet='%s', food='%s', portion=%.0fg}",
00133             getId(), getPetName(), foodType, portionGrams);
00134     }
```

References [com.mehdi.petreminder.model.Reminder.getId\(\)](#), and [com.mehdi.petreminder.model.Reminder.getPetName\(\)](#).

7.9.3.10 validate()

```
boolean com.mehdi.petreminder.model.FeedingReminder.validate ()
```

Besleme hatırlatıcısını doğrular.

Returns

foodType boş değilse ve porsiyon > 0 ise true

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 111 of file [FeedingReminder.java](#).

```
00111         {  
00112         return foodType != null && !foodType.trim().isEmpty() && portionGrams > 0;  
00113     }
```

The documentation for this class was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/model/[FeedingReminder.java](#)

7.10 com.mehdi.petreminder.annotation.Generated Interface Reference

7.10.1 Detailed Description

Annotation to mark generated or untestable code. JaCoCo automatically ignores methods/classes annotated with an annotation whose simple name contains "Generated".

Definition at line 19 of file [Generated.java](#).

The documentation for this interface was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/annotation/Generated.java

7.11 com.mehdi.petreminder.Generated Interface Reference

Marks methods/constructors to be excluded from JaCoCo coverage.

7.11.1 Detailed Description

Marks methods/constructors to be excluded from JaCoCo coverage.

@annotation [Generated](#)

JaCoCo 0.8.2+ automatically skips methods annotated with any annotation whose simple name is "Generated". Apply only to methods that are structurally untestable in a CI environment.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 30 of file [Generated.java](#).

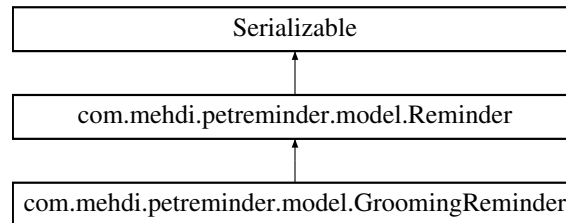
The documentation for this interface was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/[Generated.java](#)

7.12 com.mehdi.petreminder.model.GroomingReminder Class Reference

Evcil hayvan tıraş ve bakım hatırlatıcısı.

Inheritance diagram for com.mehdi.petreminder.model.GroomingReminder:



Public Member Functions

- [GroomingReminder](#) ()
Varsayılan yapıcı.
- [GroomingReminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String groomingType, boolean professional)
Temel parametrelili yapıcı.
- [GroomingReminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern, String groomingType, boolean professional, String groomerName)
Tam parametrelili yapıcı.
- String [getGroomingType](#) ()
Bakım türü getter.
- void [setGroomingType](#) (String groomingType)
Bakım türü setter.
- boolean [isProfessional](#) ()
Profesyonel mi getter.
- void [setProfessional](#) (boolean professional)
Profesyonel setter.
- String [getGroomerName](#) ()
Groomer adı getter.
- void [setGroomerName](#) (String groomerName)
Groomer adı setter.
- String [getReminderType](#) ()
Hatırlatıcı türü.
- boolean [validate](#) ()
Bakım hatırlatıcısını doğrular.
- String [getSummary](#) ()
Özet bilgi.
- String [toString](#) ()
String temsili.

Public Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- `int getId ()`
ID getter.
- `void setId (int id)`
ID setter.
- `int getPetId ()`
Pet ID getter.
- `void setPetId (int petId)`
Pet ID setter.
- `String getPetName ()`
Pet adı getter.
- `void setPetName (String petName)`
Pet adı setter.
- `LocalDateTime getScheduledTime ()`
Planlanan zaman getter.
- `void setScheduledTime (LocalDateTime scheduledTime)`
Planlanan zaman setter.
- `boolean isCompleted ()`
Tamamlandı mı getter.
- `void setCompleted (boolean completed)`
Tamamlandı setter.
- `String getDescription ()`
Açıklama getter.
- `void setDescription (String description)`
Açıklama setter.
- `String getPriority ()`
Öncelik getter.
- `void setPriority (String priority)`
Öncelik setter.
- `boolean isRecurring ()`
Tekrarlıyor mu getter.
- `void setRecurring (boolean recurring)`
Tekrarlama setter.
- `String getRecurrencePattern ()`
Tekrar deseni getter.
- `void setRecurrencePattern (String recurrencePattern)`
Tekrar deseni setter.
- `String getFormattedTime ()`
Planlanan zamanı biçimli string olarak döndürür.
- `boolean isOverdue ()`
Hatırlatıcının geçmiş mi olduğunu kontrol eder.
- `boolean equals (Object obj)`
Eşitlik kontrolü — ID bazlı.
- `int hashCode ()`
Hash kodu.

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- [Reminder](#) ()
Varsayılan yapıcı — Serialization için.
- [Reminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern)
Tam parametrelili yapıcı.

Static Protected Attributes inherited from [com.mehdi.petreminder.model.Reminder](#)

- static final DateTimeFormatter [FORMATTER](#)
Tarih saat formatı.

7.12.1 Detailed Description

Evcil hayvan tıraş ve bakım hatırlatıcısı.

OOP Inheritance: [Reminder](#)'dan miras alır.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 16 of file [GroomingReminder.java](#).

7.12.2 Constructor & Destructor Documentation

7.12.2.1 GroomingReminder() [1/3]

```
com.mehdi.petreminder.model.GroomingReminder.GroomingReminder ()
```

Varsayılan yapıcı.

Definition at line 31 of file [GroomingReminder.java](#).

```
00031 { super(); }
```

7.12.2.2 GroomingReminder() [2/3]

```
com.mehdi.petreminder.model.GroomingReminder.GroomingReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String groomingType,
    boolean professional)
```

Temel parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>groomingType</i>	Bakım türü
<i>professional</i>	Profesyonel mi

Definition at line 42 of file [GroomingReminder.java](#).

```
00044
00045         super(id, petId, petName, scheduledTime,
00046             "Bakim: " + groomingType, "DUSUK", false, "");
00047         this.groomingType = groomingType;
00048         this.professional = professional;
00049         this.groomerName = "";
00050     }
```

7.12.2.3 GroomingReminder() [3/3]

```
com.mehdi.petreminder.model.GroomingReminder.GroomingReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String description,
    String priority,
    boolean recurring,
    String recurrencePattern,
    String groomingType,
    boolean professional,
    String groomerName)
```

Tam parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID

<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>description</i>	Açıklama
<i>priority</i>	Öncelik
<i>recurring</i>	Tekrarlıyor mu
<i>recurrencePattern</i>	Tekrar deseni
<i>groomingType</i>	Bakım türü
<i>professional</i>	Profesyonel mi
<i>groomerName</i>	Groomer adı

Definition at line 66 of file [GroomingReminder.java](#).

```

00070
00071         super(id, petId, petName, scheduledTime, description,
00072             priority, recurring, recurrencePattern);
00073         this.groomingType = groomingType;
00074         this.professional = professional;
00075         this.groomerName = groomerName;
00076     }

```

7.12.3 Member Function Documentation

7.12.3.1 getGroomerName()

```
String com.mehdi.petreminder.model.GroomingReminder.getGroomerName ()
```

Groomer adı getter.

Returns

Groomer adı

Definition at line 91 of file [GroomingReminder.java](#).

```
00091 { return groomerName; }
```

7.12.3.2 getGroomingType()

```
String com.mehdi.petreminder.model.GroomingReminder.getGroomingType ()
```

Bakım türü getter.

Returns

Bakım türü

Definition at line 79 of file [GroomingReminder.java](#).

```
00079 { return groomingType; }
```


7.12.3.3 getReminderType()

```
String com.mehdi.petreminder.model.GroomingReminder.getReminderType ()
```

Hatırlatıcı türü.

Returns

"Bakim"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 98 of file [GroomingReminder.java](#).

```
00098 { return "Bakim"; }
```

7.12.3.4 getSummary()

```
String com.mehdi.petreminder.model.GroomingReminder.getSummary ()
```

Özet bilgi.

Returns

"Bakım: Tıraş (Profesyonel)"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 114 of file [GroomingReminder.java](#).

```
00114         {
00115             return String.format("Bakım: %s%s%s",
00116                 groomingType,
00117                 professional ? " (Profesyonel)" : "",
00118                 groomerName != null && !groomerName.isEmpty() ? " - " + groomerName : "");
00119         }
```

7.12.3.5 isProfessional()

```
boolean com.mehdi.petreminder.model.GroomingReminder.isProfessional ()
```

Profesyonel mi getter.

Returns

Profesyonel mi

Definition at line 85 of file [GroomingReminder.java](#).

```
00085 { return professional; }
```

7.12.3.6 setGroomerName()

```
void com.mehdi.petreminder.model.GroomingReminder.setGroomerName (
    String groomerName)
```

Groomer adı setter.

Parameters

<i>groomerName</i>	Groomer adı
--------------------	-------------

Definition at line 94 of file [GroomingReminder.java](#).

```
00094 { this.groomerName = groomerName; }
```

7.12.3.7 setGroomingType()

```
void com.mehdi.petreminder.model.GroomingReminder.setGroomingType (
    String groomingType)
```

Bakım türü setter.

Parameters

<i>groomingType</i>	Bakım türü
---------------------	------------

Definition at line 82 of file [GroomingReminder.java](#).

```
00082 { this.groomingType = groomingType; }
```

7.12.3.8 setProfessional()

```
void com.mehdi.petreminder.model.GroomingReminder.setProfessional (
    boolean professional)
```

Profesyonel setter.

Parameters

<i>professional</i>	Profesyonel mi
---------------------	----------------

Definition at line 88 of file [GroomingReminder.java](#).

```
00088 { this.professional = professional; }
```

7.12.3.9 toString()

```
String com.mehdi.petreminder.model.GroomingReminder.toString ()
```

String temsili.

Returns

Bakım hatırlatıcısı bilgisi

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 123 of file [GroomingReminder.java](#).

```
00123         {
00124         return String.format("GroomingReminder{id=%d, pet='%s', type='%s', prof=%b}",
00125             getId(), getPetName(), groomingType, professional);
00126     }
```

References [com.mehdi.petreminder.model.Reminder.getId\(\)](#), and [com.mehdi.petreminder.model.Reminder.getPetName\(\)](#).

7.12.3.10 validate()

```
boolean com.mehdi.petreminder.model.GroomingReminder.validate ()
```

Bakım hatırlatıcısını doğrular.

Returns

Bakım türü boş değilse true

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 105 of file [GroomingReminder.java](#).

```
00105         {
00106         return groomingType != null && !groomingType.trim().isEmpty();
00107     }
```

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/GroomingReminder.java](#)

7.13 com.mehdi.petreminder.gui.util.GuiConstants Class Reference

GUI bileşenlerinde kullanılan sabitler.

Static Public Attributes

- static final Color **NAV_COLOR** = new Color(45, 55, 72)
Sol nav paneli arka plan rengi.
- static final Color **NAV_HOVER** = new Color(74, 85, 104)
Nav buton hover rengi.
- static final Color **NAV_ACTIVE** = new Color(99, 102, 241)
Nav aktif buton rengi.
- static final Color **BG_COLOR** = new Color(248, 250, 252)
İçerik alanı arka plan.
- static final Color **CARD_BG** = Color.WHITE
Kart arka plan rengi.
- static final Color **PRIMARY** = new Color(99, 102, 241)
Birincil vurgu rengi (indigo).
- static final Color **DANGER** = new Color(239, 68, 68)
Tehlike rengi (kırmızı).
- static final Color **SUCCESS** = new Color(34, 197, 94)
Başarı rengi (yeşil).
- static final Color **WARNING** = new Color(245, 158, 11)
Uyarı rengi (turuncu).
- static final Color **TEXT_PRIMARY** = new Color(17, 24, 39)
Başlık metni rengi.
- static final Color **TEXT_SECONDARY** = new Color(107, 114, 128)
İkincil metin rengi.
- static final Color **BORDER_COLOR** = new Color(229, 231, 235)
Kenarlık rengi.
- static final Font **TITLE_FONT** = new Font("Segoe UI", Font.BOLD, 22)
Büyük başlık fontu.
- static final Font **SECTION_FONT** = new Font("Segoe UI", Font.BOLD, 16)
Bölüm başlık fontu.
- static final Font **BODY_FONT** = new Font("Segoe UI", Font.PLAIN, 13)
Normal metin fontu.
- static final Font **SMALL_FONT** = new Font("Segoe UI", Font.PLAIN, 11)
Küçük metin fontu.
- static final Font **NAV_FONT** = new Font("Segoe UI", Font.PLAIN, 13)
Nav buton fontu.
- static final int **NAV_WIDTH** = 200
Sol nav genişliği.
- static final int **GAP** = 12
Standart boşluk.
- static final int **GAP_LARGE** = 24
Büyük boşluk.

7.13.1 Detailed Description

GUI bileşenlerinde kullanılan sabitler.

Tüm renk, font ve boyut değerleri merkezi olarak burada tanımlanır.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 21 of file [GuiConstants.java](#).

7.13.2 Member Data Documentation

7.13.2.1 BG_COLOR

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.BG_COLOR = new Color(248, 250, 252)
[static]
```

İçerik alanı arka plan.

Definition at line 38 of file [GuiConstants.java](#).

Referenced by [com.mehdi.petreminder.gui.panels.DashboardPanel.DashboardPanel\(\)](#), [com.mehdi.petreminder.gui.panels.MedicalPa](#), [com.mehdi.petreminder.gui.panels.PetsPanel.PetsPanel\(\)](#), [com.mehdi.petreminder.gui.panels.RemindersPanel.RemindersPanel\(\)](#), [com.mehdi.petreminder.gui.panels.SettingsPanel.SettingsPanel\(\)](#), and [com.mehdi.petreminder.gui.panels.VetPanel.VetPanel\(\)](#).

7.13.2.2 BODY_FONT

```
final Font com.mehdi.petreminder.gui.util.GuiConstants.BODY_FONT = new Font("Segoe UI", Font.↔
PLAIN, 13) [static]
```

Normal metin fontu.

Definition at line 73 of file [GuiConstants.java](#).

7.13.2.3 BORDER_COLOR

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.BORDER_COLOR = new Color(229, 231,
235) [static]
```

Kenarlık rengi.

Definition at line 62 of file [GuiConstants.java](#).

7.13.2.4 CARD_BG

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.CARD_BG = Color.WHITE [static]
```

Kart arka plan rengi.

Definition at line 41 of file [GuiConstants.java](#).

7.13.2.5 DANGER

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.DANGER = new Color(239, 68, 68) [static]
```

Tehlike rengi (kırmızı).

Definition at line 47 of file [GuiConstants.java](#).

7.13.2.6 GAP

```
final int com.mehdi.petreminder.gui.util.GuiConstants.GAP = 12 [static]
```

Standart boşluk.

Definition at line 87 of file [GuiConstants.java](#).

Referenced by [com.mehdi.petreminder.gui.panels.DashboardPanel.DashboardPanel\(\)](#), [com.mehdi.petreminder.gui.panels.MedicalPa](#), [com.mehdi.petreminder.gui.panels.PetsPanel.PetsPanel\(\)](#), [com.mehdi.petreminder.gui.panels.RemindersPanel.RemindersPanel\(\)](#), and [com.mehdi.petreminder.gui.panels.VetPanel.VetPanel\(\)](#).

7.13.2.7 GAP_LARGE

```
final int com.mehdi.petreminder.gui.util.GuiConstants.GAP_LARGE = 24 [static]
```

Büyük boşluk.

Definition at line 90 of file [GuiConstants.java](#).

Referenced by [com.mehdi.petreminder.gui.panels.DashboardPanel.DashboardPanel\(\)](#), [com.mehdi.petreminder.gui.panels.MedicalPa](#), [com.mehdi.petreminder.gui.panels.PetsPanel.PetsPanel\(\)](#), [com.mehdi.petreminder.gui.panels.RemindersPanel.RemindersPanel\(\)](#), [com.mehdi.petreminder.gui.panels.SettingsPanel.SettingsPanel\(\)](#), and [com.mehdi.petreminder.gui.panels.VetPanel.VetPanel\(\)](#).

7.13.2.8 NAV_ACTIVE

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.NAV_ACTIVE = new Color(99, 102, 241) [static]
```

Nav aktif buton rengi.

Definition at line 35 of file [GuiConstants.java](#).

Referenced by [com.mehdi.petreminder.gui.panels.NavPanel.setActiveButton\(\)](#).

7.13.2.9 NAV_COLOR

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.NAV_COLOR = new Color(45, 55, 72) [static]
```

Sol nav paneli arka plan rengi.

Definition at line 29 of file [GuiConstants.java](#).

Referenced by [com.mehdi.petreminder.gui.panels.NavPanel.NavPanel\(\)](#), and [com.mehdi.petreminder.gui.panels.NavPanel.setActiveB](#)

7.13.2.10 NAV_FONT

```
final Font com.mehdi.petreminder.gui.utilGuiConstants.NAV_FONT = new Font("Segoe UI", Font.↵PLAIN, 13) [static]
```

Nav buton fontu.

Definition at line 79 of file [GuiConstants.java](#).

7.13.2.11 NAV_HOVER

```
final Color com.mehdi.petreminder.gui.utilGuiConstants.NAV_HOVER = new Color(74, 85, 104) [static]
```

Nav buton hover rengi.

Definition at line 32 of file [GuiConstants.java](#).

7.13.2.12 NAV_WIDTH

```
final int com.mehdi.petreminder.gui.utilGuiConstants.NAV_WIDTH = 200 [static]
```

Sol nav genişliği.

Definition at line 84 of file [GuiConstants.java](#).

Referenced by [com.mehdi.petreminder.gui.panels.NavPanel.NavPanel\(\)](#).

7.13.2.13 PRIMARY

```
final Color com.mehdi.petreminder.gui.utilGuiConstants.PRIMARY = new Color(99, 102, 241) [static]
```

Birincil vurgu rengi (indigo).

Definition at line 44 of file [GuiConstants.java](#).

7.13.2.14 SECTION_FONT

```
final Font com.mehdi.petreminder.gui.utilGuiConstants.SECTION_FONT = new Font("Segoe UI", Font.BOLD, 16) [static]
```

Bölüm başlık fontu.

Definition at line 70 of file [GuiConstants.java](#).

7.13.2.15 SMALL_FONT

```
final Font com.mehdi.petreminder.gui.utilGuiConstants.SMALL_FONT = new Font("Segoe UI", Font.↵  
PLAIN, 11) [static]
```

Küçük metin fontu.

Definition at line 76 of file [GuiConstants.java](#).

7.13.2.16 SUCCESS

```
final Color com.mehdi.petreminder.gui.utilGuiConstants.SUCCESS = new Color(34, 197, 94) [static]
```

Başarı rengi (yeşil).

Definition at line 50 of file [GuiConstants.java](#).

7.13.2.17 TEXT_PRIMARY

```
final Color com.mehdi.petreminder.gui.utilGuiConstants.TEXT_PRIMARY = new Color(17, 24, 39)  
[static]
```

Başlık metni rengi.

Definition at line 56 of file [GuiConstants.java](#).

7.13.2.18 TEXT_SECONDARY

```
final Color com.mehdi.petreminder.gui.utilGuiConstants.TEXT_SECONDARY = new Color(107, 114,  
128) [static]
```

İkincil metin rengi.

Definition at line 59 of file [GuiConstants.java](#).

7.13.2.19 TITLE_FONT

```
final Font com.mehdi.petreminder.gui.utilGuiConstants.TITLE_FONT = new Font("Segoe UI", Font.↵  
BOLD, 22) [static]
```

Büyük başlık fontu.

Definition at line 67 of file [GuiConstants.java](#).

7.13.2.20 WARNING

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.WARNING = new Color(245, 158, 11)
[static]
```

Uyarı rengi (turuncu).

Definition at line 53 of file [GuiConstants.java](#).

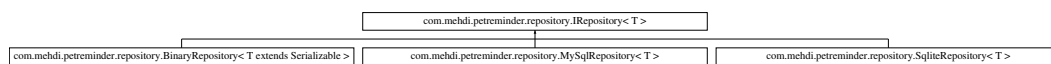
The documentation for this class was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/gui/util/[GuiConstants.java](#)

7.14 com.mehdi.petreminder.repository.IRepository< T > Interface Template Reference

Generic CRUD repository arayüzü.

Inheritance diagram for com.mehdi.petreminder.repository.IRepository< T >:



Public Member Functions

- int [save](#) (T entity)
Yeni bir varlık kaydeder.
- Optional< T > [findById](#) (int id)
ID'ye göre varlık getirir.
- List< T > [findAll](#) ()
Tüm varlıkları listeler.
- boolean [update](#) (T entity)
Mevcut varlığı günceller.
- boolean [delete](#) (int id)
ID'ye göre varlık siler.
- void [deleteAll](#) ()
Tüm varlıkları siler.
- int [count](#) ()
Varlık sayısını döndürür.
- void [close](#) ()
Repository bağlantısını/kaynaklarını kapatır.

7.14.1 Detailed Description

Generic CRUD repository arayüzü.

PDF zorunluluğu: [IRepository](#)<T> interface with [RepositoryFactory](#). OOP Abstraction: Binary / SQLite / MySQL aynı interface üzerinden kullanılır.

Parameters

<T>	Varlık türü (Pet, Reminder, User , MedicalRecord)
-----	--

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 23 of file [IRepository.java](#).

7.14.2 Member Function Documentation

7.14.2.1 close()

```
void com.mehdi.petreminder.repository.IRepository< T >.close ()
```

Repository bağlantısını/kaynaklarını kapatır.

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.M](#) and [com.mehdi.petreminder.repository.SQLiteRepository< T >](#).

7.14.2.2 count()

```
int com.mehdi.petreminder.repository.IRepository< T >.count ()
```

Varlık sayısını döndürür.

Returns

Toplam kayıt sayısı

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.M](#) and [com.mehdi.petreminder.repository.SQLiteRepository< T >](#).

7.14.2.3 delete()

```
boolean com.mehdi.petreminder.repository.IRepository< T >.delete (
    int id)
```

ID'ye göre varlık siler.

Parameters

<i>id</i>	Silinecek varlığın ID'si
-----------	--------------------------

Returns

Silme başarılıysa true

Exceptions

<i>RepositoryException</i>	Silme başarısız olursa
----------------------------	------------------------

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.M](#) and [com.mehdi.petreminder.repository.SqliteRepository< T >](#).

7.14.2.4 deleteAll()

```
void com.mehdi.petreminder.repository.IRepository< T >.deleteAll ()
```

Tüm varlıkları siler.

Exceptions

<i>RepositoryException</i>	Silme başarısız olursa
----------------------------	------------------------

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.M](#) and [com.mehdi.petreminder.repository.SqliteRepository< T >](#).

7.14.2.5 findAll()

```
List< T > com.mehdi.petreminder.repository.IRepository< T >.findAll ()
```

Tüm varlıkları listeler.

Returns

Varlık listesi (boş olabilir)

Exceptions

RepositoryException	Okuma başarısız olursa
-------------------------------------	------------------------

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.M](#) and [com.mehdi.petreminder.repository.SqliteRepository< T >](#).

7.14.2.6 findById()

```
Optional< T > com.mehdi.petreminder.repository.IRepository< T >.findById (  
    int id)
```

ID'ye göre varlık getirir.

Parameters

<i>id</i>	Varlık ID'si
-----------	--------------

Returns

Varlık (bulunamazsa empty Optional)

Exceptions

RepositoryException	Okuma başarısız olursa
-------------------------------------	------------------------

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.M](#) and [com.mehdi.petreminder.repository.SqliteRepository< T >](#).

7.14.2.7 save()

```
int com.mehdi.petreminder.repository.IRepository< T >.save (  
    T entity)
```

Yeni bir varlık kaydeder.

Parameters

<i>entity</i>	Kaydedilecek varlık
---------------	---------------------

Returns

Oluşturulan varlığın ID'si

Exceptions

RepositoryException	Kayıt başarısız olursa
-------------------------------------	------------------------

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.MemRepository< T >](#) and [com.mehdi.petreminder.repository.SqliteRepository< T >](#).

7.14.2.8 update()

```
boolean com.mehdi.petreminder.repository.IRepository< T >.update (
    T entity)
```

Mevcut varlığı günceller.

Parameters

<i>entity</i>	Güncellenecek varlık
---------------	----------------------

Returns

Güncelleme başarılıysa true

Exceptions

RepositoryException	Güncelleme başarısız olursa
-------------------------------------	-----------------------------

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.MemRepository< T >](#) and [com.mehdi.petreminder.repository.SqliteRepository< T >](#).

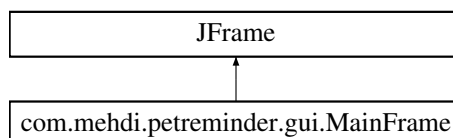
The documentation for this interface was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/IRepository.java](#)

7.15 com.mehdi.petreminder.gui.MainFrame Class Reference

Uygulamanın ana Swing penceresi.

Inheritance diagram for com.mehdi.petreminder.gui.MainFrame:



Public Member Functions

- [MainFrame](#) ()
MainFrame yapıcısı — GUI bileşenlerini başlatır.
- void [navigateTo](#) (String cardName)
Belirtilen kartı gösterir.
- void [onStorageChanged](#) ([StorageType](#) type)
Storage backend değişince tüm panelleri günceller.

Static Public Member Functions

- static void [launch](#) ()
Uygulamayı başlatır (EDT'de çalışır).

Static Public Attributes

- static final String [CARD_DASHBOARD](#) = "DASHBOARD"
Kart isimleri.
- static final String [CARD_PETS](#) = "PETS"
Pets kart adı.
- static final String [CARD_REMINDERS](#) = "REMINDERS"
Reminders kart adı.
- static final String [CARD_VET](#) = "VET"
Vet kart adı.
- static final String [CARD_MEDICAL](#) = "MEDICAL"
Medical kart adı.
- static final String [CARD_SETTINGS](#) = "SETTINGS"
Settings kart adı.

7.15.1 Detailed Description

Uygulamanın ana Swing penceresi.

PDF zorunluluğu: Swing GUI — ayrı GUI katmanı. FlatLaf light tema kullanılır (modern görünüm). Sol [NavPanel](#) + sağ CardLayout içerik alanı.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 33 of file [MainFrame.java](#).

7.15.2 Constructor & Destructor Documentation

7.15.2.1 MainFrame()

`com.mehdi.petreminder.gui.MainFrame.MainFrame ()`

[MainFrame](#) yapıcısı — GUI bileşenlerini başlatır.

Definition at line 88 of file [MainFrame.java](#).

```
00088         {
00089             super(TITLE);
00090             initLookAndFeel();
00091             initComponents();
00092             setupLayout();
00093             setupFrame();
00094             navigateTo(CARD_DASHBOARD);
00095             logger.info("MainFrame başlatıldı.");
00096         }
```

References [CARD_DASHBOARD](#), and [navigateTo\(\)](#).

Referenced by [launch\(\)](#).

7.15.3 Member Function Documentation

7.15.3.1 launch()

`void com.mehdi.petreminder.gui.MainFrame.launch () [static]`

Uygulamayı başlatır (EDT'de çalışır).

Definition at line 241 of file [MainFrame.java](#).

```
00241         {
00242             SwingUtilities.invokeLater(() -> {
00243                 MainFrame frame = new MainFrame();
00244                 frame.setVisible(true);
00245             });
00246         }
```

References [MainFrame\(\)](#).

7.15.3.2 navigateTo()

`void com.mehdi.petreminder.gui.MainFrame.navigateTo (String cardName)`

Belirtilen kartı gösterir.

Parameters

<i>cardName</i>	Kart adı (CARD_* sabitlerinden biri)
-----------------	--------------------------------------

Definition at line 196 of file [MainFrame.java](#).

```
00196         {
00197             cardLayout.show(contentPanel, cardName);
00198             navPanel.setActiveButton(cardName);
00199             refreshCurrentPanel(cardName);
00200             logger.debug("Navigasyon: {}", cardName);
00201         }
```

Referenced by [MainFrame\(\)](#).

7.15.3.3 onStorageChanged()

```
void com.mehdi.petreminder.gui.MainFrame.onStorageChanged (
    StorageType type)
```

Storage backend değişince tüm panelleri günceller.

Parameters

<i>type</i>	Yeni StorageType
-------------	----------------------------------

Definition at line 223 of file [MainFrame.java](#).

```
00223                                     {
00224         StorageConfig.setActiveBackend(type);
00225         // Status bar'ı güncelle
00226         Component[] comps = ((JPanel) getContentPane().getComponent(2)).getComponents();
00227         for (Component c : comps) {
00228             if (c instanceof JLabel && ((JLabel) c).getText().startsWith("Storage:")) {
00229                 ((JLabel) c).setText("Storage: " + type.getDisplayName());
00230             }
00231         }
00232         JOptionPane.showMessageDialog(this,
00233             "Storage backend değiştirildi:\n" + type.getDisplayName(),
00234             "Ayarlar", JOptionPane.INFORMATION_MESSAGE);
00235         logger.info("Storage değişti: {}", type);
00236     }
```

References [com.mehdi.petreminder.config.StorageType.getDisplayName\(\)](#), and [com.mehdi.petreminder.config.StorageConfig.setActiveBackend\(\)](#).

7.15.4 Member Data Documentation

7.15.4.1 CARD_DASHBOARD

```
final String com.mehdi.petreminder.gui.MainFrame.CARD_DASHBOARD = "DASHBOARD" [static]
```

Kart isimleri.

Definition at line 46 of file [MainFrame.java](#).

Referenced by [MainFrame\(\)](#).

7.15.4.2 CARD_MEDICAL

```
final String com.mehdi.petreminder.gui.MainFrame.CARD_MEDICAL = "MEDICAL" [static]
```

Medical kart adı.

Definition at line 54 of file [MainFrame.java](#).

7.15.4.3 CARD_PETS

```
final String com.mehdi.petreminder.gui.MainFrame.CARD_PETS = "PETS" [static]
```

Pets kart adı.

Definition at line 48 of file [MainFrame.java](#).

7.15.4.4 CARD_REMINDERS

```
final String com.mehdi.petreminder.gui.MainFrame.CARD_REMINDERS = "REMINDERS" [static]
```

Reminders kart adı.

Definition at line 50 of file [MainFrame.java](#).

7.15.4.5 CARD_SETTINGS

```
final String com.mehdi.petreminder.gui.MainFrame.CARD_SETTINGS = "SETTINGS" [static]
```

Settings kart adı.

Definition at line 56 of file [MainFrame.java](#).

7.15.4.6 CARD_VET

```
final String com.mehdi.petreminder.gui.MainFrame.CARD_VET = "VET" [static]
```

Vet kart adı.

Definition at line 52 of file [MainFrame.java](#).

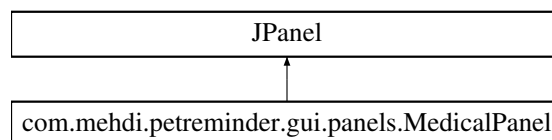
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/gui/MainFrame.java](#)

7.16 com.mehdi.petreminder.gui.panels.MedicalPanel Class Reference

Sağlık kayıtları CRUD paneli.

Inheritance diagram for com.mehdi.petreminder.gui.panels.MedicalPanel:



Public Member Functions

- [MedicalPanel](#) ([MainFrame](#) mainFrame)
MedicalPanel yapıcısı.
- void [refresh](#) ()
Tabloyu yeniler.

7.16.1 Detailed Description

Sağlık kayıtları CRUD paneli.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 28 of file [MedicalPanel.java](#).

7.16.2 Constructor & Destructor Documentation

7.16.2.1 MedicalPanel()

```
com.mehdi.petreminder.gui.panels.MedicalPanel.MedicalPanel (  
    MainFrame mainFrame)
```

[MedicalPanel](#) yapıcısı.

Parameters

<i>mainFrame</i>	Ana pencere
------------------	-------------

Definition at line 49 of file [MedicalPanel.java](#).

```
00049      {  
00050          this.mainFrame      = mainFrame;  
00051          this.medicalService = new MedicalRecordService();  
00052          setBackground(GuiConstants.BG_COLOR);  
00053          setLayout(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));  
00054          setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,  
00055                                   GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));  
00056          buildUI();  
00057      }
```

References [com.mehdi.petreminder.gui.util.GuiConstants.BG_COLOR](#), [com.mehdi.petreminder.gui.util.GuiConstants.GAP](#), and [com.mehdi.petreminder.gui.util.GuiConstants.GAP_LARGE](#).

7.16.3 Member Function Documentation

7.16.3.1 refresh()

```
void com.mehdi.petreminder.gui.panels.MedicalPanel.refresh ()
```

Tabloyu yeniler.

Definition at line 157 of file [MedicalPanel.java](#).

```
00157         {
00158             tableModel.setRowCount(0);
00159             try {
00160                 List<MedicalRecord> list = medicalService.getAllRecords();
00161                 for (MedicalRecord r : list) {
00162                     tableModel.addRow(new Object[]{
00163                         r.getId(), r.getPetName(), r.getRecordType(),
00164                         r.getDiagnosis(), r.getVeterinarianName(),
00165                         r.getFormattedDate(),
00166                         String.format("%.2f", r.getCost())
00167                     });
00168                 }
00169             } catch (Exception e) { /* sessiz */ }
00170         }
```

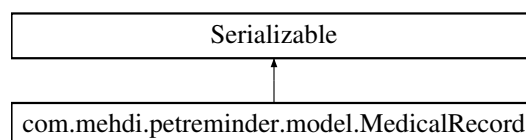
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/MedicalPanel.java](#)

7.17 com.mehdi.petreminder.model.MedicalRecord Class Reference

Evcil hayvanın sağlık kaydı.

Inheritance diagram for com.mehdi.petreminder.model.MedicalRecord:



Public Member Functions

- [MedicalRecord](#) ()
Varsayılan yapıcı.
- [MedicalRecord](#) (int id, int petId, String petName, LocalDate recordDate, String recordType, String diagnosis, String veterinarianName)
Temel parametrelili yapıcı.
- int [getId](#) ()
ID getter.
- void [setId](#) (int id)
ID setter.
- int [getPetId](#) ()
Pet ID getter.
- void [setPetId](#) (int petId)
Pet ID setter.

- String [getPetName](#) ()
Pet adı getter.
- void [setPetName](#) (String petName)
Pet adı setter.
- LocalDate [getRecordDate](#) ()
Kayıt tarihi getter.
- void [setRecordDate](#) (LocalDate recordDate)
Kayıt tarihi setter.
- String [getRecordType](#) ()
Kayıt türü getter.
- void [setRecordType](#) (String recordType)
Kayıt türü setter.
- String [getDiagnosis](#) ()
Teşhis getter.
- void [setDiagnosis](#) (String diagnosis)
Teşhis setter.
- String [getTreatment](#) ()
Tedavi getter.
- void [setTreatment](#) (String treatment)
Tedavi setter.
- String [getVeterinarianName](#) ()
Veteriner getter.
- void [setVeterinarianName](#) (String veterinarianName)
Veteriner setter.
- double [getCost](#) ()
Maliyet getter.
- void [setCost](#) (double cost)
Maliyet setter.
- LocalDate [getNextCheckDate](#) ()
Sonraki kontrol getter.
- void [setNextCheckDate](#) (LocalDate nextCheckDate)
Sonraki kontrol setter.
- String [getNotes](#) ()
Notlar getter.
- void [setNotes](#) (String notes)
Notlar setter.
- String [getVaccineName](#) ()
Aşı adı getter.
- void [setVaccineName](#) (String vaccineName)
Aşı adı setter.
- String [getFormattedDate](#) ()
Biçimli tarih döndürür.
- String [toString](#) ()
String temsili.
- boolean [equals](#) (Object obj)
Eşitlik kontrolü.
- int [hashCode](#) ()
Hash kodu.

7.17.1 Detailed Description

Evcil hayvanın sağlık kaydı.

OOP Composition: [Pet](#) nesnesiyle composition ilişkisi. OOP Encapsulation: tüm alanlar private. Serializable: BinaryRepository için.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 21 of file [MedicalRecord.java](#).

7.17.2 Constructor & Destructor Documentation

7.17.2.1 MedicalRecord() [1/2]

```
com.mehdi.petreminder.model.MedicalRecord.MedicalRecord ()
```

Varsayılan yapıcı.

Definition at line 70 of file [MedicalRecord.java](#).

```
00070 {}
```

Referenced by [equals\(\)](#).

7.17.2.2 MedicalRecord() [2/2]

```
com.mehdi.petreminder.model.MedicalRecord.MedicalRecord (
    int id,
    int petId,
    String petName,
    LocalDate recordDate,
    String recordType,
    String diagnosis,
    String veterinarianName)
```

Temel parametrelili yapıcı.

Parameters

<i>id</i>	Kayıt ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı

<i>recordDate</i>	Tarih
<i>recordType</i>	Tür
<i>diagnosis</i>	Teşhis
<i>veterinarianName</i>	Veteriner adı

Definition at line 82 of file [MedicalRecord.java](#).

```

00084
00085         this.id = id;
00086         this.petId = petId;
00087         this.petName = petName;
00088         this.recordDate = recordDate;
00089         this.recordType = recordType;
00090         this.diagnosis = diagnosis;
00091         this.veterinarianName = veterinarianName;
00092         this.treatment = "";
00093         this.cost = 0.0;
00094         this.notes = "";
00095         this.vaccineName = "";
00096     }

```

7.17.3 Member Function Documentation

7.17.3.1 equals()

```
boolean com.mehdi.petreminder.model.MedicalRecord.equals (
    Object obj)
```

Eşitlik kontrolü.

Parameters

<i>obj</i>	Karşılaştırılan nesne
------------	-----------------------

Returns

ID'ler eşitse true

Definition at line 199 of file [MedicalRecord.java](#).

```

00199
00200         if (this == obj) return true;
00201         if (obj == null || !(obj instanceof MedicalRecord)) return false;
00202         MedicalRecord other = (MedicalRecord) obj;
00203         return this.id == other.id;
00204     }

```

References [MedicalRecord\(\)](#).

7.17.3.2 getCost()

```
double com.mehdi.petreminder.model.MedicalRecord.getCost ()
```

Maliyet getter.

Returns

Maliyet (TL)

Definition at line 149 of file [MedicalRecord.java](#).

```
00149 { return cost; }
```

7.17.3.3 getDiagnosis()

```
String com.mehdi.petreminder.model.MedicalRecord.getDiagnosis ()
```

Teşhis getter.

Returns

Teşhis

Definition at line 129 of file [MedicalRecord.java](#).

```
00129 { return diagnosis; }
```

7.17.3.4 getFormattedDate()

```
String com.mehdi.petreminder.model.MedicalRecord.getFormattedDate ()
```

Biçimli tarih döndürür.

Returns

dd/MM/yyyy formatında tarih

Definition at line 178 of file [MedicalRecord.java](#).

```
00178 {  
00179     if (recordDate == null) return "Belirsiz";  
00180     return recordDate.format (DATE_FMT);  
00181 }
```

Referenced by [toString\(\)](#).

7.17.3.5 getId()

```
int com.mehdi.petreminder.model.MedicalRecord.getId ()
```

ID getter.

Returns

Kayıt ID

Definition at line 99 of file [MedicalRecord.java](#).

```
00099 { return id; }
```

Referenced by [com.mehdi.petreminder.service.MedicalRecordService.updateRecord\(\)](#).

7.17.3.6 getNextCheckDate()

`LocalDate com.mehdi.petreminder.model.MedicalRecord.getNextCheckDate ()`

Sonraki kontrol getter.

Returns

Sonraki kontrol tarihi

Definition at line 155 of file [MedicalRecord.java](#).

```
00155 { return nextCheckDate; }
```

7.17.3.7 getNotes()

`String com.mehdi.petreminder.model.MedicalRecord.getNotes ()`

Notlar getter.

Returns

Notlar

Definition at line 163 of file [MedicalRecord.java](#).

```
00163 { return notes; }
```

7.17.3.8 getPetId()

`int com.mehdi.petreminder.model.MedicalRecord.getPetId ()`

[Pet](#) ID getter.

Returns

[Pet](#) ID

Definition at line 105 of file [MedicalRecord.java](#).

```
00105 { return petId; }
```

Referenced by [com.mehdi.petreminder.service.MedicalRecordService.addRecord\(\)](#).

7.17.3.9 getPetName()

`String com.mehdi.petreminder.model.MedicalRecord.getPetName ()`

[Pet](#) adı getter.

Returns

[Pet](#) adı

Definition at line 111 of file [MedicalRecord.java](#).

```
00111 { return petName; }
```


7.17.3.10 getRecordDate()

LocalDate com.mehdi.petreminder.model.MedicalRecord.getRecordDate ()

Kayıt tarihi getter.

Returns

Kayıt tarihi

Definition at line 117 of file [MedicalRecord.java](#).

```
00117 { return recordDate; }
```

Referenced by [com.mehdi.petreminder.service.MedicalRecordService.getRecordsByPetId\(\)](#).

7.17.3.11 getRecordType()

String com.mehdi.petreminder.model.MedicalRecord.getRecordType ()

Kayıt türü getter.

Returns

Kayıt türü

Definition at line 123 of file [MedicalRecord.java](#).

```
00123 { return recordType; }
```

7.17.3.12 getTreatment()

String com.mehdi.petreminder.model.MedicalRecord.getTreatment ()

Tedavi getter.

Returns

Tedavi

Definition at line 135 of file [MedicalRecord.java](#).

```
00135 { return treatment; }
```

7.17.3.13 getVaccineName()

String com.mehdi.petreminder.model.MedicalRecord.getVaccineName ()

Aşı adı getter.

Returns

Aşı adı

Definition at line 169 of file [MedicalRecord.java](#).

```
00169 { return vaccineName; }
```

7.17.3.14 `getVeterinarianName()`

```
String com.mehdi.petreminder.model.MedicalRecord.getVeterinarianName ()
```

Veteriner getter.

Returns

Veteriner

Definition at line 141 of file [MedicalRecord.java](#).

```
00141 { return veterinarianName; }
```

7.17.3.15 `hashCode()`

```
int com.mehdi.petreminder.model.MedicalRecord.hashCode ()
```

Hash kodu.

Returns

ID'ye göre hash

Definition at line 211 of file [MedicalRecord.java](#).

```
00211 {  
00212     return Integer.hashCode(id);  
00213 }
```

7.17.3.16 `setCost()`

```
void com.mehdi.petreminder.model.MedicalRecord.setCost (  
    double cost)
```

Maliyet setter.

Parameters

<i>cost</i>	Maliyet
-------------	---------

Definition at line 152 of file [MedicalRecord.java](#).

```
00152 { this.cost = cost; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.17.3.17 setDiagnosis()

```
void com.mehdi.petreminder.model.MedicalRecord.setDiagnosis (
    String diagnosis)
```

Teşhis setter.

Parameters

<i>diagnosis</i>	Teşhis
------------------	--------

Definition at line 132 of file [MedicalRecord.java](#).

```
00132 { this.diagnosis = diagnosis; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.17.3.18 setId()

```
void com.mehdi.petreminder.model.MedicalRecord.setId (
    int id)
```

ID setter.

Parameters

<i>id</i>	Kayıt ID
-----------	----------

Definition at line 102 of file [MedicalRecord.java](#).

```
00102 { this.id = id; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.17.3.19 setNextCheckDate()

```
void com.mehdi.petreminder.model.MedicalRecord.setNextCheckDate (
    LocalDate nextCheckDate)
```

Sonraki kontrol setter.

Parameters

<i>nextCheckDate</i>	Tarih
----------------------	-------

Definition at line 158 of file [MedicalRecord.java](#).

```
00158
00159         this.nextCheckDate = nextCheckDate;
00160     }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.17.3.20 setNotes()

```
void com.mehdi.petreminder.model.MedicalRecord.setNotes (  
    String notes)
```

Notlar setter.

Parameters

<i>notes</i>	Notlar
--------------	--------

Definition at line 166 of file [MedicalRecord.java](#).

```
00166 { this.notes = notes; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.17.3.21 setPetId()

```
void com.mehdi.petreminder.model.MedicalRecord.setPetId (  
    int petId)
```

[Pet](#) ID setter.

Parameters

<i>petId</i>	Pet ID
--------------	------------------------

Definition at line 108 of file [MedicalRecord.java](#).

```
00108 { this.petId = petId; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.17.3.22 setPetName()

```
void com.mehdi.petreminder.model.MedicalRecord.setPetName (  
    String petName)
```

[Pet](#) adı setter.

Parameters

<i>petName</i>	Pet adı
----------------	-------------------------

Definition at line 114 of file [MedicalRecord.java](#).

```
00114 { this.petName = petName; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.17.3.23 setRecordDate()

```
void com.mehdi.petreminder.model.MedicalRecord.setRecordDate (
    LocalDate recordDate)
```

Kayıt tarihi setter.

Parameters

<i>recordDate</i>	Kayıt tarihi
-------------------	--------------

Definition at line 120 of file [MedicalRecord.java](#).

```
00120 { this.recordDate = recordDate; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.17.3.24 setRecordType()

```
void com.mehdi.petreminder.model.MedicalRecord.setRecordType (
    String recordType)
```

Kayıt türü setter.

Parameters

<i>recordType</i>	Kayıt türü
-------------------	------------

Definition at line 126 of file [MedicalRecord.java](#).

```
00126 { this.recordType = recordType; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.17.3.25 setTreatment()

```
void com.mehdi.petreminder.model.MedicalRecord.setTreatment (
    String treatment)
```

Tedavi setter.

Parameters

<i>treatment</i>	Tedavi
------------------	--------

Definition at line 138 of file [MedicalRecord.java](#).

```
00138 { this.treatment = treatment; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.17.3.26 setVaccineName()

```
void com.mehdi.petreminder.model.MedicalRecord.setVaccineName (  
    String vaccineName)
```

Aşı adı setter.

Parameters

<i>vaccineName</i>	Aşı adı
--------------------	---------

Definition at line 172 of file [MedicalRecord.java](#).

```
00172 { this.vaccineName = vaccineName; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.17.3.27 setVeterinarianName()

```
void com.mehdi.petreminder.model.MedicalRecord.setVeterinarianName (  
    String veterinarianName)
```

Veteriner setter.

Parameters

<i>veterinarianName</i>	Veteriner
-------------------------	-----------

Definition at line 144 of file [MedicalRecord.java](#).

```
00144  
00145         this.veterinarianName = veterinarianName; {  
00146     }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.17.3.28 toString()

```
String com.mehdi.petreminder.model.MedicalRecord.toString ()
```

String temsili.

Returns

Sağlık kaydı özeti

Definition at line 188 of file [MedicalRecord.java](#).

```
00188         {  
00189             return String.format("MedicalRecord{id=%d, pet='%s', type='%s', date='%s'}",  
00190                 id, petName, recordType, getFormattedDate());  
00191         }
```

References [getFormattedDate\(\)](#).

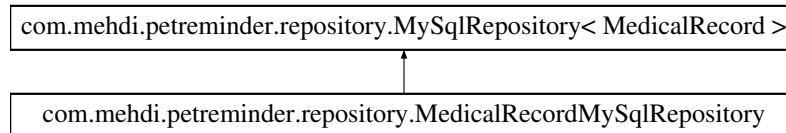
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/MedicalRecord.java](#)

7.18 com.mehdi.petreminder.repository.MedicalRecordMySqlRepository Class Reference

MySQL üzerinde [MedicalRecord](#) CRUD.

Inheritance diagram for com.mehdi.petreminder.repository.MedicalRecordMySqlRepository:



Public Member Functions

- [MedicalRecordMySqlRepository](#) ()
Varsayılan yapıcı.
- [MedicalRecordMySqlRepository](#) (Connection [connection](#))
Test yapıcısı.

Public Member Functions inherited from

[com.mehdi.petreminder.repository.MySqlRepository](#) < [MedicalRecord](#) >

- [MySqlRepository](#) ()
MySQL bağlantısını açar.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür.

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [MedicalRecord](#) m) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [MedicalRecord](#) m) throws SQLException
- [MedicalRecord](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from**[com.mehdi.petreminder.repository.MySqlRepository](#) < [MedicalRecord](#) >**

- abstract String [getTableName](#) ()
Tablo adı.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL.
- abstract String [getInsertSql](#) ()
INSERT SQL.
- abstract String [getUpdateSql](#) ()
UPDATE SQL.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members**Protected Attributes inherited from****[com.mehdi.petreminder.repository.MySqlRepository](#) < [MedicalRecord](#) >**

- Connection [connection](#)
MySQL JDBC bağlantısı.

Static Protected Attributes inherited from**[com.mehdi.petreminder.repository.MySqlRepository](#) < [MedicalRecord](#) >**

- static final Logger [logger](#)
Logger.

7.18.1 Detailed DescriptionMySQL üzerinde [MedicalRecord](#) CRUD.**Author**

Muhammed Mehdi Karagülle
 Ibrahim Demirci
 Zumre Uykun

Version

1.0

Definition at line 22 of file [MedicalRecordMySqlRepository.java](#).

7.18.2 Constructor & Destructor Documentation

7.18.2.1 MedicalRecordMySqlRepository() [1/2]

`com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.MedicalRecordMySqlRepository ()`

Varsayılan yapıcı.

Definition at line 25 of file [MedicalRecordMySqlRepository.java](#).

```
00025 { super(); }
```

7.18.2.2 MedicalRecordMySqlRepository() [2/2]

`com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.MedicalRecordMySqlRepository (Connection connection)`

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 31 of file [MedicalRecordMySqlRepository.java](#).

```
00031 { super(connection); }
```

References [com.mehdi.petreminder.repository.MySqlRepository< MedicalRecord >.connection](#).

7.18.3 Member Function Documentation

7.18.3.1 getCreateTableSql()

`String com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.getCreateTableSql ()`
[protected]

Member documentation.

Definition at line 45 of file [MedicalRecordMySqlRepository.java](#).

```
00045                                     {
00046         return "CREATE TABLE IF NOT EXISTS medical_records ("
00047             + "id INT AUTO_INCREMENT PRIMARY KEY,"
00048             + "pet_id INT NOT NULL,"
00049             + "pet_name VARCHAR(50),"
00050             + "record_date DATE NOT NULL,"
00051             + "record_type VARCHAR(30) NOT NULL,"
00052             + "diagnosis TEXT,"
00053             + "treatment TEXT,"
00054             + "veterinarian_name VARCHAR(100),"
00055             + "cost DOUBLE DEFAULT 0,"
00056             + "next_check_date DATE,"
00057             + "notes TEXT,"
00058             + "vaccine_name VARCHAR(100)"
00059             + ")";
00060     }
```

7.18.3.2 getInsertSql()

String com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.getInsertSql () [protected]

Member documentation.

Definition at line 67 of file [MedicalRecordMySqlRepository.java](#).

```
00067         {
00068         return "INSERT INTO medical_records (pet_id,pet_name,record_date,record_type,"
00069             + "diagnosis,treatment,veterinarian_name,cost,next_check_date,notes,vaccine_name)"
00070             + " VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?) ";
00071     }
```

7.18.3.3 getTableName()

String com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.getTableName () [protected]

Member documentation.

Definition at line 38 of file [MedicalRecordMySqlRepository.java](#).

```
00038 { return "medical_records"; }
```

7.18.3.4 getUpdateSql()

String com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.getUpdateSql () [protected]

Member documentation.

Definition at line 78 of file [MedicalRecordMySqlRepository.java](#).

```
00078         {
00079         return "UPDATE medical_records SET pet_id=?,pet_name=?,record_date=?,record_type=?,
00080             + "diagnosis=?,treatment=?,veterinarian_name=?,cost=?,next_check_date=?,
00081             + "notes=?,vaccine_name=? WHERE id=?";
00082     }
```

7.18.3.5 mapRow()

[MedicalRecord](#) com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow (ResultSet rs) throws SQLException [protected]

Member documentation.

Definition at line 121 of file [MedicalRecordMySqlRepository.java](#).

```
00121         {
00122         MedicalRecord m = new MedicalRecord();
00123         m.setId(rs.getInt("id"));
00124         m.setPetId(rs.getInt("pet_id"));
00125         m.setPetName(rs.getString("pet_name"));
00126         java.sql.Date rd = rs.getDate("record_date");
00127         if (rd != null) m.setRecordDate(rd.toLocalDate());
00128         m.setRecordType(rs.getString("record_type"));
00129         m.setDiagnosis(rs.getString("diagnosis"));
00130         m.setTreatment(rs.getString("treatment"));
00131         m.setVeterinarianName(rs.getString("veterinarian_name"));
00132         m.setCost(rs.getDouble("cost"));
00133         java.sql.Date ncd = rs.getDate("next_check_date");
00134         if (ncd != null) m.setNextCheckDate(ncd.toLocalDate());
00135         m.setNotes(rs.getString("notes"));
00136         m.setVaccineName(rs.getString("vaccine_name"));
00137         return m;
00138     }
```

References [com.mehdi.petreminder.model.MedicalRecord.setCost\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setDiagnosis\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setId\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setNextCheckDate\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setNotes\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setPetId\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setPetName\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setRecordDate\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setRecordType\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setTreatment\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setVaccineName\(\)](#), and [com.mehdi.petreminder.model.MedicalRecord.setVeterinarianName\(\)](#).

7.18.3.6 setInsertParams()

```
void com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.setInsertParams (
    PreparedStatement ps,
    MedicalRecord m) throws SQLException [protected]
```

Member documentation.

Definition at line 90 of file [MedicalRecordMySqlRepository.java](#).

```
00090
00091         ps.setInt(1, m.getPetId());
00092         ps.setString(2, m.getPetName());
00093         ps.setObject(3, m.getRecordDate() != null
00094             ? java.sql.Date.valueOf(m.getRecordDate()) : null);
00095         ps.setString(4, m.getRecordType());
00096         ps.setString(5, m.getDiagnosis());
00097         ps.setString(6, m.getTreatment());
00098         ps.setString(7, m.getVeterinarianName());
00099         ps.setDouble(8, m.getCost());
00100         ps.setObject(9, m.getNextCheckDate() != null
00101             ? java.sql.Date.valueOf(m.getNextCheckDate()) : null);
00102         ps.setString(10, m.getNotes());
00103         ps.setString(11, m.getVaccineName());
00104     }
```

Referenced by [setUpdateParams\(\)](#).

7.18.3.7 setUpdateParams()

```
void com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.setUpdateParams (
    PreparedStatement ps,
    MedicalRecord m) throws SQLException [protected]
```

Member documentation.

Definition at line 111 of file [MedicalRecordMySqlRepository.java](#).

```
00111
00112         setInsertParams(ps, m);
00113         ps.setInt(12, m.getId());
00114     }
```

References [setInsertParams\(\)](#).

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/MedicalRecordMySqlRepository.java](#)

7.19 com.mehdi.petreminder.service.MedicalRecordService Class Reference

[MedicalRecord](#) CRUD ve iş kuralları.

Public Member Functions

- [MedicalRecordService](#) ()
Varsayılan yapıcı.
- [MedicalRecordService](#) ([IRepository](#)< [MedicalRecord](#) > repository)
Test yapıcısı.
- [MedicalRecord](#) [addRecord](#) ([MedicalRecord](#) record)
Kayıt ekler.
- [MedicalRecord](#) [getRecordById](#) (int id)
ID ile kayıt getirir.
- List< [MedicalRecord](#) > [getAllRecords](#) ()
Tüm kayıtları listeler.
- List< [MedicalRecord](#) > [getRecordsByPetId](#) (int petId)
Pet'e ait kayıtları getirir.
- void [updateRecord](#) ([MedicalRecord](#) record)
Kayıt günceller.
- void [deleteRecord](#) (int id)
Kayıt siler.
- void [deleteAllRecords](#) ()
Tüm kayıtları siler.
- int [getRecordCount](#) ()
Toplam kayıt sayısı.
- void [close](#) ()
Repository kapatır.

7.19.1 Detailed Description

[MedicalRecord](#) CRUD ve iş kuralları.

Observer Pattern: Kayıt eklendiğinde veya silindiğinde [EventManager](#) üzerinden bildirim gönderilir.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 32 of file [MedicalRecordService.java](#).

7.19.2 Constructor & Destructor Documentation

7.19.2.1 [MedicalRecordService](#)() [1/2]

```
com.mehdi.petreminder.service.MedicalRecordService.MedicalRecordService ()
```

Varsayılan yapıcı.

Definition at line 44 of file [MedicalRecordService.java](#).

```
00044                                     {  
00045         this.repository = RepositoryFactory.createMedicalRecordRepository();  
00046     }
```

References [com.mehdi.petreminder.repository.RepositoryFactory.createMedicalRecordRepository\(\)](#).

7.19.2.2 MedicalRecordService() [2/2]

```
com.mehdi.petreminder.service.MedicalRecordService.MedicalRecordService (
    IRepository< MedicalRecord > repository)
```

Test yapıcısı.

Parameters

<i>repository</i>	Test repository
-------------------	-----------------

Definition at line 52 of file [MedicalRecordService.java](#).

```
00052                                     {
00053         this.repository = repository;
00054     }
```

7.19.3 Member Function Documentation

7.19.3.1 addRecord()

```
MedicalRecord com.mehdi.petreminder.service.MedicalRecordService.addRecord (
    MedicalRecord record)
```

Kayıt ekler.

Parameters

<i>record</i>	Eklenecek MedicalRecord
---------------	---

Returns

Eklenen kayıt

Definition at line 61 of file [MedicalRecordService.java](#).

```
00061                                     {
00062         validateRecord(record);
00063         repository.save(record);
00064         logger.info("MedicalRecord eklendi: petId={}", record.getPetId());
00065         // Observer Pattern: Saglik kaydi eklendi bildirimi
00066         EventManager.getInstance().notify(EventType.MEDICAL_RECORD_ADDED, record);
00067         return record;
00068     }
```

References [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.getPetId\(\)](#), [com.mehdi.petreminder.observer.EventType.MEDICAL_RECORD_ADDED](#), and [com.mehdi.petreminder.observer.EventManager.not](#)

7.19.3.2 close()

```
void com.mehdi.petreminder.service.MedicalRecordService.close ()
```

Repository kapatır.

Definition at line 142 of file [MedicalRecordService.java](#).

```
00142                                     {
00143         repository.close();
00144     }
```

7.19.3.3 deleteAllRecords()

```
void com.mehdi.petreminder.service.MedicalRecordService.deleteAllRecords ()
```

Tüm kayıtları siler.

Definition at line 127 of file [MedicalRecordService.java](#).

```
00127         {
00128             repository.deleteAll();
00129         }
```

7.19.3.4 deleteRecord()

```
void com.mehdi.petreminder.service.MedicalRecordService.deleteRecord (
    int id)
```

Kayıt siler.

Parameters

<i>id</i>	Kayıt ID
-----------	----------

Definition at line 116 of file [MedicalRecordService.java](#).

```
00116         {
00117             boolean deleted = repository.delete(id);
00118             if (!deleted) throw new ServiceException("Kayıt silinemedi: id=" + id);
00119             logger.info("MedicalRecord silindi: id={}", id);
00120             // Observer Pattern: Sağlık kaydı silindi bildirimi
00121             EventManager.getInstance().notify(EventType.MEDICAL_RECORD_DELETED, id);
00122         }
```

References [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [com.mehdi.petreminder.observer.EventType.MEDICAL_RECORD_DELETED](#) and [com.mehdi.petreminder.observer.EventManager.notify\(\)](#).

7.19.3.5 getAllRecords()

```
List< MedicalRecord > com.mehdi.petreminder.service.MedicalRecordService.getAllRecords ()
```

Tüm kayıtları listeler.

Returns

Liste

Definition at line 84 of file [MedicalRecordService.java](#).

```
00084         {
00085             return repository.findAll();
00086         }
```

7.19.3.6 getRecordById()

[MedicalRecord](#) com.mehdi.petreminder.service.MedicalRecordService.getRecordById (
int id)

ID ile kayıt getirir.

Parameters

<i>id</i>	Kayıt ID
-----------	----------

Returns

[MedicalRecord](#)

Definition at line 75 of file [MedicalRecordService.java](#).

```
00075                                     {
00076         return repository.findById(id)
00077         .orElseThrow(() -> new ServiceException("Kayıt bulunamadı: id=" + id));
00078     }
```

7.19.3.7 getRecordCount()

int com.mehdi.petreminder.service.MedicalRecordService.getRecordCount ()

Toplam kayıt sayısı.

Returns

Sayı

Definition at line 135 of file [MedicalRecordService.java](#).

```
00135                                     {
00136         return repository.count();
00137     }
```

7.19.3.8 getRecordsByPetId()

List< [MedicalRecord](#) > com.mehdi.petreminder.service.MedicalRecordService.getRecordsByPetId (
int petId)

Pet'e ait kayıtları getirir.

Parameters

<i>pet↔ Id</i>	Pet ID
--------------------	--------

Returns

[MedicalRecord](#) listesi

Definition at line 93 of file [MedicalRecordService.java](#).

```
00093                                     {
00094         return repository.findAll().stream()
00095         .filter(r -> r.getPetId() == petId)
00096         .sorted(Comparator.comparing(
00097             MedicalRecord::getRecordDate,
00098             Comparator.nullsLast(Comparator.reverseOrder())))
00099         .collect(Collectors.toList());
00100     }
```

References [com.mehdi.petreminder.model.MedicalRecord.getRecordDate\(\)](#).

7.19.3.9 updateRecord()

```
void com.mehdi.petreminder.service.MedicalRecordService.updateRecord (
    MedicalRecord record)
```

Kayıt günceller.

Parameters

<i>record</i>	Güncellenecek kayıt
---------------	---------------------

Definition at line 106 of file [MedicalRecordService.java](#).

```
00106                                     {
00107         validateRecord(record);
00108         boolean updated = repository.update(record);
00109         if (!updated) throw new ServiceException("Kayıt güncellenemedi: id=" + record.getId());
00110     }
```

References [com.mehdi.petreminder.model.MedicalRecord.getId\(\)](#).

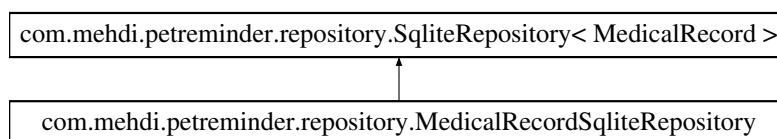
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/service/MedicalRecordService.java](#)

7.20 com.mehdi.petreminder.repository.MedicalRecordSqliteRepository Class Reference

SQLite üzerinde [MedicalRecord](#) CRUD.

Inheritance diagram for [com.mehdi.petreminder.repository.MedicalRecordSqliteRepository](#):



Public Member Functions

- [MedicalRecordSqliteRepository \(\)](#)
Varsayılan yapıcı.
- [MedicalRecordSqliteRepository \(Connection connection\)](#)
Test yapıcısı.
- [List< MedicalRecord > findByPetId \(int petId\)](#)
PetId'ye göre sağlık kayıtlarını getirir.

Public Member Functions inherited from**com.mehdi.petreminder.repository.SqliteRepository< MedicalRecord >**

- [SqliteRepository](#) ()
SQLite bağlantısını açar ve tabloyu oluşturur.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür (test için).

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [MedicalRecord](#) m) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [MedicalRecord](#) m) throws SQLException
- [MedicalRecord](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from**com.mehdi.petreminder.repository.SqliteRepository< MedicalRecord >**

- abstract String [getTableName](#) ()
Tablo adını döndürür.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL'ini döndürür.
- abstract String [getInsertSql](#) ()
INSERT SQL'ini döndürür (ID hariç).
- abstract String [getUpdateSql](#) ()
UPDATE SQL'ini döndürür.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members

Protected Attributes inherited from

[com.mehdi.petreminder.repository.SqliteRepository](#) < [MedicalRecord](#) >

- Connection [connection](#)
SQLite JDBC bağlantısı.

Static Protected Attributes inherited from

[com.mehdi.petreminder.repository.SqliteRepository](#) < [MedicalRecord](#) >

- static final Logger [logger](#)
Logger.

7.20.1 Detailed Description

SQLite üzerinde [MedicalRecord](#) CRUD.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 23 of file [MedicalRecordSqliteRepository.java](#).

7.20.2 Constructor & Destructor Documentation

7.20.2.1 [MedicalRecordSqliteRepository\(\)](#) [1/2]

```
com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.MedicalRecordSqliteRepository  
( )
```

Varsayılan yapıcı.

Definition at line 26 of file [MedicalRecordSqliteRepository.java](#).

```
00026 { super(); }
```

7.20.2.2 MedicalRecordSqliteRepository() [2/2]

com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.MedicalRecordSqliteRepository (
 Connection connection)

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 32 of file [MedicalRecordSqliteRepository.java](#).

```
00032 { super(connection); }
```

References [com.mehdi.petreminder.repository.SqliteRepository< MedicalRecord >.connection](#).

7.20.3 Member Function Documentation

7.20.3.1 findByPetId()

List< [MedicalRecord](#) > com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.findBy↵
 PetId (
 int petId)

PetId'ye göre sağlık kayıtlarını getirir.

Parameters

<i>pet↵ Id</i>	Pet ID
--------------------	--------

Returns

[MedicalRecord](#) listesi

Definition at line 144 of file [MedicalRecordSqliteRepository.java](#).

```
00144         {
00145             List<MedicalRecord> list = new ArrayList<>();
00146             try (PreparedStatement ps = connection.prepareStatement(
00147                 "SELECT * FROM medical_records WHERE pet_id = ?")) {
00148                 ps.setInt(1, petId);
00149                 ResultSet rs = ps.executeQuery();
00150                 while (rs.next()) list.add(mapRow(rs));
00151             } catch (SQLException e) {
00152                 throw new RepositoryException("findByPetId hatası", e);
00153             }
00154             return list;
00155         }
```

References [com.mehdi.petreminder.repository.SqliteRepository< MedicalRecord >.connection](#), and [mapRow\(\)](#).

7.20.3.2 getCreateTableSql()

```
String com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.getCreateTableSql ()
[protected]
```

Member documentation.

Definition at line 46 of file [MedicalRecordSqliteRepository.java](#).

```
00046      {
00047      return "CREATE TABLE IF NOT EXISTS medical_records ("
00048      + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00049      + "pet_id INTEGER NOT NULL,"
00050      + "pet_name TEXT,"
00051      + "record_date TEXT NOT NULL,"
00052      + "record_type TEXT NOT NULL,"
00053      + "diagnosis TEXT,"
00054      + "treatment TEXT,"
00055      + "veterinarian_name TEXT,"
00056      + "cost REAL DEFAULT 0,"
00057      + "next_check_date TEXT,"
00058      + "notes TEXT,"
00059      + "vaccine_name TEXT"
00060      + ")";
00061      }
```

7.20.3.3 getInsertSql()

```
String com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.getInsertSql () [protected]
```

Member documentation.

Definition at line 68 of file [MedicalRecordSqliteRepository.java](#).

```
00068      {
00069      return "INSERT INTO medical_records (pet_id,pet_name,record_date,record_type,"
00070      + "diagnosis,treatment,veterinarian_name,cost,next_check_date,notes,vaccine_name)"
00071      + " VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00072      }
```

7.20.3.4 getTableName()

```
String com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.getTableName () [protected]
```

Member documentation.

Definition at line 39 of file [MedicalRecordSqliteRepository.java](#).

```
00039 { return "medical_records"; }
```

7.20.3.5 getUpdateSql()

```
String com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.getUpdateSql () [protected]
```

Member documentation.

Definition at line 79 of file [MedicalRecordSqliteRepository.java](#).

```
00079      {
00080      return "UPDATE medical_records SET pet_id=?,pet_name=?,record_date=?,record_type=?, "
00081      + "diagnosis=?,treatment=?,veterinarian_name=?,cost=?,next_check_date=?, "
00082      + "notes=?,vaccine_name=? WHERE id=?";
00083      }
```

7.20.3.6 mapRow()

`MedicalRecord` com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.mapRow (
 `ResultSet rs`) throws `SQLException` [protected]

Member documentation.

Definition at line 120 of file `MedicalRecordSqliteRepository.java`.

```
00120                                     {
00121         MedicalRecord m = new MedicalRecord();
00122         m.setId(rs.getInt("id"));
00123         m.setPetId(rs.getInt("pet_id"));
00124         m.setPetName(rs.getString("pet_name"));
00125         String rd = rs.getString("record_date");
00126         if (rd != null) m.setRecordDate(LocalDate.parse(rd));
00127         m.setRecordType(rs.getString("record_type"));
00128         m.setDiagnosis(rs.getString("diagnosis"));
00129         m.setTreatment(rs.getString("treatment"));
00130         m.setVeterinarianName(rs.getString("veterinarian_name"));
00131         m.setCost(rs.getDouble("cost"));
00132         String ncd = rs.getString("next_check_date");
00133         if (ncd != null) m.setNextCheckDate(LocalDate.parse(ncd));
00134         m.setNotes(rs.getString("notes"));
00135         m.setVaccineName(rs.getString("vaccine_name"));
00136         return m;
00137     }
```

References `com.mehdi.petreminder.model.MedicalRecord.setCost()`, `com.mehdi.petreminder.model.MedicalRecord.setDiagnosis()`, `com.mehdi.petreminder.model.MedicalRecord.setId()`, `com.mehdi.petreminder.model.MedicalRecord.setNextCheckDate()`, `com.mehdi.petreminder.model.MedicalRecord.setNotes()`, `com.mehdi.petreminder.model.MedicalRecord.setPetId()`, `com.mehdi.petreminder.model.MedicalRecord.setPetName()`, `com.mehdi.petreminder.model.MedicalRecord.setRecordDate()`, `com.mehdi.petreminder.model.MedicalRecord.setRecordType()`, `com.mehdi.petreminder.model.MedicalRecord.setTreatment()`, `com.mehdi.petreminder.model.MedicalRecord.setVaccineName()`, and `com.mehdi.petreminder.model.MedicalRecord.setVeterinarianName()`.

Referenced by `findByPetId()`.

7.20.3.7 setInsertParams()

`void` com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.setInsertParams (
 `PreparedStatement ps`,
 `MedicalRecord m`) throws `SQLException` [protected]

Member documentation.

Definition at line 91 of file `MedicalRecordSqliteRepository.java`.

```
00091                                     {
00092         ps.setInt(1, m.getPetId());
00093         ps.setString(2, m.getPetName());
00094         ps.setString(3, m.getRecordDate() != null ? m.getRecordDate().toString() : null);
00095         ps.setString(4, m.getRecordType());
00096         ps.setString(5, m.getDiagnosis());
00097         ps.setString(6, m.getTreatment());
00098         ps.setString(7, m.getVeterinarianName());
00099         ps.setDouble(8, m.getCost());
00100         ps.setString(9, m.getNextCheckDate() != null ? m.getNextCheckDate().toString() : null);
00101         ps.setString(10, m.getNotes());
00102         ps.setString(11, m.getVaccineName());
00103     }
```

Referenced by `setUpdateParams()`.

7.20.3.8 setUpdateParams()

```
void com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.setUpdateParams (
    PreparedStatement ps,
    MedicalRecord m) throws SQLException [protected]
```

Member documentation.

Definition at line 110 of file [MedicalRecordSqliteRepository.java](#).

```
00110
00111         setInsertParams(ps, m);
00112         ps.setInt(12, m.getId());
00113     }
```

References [setInsertParams\(\)](#).

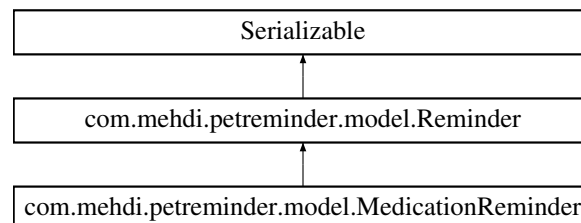
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/MedicalRecordSqliteRepository.java](#)

7.21 com.mehdi.petreminder.model.MedicationReminder Class Reference

Evcil hayvan ilaç hatırlatıcısı.

Inheritance diagram for com.mehdi.petreminder.model.MedicationReminder:



Public Member Functions

- [MedicationReminder \(\)](#)
Varsayılan yapıcı.
- [MedicationReminder \(int id, int petId, String petName, LocalDateTime scheduledTime, String medicationName, double dosage, String dosageUnit\)](#)
Temel parametrelili yapıcı.
- [MedicationReminder \(int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern, String medicationName, double dosage, String dosageUnit, boolean prescribed, String veterinarianName\)](#)
Tam parametrelili yapıcı.
- String [getMedicationName \(\)](#)
İlaç adı getter.
- void [setMedicationName \(String medicationName\)](#)
İlaç adı setter.
- double [getDosage \(\)](#)

- Doz getter.*
- void [setDosage](#) (double dosage)
- Doz setter.*
- String [getDosageUnit](#) ()
- Birim getter.*
- void [setDosageUnit](#) (String dosageUnit)
- Birim setter.*
- boolean [isPrescribed](#) ()
- Reçeteli mi getter.*
- void [setPrescribed](#) (boolean prescribed)
- Reçeteli setter.*
- String [getVeterinarianName](#) ()
- Veteriner adı getter.*
- void [setVeterinarianName](#) (String veterinarianName)
- Veteriner adı setter.*
- String [getReminderType](#) ()
- Hatırlatıcı türü.*
- boolean [validate](#) ()
- İlaç hatırlatıcısını doğrular.*
- String [getSummary](#) ()
- Özet bilgi.*
- String [toString](#) ()
- String temsili.*

Public Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- int [getId](#) ()
- ID getter.*
- void [setId](#) (int id)
- ID setter.*
- int [getPetId](#) ()
- Pet ID getter.*
- void [setPetId](#) (int petId)
- Pet ID setter.*
- String [getPetName](#) ()
- Pet adı getter.*
- void [setPetName](#) (String petName)
- Pet adı setter.*
- LocalDateTime [getScheduledTime](#) ()
- Planlanan zaman getter.*
- void [setScheduledTime](#) (LocalDateTime scheduledTime)
- Planlanan zaman setter.*
- boolean [isCompleted](#) ()
- Tamamlandı mı getter.*
- void [setCompleted](#) (boolean completed)
- Tamamlandı setter.*
- String [getDescription](#) ()
- Açıklama getter.*
- void [setDescription](#) (String description)
- Açıklama setter.*

- String [getPriority](#) ()
Öncelik getter.
- void [setPriority](#) (String priority)
Öncelik setter.
- boolean [isRecurring](#) ()
Tekrarlıyor mu getter.
- void [setRecurring](#) (boolean recurring)
Tekrarlama setter.
- String [getRecurrencePattern](#) ()
Tekrar deseni getter.
- void [setRecurrencePattern](#) (String recurrencePattern)
Tekrar deseni setter.
- String [getFormattedTime](#) ()
Planlanan zamanı biçimli string olarak döndürür.
- boolean [isOverdue](#) ()
Hatırlatıcının geçmiş mi olduğunu kontrol eder.
- boolean [equals](#) (Object obj)
Eşitlik kontrolü — ID bazlı.
- int [hashCode](#) ()
Hash kodu.

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- [Reminder](#) ()
Varsayılan yapıcı — Serialization için.
- [Reminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern)
Tam parametrelili yapıcı.

Static Protected Attributes inherited from [com.mehdi.petreminder.model.Reminder](#)

- static final DateFormatter [FORMATTER](#)
Tarih saat formatı.

7.21.1 Detailed Description

Evcil hayvan ilaç hatırlatıcısı.

OOP Inheritance: [Reminder](#)'dan miras alır.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 16 of file [MedicationReminder.java](#).

7.21.2 Constructor & Destructor Documentation

7.21.2.1 MedicationReminder() [1/3]

com.mehdi.petreminder.model.MedicationReminder.MedicationReminder ()

Varsayılan yapıcı.

Definition at line 37 of file [MedicationReminder.java](#).

```
00037 { super(); }
```

7.21.2.2 MedicationReminder() [2/3]

```
com.mehdi.petreminder.model.MedicationReminder.MedicationReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String medicationName,
    double dosage,
    String dosageUnit)
```

Temel parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>medicationName</i>	İlaç adı
<i>dosage</i>	Doz
<i>dosageUnit</i>	Birim

Definition at line 49 of file [MedicationReminder.java](#).

```
00052
00053         super(id, petId, petName, scheduledTime,
00054             "İlaç: " + medicationName, "YUKSEK", false, "");
00055         this.medicationName = medicationName;
00056         this.dosage = dosage;
00057         this.dosageUnit = dosageUnit;
00058         this.prescribed = false;
00059         this.veterinarianName = "";
00060     }
```

7.21.2.3 MedicationReminder() [3/3]

```
com.mehdi.petreminder.model.MedicationReminder.MedicationReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String description,
    String priority,
    boolean recurring,
    String recurrencePattern,
    String medicationName,
    double dosage,
    String dosageUnit,
    boolean prescribed,
    String veterinarianName)
```

Tam parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>description</i>	Açıklama
<i>priority</i>	Öncelik
<i>recurring</i>	Tekrarlıyor mu
<i>recurrencePattern</i>	Tekrar deseni
<i>medicationName</i>	İlaç adı
<i>dosage</i>	Doz
<i>dosageUnit</i>	Birim
<i>prescribed</i>	Reçeteli mi
<i>veterinarianName</i>	Veteriner adı

Definition at line 78 of file [MedicationReminder.java](#).

```
00083
00084         super(id, petId, petName, scheduledTime, description,
00085             priority, recurring, recurrencePattern);
00086         this.medicationName = medicationName;
00087         this.dosage = dosage;
00088         this.dosageUnit = dosageUnit;
00089         this.prescribed = prescribed;
00090         this.veterinarianName = veterinarianName;
00091     }
```

7.21.3 Member Function Documentation

7.21.3.1 getDosage()

```
double com.mehdi.petreminder.model.MedicationReminder.getDosage ()
```

Doz getter.

Returns

Doz

Definition at line 102 of file [MedicationReminder.java](#).

```
00102 { return dosage; }
```

7.21.3.2 getDosageUnit()

```
String com.mehdi.petreminder.model.MedicationReminder.getDosageUnit ()
```

Birim getter.

Returns

Birim

Definition at line 108 of file [MedicationReminder.java](#).

```
00108 { return dosageUnit; }
```

7.21.3.3 getMedicationName()

```
String com.mehdi.petreminder.model.MedicationReminder.getMedicationName ()
```

İlaç adı getter.

Returns

İlaç adı

Definition at line 94 of file [MedicationReminder.java](#).

```
00094 { return medicationName; }
```

7.21.3.4 getReminderType()

```
String com.mehdi.petreminder.model.MedicationReminder.getReminderType ()
```

Hatırlatıcı türü.

Returns

"İlac"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 129 of file [MedicationReminder.java](#).

```
00129 { return "İlac"; }
```

7.21.3.5 getSummary()

```
String com.mehdi.petreminder.model.MedicationReminder.getSummary ()
```

Özet bilgi.

Returns

"İlaç: Antibiyotik 250mg"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 147 of file [MedicationReminder.java](#).

```
00147         {  
00148             return String.format("İlaç: %s %.1f%s%s",  
00149                 medicationName, dosage, dosageUnit,  
00150                 prescribed ? " (Receteli)" : "");  
00151         }
```

7.21.3.6 getVeterinarianName()

```
String com.mehdi.petreminder.model.MedicationReminder.getVeterinarianName ()
```

Veteriner adı getter.

Returns

Veteriner adı

Definition at line 120 of file [MedicationReminder.java](#).

```
00120 { return veterinarianName; }
```

7.21.3.7 isPrescribed()

```
boolean com.mehdi.petreminder.model.MedicationReminder.isPrescribed ()
```

Reçeteli mi getter.

Returns

Reçeteli mi

Definition at line 114 of file [MedicationReminder.java](#).

```
00114 { return prescribed; }
```

7.21.3.8 setDosage()

```
void com.mehdi.petreminder.model.MedicationReminder.setDosage (
    double dosage)
```

Doz setter.

Parameters

<i>dosage</i>	Doz
---------------	-----

Definition at line 105 of file [MedicationReminder.java](#).

```
00105 { this.dosage = dosage; }
```

7.21.3.9 setDosageUnit()

```
void com.mehdi.petreminder.model.MedicationReminder.setDosageUnit (
    String dosageUnit)
```

Birim setter.

Parameters

<i>dosageUnit</i>	Birim
-------------------	-------

Definition at line 111 of file [MedicationReminder.java](#).

```
00111 { this.dosageUnit = dosageUnit; }
```

7.21.3.10 setMedicationName()

```
void com.mehdi.petreminder.model.MedicationReminder.setMedicationName (
    String medicationName)
```

İlaç adı setter.

Parameters

<i>medicationName</i>	İlaç adı
-----------------------	----------

Definition at line 97 of file [MedicationReminder.java](#).

```
00097                                     {
00098         this.medicationName = medicationName;
00099     }
```

7.21.3.11 setPrescribed()

```
void com.mehdi.petreminder.model.MedicationReminder.setPrescribed (
    boolean prescribed)
```

Reçeteli setter.

Parameters

<i>prescribed</i>	Reçeteli mi
-------------------	-------------

Definition at line 117 of file [MedicationReminder.java](#).

```
00117 { this.prescribed = prescribed; }
```

7.21.3.12 setVeterinarianName()

```
void com.mehdi.petreminder.model.MedicationReminder.setVeterinarianName (
    String veterinarianName)
```

Veteriner adı setter.

Parameters

<i>veterinarianName</i>	Veteriner adı
-------------------------	---------------

Definition at line 123 of file [MedicationReminder.java](#).

```
00123                                     {
00124         this.veterinarianName = veterinarianName;
00125     }
```

7.21.3.13 toString()

```
String com.mehdi.petreminder.model.MedicationReminder.toString ()
```

String temsili.

Returns

İlaç hatırlatıcısı bilgisi

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 158 of file [MedicationReminder.java](#).

```
00158         {
00159         return String.format("MedicationReminder{id=%d, pet='%s', med='%s', dose=%.1f%s}",
00160             getId(), getPetName(), medicationName, dosage, dosageUnit);
00161     }
```

References [com.mehdi.petreminder.model.Reminder.getId\(\)](#), and [com.mehdi.petreminder.model.Reminder.getPetName\(\)](#).

7.21.3.14 validate()

```
boolean com.mehdi.petreminder.model.MedicationReminder.validate ()
```

İlaç hatırlatıcısını doğrular.

Returns

İlaç adı ve doz geçerliyse true

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 136 of file [MedicationReminder.java](#).

```
00136         {
00137             return medicationName != null && !medicationName.trim().isEmpty()
00138                && dosage > 0
00139                && dosageUnit != null && !dosageUnit.trim().isEmpty();
00140         }
```

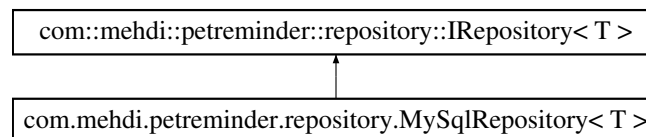
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/MedicationReminder.java](#)

7.22 com.mehdi.petreminder.repository.MySqlRepository< T > Class Template Reference

MySQL tabanlı abstract repository temel sınıfı.

Inheritance diagram for com.mehdi.petreminder.repository.MySqlRepository< T >:



Public Member Functions

- [MySqlRepository](#) ()
MySQL bağlantısını açar.
- [MySqlRepository](#) (Connection [connection](#))
Test amaçlı yapıcı.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür.

Protected Member Functions

- abstract String [getTableName](#) ()
Tablo adı.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL.
- abstract String [getInsertSql](#) ()
INSERT SQL.
- abstract String [getUpdateSql](#) ()
UPDATE SQL.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Protected Attributes

- Connection [connection](#)
MySQL JDBC bağlantısı.

Static Protected Attributes

- static final Logger [logger](#)
Logger.

7.22.1 Detailed Description

MySQL tabanlı abstract repository temel sınıfı.

[SqliteRepository](#) ile aynı Template Method Pattern kullanır. Docker Compose ile ayağa kaldırılan MySQL 8.0'a bağlanır.

Parameters

< T >	Varlık türü
-------	-------------

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 27 of file [MySqlRepository.java](#).

7.22.2 Constructor & Destructor Documentation

7.22.2.1 MySqlRepository() [1/2]

`com.mehdi.petreminder.repository.MySqlRepository< T >.MySqlRepository ()`

MySQL bağlantısını açar.

Exceptions

<i>RepositoryException</i>	Bağlantı başarısız olursa
--	---------------------------

Definition at line 40 of file [MySqlRepository.java](#).

```

00040                                     {
00041         try {
00042             Class.forName("com.mysql.cj.jdbc.Driver");
00043             connection = DriverManager.getConnection(
00044                 StorageConfig.getMysqlUrl(),
00045                 StorageConfig.getMysqlUsername(),
00046                 StorageConfig.getMysqlPassword());
00047             connection.setAutoCommit(true);
00048             createTableIfNotExists();
00049             logger.info("MySQL bağlantısı açıldı: {}", StorageConfig.getMysqlUrl());
00050         } catch (Exception e) {
00051             throw new RepositoryException("MySQL bağlantı hatası -- Docker çalışıyor mu?", e);
00052         }
00053     }

```

References [connection](#), [createTableIfNotExists\(\)](#), [com.mehdi.petreminder.config.StorageConfig.getMysqlPassword\(\)](#), [com.mehdi.petreminder.config.StorageConfig.getMysqlUrl\(\)](#), [com.mehdi.petreminder.config.StorageConfig.getMysqlUsername\(\)](#), and [logger](#).

7.22.2.2 MySqlRepository() [2/2]

`com.mehdi.petreminder.repository.MySqlRepository< T >.MySqlRepository (Connection connection)`

Test amaçlı yapıcı.

Parameters

<i>connection</i>	Test bağlantısı (H2 MySQL-mode)
-------------------	---------------------------------

Definition at line 59 of file [MySqlRepository.java](#).

```

00059                                     {
00060         this.connection = connection;
00061         try {
00062             createTableIfNotExists();
00063         } catch (Exception e) {
00064             throw new RepositoryException("Tablo oluşturma hatası", e);
00065         }
00066     }

```

References [connection](#), and [createTableIfNotExists\(\)](#).

7.22.3 Member Function Documentation

7.22.3.1 close()

```
void com.mehdi.petreminder.repository.MySqlRepository< T >.close ()
```

Repository bağlantısını/kaynaklarını kapatır.

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 217 of file [MySqlRepository.java](#).

```
00217         {
00218             try {
00219                 if (connection != null && !connection.isClosed()) {
00220                     connection.close();
00221                     logger.info("MySQL bağlantısı kapatıldı.");
00222                 }
00223             } catch (SQLException e) {
00224                 logger.warn("MySQL kapatma hatası: {}", e.getMessage());
00225             }
00226         }
```

References [connection](#), and [logger](#).

7.22.3.2 count()

```
int com.mehdi.petreminder.repository.MySqlRepository< T >.count ()
```

Varlık sayısını döndürür.

Returns

Toplam kayıt sayısı

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 204 of file [MySqlRepository.java](#).

```
00204         {
00205             try (Statement st = connection.createStatement();
00206                 ResultSet rs = st.executeQuery("SELECT COUNT(*) FROM " + getTableName())) {
00207                 if (rs.next()) return rs.getInt(1);
00208             } catch (SQLException e) {
00209                 throw new RepositoryException("MySQL count hatası", e);
00210             }
00211             return 0;
00212         }
```

References [connection](#), and [getTableName\(\)](#).

7.22.3.3 createTableIfNotExists()

```
void com.mehdi.petreminder.repository.MySqlRepository< T >.createTableIfNotExists () [protected]
```

Tabloyu oluşturur (yoksa).

Definition at line 110 of file [MySqlRepository.java](#).

```
00110         {
00111             try (Statement st = connection.createStatement()) {
00112                 st.execute(getCreateTableSql());
00113             } catch (SQLException e) {
00114                 throw new RepositoryException("Tablo oluşturma hatası", e);
00115             }
00116         }
```

References [connection](#), and [getCreateTableSql\(\)](#).

Referenced by [MySqlRepository\(\)](#), and [MySqlRepository\(\)](#).

7.22.3.4 delete()

```
boolean com.mehdi.petreminder.repository.MySqlRepository< T >.delete (
    int id)
```

ID'ye göre varlık siler.

Parameters

<i>id</i>	Silinecek varlığın ID'si
-----------	--------------------------

Returns

Silme başarılıysa true

Exceptions

RepositoryException	Silme başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 181 of file [MySqlRepository.java](#).

```
00181         {
00182             try (PreparedStatement ps = connection.prepareStatement(
00183                 "DELETE FROM " + getTableName() + " WHERE id = ?")) {
00184                 ps.setInt(1, id);
00185                 return ps.executeUpdate() > 0;
00186             } catch (SQLException e) {
00187                 throw new RepositoryException("MySQL delete hatası", e);
00188             }
00189         }
```

References [connection](#), and [getTableName\(\)](#).

7.22.3.5 deleteAll()

```
void com.mehdi.petreminder.repository.MySqlRepository< T >.deleteAll ()
```

Tüm varlıkları siler.

Exceptions

RepositoryException	Silme başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 193 of file [MySqlRepository.java](#).

```
00193         {
00194             try (Statement st = connection.createStatement()) {
00195                 st.execute("DELETE FROM " + getTableName());
00196             } catch (SQLException e) {
00197                 throw new RepositoryException("MySQL deleteAll hatası", e);
00198             }
00199         }
```

References [connection](#), and [getTableName\(\)](#).

7.22.3.6 findAll()

List< T > [com.mehdi.petreminder.repository.MySqlRepository](#)< T >.findAll ()

Tüm varlıkları listeler.

Returns

Varlık listesi (boş olabilir)

Exceptions

RepositoryException	Okuma başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository](#)< T >.

Definition at line 155 of file [MySqlRepository.java](#).

```

00155         {
00156             List<T> list = new ArrayList<>();
00157             try (Statement st = connection.createStatement();
00158                 ResultSet rs = st.executeQuery("SELECT * FROM " + getTableName())) {
00159                 while (rs.next()) list.add(mapRow(rs));
00160             } catch (SQLException e) {
00161                 throw new RepositoryException("MySQL findAll hatası", e);
00162             }
00163             return list;
00164         }

```

References [connection](#), [getTableName\(\)](#), and [mapRow\(\)](#).

7.22.3.7 findById()

Optional< T > [com.mehdi.petreminder.repository.MySqlRepository](#)< T >.findById (int id)

ID'ye göre varlık getirir.

Parameters

<i>id</i>	Varlık ID'si
-----------	--------------

Returns

Varlık (bulunamazsa empty Optional)

Exceptions

RepositoryException	Okuma başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository](#)< T >.

Definition at line 141 of file [MySqlRepository.java](#).

```

00141         {
00142             String sql = "SELECT * FROM " + getTableName() + " WHERE id = ?";
00143             try (PreparedStatement ps = connection.prepareStatement(sql)) {
00144                 ps.setInt(1, id);
00145                 ResultSet rs = ps.executeQuery();
00146                 if (rs.next()) return Optional.of(mapRow(rs));
00147             } catch (SQLException e) {
00148                 throw new RepositoryException("MySQL findById hatası", e);
00149             }
00150             return Optional.empty();
00151         }

```

References [connection](#), [getTableName\(\)](#), and [mapRow\(\)](#).

7.22.3.8 getConnection()

Connection [com.mehdi.petreminder.repository.MySqlRepository< T >.getConnection](#) ()

Bağlantıyı döndürür.

Returns

JDBC Connection

Definition at line 232 of file [MySqlRepository.java](#).

```
00232         {  
00233             return connection;  
00234         }
```

References [connection](#).

7.22.3.9 getCreateTableSql()

abstract String [com.mehdi.petreminder.repository.MySqlRepository< T >.getCreateTableSql](#) ()
[abstract], [protected]

Tablo oluşturma SQL.

Returns

CREATE TABLE sql

Referenced by [createTableIfNotExists\(\)](#).

7.22.3.10 getInsertSql()

abstract String [com.mehdi.petreminder.repository.MySqlRepository< T >.getInsertSql](#) () [abstract],
[protected]

INSERT SQL.

Returns

INSERT sql

Referenced by [save\(\)](#).

7.22.3.11 getTableName()

abstract String [com.mehdi.petreminder.repository.MySqlRepository< T >.getTableName](#) () [abstract],
[protected]

Tablo adı.

Returns

SQL tablo adı

Referenced by [count\(\)](#), [delete\(\)](#), [deleteAll\(\)](#), [findAll\(\)](#), and [findById\(\)](#).

7.22.3.12 getUpdateSql()

```
abstract String com.mehdi.petreminder.repository.MySqlRepository< T >.getUpdateSql () [abstract],  
[protected]
```

UPDATE SQL.

Returns

UPDATE sql

Referenced by [update\(\)](#).

7.22.3.13 mapRow()

```
abstract T com.mehdi.petreminder.repository.MySqlRepository< T >.mapRow (  
    ResultSet rs) throws SQLException [abstract], [protected]
```

ResultSet'ten varlık oluşturur.

Parameters

<i>rs</i>	ResultSet
-----------	-----------

Returns

Varlık nesnesi

Exceptions

<i>SQLException</i>	SQL hatası
---------------------	------------

Referenced by [findAll\(\)](#), and [findById\(\)](#).

7.22.3.14 save()

```
int com.mehdi.petreminder.repository.MySqlRepository< T >.save (  
    T entity)
```

Yeni bir varlık kaydeder.

Parameters

<i>entity</i>	Kaydedilecek varlık
---------------	---------------------

Returns

Oluşturulan varlığın ID'si

Exceptions

RepositoryException	Kayıt başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 121 of file [MySqlRepository.java](#).

```

00121         {
00122             if (entity == null) throw new RepositoryException("Entity null olamaz");
00123             try (PreparedStatement ps = connection.prepareStatement(
00124                 getInsertSql(), Statement.RETURN_GENERATED_KEYS)) {
00125                 setInsertParams(ps, entity);
00126                 ps.executeUpdate();
00127                 ResultSet keys = ps.getGeneratedKeys();
00128                 if (keys.next()) {
00129                     int id = keys.getInt(1);
00130                     setEntityId(entity, id);
00131                     return id;
00132                 }
00133                 throw new RepositoryException("MySQL ID üretilemedi");
00134             } catch (SQLException e) {
00135                 throw new RepositoryException("MySQL save hatası", e);
00136             }
00137         }

```

References [connection](#), [getInsertSql\(\)](#), [setEntityId\(\)](#), and [setInsertParams\(\)](#).

7.22.3.15 setEntityId()

```

void com.mehdi.petreminder.repository.MySqlRepository< T >.setEntityId (
    T entity,
    int id) [protected]

```

Reflection ile entity'ye id set eder.

Parameters

<i>entity</i>	Varlık
<i>id</i>	ID

Definition at line 242 of file [MySqlRepository.java](#).

```

00242         {
00243             try {
00244                 var method = entity.getClass().getMethod("setId", int.class);
00245                 method.invoke(entity, id);
00246             } catch (Exception e) {
00247                 logger.warn("setId çağrılmadı: {}", e.getMessage());
00248             }
00249         }

```

References [logger](#).

Referenced by [save\(\)](#).

7.22.3.16 setInsertParams()

```
abstract void com.mehdi.petreminder.repository.MySqlRepository< T >.setInsertParams (
    PreparedStatement ps,
    T entity) throws SQLException [abstract], [protected]
```

INSERT parametrelerini set eder.

Parameters

<i>ps</i>	PreparedStatement
<i>entity</i>	Varlık

Exceptions

<i>SQLException</i>	SQL hatası
---------------------	------------

Referenced by [save\(\)](#).

7.22.3.17 setUpdateParams()

```
abstract void com.mehdi.petreminder.repository.MySqlRepository< T >.setUpdateParams (
    PreparedStatement ps,
    T entity) throws SQLException [abstract], [protected]
```

UPDATE parametrelerini set eder.

Parameters

<i>ps</i>	PreparedStatement
<i>entity</i>	Varlık

Exceptions

<i>SQLException</i>	SQL hatası
---------------------	------------

Referenced by [update\(\)](#).

7.22.3.18 update()

```
boolean com.mehdi.petreminder.repository.MySqlRepository< T >.update (
    T entity)
```

Mevcut varlığı günceller.

Parameters

<i>entity</i>	Güncellenecek varlık
---------------	----------------------

Returns

Güncelleme başarılıysa true

Exceptions

RepositoryException	Güncelleme başarısız olursa
-------------------------------------	-----------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 169 of file [MySqlRepository.java](#).

```

00169         {
00170             if (entity == null) return false;
00171             try (PreparedStatement ps = connection.prepareStatement(getUpdateSql())) {
00172                 setUpdateParams(ps, entity);
00173                 return ps.executeUpdate() > 0;
00174             } catch (SQLException e) {
00175                 throw new RepositoryException("MySQL update hatası", e);
00176             }
00177         }

```

References [connection](#), [getUpdateSql\(\)](#), and [setUpdateParams\(\)](#).

7.22.4 Member Data Documentation

7.22.4.1 connection

Connection [com.mehdi.petreminder.repository.MySqlRepository< T >.connection](#) [protected]

MySQL JDBC bağlantısı.

Definition at line 34 of file [MySqlRepository.java](#).

Referenced by [close\(\)](#), [count\(\)](#), [createTableIfNotExists\(\)](#), [delete\(\)](#), [deleteAll\(\)](#), [findAll\(\)](#), [findById\(\)](#), [getConnection\(\)](#), [MySqlRepository\(\)](#), [save\(\)](#), and [update\(\)](#).

7.22.4.2 logger

final Logger [com.mehdi.petreminder.repository.MySqlRepository< T >.logger](#) [static], [protected]

Initial value:

```

=
    (Logger) LoggerFactory.getLogger(MySqlRepository.class)

```

Logger.

Definition at line 30 of file [MySqlRepository.java](#).

Referenced by [close\(\)](#), [MySqlRepository\(\)](#), and [setEntityId\(\)](#).

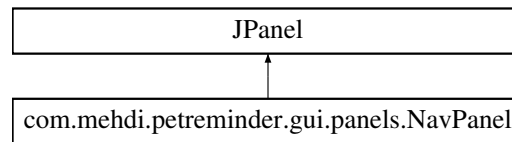
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/MySqlRepository.java](#)

7.23 com.mehdi.petreminder.gui.panels.NavPanel Class Reference

Sol navigasyon çubuğu — menü butonları.

Inheritance diagram for com.mehdi.petreminder.gui.panels.NavPanel:



Public Member Functions

- [NavPanel](#) ([MainFrame](#) mainFrame)
NavPanel yapıcısı.
- void [setActiveButton](#) (String cardName)
Aktif butonu vurgular.

7.23.1 Detailed Description

Sol navigasyon çubuğu — menü butonları.

Her buton ilgili karta navigasyon yapar. Aktif buton vurgulanır.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 27 of file [NavPanel.java](#).

7.23.2 Constructor & Destructor Documentation

7.23.2.1 NavPanel()

```
com.mehdi.petreminder.gui.panels.NavPanel.NavPanel (
    JFrame mainFrame)
```

[NavPanel](#) yapıcısı.

Parameters

<i>mainFrame</i>	Ana pencere
------------------	-------------

Definition at line 45 of file [NavPanel.java](#).

```
00045         {
00046             this.mainFrame = mainFrame;
00047             setBackground(GuiConstants.NAV_COLOR);
00048             setPreferredSize(new Dimension(GuiConstants.NAV_WIDTH, 0));
00049             setLayout(new BorderLayout(this, BorderLayout.Y_AXIS));
00050             buildNav();
00051         }
```

References [com.mehdi.petreminder.gui.util.GuiConstants.NAV_COLOR](#), and [com.mehdi.petreminder.gui.util.GuiConstants.NAV_WIDTH](#).

7.23.3 Member Function Documentation

7.23.3.1 setActiveButton()

```
void com.mehdi.petreminder.gui.panels.NavPanel.setActiveButton (
    String cardName)
```

Aktif butonu vurgular.

Parameters

<i>cardName</i>	Aktif kart adı
-----------------	----------------

Definition at line 133 of file [NavPanel.java](#).

```
00133         {
00134             if (activeCard != null && buttons.containsKey(activeCard)) {
00135                 JButton prev = buttons.get(activeCard);
00136                 prev.setBackground(GuiConstants.NAV_COLOR);
00137                 prev.setForeground(new Color(200, 210, 230));
00138             }
00139             activeCard = cardName;
00140             if (buttons.containsKey(cardName)) {
00141                 JButton active = buttons.get(cardName);
00142                 active.setBackground(GuiConstants.NAV_ACTIVE);
00143                 active.setForeground(Color.WHITE);
00144             }
00145         }
```

References [com.mehdi.petreminder.gui.util.GuiConstants.NAV_ACTIVE](#), and [com.mehdi.petreminder.gui.util.GuiConstants.NAV_COLOR](#).

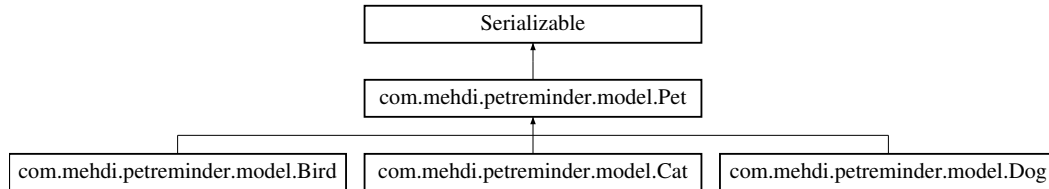
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/NavPanel.java](#)

7.24 com.mehdi.petreminder.model.Pet Class Reference

Evcil hayvanın temel soyut sinifi.

Inheritance diagram for com.mehdi.petreminder.model.Pet:



Public Member Functions

- int `getId` ()
ID getter.
- void `setId` (int id)
ID setter.
- String `getName` ()
Isim getter.
- void `setName` (String name)
Isim setter.
- String `getSpecies` ()
Tur getter.
- void `setSpecies` (String species)
Tur setter.
- LocalDate `getBirthDate` ()
Dogum tarihi getter.
- void `setBirthDate` (LocalDate birthDate)
Dogum tarihi setter.
- int `getOwnerId` ()
Sahip ID getter.
- void `setOwnerId` (int ownerId)
Sahip ID setter.
- String `getGender` ()
Cinsiyet getter.
- void `setGender` (String gender)
Cinsiyet setter.
- double `getWeight` ()
Agirlik getter.
- void `setWeight` (double weight)
Agirlik setter.
- String `getNotes` ()
Notlar getter.
- void `setNotes` (String notes)
Notlar setter.
- abstract String `makeSound` ()
Hayvana ozgu ses dondurur.
- abstract String `getIconName` ()

- `int getAge ()`
Yas hesaplar.
- `String getAgeString ()`
Yas bilgisini formatlı string olarak dondurur.
- `String toString ()`
Nesneyi string olarak temsil eder.
- `boolean equals (Object obj)`
İki `Pet` nesnesinin eşitliğini karşılaştırır.
- `int hashCode ()`
Hash kodu hesaplar.

Protected Member Functions

- `Pet ()`
Varsayılan yapıcı metod.
- `Pet (int id, String name, String species, LocalDate birthDate, int ownerId, String gender, double weight, String notes)`
Tam parametrelili yapıcı metod.

7.24.1 Detailed Description

Evcil hayvanın temel soyut sınıfı.

OOP Abstraction: abstract class ile somutlaştırılmaz. OOP Encapsulation: tüm alanlar private, getter/setter ile erişim. Serializable: BinaryRepository için zorunlu.

Author

Muhammed Mehdi Karagülle, Ibrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 22 of file [Pet.java](#).

7.24.2 Constructor & Destructor Documentation

7.24.2.1 `Pet()` [1/2]

```
com.mehdi.petreminder.model.Pet.Pet () [protected]
```

Varsayılan yapıcı metod.

Serialization için gereklidir.

Definition at line 55 of file [Pet.java](#).

```
00055 {}
```

Referenced by [equals\(\)](#).

7.24.2.2 Pet() [2/2]

```
com.mehdi.petreminder.model.Pet.Pet (
    int id,
    String name,
    String species,
    LocalDate birthDate,
    int ownerId,
    String gender,
    double weight,
    String notes) [protected]
```

Tam parametrelili yapıcı metod.

Parameters

<i>id</i>	Benzersiz kimlik numarası
<i>name</i>	Evciil hayvanın ismi
<i>species</i>	Hayvan türü
<i>birthDate</i>	Dogum tarihi
<i>ownerId</i>	Sahip kullanıcı ID'si
<i>gender</i>	Cinsiyet (Erkek/Disi)
<i>weight</i>	Ağırlık (kg)
<i>notes</i>	Ek notlar

Definition at line 68 of file [Pet.java](#).

```
00069                                     {
00070         this.id = id;
00071         this.name = name;
00072         this.species = species;
00073         this.birthDate = birthDate;
00074         this.ownerId = ownerId;
00075         this.gender = gender;
00076         this.weight = weight;
00077         this.notes = notes;
00078     }
```

7.24.3 Member Function Documentation

7.24.3.1 equals()

```
boolean com.mehdi.petreminder.model.Pet.equals (
    Object obj)
```

İki [Pet](#) nesnesinin eşitliğini karşılaştırır.

Parameters

<i>obj</i>	Karşılaştırılacak nesne
------------	-------------------------

Returns

ID'ler esitse true

Definition at line 181 of file [Pet.java](#).

```
00181         {
00182             if (this == obj) return true;
00183             if (obj == null || !(obj instanceof Pet)) return false;
00184             Pet other = (Pet) obj;
00185             return this.id == other.id;
00186         }
```

References [Pet\(\)](#).

7.24.3.2 getAge()

```
int com.mehdi.petreminder.model.Pet.getAge ()
```

Yas hesaplar.

Bugun ile dogum tarihi arasindaki farki yil cinsinden dondurur.

Returns

Yas (yil), dogum tarihi null ise -1

Definition at line 150 of file [Pet.java](#).

```
00150         {
00151             if (birthDate == null) return -1;
00152             return Period.between(birthDate, LocalDate.now()).getYears();
00153         }
```

7.24.3.3 getAgeString()

```
String com.mehdi.petreminder.model.Pet.getAgeString ()
```

Yas bilgisini formatlı string olarak dondurur.

Returns

"2 yil 3 ay" formatında yas

Definition at line 159 of file [Pet.java](#).

```
00159         {
00160             if (birthDate == null) return "Bilinmiyor";
00161             Period p = Period.between(birthDate, LocalDate.now());
00162             return p.getYears() + " yil " + p.getMonths() + " ay";
00163         }
```

Referenced by [com.mehdi.petreminder.model.Bird.toString\(\)](#), [com.mehdi.petreminder.model.Cat.toString\(\)](#), [com.mehdi.petreminder.model.Dog.toString\(\)](#), and [toString\(\)](#).

7.24.3.4 getBirthDate()

```
LocalDate com.mehdi.petreminder.model.Pet.getBirthDate ()
```

Dogum tarihi getter.

Returns

Dogum tarihi

Definition at line 101 of file [Pet.java](#).

```
00101 { return birthDate; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.setInsertParams\(\)](#).

7.24.3.5 getGender()

```
String com.mehdi.petreminder.model.Pet.getGender ()
```

Cinsiyet getter.

Returns

Cinsiyet

Definition at line 113 of file [Pet.java](#).

```
00113 { return gender; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.setInsertParams\(\)](#).

7.24.3.6 getIconName()

```
abstract String com.mehdi.petreminder.model.Pet.getIconName () [abstract]
```

Hayvana ozgu ikon adi dondurur (GUI icin).

Returns

Ikon adi (ornk. "dog.png")

Reimplemented in [com.mehdi.petreminder.model.Bird](#), [com.mehdi.petreminder.model.Cat](#), and [com.mehdi.petreminder.model.Dog](#).

7.24.3.7 getId()

```
int com.mehdi.petreminder.model.Pet.getId ()
```

ID getter.

Returns

Evcil hayvan ID'si

Definition at line 83 of file [Pet.java](#).

```
00083 { return id; }
```

Referenced by [com.mehdi.petreminder.model.Bird.toString\(\)](#), [com.mehdi.petreminder.model.Cat.toString\(\)](#), [com.mehdi.petreminder.model.Dog.toString\(\)](#), and [com.mehdi.petreminder.service.PetService.updatePet\(\)](#).

7.24.3.8 getName()

```
String com.mehdi.petreminder.model.Pet.getName ()
```

Isim getter.

Returns

Evcil hayvan ismi

Definition at line 89 of file [Pet.java](#).

```
00089 { return name; }
```

Referenced by [com.mehdi.petreminder.service.PetService.addPet\(\)](#), [com.mehdi.petreminder.model.Bird.toString\(\)](#), [com.mehdi.petreminder.model.Cat.toString\(\)](#), and [com.mehdi.petreminder.model.Dog.toString\(\)](#).

7.24.3.9 getNotes()

```
String com.mehdi.petreminder.model.Pet.getNotes ()
```

Notlar getter.

Returns

Notlar

Definition at line 125 of file [Pet.java](#).

```
00125 { return notes; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.setInsertParams\(\)](#).

7.24.3.10 getOwnerId()

```
int com.mehdi.petreminder.model.Pet.getOwnerId ()
```

Sahip ID getter.

Returns

Sahip kullanici ID'si

Definition at line 107 of file [Pet.java](#).

```
00107 { return ownerId; }
```

7.24.3.11 getSpecies()

```
String com.mehdi.petreminder.model.Pet.getSpecies ()
```

Tur getter.

Returns

Hayvan turu

Definition at line 95 of file [Pet.java](#).

```
00095 { return species; }
```

7.24.3.12 `getWeight()`

```
double com.mehdi.petreminder.model.Pet.getWeight ()
```

Agirlik getter.

Returns

Agirlik (kg)

Definition at line 119 of file [Pet.java](#).

```
00119 { return weight; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.setInsertParams\(\)](#).

7.24.3.13 `hashCode()`

```
int com.mehdi.petreminder.model.Pet.hashCode ()
```

Hash kodu hesaplar.

Returns

ID'ye gore hash kodu

Definition at line 193 of file [Pet.java](#).

```
00193         {  
00194             return Integer.hashCode(id);  
00195         }
```

7.24.3.14 `makeSound()`

```
abstract String com.mehdi.petreminder.model.Pet.makeSound () [abstract]
```

Hayvana ozgu ses dondurur.

OOP Polymorphism: her alt sinif farkli implementasyon saglar.

Returns

Hayvan sesi string'i

Reimplemented in [com.mehdi.petreminder.model.Bird](#), [com.mehdi.petreminder.model.Cat](#), and [com.mehdi.petreminder.model.Dog](#).

7.24.3.15 setBirthDate()

```
void com.mehdi.petreminder.model.Pet.setBirthDate (
    LocalDate birthDate)
```

Dogum tarihi setter.

Parameters

<i>birthDate</i>	Dogum tarihi
------------------	--------------

Definition at line 104 of file [Pet.java](#).

```
00104 { this.birthDate = birthDate; }
```

7.24.3.16 setGender()

```
void com.mehdi.petreminder.model.Pet.setGender (
    String gender)
```

Cinsiyet setter.

Parameters

<i>gender</i>	Cinsiyet
---------------	----------

Definition at line 116 of file [Pet.java](#).

```
00116 { this.gender = gender; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRe](#)

7.24.3.17 setId()

```
void com.mehdi.petreminder.model.Pet.setId (
    int id)
```

ID setter.

Parameters

<i>id</i>	Yeni ID degeri
-----------	----------------

Definition at line 86 of file [Pet.java](#).

```
00086 { this.id = id; }
```

7.24.3.18 setName()

```
void com.mehdi.petreminder.model.Pet.setName (
    String name)
```

Isim setter.

Parameters

<i>name</i>	Yeni isim
-------------	-----------

Definition at line 92 of file [Pet.java](#).

```
00092 { this.name = name; }
```

7.24.3.19 setNotes()

```
void com.mehdi.petreminder.model.Pet.setNotes (
    String notes)
```

Notlar setter.

Parameters

<i>notes</i>	Notlar
--------------	--------

Definition at line 128 of file [Pet.java](#).

```
00128 { this.notes = notes; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRe](#)

7.24.3.20 setOwnerId()

```
void com.mehdi.petreminder.model.Pet.setOwnerId (
    int ownerId)
```

Sahip ID setter.

Parameters

<i>owner↔ Id</i>	Sahip kullanici ID'si
----------------------	-----------------------

Definition at line 110 of file [Pet.java](#).

```
00110 { this.ownerId = ownerId; }
```

7.24.3.21 setSpecies()

```
void com.mehdi.petreminder.model.Pet.setSpecies (  
    String species)
```

Tur setter.

Parameters

<i>species</i>	Hayvan turu
----------------	-------------

Definition at line 98 of file [Pet.java](#).

```
00098 { this.species = species; }
```

7.24.3.22 setWeight()

```
void com.mehdi.petreminder.model.Pet.setWeight (  
    double weight)
```

Agirlik setter.

Parameters

<i>weight</i>	Agirlik (kg)
---------------	--------------

Definition at line 122 of file [Pet.java](#).

```
00122 { this.weight = weight; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRepository.mapRow\(\)](#).

7.24.3.23 toString()

```
String com.mehdi.petreminder.model.Pet.toString ()
```

Nesneyi string olarak temsil eder.

Returns

Formatli pet bilgisi

Reimplemented in [com.mehdi.petreminder.model.Bird](#), [com.mehdi.petreminder.model.Cat](#), and [com.mehdi.petreminder.model.Dog](#).

Definition at line 170 of file [Pet.java](#).

```
00170 {  
00171     return String.format("Pet{id=%d, name='%s', species='%s', age=%s}",  
00172         id, name, species, getAgeString());  
00173 }
```

References [getAgeString\(\)](#).

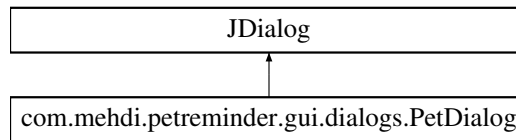
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/Pet.java](#)

7.25 com.mehdi.petreminder.gui.dialogs.PetDialog Class Reference

[Pet](#) ekleme ve düzenleme modal dialog'u.

Inheritance diagram for com.mehdi.petreminder.gui.dialogs.PetDialog:



Public Member Functions

- [PetDialog](#) (Frame parent, [Pet](#) editPet)
[PetDialog](#) yapıcısı.
- boolean [isConfirmed](#) ()
Onaylandı mı kontrolü.
- [Pet](#) [getPet](#) ()
Oluşturulan / düzenlenen pet'i döndürür.

7.25.1 Detailed Description

[Pet](#) ekleme ve düzenleme modal dialog'u.

Yeni pet için null geçilir; düzenleme için mevcut [Pet](#) nesnesi geçilir.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 25 of file [PetDialog.java](#).

7.25.2 Constructor & Destructor Documentation

7.25.2.1 PetDialog()

```
com.mehdi.petreminder.gui.dialogs.PetDialog.PetDialog (
    Frame parent,
    Pet editPet)
```

[PetDialog](#) yapıcısı.

Parameters

<i>parent</i>	Ana pencere
<i>editPet</i>	Düzenlenecek pet (yeni için null)

Definition at line 55 of file [PetDialog.java](#).

```
00055     {
00056         super(parent, editPet == null ? "Yeni Hayvan Ekle" : "Hayvanı Düzenle", true);
00057         this.editPet = editPet;
00058         buildUI();
00059         if (editPet != null) fillForm(editPet);
00060         pack();
00061         setLocationRelativeTo(parent);
00062         setMinimumSize(new Dimension(420, 500));
00063     }
```

7.25.3 Member Function Documentation

7.25.3.1 getPet()

```
Pet com.mehdi.petreminder.gui.dialogs.PetDialog.getPet ()
```

Oluşturulan / düzenlenen pet'i döndürür.

Returns

[Pet](#) nesnesi

Definition at line 285 of file [PetDialog.java](#).

```
00285 { return resultPet; }
```

7.25.3.2 isConfirmed()

```
boolean com.mehdi.petreminder.gui.dialogs.PetDialog.isConfirmed ()
```

Onaylandı mı kontrolü.

Returns

true ise kullanıcı "Kaydet" tıkladı

Definition at line 279 of file [PetDialog.java](#).

```
00279 { return confirmed; }
```

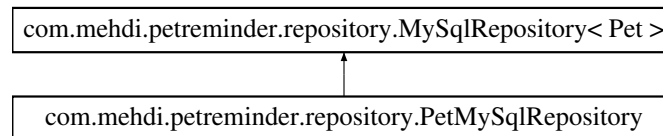
The documentation for this class was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/[PetDialog.java](#)

7.26 com.mehdi.petreminder.repository.PetMySqlRepository Class Reference

MySQL üzerinde [Pet](#) CRUD işlemleri.

Inheritance diagram for com.mehdi.petreminder.repository.PetMySqlRepository:



Public Member Functions

- [PetMySqlRepository](#) ()
Varsayılan yapıcı.
- [PetMySqlRepository](#) (Connection [connection](#))
Test yapıcısı.
- List< [Pet](#) > [findByOwnerId](#) (int ownerId)
OwnerId'ye göre petleri listeler.

Public Member Functions inherited from [com.mehdi.petreminder.repository.MySqlRepository< Pet >](#)

- [MySqlRepository](#) ()
MySQL bağlantısını açar.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür.

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [Pet](#) pet) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [Pet](#) pet) throws SQLException
- [Pet](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from [com.mehdi.petreminder.repository.MySqlRepository](#) < [Pet](#) >

- abstract String [getTableName](#) ()
Tablo adı.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL.
- abstract String [getInsertSql](#) ()
INSERT SQL.
- abstract String [getUpdateSql](#) ()
UPDATE SQL.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members

Protected Attributes inherited from [com.mehdi.petreminder.repository.MySqlRepository](#) < [Pet](#) >

- Connection [connection](#)
MySQL JDBC bağlantısı.

Static Protected Attributes inherited from [com.mehdi.petreminder.repository.MySqlRepository](#) < [Pet](#) >

- static final Logger [logger](#)
Logger.

7.26.1 Detailed Description

MySQL üzerinde [Pet](#) CRUD işlemleri.

[MySqlRepository](#)'yi extend eder; [PetSqliteRepository](#) ile aynı mantık.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 25 of file [PetMySqlRepository.java](#).

7.26.2 Constructor & Destructor Documentation

7.26.2.1 PetMySqlRepository() [1/2]

`com.mehdi.petreminder.repository.PetMySqlRepository.PetMySqlRepository ()`

Varsayılan yapıcı.

Definition at line 30 of file [PetMySqlRepository.java](#).

```
00030                                     {
00031         super();
00032     }
```

7.26.2.2 PetMySqlRepository() [2/2]

`com.mehdi.petreminder.repository.PetMySqlRepository.PetMySqlRepository (Connection connection)`

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 38 of file [PetMySqlRepository.java](#).

```
00038                                     {
00039         super(connection);
00040     }
```

References [com.mehdi.petreminder.repository.MySqlRepository< Pet >.connection](#).

7.26.3 Member Function Documentation

7.26.3.1 findByOwnerId()

`List< Pet > com.mehdi.petreminder.repository.PetMySqlRepository.findByOwnerId (int ownerId)`

OwnerId'ye göre petleri listeler.

Parameters

<i>ownerId</i>	Kullanıcı ID
----------------	--------------

Returns

Kullanıcıya ait pet listesi

Definition at line 172 of file [PetMySqlRepository.java](#).

```
00172                                     {
00173         List<Pet> list = new ArrayList<>();
00174         String sql = "SELECT * FROM pets WHERE owner_id = ?";
00175         try (PreparedStatement ps = connection.prepareStatement(sql)) {
00176             ps.setInt(1, ownerId);
00177             ResultSet rs = ps.executeQuery();
00178             while (rs.next()) list.add(mapRow(rs));
00179         } catch (SQLException e) {
00180             throw new RepositoryException("findByOwnerId hatası", e);
00181         }
00182         return list;
00183     }
```

References [com.mehdi.petreminder.repository.MySqlRepository< Pet >.connection](#), and [mapRow\(\)](#).

7.26.3.2 getCreateTableSql()

String com.mehdi.petreminder.repository.PetMySqlRepository.getCreateTableSql () [protected]

Member documentation.

Definition at line 56 of file [PetMySqlRepository.java](#).

```

00056     {
00057         return "CREATE TABLE IF NOT EXISTS pets ("
00058             + "id INT AUTO_INCREMENT PRIMARY KEY,"
00059             + "owner_id INT NOT NULL DEFAULT 1,"
00060             + "name VARCHAR(50) NOT NULL,"
00061             + "species VARCHAR(30) NOT NULL,"
00062             + "breed VARCHAR(50),"
00063             + "birth_date DATE,"
00064             + "gender VARCHAR(10),"
00065             + "weight DOUBLE DEFAULT 0,"
00066             + "notes TEXT,"
00067             + "can_talk BOOLEAN DEFAULT FALSE,"
00068             + "bird_type VARCHAR(50),"
00069             + "is_trained BOOLEAN DEFAULT FALSE,"
00070             + "is_indoor BOOLEAN DEFAULT TRUE"
00071             + ")";
00072     }

```

7.26.3.3 getInsertSql()

String com.mehdi.petreminder.repository.PetMySqlRepository.getInsertSql () [protected]

Member documentation.

Definition at line 79 of file [PetMySqlRepository.java](#).

```

00079     {
00080         return "INSERT INTO pets (owner_id,name,species,breed,birth_date,gender,"
00081             + "weight,notes,can_talk,bird_type,is_trained,is_indoor) "
00082             + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00083     }

```

7.26.3.4 getTableName()

String com.mehdi.petreminder.repository.PetMySqlRepository.getTableName () [protected]

Member documentation.

Definition at line 47 of file [PetMySqlRepository.java](#).

```

00047     {
00048         return "pets";
00049     }

```

7.26.3.5 getUpdateSql()

String com.mehdi.petreminder.repository.PetMySqlRepository.getUpdateSql () [protected]

Member documentation.

Definition at line 90 of file [PetMySqlRepository.java](#).

```

00090     {
00091         return "UPDATE pets SET owner_id=?,name=?,species=?,breed=?,birth_date=,"
00092             + "gender=?,weight=?,notes=?,can_talk=?,bird_type=?,is_trained=?,is_indoor=? "
00093             + "WHERE id=?";
00094     }

```

7.26.3.6 mapRow()

`Pet` `com.mehdi.petreminder.repository.PetMySQLRepository.mapRow (`
`ResultSet rs)` throws `SQLException` [protected]

Member documentation.

Definition at line 135 of file `PetMySQLRepository.java`.

```

00135                                     {
00136         String species = rs.getString("species");
00137         int id = rs.getInt("id");
00138         String name = rs.getString("name");
00139         Date sqlDate = rs.getDate("birth_date");
00140         LocalDate birthDate = sqlDate != null ? sqlDate.toLocalDate() : null;
00141         int ownerId = rs.getInt("owner_id");
00142
00143         Pet pet;
00144         if ("Dog".equalsIgnoreCase(species)) {
00145             Dog dog = new Dog(id, name, birthDate, ownerId);
00146             dog.setBreed(rs.getString("breed"));
00147             dog.setTrained(rs.getBoolean("is_trained"));
00148             pet = dog;
00149         } else if ("Cat".equalsIgnoreCase(species)) {
00150             Cat cat = new Cat(id, name, birthDate, ownerId);
00151             cat.setBreed(rs.getString("breed"));
00152             cat.setIndoor(rs.getBoolean("is_indoor"));
00153             pet = cat;
00154         } else {
00155             Bird bird = new Bird(id, name, birthDate, ownerId);
00156             bird.setBirdType(rs.getString("bird_type"));
00157             bird.setCanTalk(rs.getBoolean("can_talk"));
00158             pet = bird;
00159         }
00160         pet.setGender(rs.getString("gender"));
00161         pet.setWeight(rs.getDouble("weight"));
00162         pet.setNotes(rs.getString("notes"));
00163         return pet;
00164     }

```

References `com.mehdi.petreminder.model.Bird.setBirdType()`, `com.mehdi.petreminder.model.Cat.setBreed()`, `com.mehdi.petreminder.model.Dog.setBreed()`, `com.mehdi.petreminder.model.Bird.setCanTalk()`, `com.mehdi.petreminder.model.Pet.setNotes()`, `com.mehdi.petreminder.model.Cat.setIndoor()`, `com.mehdi.petreminder.model.Pet.setNotes()`, `com.mehdi.petreminder.model.Dog.setTrained()` and `com.mehdi.petreminder.model.Pet.setWeight()`.

Referenced by `findByOwnerId()`.

7.26.3.7 setInsertParams()

`void` `com.mehdi.petreminder.repository.PetMySQLRepository.setInsertParams (`
`PreparedStatement ps,`
`Pet pet)` throws `SQLException` [protected]

Member documentation.

Definition at line 102 of file `PetMySQLRepository.java`.

```

00102                                     {
00103         ps.setInt(1, pet.getOwnerId());
00104         ps.setString(2, pet.getName());
00105         ps.setString(3, pet.getSpecies());
00106         ps.setString(4, pet instanceof Dog ? ((Dog) pet).getBreed()
00107             : pet instanceof Cat ? ((Cat) pet).getBreed() : null);
00108         ps.setObject(5, pet.getBirthDate() != null
00109             ? java.sql.Date.valueOf(pet.getBirthDate()) : null);
00110         ps.setString(6, pet.getGender());
00111         ps.setDouble(7, pet.getWeight());
00112         ps.setString(8, pet.getNotes());
00113         ps.setBoolean(9, pet instanceof Bird && ((Bird) pet).isCanTalk());
00114         ps.setString(10, pet instanceof Bird ? ((Bird) pet).getBirdType() : null);
00115         ps.setBoolean(11, pet instanceof Dog && ((Dog) pet).isTrained());
00116         ps.setBoolean(12, !(pet instanceof Cat) || ((Cat) pet).isIndoor());
00117     }

```

References `com.mehdi.petreminder.model.Pet.getBirthDate()`, `com.mehdi.petreminder.model.Pet.getGender()`, `com.mehdi.petreminder.model.Pet.getNotes()`, and `com.mehdi.petreminder.model.Pet.getWeight()`.

Referenced by `setUpdateParams()`.

7.26.3.8 setUpdateParams()

```
void com.mehdi.petreminder.repository.PetMySqlRepository.setUpdateParams (
    PreparedStatement ps,
    Pet pet) throws SQLException [protected]
```

Member documentation.

Definition at line 124 of file [PetMySqlRepository.java](#).

```
00124
00125         setInsertParams(ps, pet);
00126         ps.setInt(13, pet.getId());
00127     }
```

References [setInsertParams\(\)](#).

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/PetMySqlRepository.java](#)

7.27 com.mehdi.petreminder.petreminder Class Reference

Pet Care Reminder System yardımcı/facade sınıfı.

Public Member Functions

- [petreminder \(\)](#)
Varsayılan yapıcı.
- String [getCurrentTimeFormatted \(\)](#)
Mevcut tarih-saati biçimli string olarak döndürür.
- boolean [isNullOrEmpty \(String value\)](#)
Verilen string'in boş veya null olup olmadığını kontrol eder.

7.27.1 Detailed Description

Pet Care Reminder System yardımcı/facade sınıfı.

PDF gereksinimi: Separate library (lib) and application (app) layers. Bu sınıf lib katmanının facade'ı olarak görev yapar. OOP Encapsulation: tüm alanlar private.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Date

2026-03-27

Definition at line 29 of file [petreminder.java](#).

7.27.2 Constructor & Destructor Documentation

7.27.2.1 petreminder()

```
com.mehdi.petreminder.petreminder.petreminder ()
```

Varsayılan yapıcı.

Definition at line 46 of file [petreminder.java](#).

```
00046         {
00047             logger.debug("petreminder yardimci sinifi olusturuldu.");
00048         }
```

7.27.3 Member Function Documentation

7.27.3.1 getCurrentTimeFormatted()

```
String com.mehdi.petreminder.petreminder.getCurrentTimeFormatted ()
```

Mevcut tarih-saati biçimli string olarak döndürür.

Returns

"dd/MM/yyyy HH:mm" formatında şimdiki zaman

Definition at line 54 of file [petreminder.java](#).

```
00054         {
00055             return LocalDateTime.now().format(DISPLAY_FORMAT);
00056         }
```

7.27.3.2 isNullOrEmpty()

```
boolean com.mehdi.petreminder.petreminder.isNullOrEmpty (
    String value)
```

Verilen string'in boş veya null olup olmadığını kontrol eder.

Parameters

<i>value</i>	Kontrol edilecek string
--------------	-------------------------

Returns

null veya boş ise true

Definition at line 63 of file [petreminder.java](#).

```
00063         {
00064             return value == null || value.trim().isEmpty();
00065         }
```

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/petreminder.java](#)

7.28 com.mehdi.petreminder.petreminderApp Class Reference

Pet Care Reminder System ana sınıfı.

Static Public Member Functions

- static void [main](#) (String[] args)
Ana giriş noktası.
- static void [startGui](#) ()
- static [StorageType](#) [parseStorageArg](#) (String[] args)
–storage argümanını ayrıştırır.
- static String [getVersion](#) ()
Uygulama versiyonunu döndürür.
- static String [getAppName](#) ()
Uygulama adını döndürür.

Static Public Attributes

- static final String [APP_NAME](#) = "Pet Care Reminder System"
Uygulama adı.
- static final String [APP_VERSION](#) = "1.0.0"
Versiyon.

7.28.1 Detailed Description

Pet Care Reminder System ana sınıfı.

PDF zorunluluğu: Modular Architecture. Uygulama doğrudan Swing GUI olarak başlatılır.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Date

2026-03-27

Definition at line 29 of file [petreminderApp.java](#).

7.28.2 Member Function Documentation

7.28.2.1 getAppName()

`String com.mehdi.petreminder.petreminderApp.getAppName () [static]`

Uygulama adını döndürür.

Returns

Uygulama adı

Definition at line 116 of file [petreminderApp.java](#).

```
00116 { return APP_NAME; }
```

References [APP_NAME](#).

7.28.2.2 getVersion()

`String com.mehdi.petreminder.petreminderApp.getVersion () [static]`

Uygulama versiyonunu döndürür.

Returns

Versiyon string'i

Definition at line 110 of file [petreminderApp.java](#).

```
00110 { return APP_VERSION; }
```

References [APP_VERSION](#).

7.28.2.3 main()

`void com.mehdi.petreminder.petreminderApp.main (String[] args) [static]`

Ana giriş noktası.

Parameters

<i>args</i>	Komut satırı argümanları. <code>--storage=binary</code> → Binary backend <code>--storage=sqlite</code> → SQLite backend <code>--storage=mysql</code> → MySQL backend
-------------	--

Definition at line 49 of file [petreminderApp.java](#).

```
00049                                     {
00050     logger.info("{} v{} baslatiliyor...", APP_NAME, APP_VERSION);
00051
00052     // Storage backend
00053     StorageType storageType = parseStorageArg(args);
00054     StorageConfig.setActiveBackend(storageType);
00055     logger.info("Storage backend: {}", storageType);
00056
00057     startGui();
00058
00059     logger.info("Uygulama GUI olarak baslatildi.");
00060 }
```

References [APP_NAME](#), [APP_VERSION](#), [parseStorageArg\(\)](#), and [com.mehdi.petreminder.config.StorageConfig.setActiveBackend\(\)](#).

7.28.2.4 parseStorageArg()

`StorageType` com.mehdi.petreminder.petreminderApp.parseStorageArg (
String[] args) [static]

–storage argümanını ayrıştırır.

Parameters

<code>args</code>	Argümanlar
-------------------	------------

Returns

`StorageType` (varsayılan BINARY)

Definition at line 89 of file `petreminderApp.java`.

```
00089                                     {
00090         if (args == null || args.length == 0) return StorageType.BINARY;
00091         for (String arg : args) {
00092             if (arg != null && arg.startsWith("--storage=")) {
00093                 String value = arg.substring("--storage=".length())
00094                     .trim().toUpperCase(java.util.Locale.ROOT);
00095                 try {
00096                     return StorageType.valueOf(value);
00097                 } catch (IllegalArgumentException e) {
00098                     logger.warn("Gecersiz storage: {}. BINARY kullaniliyor.", value);
00099                     return StorageType.BINARY;
00100                 }
00101             }
00102         }
00103         return StorageType.BINARY;
00104     }
```

References `com.mehdi.petreminder.config.StorageType.BINARY`.

Referenced by `main()`.

7.28.2.5 startGui()

`void` com.mehdi.petreminder.petreminderApp.startGui () [static]

Definition at line 79 of file `petreminderApp.java`.

```
00079                                     {
00080         javax.swing.SwingUtilities.invokeLater(petreminderApp::initGui);
00081     }
```

7.28.3 Member Data Documentation

7.28.3.1 APP_NAME

`final String` com.mehdi.petreminder.petreminderApp.APP_NAME = "Pet Care Reminder System" [static]

Uygulama adı.

Definition at line 36 of file `petreminderApp.java`.

Referenced by `getAppName()`, and `main()`.

7.28.3.2 APP_VERSION

```
final String com.mehdi.petreminder.petreminderApp.APP_VERSION = "1.0.0" [static]
```

Versiyon.

Definition at line 39 of file [petreminderApp.java](#).

Referenced by [getVersion\(\)](#), and [main\(\)](#).

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/petreminderApp.java](#)

7.29 com.mehdi.petreminder.observer.PetReminderObserver Interface Reference

Gozlemci arayuzu — durum degisikliklerini dinler.

Public Member Functions

- void [onEvent](#) ([EventType](#) event, Object data)
Olay gerceklestiginde cagrilan metod.

7.29.1 Detailed Description

Gozlemci arayuzu — durum degisikliklerini dinler.

Observer Design Pattern: Bu interface'i implement eden siniflar, Subject'teki degisikliklerden haberdar olur. Ornek: Storage degistiginde servisler guncellenir, Reminder eklendiginde Dashboard yenilenir.

Author

Muhammed Mehdi Karagulle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 22 of file [PetReminderObserver.java](#).

7.29.2 Member Function Documentation

7.29.2.1 onEvent()

```
void com.mehdi.petreminder.observer.PetReminderObserver.onEvent (
    EventType event,
    Object data)
```

Olay gerceklestiginde cagrilan metod.

Subject durumu degistiginde tum kayitli observer'lara bu metod uzerinden bildirim gonderilir.

Parameters

<i>event</i>	Olay turu (ornegin STORAGE_CHANGED, REMINDER_ADDED)
<i>data</i>	Olayla ilgili veri (ornegin yeni StorageType , eklenen Reminder)

The documentation for this interface was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/observer/PetReminderObserver.java](#)

7.30 com.mehdi.petreminder.service.PetService Class Reference

[Pet](#) CRUD iş mantığı katmanı.

Public Member Functions

- [PetService](#) ()
Varsayılan yapıcı — [RepositoryFactory](#) kullanır.
- [PetService](#) (IRepository< [Pet](#) > repository)
Test yapıcısı — dependency injection.
- [Pet addPet](#) ([Pet](#) pet)
Yeni pet ekler.
- [Pet getPetById](#) (int id)
ID ile pet getirir.
- List< [Pet](#) > [getAllPets](#) ()
Tüm petleri listeler.
- void [updatePet](#) ([Pet](#) pet)
[Pet](#) günceller.
- void [deletePet](#) (int id)
[Pet](#) siler.
- void [deleteAllPets](#) ()
Tüm petleri siler.
- int [getPetCount](#) ()
Toplam pet sayısını döndürür.
- List< [Pet](#) > [searchByName](#) (String name)
Ada göre pet arar (büyük/küçük harf duyarsız).
- List< [Pet](#) > [filterBySpecies](#) (String species)
Türe göre filtreler ([Dog](#), [Cat](#), [Bird](#)).
- void [close](#) ()
Repository kapatır.

7.30.1 Detailed Description

[Pet](#) CRUD iş mantığı katmanı.

OOP Composition: [IRepository](#) bağımlılığı constructor injection ile. PDF zorunluluğu: Service layer — business logic ayrılır. Observer Pattern: Her CRUD işleminde [EventManager](#) üzerinden kayıtlı observer'lara bildirim gönderilir.

Author

Muhammed Mehdi Karagülle

Ibrahim Demirci

Zumre Uykun

Version

1.0

Definition at line 33 of file [PetService.java](#).

7.30.2 Constructor & Destructor Documentation

7.30.2.1 [PetService\(\)](#) [1/2]

```
com.mehdi.petreminder.service.PetService.PetService ()
```

Varsayılan yapıcı — [RepositoryFactory](#) kullanır.

Definition at line 45 of file [PetService.java](#).

```
00045         {
00046             this.repository = RepositoryFactory.createPetRepository();
00047         }
```

References [com.mehdi.petreminder.repository.RepositoryFactory.createPetRepository\(\)](#).

7.30.2.2 [PetService\(\)](#) [2/2]

```
com.mehdi.petreminder.service.PetService.PetService (
    IRepository< Pet > repository)
```

Test yapıcısı — dependency injection.

Parameters

<i>repository</i>	Mock veya test repository'si
-------------------	------------------------------

Definition at line 53 of file [PetService.java](#).

```
00053         {
00054             this.repository = repository;
00055         }
```

7.30.3 Member Function Documentation

7.30.3.1 addPet()

```
Pet com.mehdi.petreminder.service.PetService.addPet (
    Pet pet)
```

Yeni pet ekler.

Parameters

<i>pet</i>	Eklenecek Pet nesnesi
------------	---------------------------------------

Returns

Oluşturulan [Pet](#) (ID set edilmiş)

Exceptions

ServiceException	Doğrulama başarısız olursa
----------------------------------	----------------------------

Definition at line 63 of file [PetService.java](#).

```
00063         {
00064             validatePet(pet);
00065             int id = repository.save(pet);
00066             logger.info("Pet eklendi: id={}, name={}", id, pet.getName());
00067             // Observer Pattern: Pet eklendi bildirimi
00068             EventManager.getInstance().notify(EventType.PET_ADDED, pet);
00069             return pet;
00070         }
```

References [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [com.mehdi.petreminder.model.Pet.getName\(\)](#), [com.mehdi.petreminder.observer.EventManager.notify\(\)](#), and [com.mehdi.petreminder.observer.EventType.PET_ADDED](#).

7.30.3.2 close()

```
void com.mehdi.petreminder.service.PetService.close ()
```

Repository kapatır.

Definition at line 166 of file [PetService.java](#).

```
00166         {
00167             repository.close();
00168         }
```

7.30.3.3 deleteAllPets()

```
void com.mehdi.petreminder.service.PetService.deleteAllPets ()
```

Tüm petleri siler.

Definition at line 125 of file [PetService.java](#).

```
00125         {
00126             repository.deleteAll();
00127         }
```

7.30.3.4 deletePet()

```
void com.mehdi.petreminder.service.PetService.deletePet (
    int id)
```

[Pet](#) siler.

Parameters

<i>id</i>	Silinecek Pet ID
-----------	----------------------------------

Exceptions

ServiceException	Bulunamazsa
----------------------------------	-------------

Definition at line 112 of file [PetService.java](#).

```
00112         {
00113             boolean deleted = repository.delete(id);
00114             if (!deleted) {
00115                 throw new ServiceException("Pet silinemedi: id=" + id);
00116             }
00117             logger.info("Pet silindi: id={}", id);
00118             // Observer Pattern: Pet silindi bildirimi
00119             EventManager.getInstance().notify(EventType.PET_DELETED, id);
00120         }
```

References [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [com.mehdi.petreminder.observer.EventManager.notify\(\)](#), and [com.mehdi.petreminder.observer.EventType.PET_DELETED](#).

7.30.3.5 filterBySpecies()

```
List< Pet > com.mehdi.petreminder.service.PetService.filterBySpecies (
    String species)
```

Türe göre filtreler ([Dog](#), [Cat](#), [Bird](#)).

Parameters

<i>species</i>	Tür adı
----------------	---------

Returns

Filtrelenmiş liste

Definition at line 155 of file [PetService.java](#).

```
00155         {
00156             if (species == null || species.trim().isEmpty()) return getAllPets();
00157             return repository.findAll().stream()
00158                 .filter(p -> p.getSpecies() != null
00159                     && p.getSpecies().equalsIgnoreCase(species.trim()))
00160                 .collect(Collectors.toList());
00161         }
```

References [getAllPets\(\)](#).

7.30.3.6 getAllPets()

```
List< Pet > com.mehdi.petreminder.service.PetService.getAllPets ()
```

Tüm petleri listeler.

Returns

[Pet](#) listesi

Definition at line 87 of file [PetService.java](#).

```
00087         {
00088             return repository.findAll();
00089         }
```

Referenced by [filterBySpecies\(\)](#), and [searchByName\(\)](#).

7.30.3.7 getPetById()

```
Pet com.mehdi.petreminder.service.PetService.getPetById (
    int id)
```

ID ile pet getirir.

Parameters

<i>id</i>	Pet ID
-----------	------------------------

Returns

[Pet](#) nesnesi

Exceptions

ServiceException	Bulunamazsa
----------------------------------	-------------

Definition at line 78 of file [PetService.java](#).

```
00078         {
00079             return repository.findById(id)
00080                 .orElseThrow(() -> new ServiceException("Pet bulunamadı: id=" + id));
00081         }
```

7.30.3.8 getPetCount()

```
int com.mehdi.petreminder.service.PetService.getPetCount ()
```

Toplam pet sayısını döndürür.

Returns

Sayı

Definition at line 133 of file [PetService.java](#).

```
00133         {
00134             return repository.count();
00135         }
```

7.30.3.9 searchByName()

```
List< Pet > com.mehdi.petreminder.service.PetService.searchByName (
    String name)
```

Ada göre pet arar (büyük/küçük harf duyarlı).

Parameters

<i>name</i>	Aranacak ad
-------------	-------------

Returns

Eşleşen [Pet](#) listesi

Definition at line 142 of file [PetService.java](#).

```
00142         {
00143             if (name == null || name.trim().isEmpty()) return getAllPets();
00144             String lower = name.toLowerCase().trim();
00145             return repository.findAll().stream()
00146                 .filter(p -> p.getName() != null && p.getName().toLowerCase().contains(lower))
00147                 .collect(Collectors.toList());
00148         }
```

References [getAllPets\(\)](#).

7.30.3.10 updatePet()

```
void com.mehdi.petreminder.service.PetService.updatePet (
    Pet pet)
```

[Pet](#) günceller.

Parameters

<i>pet</i>	Güncellenecek Pet
------------	-----------------------------------

Exceptions

ServiceException	Doğrulama başarısız veya bulunamazsa
----------------------------------	--------------------------------------

Definition at line 96 of file [PetService.java](#).

```
00096         {
00097             validatePet(pet);
00098             boolean updated = repository.update(pet);
00099             if (!updated) {
00100                 throw new ServiceException("Pet güncellenemedi: id=" + pet.getId());
00101             }
00102             logger.info("Pet güncellendi: id={}", pet.getId());
00103             // Observer Pattern: Pet güncellendi bildirimi
00104             EventManager.getInstance().notify(EventType.PET_UPDATED, pet);
00105         }
```

References [com.mehdi.petreminder.model.Pet.getId\(\)](#), [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [com.mehdi.petreminder.observer.EventManager.notify\(\)](#), and [com.mehdi.petreminder.observer.EventType.PET_UPDATED](#).

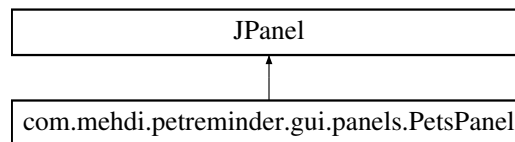
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/service/PetService.java](#)

7.31 com.mehdi.petreminder.gui.panels.PetsPanel Class Reference

Evcil hayvanlar listesi ve CRUD işlemleri.

Inheritance diagram for com.mehdi.petreminder.gui.panels.PetsPanel:



Public Member Functions

- [PetsPanel](#) ([MainFrame](#) mainFrame)
[PetsPanel](#) yapıcısı.
- void [refresh](#) ()
Tabloyu yeniler.

7.31.1 Detailed Description

Evcil hayvanlar listesi ve CRUD işlemleri.

JTable + toolbar + dialog pattern.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 29 of file [PetsPanel.java](#).

7.31.2 Constructor & Destructor Documentation

7.31.2.1 PetsPanel()

`com.mehdi.petreminder.gui.panels.PetsPanel.PetsPanel (`
 `MainFrame mainFrame)`

[PetsPanel](#) yapıcısı.

Parameters

<i>mainFrame</i>	Ana pencere
------------------	-------------

Definition at line 56 of file [PetsPanel.java](#).

```
00056      {
00057          this.mainFrame = mainFrame;
00058          this.petService = new PetService();
00059          setBackground(GuiConstants.BG_COLOR);
00060          setLayout(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));
00061          setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,
00062                                   GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));
00063          buildUI();
00064      }
```

References [com.mehdi.petreminder.gui.util.GuiConstants.BG_COLOR](#), [com.mehdi.petreminder.gui.util.GuiConstants.GAP](#), and [com.mehdi.petreminder.gui.util.GuiConstants.GAP_LARGE](#).

7.31.3 Member Function Documentation

7.31.3.1 refresh()

`void com.mehdi.petreminder.gui.panels.PetsPanel.refresh ()`

Tabloyu yeniler.

Definition at line 214 of file [PetsPanel.java](#).

```
00214      {
00215          tableModel.setRowCount(0);
00216          try {
00217              String search = searchField != null ? searchField.getText().trim() : "";
00218              String species = speciesFilter != null
00219                  ? (String) speciesFilter.getSelectedItem() : "Tümü";
00220
00221              List<Pet> pets = search.isEmpty() ? petService.getAllPets()
00222                  : petService.searchByName(search);
00223              if (!"Tümü".equals(species) && species != null) {
00224                  pets = petService.filterBySpecies(species);
00225              }
00226
00227              for (Pet p : pets) {
00228                  tableModel.addRow(new Object[]{
00229                      p.getId(),
00230                      p.getName(),
00231                      p.getSpecies(),
00232                      getBreed(p),
00233                      p.getBirthDate() != null ? p.getBirthDate().toString() : "",
00234                      p.getGender() != null ? p.getGender() : "",
00235                      String.format("%.1f", p.getWeight())
00236                  });
00237              }
00238          } catch (Exception e) {
00239              // Veri yoksa sessiz
00240          }
00241      }
```

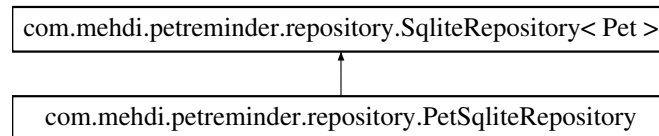
The documentation for this class was generated from the following file:

- `petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/PetsPanel.java`

7.32 com.mehdi.petreminder.repository.PetSqliteRepository Class Reference

SQLite üzerinde [Pet](#) CRUD işlemleri.

Inheritance diagram for com.mehdi.petreminder.repository.PetSqliteRepository:



Public Member Functions

- [PetSqliteRepository](#) ()
Varsayılan yapıcı — gerçek SQLite.
- [PetSqliteRepository](#) (Connection [connection](#))
Test yapıcısı.
- List< [Pet](#) > [findByOwnerId](#) (int ownerId)
OwnerId'ye göre petleri listeler.

Public Member Functions inherited from [com.mehdi.petreminder.repository.SqliteRepository< Pet >](#)

- [SqliteRepository](#) ()
SQLite bağlantısını açar ve tabloyu oluşturur.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür (test için).

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [Pet](#) pet) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [Pet](#) pet) throws SQLException
- [Pet](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from [com.mehdi.petreminder.repository.SqliteRepository](#)< [Pet](#) >

- abstract String [getTableName](#) ()
Tablo adını döndürür.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL'ini döndürür.
- abstract String [getInsertSql](#) ()
INSERT SQL'ini döndürür (ID hariç).
- abstract String [getUpdateSql](#) ()
UPDATE SQL'ini döndürür.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members

Protected Attributes inherited from [com.mehdi.petreminder.repository.SqliteRepository](#)< [Pet](#) >

- Connection [connection](#)
SQLite JDBC bağlantısı.

Static Protected Attributes inherited from [com.mehdi.petreminder.repository.SqliteRepository](#)< [Pet](#) >

- static final Logger [logger](#)
Logger.

7.32.1 Detailed Description

SQLite üzerinde [Pet](#) CRUD işlemleri.

[SqliteRepository](#)'yi extend eder; Template Method Pattern. [Dog](#) / [Cat](#) / [Bird](#) tür ayrımı species sütununa göre yapılır.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 27 of file [PetSqliteRepository.java](#).

7.32.2 Constructor & Destructor Documentation

7.32.2.1 PetSqliteDatabase() [1/2]

com.mehdi.petreminder.repository.PetSqliteDatabase.PetSqliteDatabase ()

Varsayılan yapıcı — gerçek SQLite.

Definition at line 36 of file [PetSqliteDatabase.java](#).

```
00036                                     {
00037         super();
00038     }
```

7.32.2.2 PetSqliteDatabase() [2/2]

com.mehdi.petreminder.repository.PetSqliteDatabase.PetSqliteDatabase (
Connection connection)

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 44 of file [PetSqliteDatabase.java](#).

```
00044                                     {
00045         super(connection);
00046     }
```

References [com.mehdi.petreminder.repository.SqliteDatabase< Pet >.connection](#).

7.32.3 Member Function Documentation

7.32.3.1 findById()

List< [Pet](#) > com.mehdi.petreminder.repository.PetSqliteDatabase.findById (
int ownerId)

OwnerId'ye göre petleri listeler.

Parameters

<i>ownerId</i>	Kullanıcı ID
----------------	--------------

Returns

Kullanıcıya ait pet listesi

Definition at line 185 of file [PetSqliteDatabase.java](#).

```
00185                                     {
00186         List<Pet> list = new ArrayList<>();
00187         String sql = "SELECT * FROM pets WHERE owner_id = ?";
00188         try (PreparedStatement ps = connection.prepareStatement(sql)) {
00189             ps.setInt(1, ownerId);
00190             ResultSet rs = ps.executeQuery();
00191             while (rs.next()) list.add(mapRow(rs));
00192         } catch (SQLException e) {
00193             throw new RepositoryException("findById hatası", e);
00194         }
00195         return list;
00196     }
```

References [com.mehdi.petreminder.repository.SqliteDatabase< Pet >.connection](#), and [mapRow\(\)](#).

7.32.3.2 getCreateTableSql()

String com.mehdi.petreminder.repository.PetSqliteRepository.getCreateTableSql () [protected]

Member documentation.

Definition at line 62 of file [PetSqliteRepository.java](#).

```

00062     {
00063         return "CREATE TABLE IF NOT EXISTS pets ("
00064             + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00065             + "owner_id INTEGER NOT NULL DEFAULT 1,"
00066             + "name TEXT NOT NULL,"
00067             + "species TEXT NOT NULL,"
00068             + "breed TEXT,"
00069             + "birth_date TEXT,"
00070             + "gender TEXT,"
00071             + "weight REAL DEFAULT 0,"
00072             + "notes TEXT,"
00073             + "can_talk INTEGER DEFAULT 0,"
00074             + "bird_type TEXT,"
00075             + "is_trained INTEGER DEFAULT 0,"
00076             + "is_indoor INTEGER DEFAULT 1"
00077             + ");";
00078     }

```

7.32.3.3 getInsertSql()

String com.mehdi.petreminder.repository.PetSqliteRepository.getInsertSql () [protected]

Member documentation.

Definition at line 85 of file [PetSqliteRepository.java](#).

```

00085     {
00086         return "INSERT INTO pets (owner_id,name,species,breed,birth_date,gender,"
00087             + "weight,notes,can_talk,bird_type,is_trained,is_indoor) "
00088             + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00089     }

```

7.32.3.4 getTableName()

String com.mehdi.petreminder.repository.PetSqliteRepository.getTableName () [protected]

Member documentation.

Definition at line 53 of file [PetSqliteRepository.java](#).

```

00053     {
00054         return "pets";
00055     }

```

7.32.3.5 getUpdateSql()

String com.mehdi.petreminder.repository.PetSqliteRepository.getUpdateSql () [protected]

Member documentation.

Definition at line 96 of file [PetSqliteRepository.java](#).

```

00096     {
00097         return "UPDATE pets SET owner_id=?,name=?,species=?,breed=?,birth_date=,"
00098             + "gender=?,weight=?,notes=?,can_talk=?,bird_type=?,is_trained=?,is_indoor=? "
00099             + "WHERE id=?";
00100     }

```

7.32.3.6 mapRow()

`Pet` com.mehdi.petreminder.repository.PetSqliteRepository.mapRow (
 ResultSet rs) throws SQLException [protected]

Member documentation.

Definition at line 137 of file [PetSqliteRepository.java](#).

```

00137                                     {
00138         String species = rs.getString("species");
00139         int id = rs.getInt("id");
00140         String name = rs.getString("name");
00141         String birthDateStr = rs.getString("birth_date");
00142         LocalDate birthDate = birthDateStr != null ? LocalDate.parse(birthDateStr) : null;
00143         int ownerId = rs.getInt("owner_id");
00144
00145         Pet pet;
00146         if ("Dog".equalsIgnoreCase(species) || "Kopek".equalsIgnoreCase(species)) {
00147             Dog dog = new Dog(id, name, birthDate, ownerId);
00148             dog.setBreed(rs.getString("breed"));
00149             dog.setTrained(rs.getInt("is_trained") == 1);
00150             pet = dog;
00151         } else if ("Cat".equalsIgnoreCase(species) || "Kedi".equalsIgnoreCase(species)) {
00152             Cat cat = new Cat(id, name, birthDate, ownerId);
00153             cat.setBreed(rs.getString("breed"));
00154             cat.setIndoor(rs.getInt("is_indoor") == 1);
00155             pet = cat;
00156         } else {
00157             Bird bird = new Bird(id, name, birthDate, ownerId);
00158             bird.setBirdType(rs.getString("bird_type"));
00159             bird.setCanTalk(rs.getInt("can_talk") == 1);
00160             pet = bird;
00161         }
00162         pet.setGender(rs.getString("gender"));
00163         pet.setWeight(rs.getDouble("weight"));
00164         pet.setNotes(rs.getString("notes"));
00165         return pet;
00166     }

```

References [com.mehdi.petreminder.model.Bird.setBirdType\(\)](#), [com.mehdi.petreminder.model.Cat.setBreed\(\)](#), [com.mehdi.petreminder.model.Dog.setBreed\(\)](#), [com.mehdi.petreminder.model.Bird.setCanTalk\(\)](#), [com.mehdi.petreminder.model.Pet.setNotes\(\)](#), [com.mehdi.petreminder.model.Cat.setIndoor\(\)](#), [com.mehdi.petreminder.model.Pet.setNotes\(\)](#), [com.mehdi.petreminder.model.Dog.setTrained\(\)](#) and [com.mehdi.petreminder.model.Pet.setWeight\(\)](#).

Referenced by [findByOwnerId\(\)](#).

7.32.3.7 setInsertParams()

`void` com.mehdi.petreminder.repository.PetSqliteRepository.setInsertParams (
 PreparedStatement ps,
 Pet pet) throws SQLException [protected]

Member documentation.

Definition at line 107 of file [PetSqliteRepository.java](#).

```

00107                                     {
00108         ps.setInt(1, pet.getOwnerId());
00109         ps.setString(2, pet.getName());
00110         ps.setString(3, pet.getSpecies());
00111         ps.setString(4, pet.getBreed());
00112         ps.setString(5, pet.getBirthDate() != null ? pet.getBirthDate().toString() : null);
00113         ps.setString(6, pet.getGender());
00114         ps.setDouble(7, pet.getWeight());
00115         ps.setString(8, pet.getNotes());
00116         ps.setInt(9, (pet instanceof Bird && ((Bird) pet).isCanTalk()) ? 1 : 0);
00117         ps.setString(10, pet instanceof Bird ? ((Bird) pet).getBirdType() : null);
00118         ps.setInt(11, (pet instanceof Dog && ((Dog) pet).isTrained()) ? 1 : 0);
00119         ps.setInt(12, (pet instanceof Cat && ((Cat) pet).isIndoor()) ? 1 : 0);
00120     }

```

Referenced by [setUpdateParams\(\)](#).

7.32.3.8 setUpdateParams()

```
void com.mehdi.petreminder.repository.PetSqliteRepository.setUpdateParams (
    PreparedStatement ps,
    Pet pet) throws SQLException [protected]
```

Member documentation.

Definition at line 127 of file [PetSqliteRepository.java](#).

```
00127
00128         setInsertParams(ps, pet);
00129         ps.setInt(13, pet.getId());
00130     }
```

References [setInsertParams\(\)](#).

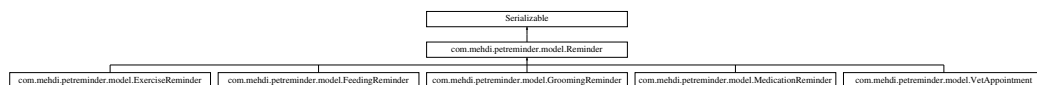
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/PetSqliteRepository.java](#)

7.33 com.mehdi.petreminder.model.Reminder Class Reference

Evcil hayvan bakım hatırlatıcısının soyut temel sınıfı.

Inheritance diagram for com.mehdi.petreminder.model.Reminder:



Public Member Functions

- int [getId](#) ()
ID getter.
- void [setId](#) (int id)
ID setter.
- int [getPetId](#) ()
Pet ID getter.
- void [setPetId](#) (int petId)
Pet ID setter.
- String [getPetName](#) ()
Pet adı getter.
- void [setPetName](#) (String petName)
Pet adı setter.
- LocalDateTime [getScheduledTime](#) ()
Planlanan zaman getter.
- void [setScheduledTime](#) (LocalDateTime scheduledTime)
Planlanan zaman setter.
- boolean [isCompleted](#) ()
Tamamlandı mı getter.
- void [setCompleted](#) (boolean completed)

- Tamamlandı setter.*
- String [getDescription](#) ()
 - Açıklama getter.*
- void [setDescription](#) (String description)
 - Açıklama setter.*
- String [getPriority](#) ()
 - Öncelik getter.*
- void [setPriority](#) (String priority)
 - Öncelik setter.*
- boolean [isRecurring](#) ()
 - Tekrarlıyor mu getter.*
- void [setRecurring](#) (boolean recurring)
 - Tekrarlama setter.*
- String [getRecurrencePattern](#) ()
 - Tekrar deseni getter.*
- void [setRecurrencePattern](#) (String recurrencePattern)
 - Tekrar deseni setter.*
- abstract String [getReminderType](#) ()
 - Hatırlatıcının türünü döndürür.*
- abstract boolean [validate](#) ()
 - Hatırlatıcının geçerliliğini doğrular.*
- abstract String [getSummary](#) ()
 - Hatırlatıcının özet bilgisini döndürür (GUI'de gösterilir).*
- String [getFormattedTime](#) ()
 - Planlanan zamanı biçimli string olarak döndürür.*
- boolean [isOverdue](#) ()
 - Hatırlatıcının geçmiş mi olduğunu kontrol eder.*
- String [toString](#) ()
 - String temsili.*
- boolean [equals](#) (Object obj)
 - Eşitlik kontrolü — ID bazlı.*
- int [hashCode](#) ()
 - Hash kodu.*

Protected Member Functions

- [Reminder](#) ()
 - Varsayılan yapıcı — Serialization için.*
- [Reminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern)
 - Tam parametrelili yapıcı.*

Static Protected Attributes

- static final DateFormatter [FORMATTER](#)
 - Tarih saat formatı.*

7.33.1 Detailed Description

Evcil hayvan bakım hatırlatıcısının soyut temel sınıfı.

OOP Abstraction: abstract class ile somutlaştırılmaz. OOP Polymorphism: [validate\(\)](#) her alt sınıfta farklı davranır.
OOP Encapsulation: tüm alanlar private.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 22 of file [Reminder.java](#).

7.33.2 Constructor & Destructor Documentation

7.33.2.1 Reminder() [1/2]

```
com.mehdi.petreminder.model.Reminder.Reminder () [protected]
```

Varsayılan yapıcı — Serialization için.

Definition at line 59 of file [Reminder.java](#).

```
00059 {}
```

Referenced by [equals\(\)](#).

7.33.2.2 Reminder() [2/2]

```
com.mehdi.petreminder.model.Reminder.Reminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String description,
    String priority,
    boolean recurring,
    String recurrencePattern) [protected]
```

Tam parametrelili yapıcı.

Parameters

<i>id</i>	Benzersiz ID
<i>petId</i>	Pet ID

<i>petName</i>	Pet adı
<i>scheduledTime</i>	Planlanan zaman
<i>description</i>	Açıklama
<i>priority</i>	Öncelik (DUSUK/ORTA/YUKSEK)
<i>recurring</i>	Tekrarlıyor mu
<i>recurrencePattern</i>	Tekrar deseni

Definition at line 72 of file [Reminder.java](#).

```

00074                                     {
00075         this.id = id;
00076         this.petId = petId;
00077         this.petName = petName;
00078         this.scheduledTime = scheduledTime;
00079         this.completed = false;
00080         this.description = description;
00081         this.priority = priority;
00082         this.recurring = recurring;
00083         this.recurrencePattern = recurrencePattern;
00084     }

```

7.33.3 Member Function Documentation

7.33.3.1 equals()

```
boolean com.mehdi.petreminder.model.Reminder.equals (
    Object obj)
```

Eşitlik kontrolü — ID bazlı.

Parameters

<i>obj</i>	Karşılaştırılan nesne
------------	-----------------------

Returns

ID'ler eşitse true

Definition at line 204 of file [Reminder.java](#).

```

00204                                     {
00205         if (this == obj) return true;
00206         if (obj == null || !(obj instanceof Reminder)) return false;
00207         Reminder other = (Reminder) obj;
00208         return this.id == other.id;
00209     }

```

References [Reminder\(\)](#).

7.33.3.2 getDescription()

```
String com.mehdi.petreminder.model.Reminder.getDescription ()
```

Açıklama getter.

Returns

Açıklama

Definition at line 121 of file [Reminder.java](#).

```
00121 { return description; }
```

7.33.3.3 getFormattedTime()

```
String com.mehdi.petreminder.model.Reminder.getFormattedTime ()
```

Planlanan zamanı biçimli string olarak döndürür.

Returns

"dd/MM/yyyy HH:mm" formatında zaman

Definition at line 174 of file [Reminder.java](#).

```
00174                                     {
00175         if (scheduledTime == null) return "Belirsiz";
00176         return scheduledTime.format (FORMATTER);
00177     }
```

References [FORMATTER](#).

Referenced by [com.mehdi.petreminder.gui.panels.VetPanel.refresh\(\)](#), and [toString\(\)](#).

7.33.3.4 getId()

```
int com.mehdi.petreminder.model.Reminder.getId ()
```

ID getter.

Returns

Hatırlatıcı ID

Definition at line 89 of file [Reminder.java](#).

```
00089 { return id; }
```

Referenced by [com.mehdi.petreminder.gui.panels.VetPanel.refresh\(\)](#), [com.mehdi.petreminder.model.ExerciseReminder.toString\(\)](#), [com.mehdi.petreminder.model.FeedingReminder.toString\(\)](#), [com.mehdi.petreminder.model.GroomingReminder.toString\(\)](#), [com.mehdi.petreminder.model.MedicationReminder.toString\(\)](#), [com.mehdi.petreminder.model.VetAppointment.toString\(\)](#), and [com.mehdi.petreminder.service.ReminderService.updateReminder\(\)](#).

7.33.3.5 getPetId()

```
int com.mehdi.petreminder.model.Reminder.getPetId ()
```

[Pet](#) ID getter.

Returns

[Pet](#) ID

Definition at line 95 of file [Reminder.java](#).

```
00095 { return petId; }
```

7.33.3.6 getPetName()

```
String com.mehdi.petreminder.model.Reminder.getPetName ()
```

Pet adı getter.

Returns

Pet adı

Definition at line 101 of file [Reminder.java](#).

```
00101 { return petName; }
```

Referenced by [com.mehdi.petreminder.gui.panels.VetPanel.refresh\(\)](#), [com.mehdi.petreminder.model.ExerciseReminder.toString\(\)](#), [com.mehdi.petreminder.model.FeedingReminder.toString\(\)](#), [com.mehdi.petreminder.model.GroomingReminder.toString\(\)](#), [com.mehdi.petreminder.model.MedicationReminder.toString\(\)](#), and [com.mehdi.petreminder.model.VetAppointment.toString\(\)](#).

7.33.3.7 getPriority()

```
String com.mehdi.petreminder.model.Reminder.getPriority ()
```

Öncelik getter.

Returns

Öncelik

Definition at line 127 of file [Reminder.java](#).

```
00127 { return priority; }
```

7.33.3.8 getRecurrencePattern()

```
String com.mehdi.petreminder.model.Reminder.getRecurrencePattern ()
```

Tekrar deseni getter.

Returns

Tekrar deseni

Definition at line 139 of file [Reminder.java](#).

```
00139 { return recurrencePattern; }
```

7.33.3.9 getReminderType()

```
abstract String com.mehdi.petreminder.model.Reminder.getReminderType () [abstract]
```

Hatırlatıcının türünü döndürür.

OOP Polymorphism: her alt sınıf farklı tür döndürür.

Returns

Hatırlatıcı türü string'i

Reimplemented in [com.mehdi.petreminder.model.ExerciseReminder](#), [com.mehdi.petreminder.model.FeedingReminder](#), [com.mehdi.petreminder.model.GroomingReminder](#), [com.mehdi.petreminder.model.MedicationReminder](#), and [com.mehdi.petreminder.model.VetAppointment](#).

Referenced by [com.mehdi.petreminder.service.ReminderService.addReminder\(\)](#), and [toString\(\)](#).

7.33.3.10 getScheduledTime()

```
LocalDateTime com.mehdi.petreminder.model.Reminder.getScheduledTime ()
```

Planlanan zaman getter.

Returns

Planlanan zaman

Definition at line 107 of file [Reminder.java](#).

```
00107 { return scheduledTime; }
```

7.33.3.11 getSummary()

```
abstract String com.mehdi.petreminder.model.Reminder.getSummary () [abstract]
```

Hatırlatıcının özet bilgisini döndürür (GUI'de gösterilir).

Returns

Özet string

Reimplemented in [com.mehdi.petreminder.model.ExerciseReminder](#), [com.mehdi.petreminder.model.FeedingReminder](#), [com.mehdi.petreminder.model.GroomingReminder](#), [com.mehdi.petreminder.model.MedicationReminder](#), and [com.mehdi.petreminder.model.VetAppointment](#).

7.33.3.12 hashCode()

```
int com.mehdi.petreminder.model.Reminder.hashCode ()
```

Hash kodu.

Returns

ID'ye göre hash

Definition at line 216 of file [Reminder.java](#).

```
00216         {  
00217             return Integer.hashCode(id);  
00218         }
```

7.33.3.13 isCompleted()

```
boolean com.mehdi.petreminder.model.Reminder.isCompleted ()
```

Tamamlandı mı getter.

Returns

true ise tamamlandı

Definition at line 115 of file [Reminder.java](#).

```
00115 { return completed; }
```

7.33.3.14 isOverdue()

```
boolean com.mehdi.petreminder.model.Reminder.isOverdue ()
```

Hatırlatıcının geçmiş mi olduğunu kontrol eder.

Returns

Planlan zaman geçmişse ve tamamlanmamışsa true

Definition at line 183 of file [Reminder.java](#).

```
00183         {  
00184             if (scheduledTime == null || completed) return false;  
00185             return LocalDateTime.now().isAfter(scheduledTime);  
00186         }
```

Referenced by [com.mehdi.petreminder.service.ReminderService.getOverdueReminders\(\)](#).

7.33.3.15 isRecurring()

```
boolean com.mehdi.petreminder.model.Reminder.isRecurring ()
```

Tekrarlıyor mu getter.

Returns

Tekrarlıyor mu

Definition at line 133 of file [Reminder.java](#).

```
00133 { return recurring; }
```

7.33.3.16 setCompleted()

```
void com.mehdi.petreminder.model.Reminder.setCompleted (
    boolean completed)
```

Tamamlandı setter.

Parameters

<i>completed</i>	Durum
------------------	-------

Definition at line 118 of file [Reminder.java](#).

```
00118 { this.completed = completed; }
```

Referenced by [com.mehdi.petreminder.repository.ReminderMySqlRepository.mapRow\(\)](#), [com.mehdi.petreminder.repository.ReminderRepository.findById\(\)](#) and [com.mehdi.petreminder.service.ReminderService.markCompleted\(\)](#).

7.33.3.17 setDescription()

```
void com.mehdi.petreminder.model.Reminder.setDescription (
    String description)
```

Açıklama setter.

Parameters

<i>description</i>	Açıklama
--------------------	----------

Definition at line 124 of file [Reminder.java](#).

```
00124 { this.description = description; }
```

7.33.3.18 setId()

```
void com.mehdi.petreminder.model.Reminder.setId (
    int id)
```

ID setter.

Parameters

<i>id</i>	Yeni ID
-----------	---------

Definition at line 92 of file [Reminder.java](#).

```
00092 { this.id = id; }
```


7.33.3.19 setPetId()

```
void com.mehdi.petreminder.model.Reminder.setPetId (
    int petId)
```

Pet ID setter.

Parameters

<i>petId</i>	Pet ID
--------------	--------

Definition at line 98 of file [Reminder.java](#).

```
00098 { this.petId = petId; }
```

7.33.3.20 setPetName()

```
void com.mehdi.petreminder.model.Reminder.setPetName (
    String petName)
```

Pet adı setter.

Parameters

<i>petName</i>	Pet adı
----------------	---------

Definition at line 104 of file [Reminder.java](#).

```
00104 { this.petName = petName; }
```

7.33.3.21 setPriority()

```
void com.mehdi.petreminder.model.Reminder.setPriority (
    String priority)
```

Öncelik setter.

Parameters

<i>priority</i>	Öncelik
-----------------	---------

Definition at line 130 of file [Reminder.java](#).

```
00130 { this.priority = priority; }
```

7.33.3.22 setRecurrencePattern()

```
void com.mehdi.petreminder.model.Reminder.setRecurrencePattern (
    String recurrencePattern)
```

Tekrar deseni setter.

Parameters

<i>recurrencePattern</i>	Tekrar deseni
--------------------------	---------------

Definition at line 142 of file [Reminder.java](#).

```
00142
00143         this.recurrencePattern = recurrencePattern;
00144     }
```

7.33.3.23 setRecurring()

```
void com.mehdi.petreminder.model.Reminder.setRecurring (
    boolean recurring)
```

Tekrarlama setter.

Parameters

<i>recurring</i>	Tekrarlıyor mu
------------------	----------------

Definition at line 136 of file [Reminder.java](#).

```
00136 { this.recurring = recurring; }
```

7.33.3.24 setScheduledTime()

```
void com.mehdi.petreminder.model.Reminder.setScheduledTime (
    LocalDateTime scheduledTime)
```

Planlanan zaman setter.

Parameters

<i>scheduledTime</i>	Planlanan zaman
----------------------	-----------------

Definition at line 110 of file [Reminder.java](#).

```
00110
00111         this.scheduledTime = scheduledTime;
00112     }
```

7.33.3.25 toString()

```
String com.mehdi.petreminder.model.Reminder.toString ()
```

String temsili.

Returns

Hatırlatıcı bilgilerini içeren string

Reimplemented in [com.mehdi.petreminder.model.ExerciseReminder](#), [com.mehdi.petreminder.model.FeedingReminder](#), [com.mehdi.petreminder.model.GroomingReminder](#), [com.mehdi.petreminder.model.MedicationReminder](#), and [com.mehdi.petreminder.model.VetAppointment](#).

Definition at line 193 of file [Reminder.java](#).

```
00193         {
00194         return String.format("Reminder{id=%d, type='%s', pet='%s', time='%s', done=%b}",
00195         id, getReminderType(), petName, getFormattedTime(), completed);
00196     }
```

References [getFormattedTime\(\)](#), and [getReminderType\(\)](#).

7.33.3.26 validate()

```
abstract boolean com.mehdi.petreminder.model.Reminder.validate () [abstract]
```

Hatırlatıcının geçerliliğini doğrular.

OOP Polymorphism: her alt sınıf kendi kuralını uygular.

Returns

Geçerliyse true

Reimplemented in [com.mehdi.petreminder.model.ExerciseReminder](#), [com.mehdi.petreminder.model.FeedingReminder](#), [com.mehdi.petreminder.model.GroomingReminder](#), [com.mehdi.petreminder.model.MedicationReminder](#), and [com.mehdi.petreminder.model.VetAppointment](#).

7.33.4 Member Data Documentation

7.33.4.1 FORMATTER

```
final DateTimeFormatter com.mehdi.petreminder.model.Reminder.FORMATTER [static], [protected]
```

Initial value:

```
=
    DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm")
```

Tarih saat formatı.

Definition at line 28 of file [Reminder.java](#).

Referenced by [getFormattedTime\(\)](#).

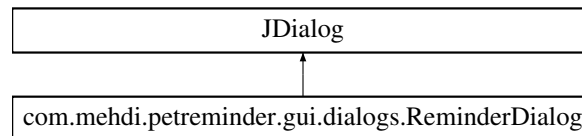
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/Reminder.java](#)

7.34 com.mehdi.petreminder.gui.dialogs.ReminderDialog Class Reference

[Reminder](#) ekleme ve düzenleme modal dialog'u.

Inheritance diagram for com.mehdi.petreminder.gui.dialogs.ReminderDialog:



Public Member Functions

- [ReminderDialog](#) (Frame parent, [Reminder](#) editReminder)
ReminderDialog yapıcısı.
- boolean [isConfirmed](#) ()
Onaylandı mı.
- [Reminder](#) [getReminder](#) ()
Oluşturulan reminder'ı döndürür.

7.34.1 Detailed Description

[Reminder](#) ekleme ve düzenleme modal dialog'u.

Tür seçimine göre dinamik form alanları gösterilir.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 25 of file [ReminderDialog.java](#).

7.34.2 Constructor & Destructor Documentation

7.34.2.1 ReminderDialog()

```
com.mehdi.petreminder.gui.dialogs.ReminderDialog.ReminderDialog (
    Frame parent,
    Reminder editReminder)
```

[ReminderDialog](#) yapıcısı.

Parameters

<i>parent</i>	Ana pencere
<i>editReminder</i>	Düzenlenecek reminder (yeni için null)

Definition at line 73 of file [ReminderDialog.java](#).

```
00073
00074         super(parent, editReminder == null ? "Yeni Hatırlatıcı" : "Hatırlatıcı Düzenle", true);
00075         this.editReminder = editReminder;
00076         buildUI();
00077         if (editReminder != null) fillForm(editReminder);
00078         pack();
00079         setLocationRelativeTo(parent);
00080         setMinimumSize(new Dimension(440, 480));
00081     }
```

7.34.3 Member Function Documentation

7.34.3.1 getReminder()

```
Reminder com.mehdi.petreminder.gui.dialogs.ReminderDialog.getReminder ()
```

Oluşturulan reminder'ı döndürür.

Returns

[Reminder](#) nesnesi

Definition at line 307 of file [ReminderDialog.java](#).

```
00307 { return resultReminder; }
```

7.34.3.2 isConfirmed()

```
boolean com.mehdi.petreminder.gui.dialogs.ReminderDialog.isConfirmed ()
```

Onaylandı mı.

Returns

true ise kaydet tıklandı

Definition at line 301 of file [ReminderDialog.java](#).

```
00301 { return confirmed; }
```

The documentation for this class was generated from the following file:

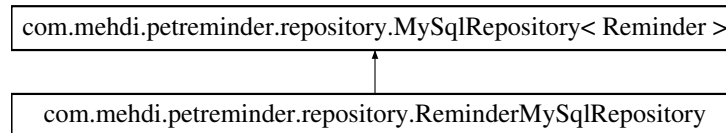
- petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/[ReminderDialog.java](#)

7.35 com.mehdi.petreminder.repository.ReminderMySqlRepository

Class Reference

MySQL üzerinde [Reminder](#) CRUD işlemleri.

Inheritance diagram for com.mehdi.petreminder.repository.ReminderMySqlRepository:



Public Member Functions

- [ReminderMySqlRepository](#) ()
Varsayılan yapıcı.
- [ReminderMySqlRepository](#) (Connection [connection](#))
Test yapıcısı.

Public Member Functions inherited from

[com.mehdi.petreminder.repository.MySqlRepository](#) < [Reminder](#) >

- [MySqlRepository](#) ()
MySQL bağlantısını açar.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür.

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [Reminder](#) r) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [Reminder](#) r) throws SQLException
- [Reminder](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from [com.mehdi.petreminder.repository.MySqlRepository](#) < [Reminder](#) >

- abstract String [getTableName](#) ()
Tablo adı.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL.
- abstract String [getInsertSql](#) ()
INSERT SQL.
- abstract String [getUpdateSql](#) ()
UPDATE SQL.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members

Protected Attributes inherited from [com.mehdi.petreminder.repository.MySqlRepository](#) < [Reminder](#) >

- Connection [connection](#)
MySQL JDBC bağlantısı.

Static Protected Attributes inherited from [com.mehdi.petreminder.repository.MySqlRepository](#) < [Reminder](#) >

- static final Logger [logger](#)
Logger.

7.35.1 Detailed Description

MySQL üzerinde [Reminder](#) CRUD işlemleri.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 23 of file [ReminderMySqlRepository.java](#).

7.35.2 Constructor & Destructor Documentation

7.35.2.1 ReminderMySqlRepository() [1/2]

`com.mehdi.petreminder.repository.ReminderMySqlRepository.ReminderMySqlRepository ()`

Varsayılan yapıcı.

Definition at line 28 of file [ReminderMySqlRepository.java](#).

```
00028                                     {
00029         super();
00030     }
```

7.35.2.2 ReminderMySqlRepository() [2/2]

`com.mehdi.petreminder.repository.ReminderMySqlRepository.ReminderMySqlRepository (Connection connection)`

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 36 of file [ReminderMySqlRepository.java](#).

```
00036                                     {
00037         super(connection);
00038     }
```

References [com.mehdi.petreminder.repository.MySqlRepository<Reminder>.connection](#).

7.35.3 Member Function Documentation

7.35.3.1 getCreateTableSql()

`String com.mehdi.petreminder.repository.ReminderMySqlRepository.getCreateTableSql () [protected]`

Member documentation.

Definition at line 54 of file [ReminderMySqlRepository.java](#).

```
00054                                     {
00055         return "CREATE TABLE IF NOT EXISTS reminders ("
00056             + "id INT AUTO_INCREMENT PRIMARY KEY,"
00057             + "pet_id INT NOT NULL,"
00058             + "pet_name VARCHAR(50),"
00059             + "reminder_type VARCHAR(30) NOT NULL,"
00060             + "scheduled_time DATETIME NOT NULL,"
00061             + "completed BOOLEAN DEFAULT FALSE,"
00062             + "description TEXT,"
00063             + "priority VARCHAR(10) DEFAULT 'ORTA',"
00064             + "recurring BOOLEAN DEFAULT FALSE,"
00065             + "recurrence_pattern VARCHAR(30),"
00066             + "food_type VARCHAR(50),"
00067             + "portion_grams DOUBLE,"
00068             + "includes_water_change BOOLEAN DEFAULT FALSE,"
00069             + "medication_name VARCHAR(100),"
00070             + "dosage DOUBLE,"
00071             + "dosage_unit VARCHAR(20),"
00072             + "prescribed BOOLEAN DEFAULT FALSE,"
```



```

00073         + "exercise_type VARCHAR(50), "
00074         + "duration_minutes INT, "
00075         + "distance_km DOUBLE, "
00076         + "grooming_type VARCHAR(50), "
00077         + "professional BOOLEAN DEFAULT FALSE, "
00078         + "groomer_name VARCHAR(100), "
00079         + "veterinarian_name VARCHAR(100), "
00080         + "clinic_name VARCHAR(100), "
00081         + "reason VARCHAR(100), "
00082         + "estimated_cost DOUBLE, "
00083         + "confirmed BOOLEAN DEFAULT FALSE, "
00084         + "clinic_phone VARCHAR(20) "
00085         + ");";
00086     }

```

7.35.3.2 getInsertSql()

String com.mehdi.petreminder.repository.ReminderMySQLRepository.getInsertSql () [protected]

Member documentation.

Definition at line 93 of file [ReminderMySQLRepository.java](#).

```

00093     {
00094         return "INSERT INTO reminders (pet_id,pet_name,reminder_type,scheduled_time,"
00095         + "completed,description,priority,recurring,recurrence_pattern,"
00096         + "food_type,portion_grams,includes_water_change,"
00097         + "medication_name,dosage,dosage_unit,prescribed,"
00098         + "exercise_type,duration_minutes,distance_km,"
00099         + "grooming_type,professional,groomer_name,"
00100         + "veterinarian_name,clinic_name,reason,estimated_cost,confirmed,clinic_phone) "
00101         + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00102     }

```

7.35.3.3 getTableName()

String com.mehdi.petreminder.repository.ReminderMySQLRepository.getTableName () [protected]

Member documentation.

Definition at line 45 of file [ReminderMySQLRepository.java](#).

```

00045     {
00046         return "reminders";
00047     }

```

7.35.3.4 getUpdateSql()

String com.mehdi.petreminder.repository.ReminderMySQLRepository.getUpdateSql () [protected]

Member documentation.

Definition at line 109 of file [ReminderMySQLRepository.java](#).

```

00109     {
00110         return "UPDATE reminders SET pet_id=?,pet_name=?,reminder_type=?,scheduled_time=,"
00111         + "completed=?,description=?,priority=?,recurring=?,recurrence_pattern=?, "
00112         + "food_type=?,portion_grams=?,includes_water_change=?, "
00113         + "medication_name=?,dosage=?,dosage_unit=?,prescribed=?, "
00114         + "exercise_type=?,duration_minutes=?,distance_km=?, "
00115         + "grooming_type=?,professional=?,groomer_name=?, "
00116         + "veterinarian_name=?,clinic_name=?,reason=?,estimated_cost=?,confirmed=?, "
00117         + "clinic_phone=? WHERE id=?";
00118     }

```

7.35.3.5 mapRow()

`Reminder` `com.mehdi.petreminder.repository.ReminderMySqlRepository.mapRow (`
`ResultSet rs)` throws `SQLException` [protected]

Member documentation.

Definition at line 174 of file `ReminderMySqlRepository.java`.

```

00174
00175         String type = rs.getString("reminder_type");
00176         int id = rs.getInt("id");
00177         int petId = rs.getInt("pet_id");
00178         String petName = rs.getString("pet_name");
00179         Timestamp ts = rs.getTimestamp("scheduled_time");
00180         LocalDateTime scheduledTime = ts != null ? ts.toLocalDateTime() : null;
00181         String description = rs.getString("description");
00182         String priority = rs.getString("priority");
00183         boolean recurring = rs.getBoolean("recurring");
00184         String recurrencePattern = rs.getString("recurrence_pattern");
00185         boolean completed = rs.getBoolean("completed");
00186
00187         Reminder reminder;
00188         switch (type != null ? type : "") {
00189             case "Besleme":
00190                 reminder = new FeedingReminder(id, petId, petName, scheduledTime,
00191                     description, priority, recurring, recurrencePattern,
00192                     rs.getString("food_type"), rs.getDouble("portion_grams"),
00193                     rs.getBoolean("includes_water_change"));
00194                 break;
00195             case "Ilac":
00196                 reminder = new MedicationReminder(id, petId, petName, scheduledTime,
00197                     description, priority, recurring, recurrencePattern,
00198                     rs.getString("medication_name"), rs.getDouble("dosage"),
00199                     rs.getString("dosage_unit"), rs.getBoolean("prescribed"), null);
00200                 break;
00201             case "Egzersiz":
00202                 reminder = new ExerciseReminder(id, petId, petName, scheduledTime,
00203                     description, priority, recurring, recurrencePattern,
00204                     rs.getString("exercise_type"), rs.getInt("duration_minutes"),
00205                     rs.getDouble("distance_km"));
00206                 break;
00207             case "Bakim":
00208                 reminder = new GroomingReminder(id, petId, petName, scheduledTime,
00209                     description, priority, recurring, recurrencePattern,
00210                     rs.getString("grooming_type"), rs.getBoolean("professional"),
00211                     rs.getString("groomer_name"));
00212                 break;
00213             case "Veteriner":
00214                 reminder = new VetAppointment(id, petId, petName, scheduledTime,
00215                     description, priority, recurring, recurrencePattern,
00216                     rs.getString("veterinarian_name"), rs.getString("clinic_name"),
00217                     rs.getString("reason"), rs.getDouble("estimated_cost"),
00218                     rs.getBoolean("confirmed"), rs.getString("clinic_phone"));
00219                 break;
00220             default:
00221                 reminder = new FeedingReminder(id, petId, petName, scheduledTime, "Mama", 100);
00222         }
00223         reminder.setCompleted(completed);
00224         return reminder;
00225     }

```

References `com.mehdi.petreminder.model.Reminder.setCompleted()`.

7.35.3.6 setInsertParams()

`void` `com.mehdi.petreminder.repository.ReminderMySqlRepository.setInsertParams (`
`PreparedStatement ps,`
`Reminder r)` throws `SQLException` [protected]

Member documentation.

Definition at line 126 of file `ReminderMySqlRepository.java`.

```

00126
00127         ps.setInt(1, r.getPetId());

```

```

00128         ps.setString(2, r.getPetName());
00129         ps.setString(3, r.getReminderType());
00130         ps.setObject(4, r.getScheduledTime() != null
00131             ? Timestamp.valueOf(r.getScheduledTime()) : null);
00132         ps.setBoolean(5, r.isCompleted());
00133         ps.setString(6, r.getDescription());
00134         ps.setString(7, r.getPriority());
00135         ps.setBoolean(8, r.isRecurring());
00136         ps.setString(9, r.getRecurrencePattern());
00137         ps.setString(10, r instanceof FeedingReminder ? ((FeedingReminder) r).getFoodType() : null);
00138         ps.setObject(11, r instanceof FeedingReminder ? ((FeedingReminder) r).getPortionGrams() :
00139             null);
00139         ps.setBoolean(12, r instanceof FeedingReminder && ((FeedingReminder)
00140             r).isIncludesWaterChange());
00140         ps.setString(13, r instanceof MedicationReminder ? ((MedicationReminder)
00141             r).getMedicationName() : null);
00141         ps.setObject(14, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosage() :
00142             null);
00142         ps.setString(15, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosageUnit() :
00143             null);
00143         ps.setBoolean(16, r instanceof MedicationReminder && ((MedicationReminder) r).isPrescribed());
00144         ps.setString(17, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getExerciseType() :
00145             null);
00145         ps.setObject(18, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDurationMinutes() :
00146             null);
00146         ps.setObject(19, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDistanceKm() :
00147             null);
00147         ps.setString(20, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomingType() :
00148             null);
00148         ps.setBoolean(21, r instanceof GroomingReminder && ((GroomingReminder) r).isProfessional());
00149         ps.setString(22, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomerName() :
00150             null);
00150         ps.setString(23, r instanceof VetAppointment ? ((VetAppointment) r).getVeterinarianName() :
00151             null);
00151         ps.setString(24, r instanceof VetAppointment ? ((VetAppointment) r).getClinicName() : null);
00152         ps.setString(25, r instanceof VetAppointment ? ((VetAppointment) r).getReason() : null);
00153         ps.setObject(26, r instanceof VetAppointment ? ((VetAppointment) r).getEstimatedCost() :
00154             null);
00154         ps.setBoolean(27, r instanceof VetAppointment && ((VetAppointment) r).isConfirmed());
00155         ps.setString(28, r instanceof VetAppointment ? ((VetAppointment) r).getClinicPhone() : null);
00156     }

```

Referenced by [setUpdateParams\(\)](#).

7.35.3.7 setUpdateParams()

```

void com.mehdi.petreminder.repository.ReminderMySQLRepository.setUpdateParams (
    PreparedStatement ps,
    Reminder r) throws SQLException [protected]

```

Member documentation.

Definition at line 163 of file [ReminderMySQLRepository.java](#).

```

00163         {
00164             setInsertParams(ps, r);
00165             ps.setInt(29, r.getId());
00166         }

```

References [setInsertParams\(\)](#).

The documentation for this class was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/repository/[ReminderMySQLRepository.java](#)

7.36 com.mehdi.petreminder.service.ReminderService Class Reference

[Reminder](#) CRUD ve iş kuralları.

Public Member Functions

- [ReminderService](#) ()
Varsayılan yapıcı.
- [ReminderService](#) ([IRepository](#)< [Reminder](#) > repository)
Test yapıcısı.
- [Reminder](#) [addReminder](#) ([Reminder](#) reminder)
Yeni hatırlatıcı ekler.
- [Reminder](#) [getReminderByld](#) (int id)
ID ile hatırlatıcı getirir.
- List< [Reminder](#) > [getAllReminders](#) ()
Tüm hatırlatıcıları listeler.
- List< [Reminder](#) > [getPendingReminders](#) ()
Bekleyen (tamamlanmamış) hatırlatıcıları getirir.
- List< [Reminder](#) > [getOverdueReminders](#) ()
Gecikmiş hatırlatıcıları getirir.
- List< [Reminder](#) > [getRemindersByPetId](#) (int petId)
Pet'e ait hatırlatıcıları getirir.
- void [markCompleted](#) (int id)
Hatırlatıcıyı tamamlandı olarak işaretler.
- void [updateReminder](#) ([Reminder](#) reminder)
Hatırlatıcı günceller.
- void [deleteReminder](#) (int id)
Hatırlatıcı siler.
- void [deleteAllReminders](#) ()
Tüm hatırlatıcıları siler.
- int [getReminderCount](#) ()
Toplam hatırlatıcı sayısı.
- void [close](#) ()
Repository kapatır.

7.36.1 Detailed Description

[Reminder](#) CRUD ve iş kuralları.

Observer Pattern: [Reminder](#) eklendiğinde, tamamlandığında veya silindiğinde [EventManager](#) üzerinden kayıtlı observer'lara bildirim gönderilir. Böylece Dashboard gibi bileşenler otomatik güncellenir.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 32 of file [ReminderService.java](#).

7.36.2 Constructor & Destructor Documentation

7.36.2.1 ReminderService() [1/2]

com.mehdi.petreminder.service.ReminderService.ReminderService ()

Varsayılan yapıcı.

Definition at line 44 of file [ReminderService.java](#).

```
00044         {
00045             this.repository = RepositoryFactory.createReminderRepository();
00046         }
```

References [com.mehdi.petreminder.repository.RepositoryFactory.createReminderRepository\(\)](#).

7.36.2.2 ReminderService() [2/2]

com.mehdi.petreminder.service.ReminderService.ReminderService (
 [IRepository](#)< [Reminder](#) > repository)

Test yapıcısı.

Parameters

<i>repository</i>	Test repository
-------------------	-----------------

Definition at line 52 of file [ReminderService.java](#).

```
00052         {
00053             this.repository = repository;
00054         }
```

7.36.3 Member Function Documentation

7.36.3.1 addReminder()

[Reminder](#) com.mehdi.petreminder.service.ReminderService.addReminder (
 [Reminder](#) reminder)

Yeni hatırlatıcı ekler.

Parameters

<i>reminder</i>	Eklenecek Reminder
-----------------	------------------------------------

Returns

Oluşturulan [Reminder](#)

Exceptions

ServiceException

Doğrulama başarısız

Definition at line 62 of file [ReminderService.java](#).

```

00062                                     {
00063         validateReminder(reminder);
00064         int id = repository.save(reminder);
00065         logger.info("Reminder eklendi: id={}, type={}", id, reminder.getReminderType());
00066         // Observer Pattern: Reminder eklendi bildirimi
00067         EventManager.getInstance().notify(EventType.REMINDER_ADDED, reminder);
00068         return reminder;
00069     }
```

References [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [com.mehdi.petreminder.model.Reminder.getReminderType\(\)](#), [com.mehdi.petreminder.observer.EventManager.notify\(\)](#), and [com.mehdi.petreminder.observer.EventType.REMINDER_ADDED](#).

7.36.3.2 close()

```
void com.mehdi.petreminder.service.ReminderService.close ()
```

Repository kapatır.

Definition at line 179 of file [ReminderService.java](#).

```

00179         {
00180         repository.close();
00181     }
```

7.36.3.3 deleteAllReminders()

```
void com.mehdi.petreminder.service.ReminderService.deleteAllReminders ()
```

Tüm hatırlatıcıları siler.

Definition at line 164 of file [ReminderService.java](#).

```

00164         {
00165         repository.deleteAll();
00166     }
```

7.36.3.4 deleteReminder()

```
void com.mehdi.petreminder.service.ReminderService.deleteReminder (
    int id)
```

Hatırlatıcı siler.

Parameters

<i>id</i>	Silinecek ID
-----------	--------------

Definition at line 151 of file [ReminderService.java](#).

```
00151         {
00152             boolean deleted = repository.delete(id);
00153             if (!deleted) {
00154                 throw new ServiceException("Reminder silinemedi: id=" + id);
00155             }
00156             logger.info("Reminder silindi: id={}", id);
00157             // Observer Pattern: Reminder silindi bildirimi
00158             EventManager.getInstance().notify(EventType.REMINDER_DELETED, id);
00159         }
```

References [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [com.mehdi.petreminder.observer.EventManager.notify\(\)](#), and [com.mehdi.petreminder.observer.EventType.REMINDER_DELETED](#).

7.36.3.5 getAllReminders()

```
List< Reminder > com.mehdi.petreminder.service.ReminderService.getAllReminders ()
```

Tüm hatırlatıcıları listeler.

Returns

[Reminder](#) listesi

Definition at line 85 of file [ReminderService.java](#).

```
00085         {
00086             return repository.findAll();
00087         }
```

7.36.3.6 getOverdueReminders()

```
List< Reminder > com.mehdi.petreminder.service.ReminderService.getOverdueReminders ()
```

Gecikmiş hatırlatıcıları getirir.

Returns

Overdue reminder listesi

Definition at line 105 of file [ReminderService.java](#).

```
00105         {
00106             return repository.findAll().stream()
00107                 .filter(Reminder::isOverdue)
00108                 .collect(Collectors.toList());
00109         }
```

References [com.mehdi.petreminder.model.Reminder.isOverdue\(\)](#).

7.36.3.7 getPendingReminders()

```
List< Reminder > com.mehdi.petreminder.service.ReminderService.getPendingReminders ()
```

Bekleyen (tamamlanmamış) hatırlatıcıları getirir.

Returns

Pending reminder listesi

Definition at line 93 of file [ReminderService.java](#).

```
00093         {
00094             return repository.findAll().stream()
00095                 .filter(r -> !r.isCompleted())
00096                 .sorted(Comparator.comparing(r -> r.getScheduledTime() != null
00097                     ? r.getScheduledTime() : LocalDateTime.MAX))
00098                 .collect(Collectors.toList());
00099         }
```

7.36.3.8 getReminderById()

```
Reminder com.mehdi.petreminder.service.ReminderService.getReminderById (
    int id)
```

ID ile hatırlatıcı getirir.

Parameters

<i>id</i>	Reminder ID
-----------	-------------

Returns

Reminder nesnesi

Definition at line 76 of file [ReminderService.java](#).

```
00076                                     {
00077         return repository.findById(id)
00078             .orElseThrow(() -> new ServiceException("Reminder bulunamadı: id=" + id));
00079     }
```

Referenced by [markCompleted\(\)](#).

7.36.3.9 getReminderCount()

```
int com.mehdi.petreminder.service.ReminderService.getReminderCount ()
```

Toplam hatırlatıcı sayısı.

Returns

Sayı

Definition at line 172 of file [ReminderService.java](#).

```
00172                                     {
00173         return repository.count();
00174     }
```

7.36.3.10 getRemindersByPetId()

```
List< Reminder > com.mehdi.petreminder.service.ReminderService.getRemindersByPetId (
    int petId)
```

Pet'e ait hatırlatıcıları getirir.

Parameters

<i>pet↔ Id</i>	Pet ID
--------------------	--------

Returns

Reminder listesi

Definition at line 116 of file [ReminderService.java](#).

```
00116                                     {
00117         return repository.findAll().stream()
00118             .filter(r -> r.getPetId() == petId)
00119             .collect(Collectors.toList());
00120     }
```


7.36.3.11 markCompleted()

```
void com.mehdi.petreminder.service.ReminderService.markCompleted (
    int id)
```

Hatırlatıcıyı tamamlandı olarak işaretler.

Parameters

<i>id</i>	Reminder ID
-----------	-------------

Definition at line 126 of file [ReminderService.java](#).

```
00126      {
00127          Reminder r = getReminderById(id);
00128          r.setCompleted(true);
00129          repository.update(r);
00130          logger.info("Reminder tamamlandı id={}", id);
00131          // Observer Pattern: Reminder tamamlandı bildirimi
00132          EventManager.getInstance().notify(EventType.REMINDER_COMPLETED, r);
00133      }
```

References [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [getReminderById\(\)](#), [com.mehdi.petreminder.observer.EventManager.notify\(\)](#), [com.mehdi.petreminder.observer.EventType.REMINDER_COMPLETED](#), and [com.mehdi.petreminder.model.Reminder.setCompleted\(\)](#).

7.36.3.12 updateReminder()

```
void com.mehdi.petreminder.service.ReminderService.updateReminder (
    Reminder reminder)
```

Hatırlatıcı günceller.

Parameters

<i>reminder</i>	Güncellenecek Reminder
-----------------	--

Definition at line 139 of file [ReminderService.java](#).

```
00139      {
00140          validateReminder(reminder);
00141          boolean updated = repository.update(reminder);
00142          if (!updated) {
00143              throw new ServiceException("Reminder güncellenemedi: id=" + reminder.getId());
00144          }
00145      }
```

References [com.mehdi.petreminder.model.Reminder.getId\(\)](#).

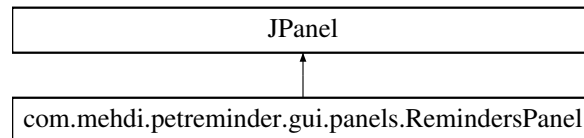
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/service/ReminderService.java](#)

7.37 com.mehdi.petreminder.gui.panels.RemindersPanel Class Reference

Hatırlatıcılar CRUD paneli.

Inheritance diagram for com.mehdi.petreminder.gui.panels.RemindersPanel:



Public Member Functions

- [RemindersPanel](#) ([MainFrame](#) mainFrame)
[RemindersPanel](#) yapıcısı.
- void [refresh](#) ()
Tabloyu yeniler.

7.37.1 Detailed Description

Hatırlatıcılar CRUD paneli.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 28 of file [RemindersPanel.java](#).

7.37.2 Constructor & Destructor Documentation

7.37.2.1 RemindersPanel()

```
com.mehdi.petreminder.gui.panels.RemindersPanel.RemindersPanel (  
    MainFrame mainFrame)
```

[RemindersPanel](#) yapıcısı.

Parameters

<i>mainFrame</i>	Ana pencere
------------------	-------------

Definition at line 52 of file [RemindersPanel.java](#).

```
00052      {
00053          this.mainFrame      = mainFrame;
00054          this.reminderService = new ReminderService();
00055          setBackground(GuiConstants.BG_COLOR);
00056          setLayout(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));
00057          setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,
00058                                   GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));
00059          buildUI();
00060      }
```

References [com.mehdi.petreminder.gui.util.GuiConstants.BG_COLOR](#), [com.mehdi.petreminder.gui.util.GuiConstants.GAP](#), and [com.mehdi.petreminder.gui.util.GuiConstants.GAP_LARGE](#).

7.37.3 Member Function Documentation

7.37.3.1 refresh()

```
void com.mehdi.petreminder.gui.panels.RemindersPanel.refresh ()
```

Tabloyu yeniler.

Definition at line 204 of file [RemindersPanel.java](#).

```
00204      {
00205          tableModel.setRowCount(0);
00206          try {
00207              List<Reminder> list = chkOnlyPending != null && chkOnlyPending.isSelected()
00208                  ? reminderService.getPendingReminders()
00209                  : reminderService.getAllReminders();
00210              for (Reminder r : list) {
00211                  String status = r.isCompleted() ? " Tamamlandı"
00212                      : r.isOverdue() ? " Gecikmiş" : " Bekliyor";
00213                  tableModel.addRow(new Object[]{
00214                      r.getId(), r.getPetName(), r.getReminderType(),
00215                      r.getPriority(), r.getFormattedTime(), status
00216                  });
00217              }
00218          } catch (Exception e) { /* sessiz */ }
00219      }
```

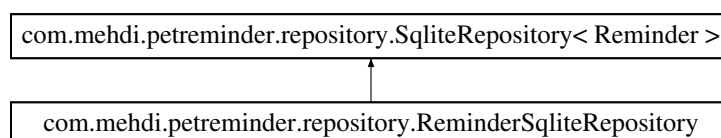
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/RemindersPanel.java](#)

7.38 com.mehdi.petreminder.repository.ReminderSqliteRepository Class Reference

SQLite üzerinde [Reminder](#) (tüm alt türler) CRUD işlemleri.

Inheritance diagram for com.mehdi.petreminder.repository.ReminderSqliteRepository:



Public Member Functions

- [ReminderSqliteRepository](#) ()
Varsayılan yapıcı.
- [ReminderSqliteRepository](#) (Connection [connection](#))
Test yapıcısı.
- List< [Reminder](#) > [findByPetId](#) (int petId)
PetId'ye göre hatırlatıcıları getirir.

Public Member Functions inherited from

[com.mehdi.petreminder.repository.SqliteRepository](#)< [Reminder](#) >

- [SqliteRepository](#) ()
SQLite bağlantısını açar ve tabloyu oluşturur.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür (test için).

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [Reminder](#) r) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [Reminder](#) r) throws SQLException
- [Reminder](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from

[com.mehdi.petreminder.repository.SqliteRepository](#)< [Reminder](#) >

- abstract String [getTableName](#) ()
Tablo adını döndürür.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL'ini döndürür.
- abstract String [getInsertSql](#) ()
INSERT SQL'ini döndürür (ID hariç).
- abstract String [getUpdateSql](#) ()
UPDATE SQL'ini döndürür.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members

Protected Attributes inherited from

[com.mehdi.petreminder.repository.SqliteRepository](#)< [Reminder](#) >

- Connection [connection](#)
SQLite JDBC bağlantısı.

Static Protected Attributes inherited from

[com.mehdi.petreminder.repository.SqliteRepository](#)< [Reminder](#) >

- static final Logger [logger](#)
Logger.

7.38.1 Detailed Description

SQLite üzerinde [Reminder](#) (tüm alt türler) CRUD işlemleri.

Tüm [Reminder](#) alt sınıfları tek tabloda (reminder_type sütunu ile) saklanır.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 24 of file [ReminderSqliteRepository.java](#).

7.38.2 Constructor & Destructor Documentation

7.38.2.1 ReminderSqliteRepository() [1/2]

```
com.mehdi.petreminder.repository.ReminderSqliteRepository.ReminderSqliteRepository ()
```

Varsayılan yapıcı.

Definition at line 29 of file [ReminderSqliteRepository.java](#).

```
00029                                     {  
00030     super ();  
00031 }
```

7.38.2.2 ReminderSqliteRepository() [2/2]

```
com.mehdi.petreminder.repository.ReminderSqliteRepository.ReminderSqliteRepository (
    Connection connection)
```

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 37 of file [ReminderSqliteRepository.java](#).

```
00037                                     {
00038         super(connection);
00039     }
```

References [com.mehdi.petreminder.repository.SqliteRepository< Reminder >.connection](#).

7.38.3 Member Function Documentation

7.38.3.1 findByPetId()

```
List< Reminder > com.mehdi.petreminder.repository.ReminderSqliteRepository.findByPetId (
    int petId)
```

PetId'ye göre hatırlatıcıları getirir.

Parameters

<i>petId</i>	Pet ID
--------------	--------

Returns

Hatırlatıcı listesi

Definition at line 244 of file [ReminderSqliteRepository.java](#).

```
00244                                     {
00245         List<Reminder> list = new ArrayList<>();
00246         try (PreparedStatement ps = connection.prepareStatement(
00247             "SELECT * FROM reminders WHERE pet_id = ?")) {
00248             ps.setInt(1, petId);
00249             ResultSet rs = ps.executeQuery();
00250             while (rs.next()) list.add(mapRow(rs));
00251         } catch (SQLException e) {
00252             throw new RepositoryException("findByPetId hatası", e);
00253         }
00254         return list;
00255     }
```

References [com.mehdi.petreminder.repository.SqliteRepository< Reminder >.connection](#), and [mapRow\(\)](#).

7.38.3.2 getCreateTableSql()

String com.mehdi.petreminder.repository.ReminderSqliteRepository.getCreateTableSql () [protected]

Member documentation.

Definition at line 55 of file [ReminderSqliteRepository.java](#).

```

00055         {
00056             return "CREATE TABLE IF NOT EXISTS reminders ("
00057                 + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00058                 + "pet_id INTEGER NOT NULL,"
00059                 + "pet_name TEXT,"
00060                 + "reminder_type TEXT NOT NULL,"
00061                 + "scheduled_time TEXT NOT NULL,"
00062                 + "completed INTEGER DEFAULT 0,"
00063                 + "description TEXT,"
00064                 + "priority TEXT DEFAULT 'ORTA',"
00065                 + "recurring INTEGER DEFAULT 0,"
00066                 + "recurrence_pattern TEXT,"
00067                 + "food_type TEXT,"
00068                 + "portion_grams REAL,"
00069                 + "includes_water_change INTEGER DEFAULT 0,"
00070                 + "medication_name TEXT,"
00071                 + "dosage REAL,"
00072                 + "dosage_unit TEXT,"
00073                 + "prescribed INTEGER DEFAULT 0,"
00074                 + "exercise_type TEXT,"
00075                 + "duration_minutes INTEGER,"
00076                 + "distance_km REAL,"
00077                 + "grooming_type TEXT,"
00078                 + "professional INTEGER DEFAULT 0,"
00079                 + "groomer_name TEXT,"
00080                 + "veterinarian_name TEXT,"
00081                 + "clinic_name TEXT,"
00082                 + "reason TEXT,"
00083                 + "estimated_cost REAL,"
00084                 + "confirmed INTEGER DEFAULT 0,"
00085                 + "clinic_phone TEXT"
00086                 + ");";
00087         }

```

7.38.3.3 getInsertSql()

String com.mehdi.petreminder.repository.ReminderSqliteRepository.getInsertSql () [protected]

Member documentation.

Definition at line 94 of file [ReminderSqliteRepository.java](#).

```

00094         {
00095             return "INSERT INTO reminders (pet_id,pet_name,reminder_type,scheduled_time,"
00096                 + "completed,description,priority,recurring,recurrence_pattern,"
00097                 + "food_type,portion_grams,includes_water_change,"
00098                 + "medication_name,dosage,dosage_unit,prescribed,"
00099                 + "exercise_type,duration_minutes,distance_km,"
00100                 + "grooming_type,professional,groomer_name,"
00101                 + "veterinarian_name,clinic_name,reason,estimated_cost,confirmed,clinic_phone) "
00102                 + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00103         }

```

7.38.3.4 getTableName()

String com.mehdi.petreminder.repository.ReminderSqliteRepository.getTableName () [protected]

Member documentation.

Definition at line 46 of file [ReminderSqliteRepository.java](#).

```

00046         {
00047             return "reminders";
00048         }

```

7.38.3.5 getUpdateSql()

String com.mehdi.petreminder.repository.ReminderSqliteRepository.getUpdateSql () [protected]

Member documentation.

Definition at line 110 of file [ReminderSqliteRepository.java](#).

```
00110         {
00111             return "UPDATE reminders SET pet_id=?,pet_name=?,reminder_type=?,scheduled_time=,"
00112                 + "completed=?,description=?,priority=?,recurring=?,recurrence_pattern=,"
00113                 + "food_type=?,portion_grams=?,includes_water_change=,"
00114                 + "medication_name=?,dosage=?,dosage_unit=?,prescribed=,"
00115                 + "exercise_type=?,duration_minutes=?,distance_km=,"
00116                 + "grooming_type=?,professional=?,groomer_name=,"
00117                 + "veterinarian_name=?,clinic_name=?,reason=?,estimated_cost=?,confirmed=,"
00118                 + "clinic_phone=? WHERE id=?";
00119         }
```

7.38.3.6 mapRow()

[Reminder](#) com.mehdi.petreminder.repository.ReminderSqliteRepository.mapRow (
 ResultSet rs) throws SQLException [protected]

Member documentation.

Definition at line 179 of file [ReminderSqliteRepository.java](#).

```
00179         {
00180             String type = rs.getString("reminder_type");
00181             int id = rs.getInt("id");
00182             int petId = rs.getInt("pet_id");
00183             String petName = rs.getString("pet_name");
00184             String timeStr = rs.getString("scheduled_time");
00185             LocalDateTime scheduledTime = timeStr != null ? LocalDateTime.parse(timeStr) : null;
00186             String description = rs.getString("description");
00187             String priority = rs.getString("priority");
00188             boolean recurring = rs.getInt("recurring") == 1;
00189             String recurrencePattern = rs.getString("recurrence_pattern");
00190             boolean completed = rs.getInt("completed") == 1;
00191
00192             Reminder reminder;
00193             switch (type) {
00194                 case "Besleme":
00195                     FeedingReminder fr = new FeedingReminder(id, petId, petName, scheduledTime,
00196                         description, priority, recurring, recurrencePattern,
00197                         rs.getString("food_type"), rs.getDouble("portion_grams"),
00198                         rs.getInt("includes_water_change") == 1);
00199                     reminder = fr;
00200                     break;
00201                 case "Ilac":
00202                     MedicationReminder mr = new MedicationReminder(id, petId, petName, scheduledTime,
00203                         description, priority, recurring, recurrencePattern,
00204                         rs.getString("medication_name"), rs.getDouble("dosage"),
00205                         rs.getString("dosage_unit"), rs.getInt("prescribed") == 1, null);
00206                     reminder = mr;
00207                     break;
00208                 case "Egzersiz":
00209                     ExerciseReminder er = new ExerciseReminder(id, petId, petName, scheduledTime,
00210                         description, priority, recurring, recurrencePattern,
00211                         rs.getString("exercise_type"), rs.getInt("duration_minutes"),
00212                         rs.getDouble("distance_km"));
00213                     reminder = er;
00214                     break;
00215                 case "Bakim":
00216                     GroomingReminder gr = new GroomingReminder(id, petId, petName, scheduledTime,
00217                         description, priority, recurring, recurrencePattern,
00218                         rs.getString("grooming_type"), rs.getInt("professional") == 1,
00219                         rs.getString("groomer_name"));
00220                     reminder = gr;
00221                     break;
00222                 case "Veteriner":
00223                     VetAppointment va = new VetAppointment(id, petId, petName, scheduledTime,
00224                         description, priority, recurring, recurrencePattern,
00225                         rs.getString("veterinarian_name"), rs.getString("clinic_name"),
00226                         rs.getString("reason"), rs.getDouble("estimated_cost"),
00227                         rs.getInt("confirmed") == 1, rs.getString("clinic_phone"));
```



```

00228         reminder = va;
00229         break;
00230     default:
00231         FeedingReminder def = new FeedingReminder(id, petId, petName, scheduledTime,
00232             "Mama", 100);
00233         reminder = def;
00234     }
00235     reminder.setCompleted(completed);
00236     return reminder;
00237 }

```

References [com.mehdi.petreminder.model.Reminder.setCompleted\(\)](#).

Referenced by [findByPetId\(\)](#).

7.38.3.7 setInsertParams()

```

void com.mehdi.petreminder.repository.ReminderSqliteRepository.setInsertParams (
    PreparedStatement ps,
    Reminder r) throws SQLException [protected]

```

Member documentation.

Definition at line 127 of file [ReminderSqliteRepository.java](#).

```

00127     {
00128         ps.setInt(1, r.getPetId());
00129         ps.setString(2, r.getPetName());
00130         ps.setString(3, r.getReminderType());
00131         ps.setString(4, r.getScheduledTime() != null ? r.getScheduledTime().toString() : null);
00132         ps.setInt(5, r.isCompleted() ? 1 : 0);
00133         ps.setString(6, r.getDescription());
00134         ps.setString(7, r.getPriority());
00135         ps.setInt(8, r.isRecurring() ? 1 : 0);
00136         ps.setString(9, r.getRecurrencePattern());
00137         // FeedingReminder
00138         ps.setString(10, r instanceof FeedingReminder ? ((FeedingReminder) r).getFoodType() : null);
00139         ps.setObject(11, r instanceof FeedingReminder ? ((FeedingReminder) r).getPortionGrams() :
null);
00140         ps.setInt(12, r instanceof FeedingReminder && ((FeedingReminder) r).isIncludesWaterChange() ?
1 : 0);
00141         // MedicationReminder
00142         ps.setString(13, r instanceof MedicationReminder ? ((MedicationReminder)
r).getMedicationName() : null);
00143         ps.setObject(14, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosage() :
null);
00144         ps.setString(15, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosageUnit() :
null);
00145         ps.setInt(16, r instanceof MedicationReminder && ((MedicationReminder) r).isPrescribed() ? 1 :
0);
00146         // ExerciseReminder
00147         ps.setString(17, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getExerciseType() :
null);
00148         ps.setObject(18, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDurationMinutes() :
null);
00149         ps.setObject(19, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDistanceKm() :
null);
00150         // GroomingReminder
00151         ps.setString(20, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomingType() :
null);
00152         ps.setInt(21, r instanceof GroomingReminder && ((GroomingReminder) r).isProfessional() ? 1 :
0);
00153         ps.setString(22, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomerName() :
null);
00154         // VetAppointment
00155         ps.setString(23, r instanceof VetAppointment ? ((VetAppointment) r).getVeterinarianName() :
null);
00156         ps.setString(24, r instanceof VetAppointment ? ((VetAppointment) r).getClinicName() : null);
00157         ps.setString(25, r instanceof VetAppointment ? ((VetAppointment) r).getReason() : null);
00158         ps.setObject(26, r instanceof VetAppointment ? ((VetAppointment) r).getEstimatedCost() :
null);
00159         ps.setInt(27, r instanceof VetAppointment && ((VetAppointment) r).isConfirmed() ? 1 : 0);
00160         ps.setString(28, r instanceof VetAppointment ? ((VetAppointment) r).getClinicPhone() : null);
00161     }

```

Referenced by [setUpUpdateParams\(\)](#).

7.38.3.8 setUpdateParams()

```
void com.mehdi.petreminder.repository.ReminderSqliteRepository.setUpdateParams (
    PreparedStatement ps,
    Reminder r) throws SQLException [protected]
```

Member documentation.

Definition at line 168 of file [ReminderSqliteRepository.java](#).

```
00168                                     {
00169         setInsertParams(ps, r);
00170         ps.setInt(29, r.getId());
00171     }
```

References [setInsertParams\(\)](#).

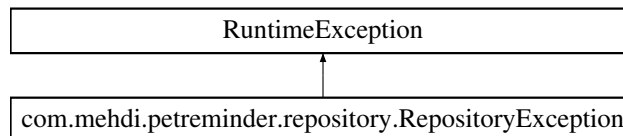
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/ReminderSqliteRepository.java](#)

7.39 com.mehdi.petreminder.repository.RepositoryException Class Reference

Repository işlemlerinde fırlatılan unchecked istisna.

Inheritance diagram for com.mehdi.petreminder.repository.RepositoryException:



Public Member Functions

- [RepositoryException](#) (String message)
Mesajlı yapıcı.
- [RepositoryException](#) (String message, Throwable cause)
Mesaj ve neden'li yapıcı.

7.39.1 Detailed Description

Repository işlemlerinde fırlatılan unchecked istisna.

OOP: RuntimeException türetilmiş özel istisna.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 16 of file [RepositoryException.java](#).

7.39.2 Constructor & Destructor Documentation

7.39.2.1 RepositoryException() [1/2]

```
com.mehdi.petreminder.repository.RepositoryException.RepositoryException (
    String message)
```

Mesajlı yapıcı.

Parameters

<i>message</i>	Hata mesajı
----------------	-------------

Definition at line 25 of file [RepositoryException.java](#).

```
00025                                     {
00026         super(message);
00027     }
```

7.39.2.2 RepositoryException() [2/2]

```
com.mehdi.petreminder.repository.RepositoryException.RepositoryException (
    String message,
    Throwable cause)
```

Mesaj ve neden'li yapıcı.

Parameters

<i>message</i>	Hata mesajı
<i>cause</i>	Altta yatan istisna

Definition at line 34 of file [RepositoryException.java](#).

```
00034                                     {
00035         super(message, cause);
00036     }
```

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/RepositoryException.java](#)

7.40 com.mehdi.petreminder.repository.RepositoryFactory Class Reference

Aktif storage backend'e göre repository üretir.

Static Public Member Functions

- static [IRepository< Pet > createPetRepository \(\)](#)
Aktif backend için [Pet](#) repository üretir.
- static [IRepository< Reminder > createReminderRepository \(\)](#)
Aktif backend için [Reminder](#) repository üretir.
- static [IRepository< User > createUserRepository \(\)](#)
Aktif backend için [User](#) repository üretir.
- static [IRepository< MedicalRecord > createMedicalRecordRepository \(\)](#)
Aktif backend için [MedicalRecord](#) repository üretir.
- static [StorageType getActiveType \(\)](#)
Aktif backend tipini döndürür.

7.40.1 Detailed Description

Aktif storage backend'e göre repository üretir.

PDF zorunluluğu: [RepositoryFactory](#) pattern — runtime switch. Factory Method Pattern: [createPetRepository\(\)](#), [createReminderRepository\(\)](#) vb. [StorageConfig.getActiveBackend\(\)](#) 'a göre doğru implementasyonu döner.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 27 of file [RepositoryFactory.java](#).

7.40.2 Member Function Documentation

7.40.2.1 createMedicalRecordRepository()

```
IRepository< MedicalRecord > com.mehdi.petreminder.repository.RepositoryFactory.createMedicalRecordRepository () [static]
```

Aktif backend için [MedicalRecord](#) repository üretir.

Returns

[IRepository<MedicalRecord>](#) implementasyonu

Definition at line 93 of file [RepositoryFactory.java](#).

```
00093                                     {
00094         StorageType type = StorageConfig.getActiveBackend();
00095         logger.info("MedicalRecord repository oluşturuluyor: {}", type);
00096         switch (type) {
00097             case SQLITE: return new MedicalRecordSqliteRepository();
00098             case MYSQL:  return new MedicalRecordMySQLRepository();
00099             case BINARY: return new BinaryRepository<>(BINARY_DIR, "medical_records");
00100             default:     return new BinaryRepository<>(BINARY_DIR, "medical_records");
00101         }
00102     }
```

References [com.mehdi.petreminder.config.StorageConfig.getActiveBackend\(\)](#).

Referenced by [com.mehdi.petreminder.service.MedicalRecordService.MedicalRecordService\(\)](#).

7.40.2.2 createPetRepository()

```
IRepository< Pet > com.mehdi.petreminder.repository.RepositoryFactory.createPetRepository ()
[static]
```

Aktif backend için [Pet](#) repository üretir.

Returns

[IRepository<Pet>](#) implementasyonu

Exceptions

RepositoryException	Backend başlatılamazsa
-------------------------------------	------------------------

Definition at line 45 of file [RepositoryFactory.java](#).

```

00045                                     {
00046         StorageType type = StorageConfig.getActiveBackend();
00047         logger.info("Pet repository oluşturuluyor: {}", type);
00048         switch (type) {
00049             case SQLITE: return new PetSqliteRepository();
00050             case MYSQL:  return new PetMySQLRepository();
00051             case BINARY:
00052                 return new BinaryRepository<>(BINARY_DIR, "pets");
00053         }
00054     }
```

References [com.mehdi.petreminder.config.StorageConfig.getActiveBackend\(\)](#).

Referenced by [com.mehdi.petreminder.service.PetService.PetService\(\)](#).

7.40.2.3 createReminderRepository()

```
IRepository< Reminder > com.mehdi.petreminder.repository.RepositoryFactory.createReminder↵
Repository () [static]
```

Aktif backend için [Reminder](#) repository üretir.

Returns

[IRepository<Reminder>](#) implementasyonu

Definition at line 61 of file [RepositoryFactory.java](#).

```

00061                                     {
00062         StorageType type = StorageConfig.getActiveBackend();
00063         logger.info("Reminder repository oluşturuluyor: {}", type);
00064         switch (type) {
00065             case SQLITE: return new ReminderSqliteRepository();
00066             case MYSQL:  return new ReminderMySQLRepository();
00067             case BINARY:
00068                 return new BinaryRepository<>(BINARY_DIR, "reminders");
00069         }
00070     }
```

References [com.mehdi.petreminder.config.StorageConfig.getActiveBackend\(\)](#).

Referenced by [com.mehdi.petreminder.service.ReminderService.ReminderService\(\)](#).

7.40.2.4 createUserRepository()

`IRepository< User > com.mehdi.petreminder.repository.RepositoryFactory.createUserRepository ()`
[static]

Aktif backend için `User` repository üretir.

Returns

`IRepository<User>` implementasyonu

Definition at line 77 of file `RepositoryFactory.java`.

```
00077                                     {
00078         StorageType type = StorageConfig.getActiveBackend();
00079         logger.info("User repository oluşturuluyor: {}", type);
00080         switch (type) {
00081             case SQLITE: return new UserSqliteRepository();
00082             case MYSQL:  return new UserMySqlRepository();
00083             case BINARY:
00084                 default: return new BinaryRepository<>(BINARY_DIR, "users");
00085         }
00086     }
```

References `com.mehdi.petreminder.config.StorageConfig.getActiveBackend()`.

7.40.2.5 getActiveType()

`StorageType com.mehdi.petreminder.repository.RepositoryFactory.getActiveType ()` [static]

Aktif backend tipini döndürür.

Returns

Aktif `StorageType`

Definition at line 108 of file `RepositoryFactory.java`.

```
00108                                     {
00109         return StorageConfig.getActiveBackend();
00110     }
```

References `com.mehdi.petreminder.config.StorageConfig.getActiveBackend()`.

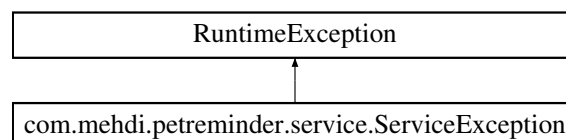
The documentation for this class was generated from the following file:

- `petreminder-app/src/main/java/com/mehdi/petreminder/repository/RepositoryFactory.java`

7.41 com.mehdi.petreminder.service.ServiceException Class Reference

Servis katmanında fırlatılan unchecked istisna.

Inheritance diagram for `com.mehdi.petreminder.service.ServiceException`:



Public Member Functions

- [ServiceException](#) (String message)
Mesajlı yapıcı.
- [ServiceException](#) (String message, Throwable cause)
Mesaj ve neden'li yapıcı.

7.41.1 Detailed Description

Servis katmanında fırlatılan unchecked istisna.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 15 of file [ServiceException.java](#).

7.41.2 Constructor & Destructor Documentation

7.41.2.1 ServiceException() [1/2]

```
com.mehdi.petreminder.service.ServiceException.ServiceException (  
    String message)
```

Mesajlı yapıcı.

Parameters

<i>message</i>	Hata mesajı
----------------	-------------

Definition at line 24 of file [ServiceException.java](#).

```
00024      {  
00025          super (message);  
00026      }
```

7.41.2.2 ServiceException() [2/2]

```
com.mehdi.petreminder.service.ServiceException.ServiceException (
    String message,
    Throwable cause)
```

Mesaj ve neden'li yapıcı.

Parameters

<i>message</i>	Hata mesajı
<i>cause</i>	Neden

Definition at line 33 of file [ServiceException.java](#).

```
00033                                     {
00034         super(message, cause);
00035     }
```

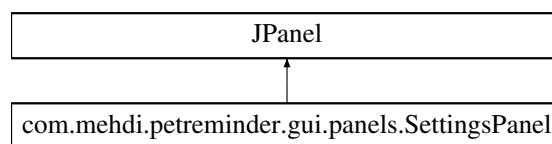
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/service/ServiceException.java](#)

7.42 com.mehdi.petreminder.gui.panels.SettingsPanel Class Reference

Storage backend ve uygulama ayarları.

Inheritance diagram for com.mehdi.petreminder.gui.panels.SettingsPanel:



Public Member Functions

- [SettingsPanel](#) ([MainFrame](#) mainFrame)
SettingsPanel yapıcısı.
- void [refresh](#) ()
Paneli yeniler.

7.42.1 Detailed Description

Storage backend ve uygulama ayarları.

PDF zorunluluğu: Runtime storage switching — no restart required.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 25 of file [SettingsPanel.java](#).

7.42.2 Constructor & Destructor Documentation

7.42.2.1 SettingsPanel()

com.mehdi.petreminder.gui.panels.SettingsPanel.SettingsPanel (
[MainFrame](#) *mainFrame*)

[SettingsPanel](#) yapıcısı.

Parameters

<i>mainFrame</i>	Ana pencere
------------------	-------------

Definition at line 43 of file [SettingsPanel.java](#).

```
00043                                     {
00044         this.mainFrame = mainFrame;
00045         setBackground(GuiConstants.BG\_COLOR);
00046         setLayout(new BorderLayout());
00047         setBorder(new EmptyBorder(GuiConstants.GAP\_LARGE, GuiConstants.GAP\_LARGE,
00048             GuiConstants.GAP\_LARGE, GuiConstants.GAP\_LARGE));
00049         buildUI();
00050     }
```

References [com.mehdi.petreminder.gui.util.GuiConstants.BG_COLOR](#), and [com.mehdi.petreminder.gui.util.GuiConstants.GAP_LARGE](#).

7.42.3 Member Function Documentation

7.42.3.1 refresh()

void com.mehdi.petreminder.gui.panels.SettingsPanel.refresh ()

Paneli yeniler.

Definition at line 178 of file [SettingsPanel.java](#).

```
00178         {
00179         if (lblCurrent != null) {
00180             lblCurrent.setText(StorageConfig.getActiveBackend\(\).getDisplayName\(\));
00181         }
00182         if (cmbStorage != null) {
00183             cmbStorage.setSelectedItem(StorageConfig.getActiveBackend\(\));
00184         }
00185     }
```

References [com.mehdi.petreminder.config.StorageConfig.getActiveBackend\(\)](#), and [com.mehdi.petreminder.config.StorageType.getDisplayName\(\)](#).

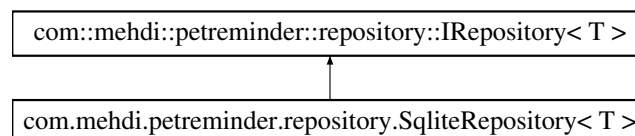
The documentation for this class was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/[SettingsPanel.java](#)

7.43 com.mehdi.petreminder.repository.SQLiteRepository< T > Class Template Reference

SQLite tabanlı abstract repository temel sınıfı.

Inheritance diagram for com.mehdi.petreminder.repository.SQLiteRepository< T >:



Public Member Functions

- [SQLiteRepository](#) ()
SQLite bağlantısını açar ve tabloyu oluşturur.
- [SQLiteRepository](#) (Connection [connection](#))
Test amaçlı yapıcı — dışarıdan bağlantı verilir.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür (test için).

Protected Member Functions

- abstract String [getTableName](#) ()
Tablo adını döndürür.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL'ini döndürür.
- abstract String [getInsertSql](#) ()
INSERT SQL'ini döndürür (ID hariç).
- abstract String [getUpdateSql](#) ()
UPDATE SQL'ini döndürür.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Protected Attributes

- Connection [connection](#)
SQLite JDBC bağlantısı.

Static Protected Attributes

- static final Logger [logger](#)
Logger.

7.43.1 Detailed Description

SQLite tabanlı abstract repository temel sınıfı.

Alt sınıflar ([PetSqliteRepository](#) vb.) bu sınıfı extend eder ve `mapRow` / `getTableName` / `getInsertSql` / `getUpdateSql` metodlarını implemente eder. Template Method Pattern kullanılmıştır.

Parameters

< T >	Varlık türü
-------	-------------

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 27 of file [SqliteRepository.java](#).

7.43.2 Constructor & Destructor Documentation

7.43.2.1 SqliteRepository() [1/2]

```
com.mehdi.petreminder.repository.SqliteRepository< T >.SqliteRepository ()
```

SQLite bağlantısını açar ve tabloyu oluşturur.

Exceptions

RepositoryException	Bağlantı başarısız olursa
-------------------------------------	---------------------------

Definition at line 40 of file [SqliteRepository.java](#).

```
00040         {
00041             try {
00042                 Class.forName("org.sqlite.JDBC");
00043                 String url = "jdbc:sqlite:" + StorageConfig.getSqliteFilePath();
00044                 connection = DriverManager.getConnection(url);
00045                 connection.setAutoCommit(true);
00046                 createTableIfNotExists();
00047                 logger.info("SQLite bağlantısı açıldı: {}", StorageConfig.getSqliteFilePath());
00048             } catch (Exception e) {
00049                 throw new RepositoryException("SQLite bağlantı hatası", e);
00050             }
00051         }
```

References [connection](#), [createTableIfNotExists\(\)](#), [com.mehdi.petreminder.config.StorageConfig.getSqliteFilePath\(\)](#), and [logger](#).

7.43.2.2 SQLiteRepository() [2/2]

```
com.mehdi.petreminder.repository.SQLiteRepository< T >.SQLiteRepository (
    Connection connection)
```

Test amaçlı yapıcı — dışarıdan bağlantı verilir.

Parameters

<i>connection</i>	Test bağlantısı (H2 veya SQLite in-memory)
-------------------	--

Definition at line 57 of file [SQLiteRepository.java](#).

```
00057
00058         this.connection = connection;
00059         try {
00060             createTableIfNotExists();
00061         } catch (Exception e) {
00062             throw new RepositoryException("Tablo oluşturma hatası", e);
00063         }
00064     }
```

References [connection](#), and [createTableIfNotExists\(\)](#).

7.43.3 Member Function Documentation

7.43.3.1 close()

```
void com.mehdi.petreminder.repository.SQLiteRepository< T >.close ()
```

Repository bağlantısını/kaynaklarını kapatır.

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 253 of file [SQLiteRepository.java](#).

```
00253
00254         try {
00255             if (connection != null && !connection.isClosed()) {
00256                 connection.close();
00257                 logger.info("SQLite bağlantısı kapatıldı.");
00258             }
00259         } catch (SQLException e) {
00260             logger.warn("SQLite kapatma hatası: {}", e.getMessage());
00261         }
00262     }
```

References [connection](#), and [logger](#).

7.43.3.2 count()

```
int com.mehdi.petreminder.repository.SQLiteRepository< T >.count ()
```

Varlık sayısını döndürür.

Returns

Toplam kayıt sayısı

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 238 of file [SQLiteRepository.java](#).

```
00238
00239         String sql = "SELECT COUNT(*) FROM " + getTableName();
00240         try (Statement st = connection.createStatement();
00241             ResultSet rs = st.executeQuery(sql)) {
00242             if (rs.next()) return rs.getInt(1);
00243         } catch (SQLException e) {
00244             throw new RepositoryException("SQLite count hatası", e);
00245         }
00246         return 0;
00247     }
```

References [connection](#), and [getTableName\(\)](#).

7.43.3.3 createTableIfNotExists()

```
void com.mehdi.petreminder.repository.SQLiteRepository< T >.createTableIfNotExists () [protected]
```

Tabloyu oluşturur (yoksa).

Definition at line 119 of file [SQLiteRepository.java](#).

```
00119         {
00120             try (Statement st = connection.createStatement()) {
00121                 String sql = getCreateTableSql();
00122                 // H2 does not support SQLite's AUTOINCREMENT; use AUTO_INCREMENT instead
00123                 String dbProduct = connection.getMetaData().getDatabaseProductName();
00124                 if ("H2".equalsIgnoreCase(dbProduct)) {
00125                     sql = sql.replace("AUTOINCREMENT", "AUTO_INCREMENT");
00126                 }
00127                 st.execute(sql);
00128             } catch (SQLException e) {
00129                 throw new RepositoryException("Tablo oluşturma hatası", e);
00130             }
00131         }
```

References [connection](#), and [getCreateTableSql\(\)](#).

Referenced by [SQLiteRepository\(\)](#), and [SQLiteRepository\(\)](#).

7.43.3.4 delete()

```
boolean com.mehdi.petreminder.repository.SQLiteRepository< T >.delete (
    int id)
```

ID'ye göre varlık siler.

Parameters

<i>id</i>	Silinecek varlığın ID'si
-----------	--------------------------

Returns

Silme başarılıysa true

Exceptions

RepositoryException	Silme başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 211 of file [SQLiteRepository.java](#).

```
00211         {
00212             String sql = "DELETE FROM " + getTableName() + " WHERE id = ?";
00213             try (PreparedStatement ps = connection.prepareStatement(sql)) {
00214                 ps.setInt(1, id);
00215                 int rows = ps.executeUpdate();
00216                 return rows > 0;
00217             } catch (SQLException e) {
00218                 throw new RepositoryException("SQLite delete hatası", e);
00219             }
00220         }
```

References [connection](#), and [getTableName\(\)](#).

7.43.3.5 deleteAll()

```
void com.mehdi.petreminder.repository.SQLiteRepository< T >.deleteAll ()
```

Tüm varlıkları siler.

Exceptions

<i>RepositoryException</i>	Silme başarısız olursa
--	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 226 of file [SQLiteRepository.java](#).

```
00226      {
00227          try (Statement st = connection.createStatement()) {
00228              st.execute("DELETE FROM " + getTableName());
00229          } catch (SQLException e) {
00230              throw new RepositoryException("SQLite deleteAll hatası", e);
00231          }
00232      }
```

References [connection](#), and [getTableName\(\)](#).

7.43.3.6 findAll()

```
List< T > com.mehdi.petreminder.repository.SQLiteRepository< T >.findAll ()
```

Tüm varlıkları listeler.

Returns

Varlık listesi (boş olabilir)

Exceptions

<i>RepositoryException</i>	Okuma başarısız olursa
--	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 178 of file [SQLiteRepository.java](#).

```
00178      {
00179          String sql = "SELECT * FROM " + getTableName();
00180          List<T> list = new ArrayList<>();
00181          try (Statement st = connection.createStatement();
00182               ResultSet rs = st.executeQuery(sql)) {
00183              while (rs.next()) {
00184                  list.add(mapRow(rs));
00185              }
00186          } catch (SQLException e) {
00187              throw new RepositoryException("SQLite findAll hatası", e);
00188          }
00189          return list;
00190      }
```

References [connection](#), [getTableName\(\)](#), and [mapRow\(\)](#).

7.43.3.7 findById()

Optional< T > [com.mehdi.petreminder.repository.SqliteRepository](#)< T >.findById (
int id)

ID'ye göre varlık getirir.

Parameters

<i>id</i>	Varlık ID'si
-----------	--------------

Returns

Varlık (bulunamazsa empty Optional)

Exceptions

RepositoryException	Okuma başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository](#)< T >.

Definition at line 160 of file [SqliteRepository.java](#).

```

00160                                     {
00161         String sql = "SELECT * FROM " + getTableName() + " WHERE id = ?";
00162         try (PreparedStatement ps = connection.prepareStatement(sql)) {
00163             ps.setInt(1, id);
00164             ResultSet rs = ps.executeQuery();
00165             if (rs.next()) {
00166                 return Optional.of(mapRow(rs));
00167             }
00168         } catch (SQLException e) {
00169             throw new RepositoryException("SQLite findById hatası", e);
00170         }
00171         return Optional.empty();
00172     }

```

References [connection](#), [getTableName\(\)](#), and [mapRow\(\)](#).

7.43.3.8 getConnection()

Connection [com.mehdi.petreminder.repository.SqliteRepository](#)< T >.getConnection ()

Bağlantıyı döndürür (test için).

Returns

JDBC Connection

Definition at line 268 of file [SqliteRepository.java](#).

```

00268                                     {
00269         return connection;
00270     }

```

References [connection](#).

7.43.3.9 getCreateTableSql()

```
abstract String com.mehdi.petreminder.repository.SqliteRepository< T >.getCreateTableSql ()  
[abstract], [protected]
```

Tablo oluşturma SQL'ini döndürür.

Returns

CREATE TABLE IF NOT EXISTS SQL

Referenced by [createTableIfNotExists\(\)](#).

7.43.3.10 getInsertSql()

```
abstract String com.mehdi.petreminder.repository.SqliteRepository< T >.getInsertSql () [abstract],  
[protected]
```

INSERT SQL'ini döndürür (ID hariç).

Returns

INSERT INTO sql

Referenced by [save\(\)](#).

7.43.3.11 getTableName()

```
abstract String com.mehdi.petreminder.repository.SqliteRepository< T >.getTableName () [abstract],  
[protected]
```

Tablo adını döndürür.

Returns

SQL tablo adı

Referenced by [count\(\)](#), [delete\(\)](#), [deleteAll\(\)](#), [findAll\(\)](#), and [findById\(\)](#).

7.43.3.12 getUpdateSql()

```
abstract String com.mehdi.petreminder.repository.SqliteRepository< T >.getUpdateSql () [abstract],  
[protected]
```

UPDATE SQL'ini döndürür.

Returns

UPDATE sql

Referenced by [update\(\)](#).

7.43.3.13 mapRow()

```
abstract T com.mehdi.petreminder.repository.SqliteRepository< T >.mapRow (
    ResultSet rs) throws SQLException [abstract], [protected]
```

ResultSet'ten varlık oluşturur.

Parameters

<i>rs</i>	ResultSet
-----------	-----------

Returns

Varlık nesnesi

Exceptions

<i>SQLException</i>	SQL hatası
---------------------	------------

Referenced by [findAll\(\)](#), and [findById\(\)](#).

7.43.3.14 save()

```
int com.mehdi.petreminder.repository.SqliteRepository< T >.save (
    T entity)
```

Yeni bir varlık kaydeder.

Parameters

<i>entity</i>	Kaydedilecek varlık
---------------	---------------------

Returns

Oluşturulan varlığın ID'si

Exceptions

<i>RepositoryException</i>	Kayıt başarısız olursa
----------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 137 of file [SqliteRepository.java](#).

```
00137         {
00138             if (entity == null) throw new RepositoryException("Entity null olamaz");
00139             try (PreparedStatement ps = connection.prepareStatement(
00140                 getInsertSql(), Statement.RETURN_GENERATED_KEYS)) {
00141                 setInsertParams(ps, entity);
00142                 ps.executeUpdate();
00143                 ResultSet keys = ps.getGeneratedKeys();
```

```

00144         if (keys.next()) {
00145             int id = keys.getInt(1);
00146             setEntityId(entity, id);
00147             logger.debug("SQLite kayıt: id={}", id);
00148             return id;
00149         }
00150         throw new RepositoryException("ID üretilemedi");
00151     } catch (SQLException e) {
00152         throw new RepositoryException("SQLite save hatası", e);
00153     }
00154 }

```

References [connection](#), [getInsertSql\(\)](#), [logger](#), [setEntityId\(\)](#), and [setInsertParams\(\)](#).

7.43.3.15 setEntityId()

```

void com.mehdi.petreminder.repository.SQLiteRepository< T >.setEntityId (
    T entity,
    int id) [protected]

```

Reflection ile entity'ye id set eder.

Parameters

<i>entity</i>	Varlık
<i>id</i>	ID

Definition at line 277 of file [SQLiteRepository.java](#).

```

00277         {
00278         try {
00279             var method = entity.getClass().getMethod("setId", int.class);
00280             method.invoke(entity, id);
00281         } catch (Exception e) {
00282             logger.warn("setId çağrılmadı: {}", e.getMessage());
00283         }
00284     }

```

References [logger](#).

Referenced by [save\(\)](#).

7.43.3.16 setInsertParams()

```

abstract void com.mehdi.petreminder.repository.SQLiteRepository< T >.setInsertParams (
    PreparedStatement ps,
    T entity) throws SQLException [abstract], [protected]

```

PreparedStatement'a INSERT parametrelerini set eder.

Parameters

<i>ps</i>	PreparedStatement
<i>entity</i>	Varlık

Exceptions

<i>SQLException</i>	SQL hatası
---------------------	------------

Referenced by [save\(\)](#).

7.43.3.17 setUpdateParams()

```
abstract void com.mehdi.petreminder.repository.SqliteRepository< T >.setUpdateParams (
    PreparedStatement ps,
    T entity) throws SQLException [abstract], [protected]
```

PreparedStatement'a UPDATE parametrelerini set eder.

Parameters

<i>ps</i>	PreparedStatement
<i>entity</i>	Varlık

Exceptions

<i>SQLException</i>	SQL hatası
---------------------	------------

Referenced by [update\(\)](#).

7.43.3.18 update()

```
boolean com.mehdi.petreminder.repository.SqliteRepository< T >.update (
    T entity)
```

Mevcut varlığı günceller.

Parameters

<i>entity</i>	Güncellenecek varlık
---------------	----------------------

Returns

Güncelleme başarılıysa true

Exceptions

RepositoryException	Güncelleme başarısız olursa
-------------------------------------	-----------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 196 of file [SqliteRepository.java](#).

```

00196         {
00197             if (entity == null) return false;
00198             try (PreparedStatement ps = connection.prepareStatement(getUpdateSql())) {
00199                 setUpdateParams(ps, entity);
00200                 int rows = ps.executeUpdate();
00201                 return rows > 0;
00202             } catch (SQLException e) {
00203                 throw new RepositoryException("SQLite update hatası", e);
00204             }
00205         }

```

References [connection](#), [getUpdateSql\(\)](#), and [setUpdateParams\(\)](#).

7.43.4 Member Data Documentation

7.43.4.1 connection

Connection [com.mehdi.petreminder.repository.SqliteRepository](#)< T >.connection [protected]

SQLite JDBC bağlantısı.

Definition at line 34 of file [SqliteRepository.java](#).

Referenced by [close\(\)](#), [count\(\)](#), [createTableIfNotExists\(\)](#), [delete\(\)](#), [deleteAll\(\)](#), [findAll\(\)](#), [findById\(\)](#), [getConnection\(\)](#), [save\(\)](#), [SqliteRepository\(\)](#), [SqliteRepository\(\)](#), and [update\(\)](#).

7.43.4.2 logger

final Logger [com.mehdi.petreminder.repository.SqliteRepository](#)< T >.logger [static], [protected]

Initial value:

```

=
    (Logger) LoggerFactory.getLogger(SqliteRepository.class)

```

Logger.

Definition at line 30 of file [SqliteRepository.java](#).

Referenced by [close\(\)](#), [save\(\)](#), [setEntityId\(\)](#), and [SqliteRepository\(\)](#).

The documentation for this class was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/repository/[SqliteRepository.java](#)

7.44 com.mehdi.petreminder.config.StorageConfig Class Reference

Aktif storage backend'i yönetir.

Static Public Member Functions

- static [StorageType](#) [getActiveBackend](#) ()
Aktif backend'i döndürür.
- static void [setActiveBackend](#) ([StorageType](#) type)
Aktif backend'i değiştirir ve kaydeder.
- static String [getMysqlUrl](#) ()
MySQL URL'ini döndürür.
- static void [setMysqlUrl](#) (String url)
MySQL URL'ini set eder.
- static String [getMysqlUsername](#) ()
MySQL kullanıcı adını döndürür.
- static String [getMysqlPassword](#) ()
MySQL şifresini döndürür.
- static String [getSqliteFilePath](#) ()
SQLite dosya yolunu döndürür.
- static String [getBinaryDirectory](#) ()
Binary dizin yolunu döndürür.
- static void [loadFromFile](#) ()
Ayarları properties dosyasından yükler.
- static void [saveToFile](#) ()
Ayarları properties dosyasına kaydeder.
- static void [reset](#) ()
Config'i sıfırlar (test için).

7.44.1 Detailed Description

Aktif storage backend'i yönetir.

PDF zorunluluğu: Runtime storage switching — no restart required. config/storage.properties dosyasına kaydeder/yükler.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 25 of file [StorageConfig.java](#).

7.44.2 Member Function Documentation

7.44.2.1 getActiveBackend()

`StorageType` `com.mehdi.petreminder.config.StorageConfig.getActiveBackend ()` [static]

Aktif backend'i döndürür.

Returns

`StorageType`

Definition at line 64 of file `StorageConfig.java`.

```
00064                                     {
00065         return activeBackend;
00066     }
```

Referenced by `com.mehdi.petreminder.repository.RepositoryFactory.createMedicalRecordRepository()`, `com.mehdi.petreminder.repository.RepositoryFactory.createReminderRepository()`, `com.mehdi.petreminder.repository.RepositoryFactory.getActiveType()`, and `com.mehdi.petreminder.gui.panels.SettingsPanel.refresh()`.

7.44.2.2 getBinaryDirectory()

`String` `com.mehdi.petreminder.config.StorageConfig.getBinaryDirectory ()` [static]

Binary dizin yolunu döndürür.

Returns

Dizin yolu

Definition at line 114 of file `StorageConfig.java`.

```
00114 { return binaryDirectory; }
```

7.44.2.3 getMysqlPassword()

`String` `com.mehdi.petreminder.config.StorageConfig.getMysqlPassword ()` [static]

MySQL şifresini döndürür.

Returns

password

Definition at line 102 of file `StorageConfig.java`.

```
00102 { return mysqlPassword; }
```

Referenced by `com.mehdi.petreminder.repository.MySqlRepository< T >.MySqlRepository()`.

7.44.2.4 getMysqlUrl()

```
String com.mehdi.petreminder.config.StorageConfig.getMysqlUrl () [static]
```

MySQL URL'ini döndürür.

Returns

JDBC URL

Definition at line 84 of file [StorageConfig.java](#).

```
00084 { return mysqlUrl; }
```

Referenced by [com.mehdi.petreminder.repository.MySqlRepository< T >.MySqlRepository\(\)](#).

7.44.2.5 getMysqlUsername()

```
String com.mehdi.petreminder.config.StorageConfig.getMysqlUsername () [static]
```

MySQL kullanıcı adını döndürür.

Returns

username

Definition at line 96 of file [StorageConfig.java](#).

```
00096 { return mysqlUsername; }
```

Referenced by [com.mehdi.petreminder.repository.MySqlRepository< T >.MySqlRepository\(\)](#).

7.44.2.6 getSqliteFilePath()

```
String com.mehdi.petreminder.config.StorageConfig.getSqliteFilePath () [static]
```

SQLite dosya yolunu döndürür.

Returns

Dosya yolu

Definition at line 108 of file [StorageConfig.java](#).

```
00108 { return sqliteFilePath; }
```

Referenced by [com.mehdi.petreminder.repository.SqliteRepository< T >.SqliteRepository\(\)](#).

7.44.2.7 loadFromFile()

```
void com.mehdi.petreminder.config.StorageConfig.loadFromFile () [static]
```

Ayarları properties dosyasından yükler.

Definition at line 120 of file [StorageConfig.java](#).

```
00120     {
00121         File file = new File(CONFIG_FILE);
00122         if (!file.exists()) {
00123             logger.debug("Config dosyası bulunamadı, varsayılanlar kullanılıyor.");
00124             return;
00125         }
00126         Properties props = new Properties();
00127         try (FileInputStream fis = new FileInputStream(file)) {
00128             props.load(fis);
00129             String backend = props.getProperty("storage.backend", "BINARY");
00130             try {
00131                 activeBackend = StorageType.valueOf(backend.toUpperCase());
00132             } catch (IllegalArgumentException e) {
00133                 activeBackend = StorageType.BINARY;
00134             }
00135             mysqlUrl = props.getProperty("mysql.url", mysqlUrl);
00136             mysqlUsername = props.getProperty("mysql.username", mysqlUsername);
00137             mysqlPassword = props.getProperty("mysql.password", mysqlPassword);
00138             sqliteFilePath = props.getProperty("sqlite.path", sqliteFilePath);
00139             binaryDirectory = props.getProperty("binary.dir", binaryDirectory);
00140             logger.info("Config yüklendi: backend={}, activeBackend");
00141         } catch (IOException e) {
00142             logger.warn("Config yüklenemedi: {}", e.getMessage());
00143         }
00144     }
```

References [com.mehdi.petreminder.config.StorageType.BINARY](#).

7.44.2.8 reset()

```
void com.mehdi.petreminder.config.StorageConfig.reset () [static]
```

Config'i sıfırlar (test için).

Definition at line 171 of file [StorageConfig.java](#).

```
00171     {
00172         activeBackend = StorageType.BINARY;
00173         mysqlUrl =
00174             "jdbc:mysql://localhost:3306/petreminder_db?useSSL=false&serverTimezone=UTC";
00175         mysqlUsername = "petreminder_user";
00176         mysqlPassword = "petreminder_pass";
00177         sqliteFilePath = "data/petreminder.db";
00178         binaryDirectory = "data/binary";
00179     }
```

References [com.mehdi.petreminder.config.StorageType.BINARY](#).

7.44.2.9 saveToFile()

```
void com.mehdi.petreminder.config.StorageConfig.saveToFile () [static]
```

Ayarları properties dosyasına kaydeder.

Definition at line 150 of file [StorageConfig.java](#).

```
00150     {
00151         File dir = new File("config");
00152         if (!dir.exists()) dir.mkdirs();
00153         Properties props = new Properties();
00154         props.setProperty("storage.backend", activeBackend.name());
00155         props.setProperty("mysql.url", mysqlUrl);
```



```

00156         props.setProperty("mysql.username", mysqlUsername);
00157         props.setProperty("mysql.password", mysqlPassword);
00158         props.setProperty("sqlite.path", sqliteFilePath);
00159         props.setProperty("binary.dir", binaryDirectory);
00160         try (FileOutputStream fos = new FileOutputStream(CONFIG_FILE)) {
00161             props.store(fos, "PetReminder Storage Configuration");
00162             logger.debug("Config kaydedildi.");
00163         } catch (IOException e) {
00164             logger.warn("Config kaydedilemedi: {}", e.getMessage());
00165         }
00166     }

```

Referenced by [setActiveBackend\(\)](#).

7.44.2.10 setActiveBackend()

```

void com.mehdi.petreminder.config.StorageConfig.setActiveBackend (
    StorageType type) [static]

```

Aktif backend'i değiştirir ve kaydeder.

Parameters

<i>type</i>	Yeni StorageType
-------------	----------------------------------

Definition at line 72 of file [StorageConfig.java](#).

```

00072                                     {
00073         if (type != null) {
00074             activeBackend = type;
00075             saveToFile();
00076             logger.info("Storage backend değiştirildi: {}", type);
00077         }
00078     }

```

References [saveToFile\(\)](#).

Referenced by [com.mehdi.petreminder.petreminderApp.main\(\)](#), and [com.mehdi.petreminder.gui.MainFrame.onStorageChanged\(\)](#).

7.44.2.11 setMysqlUrl()

```

void com.mehdi.petreminder.config.StorageConfig.setMysqlUrl (
    String url) [static]

```

MySQL URL'ini set eder.

Parameters

<i>url</i>	JDBC URL
------------	----------

Definition at line 90 of file [StorageConfig.java](#).

```

00090 { mysqlUrl = url; }

```

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/config/StorageConfig.java](#)

7.45 com.mehdi.petreminder.config.StorageType Enum Reference

Public Member Functions

- [StorageType](#) (String displayName)
Yapıcı.
- String [getDisplayName](#) ()
Gösterim adını döndürür.
- String [toString](#) ()
toString override.

Public Attributes

- [BINARY](#) =("Binary Dosya")
Binary dosya (.bin).
- [SQLITE](#) =("SQLite")
SQLite veritabanı
- [MYSQL](#) =("MySQL (Docker)")
MySQL veritabanı (Docker).

7.45.1 Detailed Description

Enum documentation.

Definition at line 18 of file [StorageType.java](#).

7.45.2 Constructor & Destructor Documentation

7.45.2.1 StorageType()

```
com.mehdi.petreminder.config.StorageType.StorageType (  
    String displayName)
```

Yapıcı.

Parameters

<i>displayName</i>	Gösterim adı
--------------------	--------------

Definition at line 36 of file [StorageType.java](#).

```
00036         {  
00037     this.displayName = displayName;  
00038 }
```

7.45.3 Member Function Documentation

7.45.3.1 getDisplayName()

```
String com.mehdi.petreminder.config.StorageType.getDisplayName ()
```

Gösterim adını döndürür.

Returns

displayName

Definition at line 44 of file [StorageType.java](#).

```
00044                                     {
00045         return displayName;
00046     }
```

Referenced by [com.mehdi.petreminder.gui.MainFrame.onStorageChanged\(\)](#), and [com.mehdi.petreminder.gui.panels.SettingsPanel.re](#)

7.45.3.2 toString()

```
String com.mehdi.petreminder.config.StorageType.toString ()
```

toString override.

Returns

displayName

Definition at line 53 of file [StorageType.java](#).

```
00053                                     {
00054         return displayName;
00055     }
```

7.45.4 Member Data Documentation

7.45.4.1 BINARY

```
com.mehdi.petreminder.config.StorageType.BINARY = ("Binary Dosya")
```

Binary dosya (.bin).

Definition at line 21 of file [StorageType.java](#).

Referenced by [com.mehdi.petreminder.config.StorageConfig.loadFromFile\(\)](#), [com.mehdi.petreminder.petreminderApp.parseStorageA](#) and [com.mehdi.petreminder.config.StorageConfig.reset\(\)](#).

7.45.4.2 MYSQL

```
com.mehdi.petreminder.config.StorageType.MYSQL = ("MySQL (Docker) ")
```

MySQL veritabanı (Docker).

Definition at line 27 of file [StorageType.java](#).

7.45.4.3 SQLITE

```
com.mehdi.petreminder.config.StorageType.SQLITE = ("SQLite")
```

SQLite veritabanı

Definition at line 24 of file [StorageType.java](#).

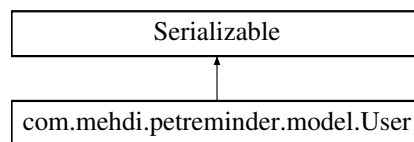
The documentation for this enum was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/config/StorageType.java](#)

7.46 com.mehdi.petreminder.model.User Class Reference

Uygulamayı kullanan evcil hayvan sahibi.

Inheritance diagram for com.mehdi.petreminder.model.User:



Public Member Functions

- [User](#) ()
Varsayılan yapıcı.
- [User](#) (int id, String username, String email, String passwordHash, String fullName)
Temel parametrelili yapıcı.
- int [getId](#) ()
ID getter.
- void [setId](#) (int id)
ID setter.
- String [getUsername](#) ()
Kullanıcı adı getter.
- void [setUsername](#) (String username)
Kullanıcı adı setter.
- String [getEmail](#) ()
E-posta getter.
- void [setEmail](#) (String email)
E-posta setter.
- String [getPasswordHash](#) ()
Şifre hash getter.
- void [setPasswordHash](#) (String passwordHash)
Şifre hash setter.
- String [getFullName](#) ()
Tam ad getter.
- void [setFullName](#) (String fullName)

- Tam ad setter.*
- LocalDateTime [getCreatedAt](#) ()
- Kayıt tarihi getter.*
- void [setCreatedAt](#) (LocalDateTime createdAt)
- Kayıt tarihi setter.*
- LocalDateTime [getLastLogin](#) ()
- Son giriş getter.*
- void [setLastLogin](#) (LocalDateTime lastLogin)
- Son giriş setter.*
- boolean [isActive](#) ()
- Aktif mi getter.*
- void [setActive](#) (boolean active)
- Aktif setter.*
- String [toString](#) ()
- String temsili — şifre hash'i GİZLENİR (KVKK).*
- boolean [equals](#) (Object obj)
- Eşitlik kontrolü — ID bazlı.*
- int [hashCode](#) ()
- Hash kodu.*

7.46.1 Detailed Description

Uygulamayı kullanan evcil hayvan sahibi.

OOP Encapsulation: tüm alanlar private. KVKK: şifre hash'lenir, kişisel veriler korunur. Serializable: Binary↔ Repository için.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 20 of file [User.java](#).

7.46.2 Constructor & Destructor Documentation

7.46.2.1 User() [1/2]

```
com.mehdi.petreminder.model.User.User ()
```

Varsayılan yapıcı.

Definition at line 50 of file [User.java](#).

```
00050 {}
```

Referenced by [equals\(\)](#).

7.46.2.2 User() [2/2]

```
com.mehdi.petreminder.model.User.User (
    int id,
    String username,
    String email,
    String passwordHash,
    String fullName)
```

Temel parametrelili yapıcı.

Parameters

<i>id</i>	Kullanıcı ID
<i>username</i>	Kullanıcı adı
<i>email</i>	E-posta
<i>passwordHash</i>	Şifre hash'i
<i>fullName</i>	Tam ad

Definition at line 60 of file [User.java](#).

```
00061                                     {
00062         this.id = id;
00063         this.username = username;
00064         this.email = email;
00065         this.passwordHash = passwordHash;
00066         this.fullName = fullName;
00067         this.createdAt = LocalDateTime.now();
00068         this.active = true;
00069     }
```

7.46.3 Member Function Documentation

7.46.3.1 equals()

```
boolean com.mehdi.petreminder.model.User.equals (
    Object obj)
```

Eşitlik kontrolü — ID bazlı.

Parameters

<i>obj</i>	Karşılaştırılan nesne
------------	-----------------------

Returns

ID'ler eşitse true

Definition at line 135 of file [User.java](#).

```
00135                                     {
00136         if (this == obj) return true;
00137         if (obj == null || !(obj instanceof User)) return false;
00138         User other = (User) obj;
00139         return this.id == other.id;
00140     }
```

References [User\(\)](#).

7.46.3.2 getCreatedAt()

LocalDateTime com.mehdi.petreminder.model.User.getCreatedAt ()

Kayıt tarihi getter.

Returns

Kayıt tarihi

Definition at line 102 of file [User.java](#).

```
00102 { return createdAt; }
```

7.46.3.3 getEmail()

String com.mehdi.petreminder.model.User.getEmail ()

E-posta getter.

Returns

E-posta

Definition at line 84 of file [User.java](#).

```
00084 { return email; }
```

7.46.3.4 getFullName()

String com.mehdi.petreminder.model.User.getFullName ()

Tam ad getter.

Returns

Tam ad

Definition at line 96 of file [User.java](#).

```
00096 { return fullName; }
```

7.46.3.5 getId()

int com.mehdi.petreminder.model.User.getId ()

ID getter.

Returns

Kullanıcı ID

Definition at line 72 of file [User.java](#).

```
00072 { return id; }
```

7.46.3.6 getLastLogin()

LocalDateTime com.mehdi.petreminder.model.User.getLastLogin ()

Son giriş getter.

Returns

Son giriş

Definition at line 108 of file [User.java](#).

```
00108 { return lastLogin; }
```

7.46.3.7 getPasswordHash()

String com.mehdi.petreminder.model.User.getPasswordHash ()

Şifre hash getter.

Returns

Şifre hash'i

Definition at line 90 of file [User.java](#).

```
00090 { return passwordHash; }
```

7.46.3.8 getUsername()

String com.mehdi.petreminder.model.User.getUsername ()

Kullanıcı adı getter.

Returns

Kullanıcı adı

Definition at line 78 of file [User.java](#).

```
00078 { return username; }
```

7.46.3.9 hashCode()

int com.mehdi.petreminder.model.User.hashCode ()

Hash kodu.

Returns

ID'ye göre hash

Definition at line 147 of file [User.java](#).

```
00147         {
00148             return Integer.hashCode(id);
00149         }
```


7.46.3.10 isActive()

```
boolean com.mehdi.petreminder.model.User.isActive ()
```

Aktif mi getter.

Returns

Aktif mi

Definition at line 114 of file [User.java](#).

```
00114 { return active; }
```

7.46.3.11 setActive()

```
void com.mehdi.petreminder.model.User.setActive (  
    boolean active)
```

Aktif setter.

Parameters

<i>active</i>	Aktif mi
---------------	----------

Definition at line 117 of file [User.java](#).

```
00117 { this.active = active; }
```

Referenced by [com.mehdi.petreminder.repository.UserMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.UserSqliteRepository.mapRow\(\)](#).

7.46.3.12 setCreatedAt()

```
void com.mehdi.petreminder.model.User.setCreatedAt (  
    LocalDateTime createdAt)
```

Kayıt tarihi setter.

Parameters

<i>createdAt</i>	Kayıt tarihi
------------------	--------------

Definition at line 105 of file [User.java](#).

```
00105 { this.createdAt = createdAt; }
```

Referenced by [com.mehdi.petreminder.repository.UserMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.UserSqliteRepository.mapRow\(\)](#).

7.46.3.13 setEmail()

```
void com.mehdi.petreminder.model.User.setEmail (
    String email)
```

E-posta setter.

Parameters

<i>email</i>	E-posta
--------------	---------

Definition at line 87 of file [User.java](#).

```
00087 { this.email = email; }
```

Referenced by [com.mehdi.petreminder.repository.UserMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.UserSqliteRepository.mapRow\(\)](#).

7.46.3.14 setFullName()

```
void com.mehdi.petreminder.model.User.setFullName (
    String fullName)
```

Tam ad setter.

Parameters

<i>fullName</i>	Tam ad
-----------------	--------

Definition at line 99 of file [User.java](#).

```
00099 { this.fullName = fullName; }
```

Referenced by [com.mehdi.petreminder.repository.UserMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.UserSqliteRepository.mapRow\(\)](#).

7.46.3.15 setId()

```
void com.mehdi.petreminder.model.User.setId (
    int id)
```

ID setter.

Parameters

<i>id</i>	Kullanıcı ID
-----------	--------------

Definition at line 75 of file [User.java](#).

```
00075 { this.id = id; }
```

Referenced by [com.mehdi.petreminder.repository.UserMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.UserSqliteRepository.mapRow\(\)](#).

7.46.3.16 setLastLogin()

```
void com.mehdi.petreminder.model.User.setLastLogin (
    LocalDateTime lastLogin)
```

Son giriş setter.

Parameters

<i>lastLogin</i>	Son giriş
------------------	-----------

Definition at line 111 of file [User.java](#).

```
00111 { this.lastLogin = lastLogin; }
```

7.46.3.17 setPasswordHash()

```
void com.mehdi.petreminder.model.User.setPasswordHash (
    String passwordHash)
```

Şifre hash setter.

Parameters

<i>passwordHash</i>	Şifre hash'i
---------------------	--------------

Definition at line 93 of file [User.java](#).

```
00093 { this.passwordHash = passwordHash; }
```

Referenced by [com.mehdi.petreminder.repository.UserMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.UserSqliteRepository.mapRow\(\)](#).

7.46.3.18 setUsername()

```
void com.mehdi.petreminder.model.User.setUsername (
    String username)
```

Kullanıcı adı setter.

Parameters

<i>username</i>	Kullanıcı adı
-----------------	---------------

Definition at line 81 of file [User.java](#).

```
00081 { this.username = username; }
```

Referenced by [com.mehdi.petreminder.repository.UserMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.UserSqliteRepository.mapRow\(\)](#).

7.46.3.19 toString()

String com.mehdi.petreminder.model.User.toString ()

String temsili — şifre hash'i GİZLENİR (KVKK).

Returns

Kullanıcı bilgisi, şifre hariç

Definition at line 124 of file [User.java](#).

```
00124 {
00125     return String.format("User{id=%d, username='%s', email='%s', active=%b}",
00126         id, username, email, active);
00127 }
```

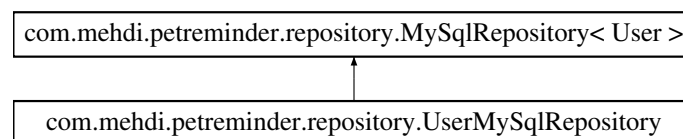
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/User.java](#)

7.47 com.mehdi.petreminder.repository.UserMySqlRepository Class Reference

MySQL üzerinde [User](#) CRUD işlemleri.

Inheritance diagram for com.mehdi.petreminder.repository.UserMySqlRepository:



Public Member Functions

- [UserMySqlRepository](#) ()
Varsayılan yapıcı.
- [UserMySqlRepository](#) (Connection [connection](#))
Test yapıcısı.

Public Member Functions inherited from [com.mehdi.petreminder.repository.MySqlRepository< User >](#)

- [MySqlRepository](#) ()
MySQL bağlantısını açar.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür.

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [User](#) u) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [User](#) u) throws SQLException
- [User](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from
[com.mehdi.petreminder.repository.MySqlRepository](#) < [User](#) >

- abstract String [getTableName](#) ()
Tablo adı.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL.
- abstract String [getInsertSql](#) ()
INSERT SQL.
- abstract String [getUpdateSql](#) ()
UPDATE SQL.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members
Protected Attributes inherited from
[com.mehdi.petreminder.repository.MySqlRepository](#) < [User](#) >

- Connection [connection](#)
MySQL JDBC bağlantısı.

Static Protected Attributes inherited from
[com.mehdi.petreminder.repository.MySqlRepository](#) < [User](#) >

- static final Logger [logger](#)
Logger.

7.47.1 Detailed Description

MySQL üzerinde [User](#) CRUD işlemleri.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 22 of file [UserMySqlRepository.java](#).

7.47.2 Constructor & Destructor Documentation

7.47.2.1 UserMySqlRepository() [1/2]

```
com.mehdi.petreminder.repository.UserMySqlRepository.UserMySqlRepository ()
```

Varsayılan yapıcı.

Definition at line 25 of file [UserMySqlRepository.java](#).

```
00025 { super(); }
```

7.47.2.2 UserMySqlRepository() [2/2]

```
com.mehdi.petreminder.repository.UserMySqlRepository.UserMySqlRepository (  
    Connection connection)
```

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 31 of file [UserMySqlRepository.java](#).

```
00031 { super(connection); }
```

References [com.mehdi.petreminder.repository.MySqlRepository< User >.connection](#).

7.47.3 Member Function Documentation

7.47.3.1 getCreateTableSql()

String com.mehdi.petreminder.repository.UserMySqlRepository.getCreateTableSql () [protected]

Member documentation.

Definition at line 45 of file [UserMySqlRepository.java](#).

```
00045         {
00046             return "CREATE TABLE IF NOT EXISTS users ("
00047                 + "id INT AUTO_INCREMENT PRIMARY KEY,"
00048                 + "username VARCHAR(50) NOT NULL UNIQUE,"
00049                 + "email VARCHAR(100) NOT NULL UNIQUE,"
00050                 + "password_hash VARCHAR(255) NOT NULL,"
00051                 + "full_name VARCHAR(100),"
00052                 + "created_at DATETIME,"
00053                 + "last_login DATETIME,"
00054                 + "active BOOLEAN DEFAULT TRUE"
00055                 + ")";
00056     }
```

7.47.3.2 getInsertSql()

String com.mehdi.petreminder.repository.UserMySqlRepository.getInsertSql () [protected]

Member documentation.

Definition at line 63 of file [UserMySqlRepository.java](#).

```
00063         {
00064             return "INSERT INTO users (username,email,password_hash,full_name,created_at,active)"
00065                 + " VALUES (?, ?, ?, ?, ?, ?)";
00066     }
```

7.47.3.3 getTableName()

String com.mehdi.petreminder.repository.UserMySqlRepository.getTableName () [protected]

Member documentation.

Definition at line 38 of file [UserMySqlRepository.java](#).

```
00038 { return "users"; }
```

7.47.3.4 getUpdateSql()

String com.mehdi.petreminder.repository.UserMySqlRepository.getUpdateSql () [protected]

Member documentation.

Definition at line 73 of file [UserMySqlRepository.java](#).

```
00073         {
00074             return "UPDATE users SET username=?,email=?,password_hash=?,full_name=?, "
00075                 + "created_at=?,active=? WHERE id=?";
00076     }
```

7.47.3.5 mapRow()

`User` com.mehdi.petreminder.repository.UserMySqlRepository.mapRow (ResultSet rs) throws SQLException [protected]

Member documentation.

Definition at line 109 of file [UserMySqlRepository.java](#).

```
00109                                     {
00110         User u = new User();
00111         u.setId(rs.getInt("id"));
00112         u.setUsername(rs.getString("username"));
00113         u.setEmail(rs.getString("email"));
00114         u.setPasswordHash(rs.getString("password_hash"));
00115         u.setFullName(rs.getString("full_name"));
00116         Timestamp ts = rs.getTimestamp("created_at");
00117         if (ts != null) u.setCreatedAt(ts.toLocalDateTime());
00118         u.setActive(rs.getBoolean("active"));
00119         return u;
00120     }
```

References [com.mehdi.petreminder.model.User.setActive\(\)](#), [com.mehdi.petreminder.model.User.setCreatedAt\(\)](#), [com.mehdi.petreminder.model.User.setEmail\(\)](#), [com.mehdi.petreminder.model.User.setFullName\(\)](#), [com.mehdi.petreminder.model.User.setPasswordHash\(\)](#), and [com.mehdi.petreminder.model.User.setUsername\(\)](#).

7.47.3.6 setInsertParams()

void com.mehdi.petreminder.repository.UserMySqlRepository.setInsertParams (PreparedStatement ps, User u) throws SQLException [protected]

Member documentation.

Definition at line 84 of file [UserMySqlRepository.java](#).

```
00084                                     {
00085         ps.setString(1, u.getUsername());
00086         ps.setString(2, u.getEmail());
00087         ps.setString(3, u.getPasswordHash());
00088         ps.setString(4, u.getFullName());
00089         ps.setObject(5, u.getCreatedAt() != null
00090             ? Timestamp.valueOf(u.getCreatedAt()) : null);
00091         ps.setBoolean(6, u.isActive());
00092     }
```

Referenced by [setUpdateParams\(\)](#).

7.47.3.7 setUpdateParams()

void com.mehdi.petreminder.repository.UserMySqlRepository.setUpdateParams (PreparedStatement ps, User u) throws SQLException [protected]

Member documentation.

Definition at line 99 of file [UserMySqlRepository.java](#).

```
00099                                     {
00100         setInsertParams(ps, u);
00101         ps.setInt(7, u.getId());
00102     }
```

References [setInsertParams\(\)](#).

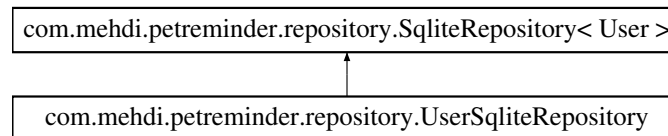
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/UserMySqlRepository.java](#)

7.48 com.mehdi.petreminder.repository.UserSqliteRepository Class Reference

SQLite üzerinde [User](#) CRUD işlemleri.

Inheritance diagram for com.mehdi.petreminder.repository.UserSqliteRepository:



Public Member Functions

- [UserSqliteRepository](#) ()
Varsayılan yapıcı.
- [UserSqliteRepository](#) (Connection [connection](#))
Test yapıcısı.
- [User findByUsername](#) (String username)
Username ile kullanıcı arar.

Public Member Functions inherited from [com.mehdi.petreminder.repository.SqliteRepository< User >](#)

- [SqliteRepository](#) ()
SQLite bağlantısını açar ve tabloyu oluşturur.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür (test için).

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [User](#) u) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [User](#) u) throws SQLException
- [User](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from [com.mehdi.petreminder.repository.SqliteRepository](#)< [User](#) >

- abstract String [getTableName](#) ()
Tablo adını döndürür.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL'ini döndürür.
- abstract String [getInsertSql](#) ()
INSERT SQL'ini döndürür (ID hariç).
- abstract String [getUpdateSql](#) ()
UPDATE SQL'ini döndürür.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members

Protected Attributes inherited from [com.mehdi.petreminder.repository.SqliteRepository](#)< [User](#) >

- Connection [connection](#)
SQLite JDBC bağlantısı.

Static Protected Attributes inherited from [com.mehdi.petreminder.repository.SqliteRepository](#)< [User](#) >

- static final Logger [logger](#)
Logger.

7.48.1 Detailed Description

SQLite üzerinde [User](#) CRUD işlemleri.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 22 of file [UserSqliteRepository.java](#).

7.48.2 Constructor & Destructor Documentation

7.48.2.1 UserSqliteRepository() [1/2]

`com.mehdi.petreminder.repository.UserSqliteRepository.UserSqliteRepository ()`

Varsayılan yapıcı.

Definition at line 25 of file [UserSqliteRepository.java](#).

```
00025 { super(); }
```

7.48.2.2 UserSqliteRepository() [2/2]

`com.mehdi.petreminder.repository.UserSqliteRepository.UserSqliteRepository (Connection connection)`

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 31 of file [UserSqliteRepository.java](#).

```
00031 { super(connection); }
```

References [com.mehdi.petreminder.repository.SqliteRepository< User >.connection](#).

7.48.3 Member Function Documentation

7.48.3.1 findByUsername()

[User](#) `com.mehdi.petreminder.repository.UserSqliteRepository.findByUsername (String username)`

Username ile kullanıcı arar.

Parameters

<i>username</i>	Kullanıcı adı
-----------------	---------------

Returns

[User](#) veya null

Definition at line 127 of file [UserSqliteRepository.java](#).

```
00127     {
00128         try (PreparedStatement ps = connection.prepareStatement(
00129             "SELECT * FROM users WHERE username = ?")) {
00130             ps.setString(1, username);
00131             ResultSet rs = ps.executeQuery();
00132             if (rs.next()) return mapRow(rs);
00133         } catch (SQLException e) {
00134             throw new RepositoryException("findByUsername hatası", e);
00135         }
00136         return null;
00137     }
```

References [com.mehdi.petreminder.repository.SqliteRepository< User >.connection](#), and [mapRow\(\)](#).

7.48.3.2 getCreateTableSql()

```
String com.mehdi.petreminder.repository.UserSqliteRepository.getCreateTableSql () [protected]
```

Member documentation.

Definition at line 45 of file [UserSqliteRepository.java](#).

```
00045     {
00046         return "CREATE TABLE IF NOT EXISTS users ("
00047             + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00048             + "username TEXT NOT NULL UNIQUE,"
00049             + "email TEXT NOT NULL UNIQUE,"
00050             + "password_hash TEXT NOT NULL,"
00051             + "full_name TEXT,"
00052             + "created_at TEXT,"
00053             + "last_login TEXT,"
00054             + "active INTEGER DEFAULT 1"
00055             + ")";
00056     }
```

7.48.3.3 getInsertSql()

```
String com.mehdi.petreminder.repository.UserSqliteRepository.getInsertSql () [protected]
```

Member documentation.

Definition at line 63 of file [UserSqliteRepository.java](#).

```
00063     {
00064         return "INSERT INTO users (username,email,password_hash,full_name,created_at,active)"
00065             + " VALUES (?, ?, ?, ?, ?, ?)";
00066     }
```

7.48.3.4 getTableName()

```
String com.mehdi.petreminder.repository.UserSqliteRepository.getTableName () [protected]
```

Member documentation.

Definition at line 38 of file [UserSqliteRepository.java](#).

```
00038 { return "users"; }
```

7.48.3.5 getUpdateSql()

```
String com.mehdi.petreminder.repository.UserSqliteRepository.getUpdateSql () [protected]
```

Member documentation.

Definition at line 73 of file [UserSqliteRepository.java](#).

```
00073     {
00074         return "UPDATE users SET username=?,email=?,password_hash=?,full_name=?, "
00075             + "created_at=?,active=? WHERE id=?";
00076     }
```

7.48.3.6 mapRow()

`User` com.mehdi.petreminder.repository.UserSqliteRepository.mapRow (ResultSet rs) throws SQLException [protected]

Member documentation.

Definition at line 109 of file [UserSqliteRepository.java](#).

```
00109                                     {
00110         User u = new User();
00111         u.setId(rs.getInt("id"));
00112         u.setUsername(rs.getString("username"));
00113         u.setEmail(rs.getString("email"));
00114         u.setPasswordHash(rs.getString("password_hash"));
00115         u.setFullName(rs.getString("full_name"));
00116         String createdAt = rs.getString("created_at");
00117         if (createdAt != null) u.setCreatedAt(LocalDate.parse(createdAt));
00118         u.setActive(rs.getInt("active") == 1);
00119         return u;
00120     }
```

References [com.mehdi.petreminder.model.User.setActive\(\)](#), [com.mehdi.petreminder.model.User.setCreatedAt\(\)](#), [com.mehdi.petreminder.model.User.setEmail\(\)](#), [com.mehdi.petreminder.model.User.setFullName\(\)](#), [com.mehdi.petreminder.model.User.setPasswordHash\(\)](#), and [com.mehdi.petreminder.model.User.setUsername\(\)](#).

Referenced by [findByUsername\(\)](#).

7.48.3.7 setInsertParams()

void com.mehdi.petreminder.repository.UserSqliteRepository.setInsertParams (PreparedStatement ps, `User` u) throws SQLException [protected]

Member documentation.

Definition at line 84 of file [UserSqliteRepository.java](#).

```
00084                                     {
00085         ps.setString(1, u.getUsername());
00086         ps.setString(2, u.getEmail());
00087         ps.setString(3, u.getPasswordHash());
00088         ps.setString(4, u.getFullName());
00089         ps.setString(5, u.getCreatedAt() != null ? u.getCreatedAt().toString() : null);
00090         ps.setInt(6, u.isActive() ? 1 : 0);
00091     }
```

Referenced by [setUpdateParams\(\)](#).

7.48.3.8 setUpdateParams()

void com.mehdi.petreminder.repository.UserSqliteRepository.setUpdateParams (PreparedStatement ps, `User` u) throws SQLException [protected]

Member documentation.

Definition at line 98 of file [UserSqliteRepository.java](#).

```
00098                                     {
00099         setInsertParams(ps, u);
00100         ps.setInt(7, u.getId());
00101     }
```

References [setInsertParams\(\)](#).

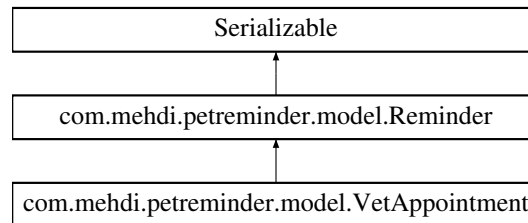
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/UserSqliteRepository.java](#)

7.49 com.mehdi.petreminder.model.VetAppointment Class Reference

Veteriner randevusu — [Reminder](#)'dan türetilmiş somut sınıf.

Inheritance diagram for com.mehdi.petreminder.model.VetAppointment:



Public Member Functions

- [VetAppointment](#) ()
Varsayılan yapıcı.
- [VetAppointment](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String veterinarianName, String clinicName, String reason)
Temel parametrelili yapıcı.
- [VetAppointment](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern, String veterinarianName, String clinicName, String reason, double estimatedCost, boolean confirmed, String clinicPhone)
Tam parametrelili yapıcı.
- String [getVeterinarianName](#) ()
Veteriner adı getter.
- void [setVeterinarianName](#) (String veterinarianName)
Veteriner adı setter.
- String [getClinicName](#) ()
Klinik adı getter.
- void [setClinicName](#) (String clinicName)
Klinik adı setter.
- String [getReason](#) ()
Muayene sebebi getter.
- void [setReason](#) (String reason)
Muayene sebebi setter.
- double [getEstimatedCost](#) ()
Tahmini maliyet getter.
- void [setEstimatedCost](#) (double estimatedCost)
Tahmini maliyet setter.
- boolean [isConfirmed](#) ()
Onay durumu getter.
- void [setConfirmed](#) (boolean confirmed)
Onay durumu setter.
- String [getClinicPhone](#) ()
Klinik telefonu getter.
- void [setClinicPhone](#) (String clinicPhone)
Klinik telefonu setter.
- String [getReminderType](#) ()

- *Hatırlatıcı türü.*
- boolean `validate` ()
- *Veteriner randevusunu doğrular.*
- String `getSummary` ()
- *Özet bilgi.*
- String `toString` ()
- *String temsili.*

Public Member Functions inherited from `com.mehdi.petreminder.model.Reminder`

- int `getId` ()
- *ID getter.*
- void `setId` (int id)
- *ID setter.*
- int `getPetId` ()
- *Pet ID getter.*
- void `setPetId` (int petId)
- *Pet ID setter.*
- String `getPetName` ()
- *Pet adı getter.*
- void `setPetName` (String petName)
- *Pet adı setter.*
- LocalDateTime `getScheduledTime` ()
- *Planlanan zaman getter.*
- void `setScheduledTime` (LocalDateTime scheduledTime)
- *Planlanan zaman setter.*
- boolean `isCompleted` ()
- *Tamamlandı mı getter.*
- void `setCompleted` (boolean completed)
- *Tamamlandı setter.*
- String `getDescription` ()
- *Açıklama getter.*
- void `setDescription` (String description)
- *Açıklama setter.*
- String `getPriority` ()
- *Öncelik getter.*
- void `setPriority` (String priority)
- *Öncelik setter.*
- boolean `isRecurring` ()
- *Tekrarlıyor mu getter.*
- void `setRecurring` (boolean recurring)
- *Tekrarlama setter.*
- String `getRecurrencePattern` ()
- *Tekrar deseni getter.*
- void `setRecurrencePattern` (String recurrencePattern)
- *Tekrar deseni setter.*
- String `getFormattedTime` ()
- *Planlanan zamanı biçimli string olarak döndürür.*
- boolean `isOverdue` ()
- *Hatırlatıcının geçmiş mi olduğunu kontrol eder.*
- boolean `equals` (Object obj)
- *Eşitlik kontrolü — ID bazlı.*
- int `hashCode` ()
- *Hash kodu.*

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- [Reminder](#) ()
Varsayılan yapıcı — Serialization için.
- [Reminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern)
Tam parametrelili yapıcı.

Static Protected Attributes inherited from [com.mehdi.petreminder.model.Reminder](#)

- static final DateTimeFormatter [FORMATTER](#)
Tarih saat formatı.

7.49.1 Detailed Description

Veteriner randevusu — [Reminder](#)'dan türetilmiş somut sınıf.

OOP Inheritance: [Reminder](#)'dan miras alır. OOP Polymorphism: [getReminderType\(\)](#), [validate\(\)](#), [getSummary\(\)](#) override.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 17 of file [VetAppointment.java](#).

7.49.2 Constructor & Destructor Documentation

7.49.2.1 VetAppointment() [1/3]

```
com.mehdi.petreminder.model.VetAppointment.VetAppointment ()
```

Varsayılan yapıcı.

Definition at line 41 of file [VetAppointment.java](#).

```
00041 { super(); }
```


7.49.2.2 VetAppointment() [2/3]

```
com.mehdi.petreminder.model.VetAppointment.VetAppointment (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String veterinarianName,
    String clinicName,
    String reason)
```

Temel parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Randevu zamanı
<i>veterinarianName</i>	Veteriner adı
<i>clinicName</i>	Klinik adı
<i>reason</i>	Muayene sebebi

Definition at line 53 of file [VetAppointment.java](#).

```
00056         {
00057             super(id, petId, petName, scheduledTime,
00058                 "Veteriner: " + reason, "YUKSEK", false, "");
00059             this.veterinarianName = veterinarianName;
00060             this.clinicName = clinicName;
00061             this.reason = reason;
00062             this.estimatedCost = 0.0;
00063             this.confirmed = false;
00064             this.clinicPhone = "";
00065         }
```

7.49.2.3 VetAppointment() [3/3]

```
com.mehdi.petreminder.model.VetAppointment.VetAppointment (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String description,
    String priority,
    boolean recurring,
    String recurrencePattern,
    String veterinarianName,
    String clinicName,
    String reason,
    double estimatedCost,
    boolean confirmed,
    String clinicPhone)
```

Tam parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>description</i>	Açıklama
<i>priority</i>	Öncelik
<i>recurring</i>	Tekrarlıyor mu
<i>recurrencePattern</i>	Tekrar deseni
<i>veterinarianName</i>	Veteriner adı
<i>clinicName</i>	Klinik adı
<i>reason</i>	Muayene sebebi
<i>estimatedCost</i>	Tahmini maliyet
<i>confirmed</i>	Onaylandı mı
<i>clinicPhone</i>	Klinik telefonu

Definition at line 84 of file [VetAppointment.java](#).

```

00090                                     {
00091         super(id, petId, petName, scheduledTime, description,
00092               priority, recurring, recurrencePattern);
00093         this.veterinarianName = veterinarianName;
00094         this.clinicName = clinicName;
00095         this.reason = reason;
00096         this.estimatedCost = estimatedCost;
00097         this.confirmed = confirmed;
00098         this.clinicPhone = clinicPhone;
00099     }

```

7.49.3 Member Function Documentation**7.49.3.1 getClinicName()**

```
String com.mehdi.petreminder.model.VetAppointment.getClinicName ()
```

Klinik adı getter.

Returns

Klinik adı

Definition at line 110 of file [VetAppointment.java](#).

```
00110 { return clinicName; }
```

Referenced by [com.mehdi.petreminder.gui.panels.VetPanel.refresh\(\)](#).

7.49.3.2 getClinicPhone()

`String com.mehdi.petreminder.model.VetAppointment.getClinicPhone ()`

Klinik telefonu getter.

Returns

Telefon

Definition at line 134 of file [VetAppointment.java](#).

```
00134 { return clinicPhone; }
```

7.49.3.3 getEstimatedCost()

`double com.mehdi.petreminder.model.VetAppointment.getEstimatedCost ()`

Tahmini maliyet getter.

Returns

Tahmini maliyet (TL)

Definition at line 122 of file [VetAppointment.java](#).

```
00122 { return estimatedCost; }
```

7.49.3.4 getReason()

`String com.mehdi.petreminder.model.VetAppointment.getReason ()`

Muayene sebebi getter.

Returns

Muayene sebebi

Definition at line 116 of file [VetAppointment.java](#).

```
00116 { return reason; }
```

Referenced by [com.mehdi.petreminder.gui.panels.VetPanel.refresh\(\)](#).

7.49.3.5 getReminderType()

`String com.mehdi.petreminder.model.VetAppointment.getReminderType ()`

Hatırlatıcı türü.

Returns

"Veteriner"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 141 of file [VetAppointment.java](#).

```
00141 { return "Veteriner"; }
```

7.49.3.6 getSummary()

```
String com.mehdi.petreminder.model.VetAppointment.getSummary ()
```

Özet bilgi.

Returns

"Veteriner: Dr.Ali - Rize Vet Kliniği (Rutin Muayene)"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 159 of file [VetAppointment.java](#).

```
00159         {  
00160             return String.format("Veteriner: %s - %s (%s)%s",  
00161                 veterinarianName, clinicName, reason,  
00162                 confirmed ? " [Onaylandı]" : " [Bekliyor]");  
00163         }
```

7.49.3.7 getVeterinarianName()

```
String com.mehdi.petreminder.model.VetAppointment.getVeterinarianName ()
```

Veteriner adı getter.

Returns

Veteriner adı

Definition at line 102 of file [VetAppointment.java](#).

```
00102 { return veterinarianName; }
```

Referenced by [com.mehdi.petreminder.gui.panels.VetPanel.refresh\(\)](#).

7.49.3.8 isConfirmed()

```
boolean com.mehdi.petreminder.model.VetAppointment.isConfirmed ()
```

Onay durumu getter.

Returns

Onaylandı mı

Definition at line 128 of file [VetAppointment.java](#).

```
00128 { return confirmed; }
```

Referenced by [com.mehdi.petreminder.gui.panels.VetPanel.refresh\(\)](#).

7.49.3.9 setClinicName()

```
void com.mehdi.petreminder.model.VetAppointment.setClinicName (  
    String clinicName)
```

Klinik adı setter.

Parameters

<i>clinicName</i>	Klinik adı
-------------------	------------

Definition at line 113 of file [VetAppointment.java](#).

```
00113 { this.clinicName = clinicName; }
```

7.49.3.10 setClinicPhone()

```
void com.mehdi.petreminder.model.VetAppointment.setClinicPhone (  
    String clinicPhone)
```

Klinik telefonu setter.

Parameters

<i>clinicPhone</i>	Telefon
--------------------	---------

Definition at line 137 of file [VetAppointment.java](#).

```
00137 { this.clinicPhone = clinicPhone; }
```

7.49.3.11 setConfirmed()

```
void com.mehdi.petreminder.model.VetAppointment.setConfirmed (  
    boolean confirmed)
```

Onay durumu setter.

Parameters

<i>confirmed</i>	Onaylandı mı
------------------	--------------

Definition at line 131 of file [VetAppointment.java](#).

```
00131 { this.confirmed = confirmed; }
```

7.49.3.12 setEstimatedCost()

```
void com.mehdi.petreminder.model.VetAppointment.setEstimatedCost (
    double estimatedCost)
```

Tahmini maliyet setter.

Parameters

<i>estimatedCost</i>	Tahmini maliyet
----------------------	-----------------

Definition at line 125 of file [VetAppointment.java](#).

```
00125 { this.estimatedCost = estimatedCost; }
```

7.49.3.13 setReason()

```
void com.mehdi.petreminder.model.VetAppointment.setReason (
    String reason)
```

Muayene sebebi setter.

Parameters

<i>reason</i>	Muayene sebebi
---------------	----------------

Definition at line 119 of file [VetAppointment.java](#).

```
00119 { this.reason = reason; }
```

7.49.3.14 setVeterinarianName()

```
void com.mehdi.petreminder.model.VetAppointment.setVeterinarianName (
    String veterinarianName)
```

Veteriner adı setter.

Parameters

<i>veterinarianName</i>	Veteriner adı
-------------------------	---------------

Definition at line 105 of file [VetAppointment.java](#).

```
00105
00106         this.veterinarianName = veterinarianName;
00107     }
```

7.49.3.15 toString()

```
String com.mehdi.petreminder.model.VetAppointment.toString ()
```

String temsili.

Returns

[VetAppointment](#) bilgisi

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 167 of file [VetAppointment.java](#).

```
00167         {
00168             return String.format(
00169                 "VetAppointment{id=%d, pet='%s', vet='%s', clinic='%s', reason='%s'}",
00170                 getId(), getPetName(), veterinarianName, clinicName, reason);
00171         }
```

References [com.mehdi.petreminder.model.Reminder.getId\(\)](#), and [com.mehdi.petreminder.model.Reminder.getPetName\(\)](#).

7.49.3.16 validate()

```
boolean com.mehdi.petreminder.model.VetAppointment.validate ()
```

Veteriner randevusunu doğrular.

Returns

Veteriner ve klinik bilgileri dolu ise true

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 148 of file [VetAppointment.java](#).

```
00148         {
00149             return veterinarianName != null && !veterinarianName.trim().isEmpty()
00150                 && clinicName != null && !clinicName.trim().isEmpty()
00151                 && reason != null && !reason.trim().isEmpty();
00152         }
```

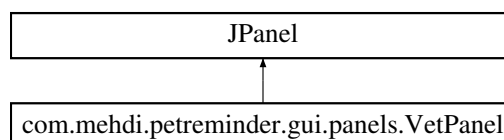
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/VetAppointment.java](#)

7.50 com.mehdi.petreminder.gui.panels.VetPanel Class Reference

Veteriner randevuları CRUD.

Inheritance diagram for [com.mehdi.petreminder.gui.panels.VetPanel](#):



Public Member Functions

- [VetPanel](#) ([MainFrame](#) mainFrame)
VetPanel yapıcısı.
- void [refresh](#) ()
Tabloyu yeniler.

7.50.1 Detailed Description

Veteriner randevuları CRUD.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 29 of file [VetPanel.java](#).

7.50.2 Constructor & Destructor Documentation

7.50.2.1 VetPanel()

```
com.mehdi.petreminder.gui.panels.VetPanel.VetPanel (
    MainFrame mainFrame)
```

[VetPanel](#) yapıcısı.

Parameters

<i>mainFrame</i>	Ana pencere
------------------	-------------

Definition at line 50 of file [VetPanel.java](#).

```
00050      {
00051          this.mainFrame      = mainFrame;
00052          this.reminderService = new ReminderService();
00053          setBackground(GuiConstants.BG\_COLOR);
00054          setLayout(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));
00055          setBorder(new EmptyBorder(GuiConstants.GAP\_LARGE, GuiConstants.GAP\_LARGE,
00056                                  GuiConstants.GAP\_LARGE, GuiConstants.GAP\_LARGE));
00057          buildUI();
00058      }
```

References [com.mehdi.petreminder.gui.util.GuiConstants.BG_COLOR](#), [com.mehdi.petreminder.gui.util.GuiConstants.GAP](#), and [com.mehdi.petreminder.gui.util.GuiConstants.GAP_LARGE](#).

7.50.3 Member Function Documentation

7.50.3.1 refresh()

```
void com.mehdi.petreminder.gui.panels.VetPanel.refresh ()
```

Tabloyu yeniler.

Definition at line 171 of file [VetPanel.java](#).

```
00171         {
00172             tableModel.setRowCount(0);
00173             try {
00174                 List<Reminder> all = reminderService.getAllReminders();
00175                 List<VetAppointment> vets = all.stream()
00176                     .filter(r -> r instanceof VetAppointment)
00177                     .map(r -> (VetAppointment) r)
00178                     .collect(Collectors.toList());
00179                 for (VetAppointment va : vets) {
00180                     tableModel.addRow(new Object[]{
00181                         va.getId(), va.getPetName(), va.getVeterinarianName(),
00182                         va.getClinicName(), va.getReason(), va.getFormattedTime(),
00183                         va.isConfirmed() ? "" : ""
00184                     });
00185                 }
00186             } catch (Exception e) { /* sessiz */ }
00187         }
```

References [com.mehdi.petreminder.model.VetAppointment.getClinicName\(\)](#), [com.mehdi.petreminder.model.Reminder.getFormattedTime\(\)](#), [com.mehdi.petreminder.model.Reminder.getId\(\)](#), [com.mehdi.petreminder.model.Reminder.getPetName\(\)](#), [com.mehdi.petreminder.model.VetAppointment.getReason\(\)](#), [com.mehdi.petreminder.model.VetAppointment.getVeterinarianName\(\)](#), and [com.mehdi.petreminder.model.VetAppointment.isConfirmed\(\)](#).

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/VetPanel.java](#)

Chapter 8

File Documentation

8.1 Generated.java

```
00001 package com.mehdi.petreminder.annotation;
00002
00003 import java.lang.annotation.Documented;
00004 import java.lang.annotation.Retention;
00005 import java.lang.annotation.RetentionPolicy;
00006 import java.lang.annotation.Target;
00007 import static java.lang.annotation.ElementType.METHOD;
00008 import static java.lang.annotation.ElementType.TYPE;
00009 import static java.lang.annotation.ElementType.CONSTRUCTOR;
00010
00016 @Documented
00017 @Retention(RetentionPolicy.RUNTIME)
00018 @Target({TYPE, METHOD, CONSTRUCTOR})
00019 public @interface Generated {
00020 }
```

8.2 petreminder-app/src/main/java/com/mehdi/petreminder/Generated.java File Reference ↩

JaCoCo coverage exclusion annotation.

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
```

Classes

- interface [com.mehdi.petreminder.Generated](#)
Marks methods/constructors to be excluded from JaCoCo coverage.

8.2.1 Detailed Description

JaCoCo coverage exclusion annotation.

JaCoCo 0.8.2+ skips any method/class annotated with an annotation whose simple name is "Generated". This is used to exclude structurally unreachable code paths (e.g. methods requiring a real system terminal, Swing display, or Docker MySQL server) from coverage reporting, so the project can reach 100% on testable code.

Definition in file [Generated.java](#).

8.3 Generated.java

[Go to the documentation of this file.](#)

```
00001
00010 package com.mehdi.petreminder;
00011
00012 import java.lang.annotation.ElementType;
00013 import java.lang.annotation.Retention;
00014 import java.lang.annotation.RetentionPolicy;
00015 import java.lang.annotation.Target;
00016
00028 @Retention(RetentionPolicy.RUNTIME)
00029 @Target({ElementType.METHOD, ElementType.CONSTRUCTOR, ElementType.TYPE})
00030 public @interface Generated {
00031 }
```

8.4 petreminder-app/src/main/java/com/mehdi/petreminder/config/StorageConfig.java File Reference

Storage backend konfigürasyonu — runtime switch destekler.

```
import com.mehdi.petreminder.annotation.Generated;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.io.*;
import java.util.Properties;
```

Classes

- class [com.mehdi.petreminder.config.StorageConfig](#)
Aktif storage backend'i yönetir.

8.4.1 Detailed Description

Storage backend konfigürasyonu — runtime switch destekler.

Definition in file [StorageConfig.java](#).

8.5 StorageConfig.java

[Go to the documentation of this file.](#)

```
00001
00005 package com.mehdi.petreminder.config;
00006
00007 import com.mehdi.petreminder.annotation.Generated;
00008
00009 import org.slf4j.LoggerFactory;
00010 import ch.qos.logback.classic.Logger;
00011
00012 import java.io.*;
00013 import java.util.Properties;
00014
00025 public class StorageConfig {
00026
00028     private static final Logger logger =
00029         (Logger) LoggerFactory.getLogger(StorageConfig.class);
```

```

00030
00032     private static final String CONFIG_FILE = "config/storage.properties";
00033
00035     private static StorageType activeBackend = StorageType.BINARY;
00036
00038     private static String mysqlUrl =
00039         "jdbc:mysql://localhost:3306/petreminder_db?useSSL=false&serverTimezone=UTC";
00040
00042     private static String mysqlUsername = "petreminder_user";
00043
00045     private static String mysqlPassword = "petreminder_pass";
00046
00048     private static String sqliteFilePath = "data/petreminder.db";
00049
00051     private static String binaryDirectory = "data/binary";
00052
00053     static {
00054         loadFromFile();
00055     }
00056
00058     private StorageConfig() { }
00059
00064     public static StorageType getActiveBackend() {
00065         return activeBackend;
00066     }
00067
00072     public static void setActiveBackend(StorageType type) {
00073         if (type != null) {
00074             activeBackend = type;
00075             saveToFile();
00076             logger.info("Storage backend değiştirildi: {}", type);
00077         }
00078     }
00079
00084     public static String getMySQLUrl() { return mysqlUrl; }
00085
00090     public static void setMySQLUrl(String url) { mysqlUrl = url; }
00091
00096     public static String getMySQLUsername() { return mysqlUsername; }
00097
00102     public static String getMySQLPassword() { return mysqlPassword; }
00103
00108     public static String getSqliteFilePath() { return sqliteFilePath; }
00109
00114     public static String getBinaryDirectory() { return binaryDirectory; }
00115
00119     @com.mehdi.petreminder.annotation.Generated
00120     public static void loadFromFile() {
00121         File file = new File(CONFIG_FILE);
00122         if (!file.exists()) {
00123             logger.debug("Config dosyası bulunamadı, varsayılanlar kullanılıyor.");
00124             return;
00125         }
00126         Properties props = new Properties();
00127         try (FileInputStream fis = new FileInputStream(file)) {
00128             props.load(fis);
00129             String backend = props.getProperty("storage.backend", "BINARY");
00130             try {
00131                 activeBackend = StorageType.valueOf(backend.toUpperCase());
00132             } catch (IllegalArgumentException e) {
00133                 activeBackend = StorageType.BINARY;
00134             }
00135             mysqlUrl = props.getProperty("mysql.url", mysqlUrl);
00136             mysqlUsername = props.getProperty("mysql.username", mysqlUsername);
00137             mysqlPassword = props.getProperty("mysql.password", mysqlPassword);
00138             sqliteFilePath = props.getProperty("sqlite.path", sqliteFilePath);
00139             binaryDirectory = props.getProperty("binary.dir", binaryDirectory);
00140             logger.info("Config yüklendi: backend={}, activeBackend");
00141         } catch (IOException e) {
00142             logger.warn("Config yüklenemedi: {}", e.getMessage());
00143         }
00144     }
00145
00149     @com.mehdi.petreminder.annotation.Generated
00150     public static void saveToFile() {
00151         File dir = new File("config");
00152         if (!dir.exists()) dir.mkdirs();
00153         Properties props = new Properties();
00154         props.setProperty("storage.backend", activeBackend.name());
00155         props.setProperty("mysql.url", mysqlUrl);
00156         props.setProperty("mysql.username", mysqlUsername);
00157         props.setProperty("mysql.password", mysqlPassword);
00158         props.setProperty("sqlite.path", sqliteFilePath);
00159         props.setProperty("binary.dir", binaryDirectory);
00160         try (FileOutputStream fos = new FileOutputStream(CONFIG_FILE)) {
00161             props.store(fos, "PetReminder Storage Configuration");
00162             logger.debug("Config kaydedildi.");

```

```

00163         } catch (IOException e) {
00164             logger.warn("Config kaydedilemedi: {}", e.getMessage());
00165         }
00166     }
00167
00171     public static void reset() {
00172         activeBackend = StorageType.BINARY;
00173         mysqlUrl =
00174             "jdbc:mysql://localhost:3306/petreminder_db?useSSL=false&serverTimezone=UTC";
00175         mysqlUsername = "petreminder_user";
00176         mysqlPassword = "petreminder_pass";
00177         sqliteFilePath = "data/petreminder.db";
00178         binaryDirectory = "data/binary";
00179     }

```

8.6 petreminder-app/src/main/java/com/mehdi/petreminder/config/StorageType.java File Reference

Storage backend türleri.

Classes

- enum [com.mehdi.petreminder.config.StorageType](#)

8.6.1 Detailed Description

Storage backend türleri.

Definition in file [StorageType.java](#).

8.7 StorageType.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.config;
00006
00018 public enum StorageType {
00019
00021     BINARY("Binary Dosya"),
00022
00024     SQLITE("SQLite"),
00025
00027     MYSQL("MySQL (Docker)");
00028
00030     private final String displayName;
00031
00036     StorageType(String displayName) {
00037         this.displayName = displayName;
00038     }
00039
00044     public String getDisplayName() {
00045         return displayName;
00046     }
00047
00052     @Override
00053     public String toString() {
00054         return displayName;
00055     }
00056 }

```

8.8 petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/PetDialog.java File Reference ↩

[Pet](#) ekleme / düzenleme diyalogu.

```
import com.mehdi.petreminder.gui.util.GuiConstants;
import com.mehdi.petreminder.model.*;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.time.LocalDate;
import java.time.format.DateTimeParseException;
```

Classes

- class [com.mehdi.petreminder.gui.dialogs.PetDialog](#)
[Pet](#) ekleme ve düzenleme modal dialog'u.

8.8.1 Detailed Description

[Pet](#) ekleme / düzenleme diyalogu.

Definition in file [PetDialog.java](#).

8.9 PetDialog.java

[Go to the documentation of this file.](#)

```
00001
00005 package com.mehdi.petreminder.gui.dialogs;
00006
00007 import com.mehdi.petreminder.gui.util.GuiConstants;
00008 import com.mehdi.petreminder.model.*;
00009
00010 import javax.swing.*;
00011 import javax.swing.border.EmptyBorder;
00012 import java.awt.*;
00013 import java.time.LocalDate;
00014 import java.time.format.DateTimeParseException;
00015
00025 public class PetDialog extends JDialog {
00026
00028     private static final long serialVersionUID = 1L;
00029
00031     private boolean confirmed = false;
00032
00034     private Pet resultPet;
00035
00037     private JTextField txtName, txtBreed, txtBirthDate, txtGender, txtWeight, txtNotes;
00039     private JComboBox<String> cmbSpecies;
00041     private JCheckBox chkTrained;
00043     private JCheckBox chkIndoor;
00045     private JCheckBox chkCanTalk;
00046
00048     private final Pet editPet;
00049
00055     public PetDialog(Frame parent, Pet editPet) {
00056         super(parent, editPet == null ? "Yeni Hayvan Ekle" : "Hayvanı Düzenle", true);
00057         this.editPet = editPet;
00058         buildUI();
00059         if (editPet != null) fillForm(editPet);
00060         pack();
```

```

00061         setLocationRelativeTo(parent);
00062         setMinimumSize(new Dimension(420, 500));
00063     }
00064
00065     private void buildUI() {
00066         JPanel main = new JPanel(new BorderLayout(8, 8));
00067         main.setBorder(new EmptyBorder(20, 20, 20, 20));
00071
00072         main.add(buildForm(), BorderLayout.CENTER);
00073         main.add(buildButtons(), BorderLayout.SOUTH);
00074         setContentPane(main);
00075     }
00076
00077     private JPanel buildForm() {
00078         JPanel p = new JPanel(new GridBagLayout());
00079         GridBagConstraints gbc = new GridBagConstraints();
00080         gbc.insets = new Insets(5, 5, 5, 5);
00081         gbc.anchor = GridBagConstraints.WEST;
00082         gbc.fill = GridBagConstraints.HORIZONTAL;
00083
00084         int row = 0;
00085         txtName = addField(p, gbc, row++, "Ad *:", new JTextField(20));
00086         cmbSpecies = new JComboBox<>(new String[]{"Dog", "Cat", "Bird"});
00087         cmbSpecies.addActionListener(e -> updateSpecificFields());
00088         addLabel(p, gbc, row, "Tür *:");
00089         gbc.gridx = 1; gbc.gridy = row++;
00090         p.add(cmbSpecies, gbc);
00091
00092         txtBreed = addField(p, gbc, row++, "İrk / Tür:", new JTextField(20));
00093         txtBirthDate = addField(p, gbc, row++, "Doğum Tarihi\n(yyyy-MM-dd):", new JTextField(20));
00094         txtGender = addField(p, gbc, row++, "Cinsiyet:", new JTextField(20));
00095         txtWeight = addField(p, gbc, row++, "Ağırlık (kg):", new JTextField(20));
00096
00097         chkTrained = new JCheckBox("Eğitimli");
00098         chkIndoor = new JCheckBox("Ev Kedisi");
00099         chkCanTalk = new JCheckBox("Konuşuyor");
00100
00101         addLabel(p, gbc, row, "Özellik:");
00102         gbc.gridx = 1; gbc.gridy = row++;
00103         JPanel checkPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));
00104         checkPanel.setOpaque(false);
00105         checkPanel.add(chkTrained);
00106         checkPanel.add(chkIndoor);
00107         checkPanel.add(chkCanTalk);
00108         p.add(checkPanel, gbc);
00109
00110         txtNotes = addField(p, gbc, row, "Notlar:", new JTextField(20));
00111         updateSpecificFields();
00112         return p;
00113     }
00114
00115     private void updateSpecificFields() {
00116         String species = (String) cmbSpecies.getSelectedItem();
00117         chkTrained.setEnabled("Dog".equals(species));
00118         chkIndoor.setEnabled("Cat".equals(species));
00119         chkCanTalk.setEnabled("Bird".equals(species));
00120     }
00121
00122     private JTextField addField(JPanel p, GridBagConstraints gbc,
00123                                int row, String label, JTextField field) {
00124         addLabel(p, gbc, row, label);
00125         gbc.gridx = 1; gbc.gridy = row;
00126         p.add(field, gbc);
00127         return field;
00128     }
00129
00130     private void addLabel(JPanel p, GridBagConstraints gbc, int row, String text) {
00131         gbc.gridx = 0; gbc.gridy = row;
00132         JLabel lbl = new JLabel(text.replace("\n", " "));
00133         lbl.setFont(GuiConstants.BODY_FONT);
00134         p.add(lbl, gbc);
00135     }
00136
00137     private JPanel buildButtons() {
00138         JPanel p = new JPanel(new FlowLayout(FlowLayout.RIGHT));
00139         JButton btnOk = new JButton("Kaydet");
00140         btnOk.setBackground(GuiConstants.PRIMARY);
00141         btnOk.setForeground(Color.WHITE);
00142         btnOk.setFocusPainted(false);
00143         btnOk.addActionListener(e -> onSave());
00144
00145         JButton btnCancel = new JButton("İptal");
00146         btnCancel.addActionListener(e -> dispose());
00147
00148         p.add(btnCancel);
00149         p.add(btnOk);
00150         return p;
00151     }

```



```

00178     }
00179
00184     private void fillForm(Pet pet) {
00185         txtName.setText(pet.getName());
00186         cmbSpecies.setSelectedItem(pet.getSpecies());
00187         txtBreed.setText(getBreed(pet));
00188         if (pet.getBirthDate() != null) {
00189             txtBirthDate.setText(pet.getBirthDate().toString());
00190         }
00191         txtGender.setText(pet.getGender() != null ? pet.getGender() : "");
00192         txtWeight.setText(String.valueOf(pet.getWeight()));
00193         txtNotes.setText(pet.getNotes() != null ? pet.getNotes() : "");
00194         if (pet instanceof Dog) chkTrained.setSelected(((Dog) pet).isTrained());
00195         if (pet instanceof Cat) chkIndoor.setSelected(((Cat) pet).isIndoor());
00196         if (pet instanceof Bird) chkCanTalk.setSelected(((Bird) pet).isCanTalk());
00197         updateSpecificFields();
00198     }
00199
00205     private String getBreed(Pet pet) {
00206         if (pet instanceof Dog) return ((Dog) pet).getBreed() != null ? ((Dog) pet).getBreed() : "";
00207         if (pet instanceof Cat) return ((Cat) pet).getBreed() != null ? ((Cat) pet).getBreed() : "";
00208         if (pet instanceof Bird) return ((Bird) pet).getBirdType() != null ? ((Bird)
pet).getBirdType() : "";
00209         return "";
00210     }
00211
00215     private void onSave() {
00216         String name = txtName.getText().trim();
00217         if (name.isEmpty()) {
00218             JOptionPane.showMessageDialog(this, "Ad boş olamaz!", "Hata",
JOptionPane.ERROR_MESSAGE);
00219             return;
00220         }
00221
00222         LocalDate birthDate = null;
00223         String bdStr = txtBirthDate.getText().trim();
00224         if (!bdStr.isEmpty()) {
00225             try {
00226                 birthDate = LocalDate.parse(bdStr);
00227             } catch (DateTimeParseException e) {
00228                 JOptionPane.showMessageDialog(this, "Tarih formatı: yyyy-MM-dd", "Hata",
JOptionPane.ERROR_MESSAGE);
00229                 return;
00230             }
00231         }
00232     }
00233
00234         double weight = 0;
00235         try {
00236             if (!txtWeight.getText().trim().isEmpty()) {
00237                 weight = Double.parseDouble(txtWeight.getText().trim());
00238             }
00239         } catch (NumberFormatException e) {
00240             JOptionPane.showMessageDialog(this, "Ağırlık sayısal olmalı", "Hata",
JOptionPane.ERROR_MESSAGE);
00241             return;
00242         }
00243     }
00244
00245         String species = (String) cmbSpecies.getSelectedItem();
00246         int ownerId = editPet != null ? editPet.getOwnerId() : 1;
00247
00248         Pet pet;
00249         if ("Dog".equals(species)) {
00250             Dog dog = new Dog(editPet != null ? editPet.getId() : 0, name, birthDate, ownerId);
00251             dog.setBreed(txtBreed.getText().trim());
00252             dog.setTrained(chkTrained.isSelected());
00253             pet = dog;
00254         } else if ("Cat".equals(species)) {
00255             Cat cat = new Cat(editPet != null ? editPet.getId() : 0, name, birthDate, ownerId);
00256             cat.setBreed(txtBreed.getText().trim());
00257             cat.setIndoor(chkIndoor.isSelected());
00258             pet = cat;
00259         } else {
00260             Bird bird = new Bird(editPet != null ? editPet.getId() : 0, name, birthDate, ownerId);
00261             bird.setBirdType(txtBreed.getText().trim());
00262             bird.setCanTalk(chkCanTalk.isSelected());
00263             pet = bird;
00264         }
00265
00266         pet.setGender(txtGender.getText().trim());
00267         pet.setWeight(weight);
00268         pet.setNotes(txtNotes.getText().trim());
00269
00270         resultPet = pet;
00271         confirmed = true;
00272         dispose();
00273     }
00274
00279     public boolean isConfirmed() { return confirmed; }

```

```

00280
00285     public Pet getPet() { return resultPet; }
00286 }

```

8.10 petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/ReminderDialog.java File Reference

Hatırlatıcı ekleme / düzenleme diyalogu.

```

import com.mehdi.petreminder.gui.util.GuiConstants;
import com.mehdi.petreminder.model.*;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeParseException;

```

Classes

- class [com.mehdi.petreminder.gui.dialogs.ReminderDialog](#)
Reminder ekleme ve düzenleme modal dialog'u.

8.10.1 Detailed Description

Hatırlatıcı ekleme / düzenleme diyalogu.

Definition in file [ReminderDialog.java](#).

8.11 ReminderDialog.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.gui.dialogs;
00006
00007 import com.mehdi.petreminder.gui.util.GuiConstants;
00008 import com.mehdi.petreminder.model.*;
00009
00010 import javax.swing.*;
00011 import javax.swing.border.EmptyBorder;
00012 import java.awt.*;
00013 import java.time.LocalDateTime;
00014 import java.time.format.DateTimeParseException;
00015
00025 public class ReminderDialog extends JDialog {
00026
00028     private static final long serialVersionUID = 1L;
00029
00031     private boolean confirmed = false;
00032
00034     private Reminder resultReminder;
00035
00036     // Ortak alanlar
00038     private JTextField txtPetId;
00040     private JTextField txtPetName;
00042     private JTextField txtTime;

```

```

00044     private JComboBox<String> cmbPriority;
00046     private JComboBox<String> cmbType;
00048     private JTextField txtDesc;
00049
00050     // Tür-özel alanlar
00052     private JTextField txtFoodType, txtPortionGrams;
00054     private JTextField txtMedName, txtDosage, txtDosageUnit;
00056     private JTextField txtExerciseType, txtDuration;
00058     private JTextField txtGroomingType;
00060     private JTextField txtVetName, txtClinic, txtReason;
00061
00063     private JPanel specificPanel;
00064
00066     private final Reminder editReminder;
00067
00073     public ReminderDialog(Frame parent, Reminder editReminder) {
00074         super(parent, editReminder == null ? "Yeni Hatırlatıcı" : "Hatırlatıcı Düzenle", true);
00075         this.editReminder = editReminder;
00076         buildUI();
00077         if (editReminder != null) fillForm(editReminder);
00078         pack();
00079         setLocationRelativeTo(parent);
00080         setMinimumSize(new Dimension(440, 480));
00081     }
00082
00086     private void buildUI() {
00087         JPanel main = new JPanel(new BorderLayout(8, 8));
00088         main.setBorder(new EmptyBorder(20, 20, 20, 20));
00089         main.add(buildCommonForm(), BorderLayout.NORTH);
00090         specificPanel = new JPanel(new GridBagLayout());
00091         main.add(new JScrollPane(specificPanel), BorderLayout.CENTER);
00092         main.add(buildButtons(), BorderLayout.SOUTH);
00093         setContentPane(main);
00094     }
00095
00100     private JPanel buildCommonForm() {
00101         JPanel p = new JPanel(new GridBagLayout());
00102         GridBagConstraints gbc = new GridBagConstraints();
00103         gbc.insets = new Insets(4, 4, 4, 4);
00104         gbc.fill = GridBagConstraints.HORIZONTAL;
00105         gbc.anchor = GridBagConstraints.WEST;
00106
00107         int row = 0;
00108         txtPetId = addField(p, gbc, row++, "Pet ID:", new JTextField("1", 10));
00109         txtPetName = addField(p, gbc, row++, "Pet Adı:", new JTextField(10));
00110         txtTime = addField(p, gbc, row++, "Zaman (yyyy-MM-ddTHH:mm):",
00111             new JTextField(LocalDate.now().plusHours(1).toString().substring(0, 16), 10));
00112
00113         cmbPriority = new JComboBox<>(new String[]{"YUKSEK", "ORTA", "DUSUK"});
00114         cmbPriority.setSelectedItem("ORTA");
00115         addLabel(p, gbc, row, "Öncelik:");
00116         gbc.gridx = 1; gbc.gridy = row++;
00117         p.add(cmbPriority, gbc);
00118
00119         cmbType = new JComboBox<>(new String[]{"Besleme", "İlac", "Egzersiz", "Bakim", "Veteriner"});
00120         cmbType.addActionListener(e -> updateSpecificFields());
00121         addLabel(p, gbc, row, "Tür:");
00122         gbc.gridx = 1; gbc.gridy = row++;
00123         p.add(cmbType, gbc);
00124
00125         txtDesc = addField(p, gbc, row, "Açıklama:", new JTextField(10));
00126         return p;
00127     }
00128
00132     private void updateSpecificFields() {
00133         specificPanel.removeAll();
00134         String type = (String) cmbType.getSelectedItem();
00135         GridBagConstraints gbc = new GridBagConstraints();
00136         gbc.insets = new Insets(4, 4, 4, 4);
00137         gbc.fill = GridBagConstraints.HORIZONTAL;
00138         gbc.anchor = GridBagConstraints.WEST;
00139
00140         int row = 0;
00141         switch (type != null ? type : "") {
00142             case "Besleme":
00143                 txtFoodType = addField(specificPanel, gbc, row++, "Besin Türü:", new
00144                     JTextField(10));
00145                 txtPortionGrams = addField(specificPanel, gbc, row, "Porsiyon (g):", new
00146                     JTextField("100", 10));
00147                 break;
00148             case "İlac":
00149                 txtMedName = addField(specificPanel, gbc, row++, "İlaç Adı:", new JTextField(10));
00150                 txtDosage = addField(specificPanel, gbc, row++, "Doz:", new JTextField("1.0", 10));
00151                 txtDosageUnit = addField(specificPanel, gbc, row, "Birim:", new JTextField("mg", 10));
00152                 break;
00153             case "Egzersiz":
00154                 txtExerciseType = addField(specificPanel, gbc, row++, "Egzersiz Türü:", new

```

```

        JTextField("Yürüyüş", 10));
00153         txtDuration      = addField(specificPanel, gbc, row, "Süre (dk):", new JTextField("30",
10));
00154         break;
00155         case "Bakim":
00156             txtGroomingType = addField(specificPanel, gbc, row, "Bakım Türü:", new
JTextField("Tıraş", 10));
00157             break;
00158             case "Veteriner":
00159                 txtVetName = addField(specificPanel, gbc, row++, "Veteriner:", new JTextField(10));
00160                 txtClinic  = addField(specificPanel, gbc, row++, "Klinik:", new JTextField(10));
00161                 txtReason  = addField(specificPanel, gbc, row, "Sebebi:", new JTextField("Kontrol",
10));
00162             break;
00163             default:
00164                 break;
00165         }
00166         specificPanel.revalidate();
00167         specificPanel.repaint();
00168         pack();
00169     }
00170
00171     private void fillForm(Reminder r) {
00172         txtPetId.setText(String.valueOf(r.getPetId()));
00173         txtPetName.setText(r.getPetName() != null ? r.getPetName() : "");
00174         if (r.getScheduledTime() != null) {
00175             txtTime.setText(r.getScheduledTime().toString().substring(0, 16));
00176         }
00177         cmbPriority.setSelectedItem(r.getPriority());
00178         cmbType.setSelectedItem(r.getReminderType());
00179         updateSpecificFields();
00180         txtDesc.setText(r.getDescription() != null ? r.getDescription() : "");
00181     }
00182
00183     private void onSave() {
00184         try {
00185             int petId = Integer.parseInt(txtPetId.getText().trim());
00186             String petName = txtPetName.getText().trim();
00187             LocalDateTime time = LocalDateTime.parse(txtTime.getText().trim());
00188             String priority = (String) cmbPriority.getSelectedItem();
00189             String type = (String) cmbType.getSelectedItem();
00190             String desc = txtDesc.getText().trim();
00191             int editId = editReminder != null ? editReminder.getId() : 0;
00192
00193             Reminder r;
00194             switch (type != null ? type : "") {
00195                 case "Besleme":
00196                     String food = txtFoodType != null ? txtFoodType.getText().trim() : "Mama";
00197                     double portion = txtPortionGrams != null
00198                         ? Double.parseDouble(txtPortionGrams.getText().trim()) : 100;
00199                     r = new FeedingReminder(editId, petId, petName, time, desc,
00200                         priority, false, "", food, portion, false);
00201                     break;
00202                 case "İlaç":
00203                     String med = txtMedName != null ? txtMedName.getText().trim() : "İlaç";
00204                     double dose = txtDosage != null
00205                         ? Double.parseDouble(txtDosage.getText().trim()) : 1.0;
00206                     String unit = txtDosageUnit != null ? txtDosageUnit.getText().trim() : "mg";
00207                     r = new MedicationReminder(editId, petId, petName, time, desc,
00208                         priority, false, "", med, dose, unit, false, null);
00209                     break;
00210                 case "Egzersiz":
00211                     String exType = txtExerciseType != null ? txtExerciseType.getText().trim() :
"Yürüyüş";
00212                     int dur = txtDuration != null
00213                         ? Integer.parseInt(txtDuration.getText().trim()) : 30;
00214                     r = new ExerciseReminder(editId, petId, petName, time, desc,
00215                         priority, false, "", exType, dur, 0);
00216                     break;
00217                 case "Bakim":
00218                     String gType = txtGroomingType != null ? txtGroomingType.getText().trim() :
"Tıraş";
00219                     r = new GroomingReminder(editId, petId, petName, time, desc,
00220                         priority, false, "", gType, false, "");
00221                     break;
00222                 case "Veteriner":
00223                     String vet = txtVetName != null ? txtVetName.getText().trim() : "Veteriner";
00224                     String clinic = txtClinic != null ? txtClinic.getText().trim() : "Klinik";
00225                     String reason = txtReason != null ? txtReason.getText().trim() : "Kontrol";
00226                     r = new VetAppointment(editId, petId, petName, time, desc,
00227                         priority, false, "", vet, clinic, reason, 0, false, "");
00228                     break;
00229                 default:
00230                     r = new FeedingReminder(editId, petId, petName, time, "Mama", 100);
00231             }
00232             resultReminder = r;
00233             confirmed = true;
00234         }
00235     }
00236 }

```

```

00241         dispose();
00242     } catch (NumberFormatException e) {
00243         JOptionPane.showMessageDialog(this, "Sayısal alanları kontrol edin.", "Hata",
00244             JOptionPane.ERROR_MESSAGE);
00245     } catch (DateTimeParseException e) {
00246         JOptionPane.showMessageDialog(this, "Zaman formatı: yyyy-MM-ddTHH:mm", "Hata",
00247             JOptionPane.ERROR_MESSAGE);
00248     }
00249 }
00250
00251 private JPanel buildButtons() {
00252     JPanel p = new JPanel(new FlowLayout(FlowLayout.RIGHT));
00253     JButton ok = new JButton("Kaydet");
00254     ok.setBackground(GuiConstants.PRIMARY);
00255     ok.setForeground(Color.WHITE);
00256     ok.setFocusPainted(false);
00257     ok.addActionListener(e -> onSave());
00258     JButton cancel = new JButton("İptal");
00259     cancel.addActionListener(e -> dispose());
00260     p.add(cancel);
00261     p.add(ok);
00262     return p;
00263 }
00264
00265 private JTextField addField(JPanel p, GridBagConstraints gbc, int row, String label, JTextField
field) {
00266     addLabel(p, gbc, row, label);
00267     gbc.gridx = 1; gbc.gridy = row;
00268     p.add(field, gbc);
00269     return field;
00270 }
00271
00272 private void addLabel(JPanel p, GridBagConstraints gbc, int row, String text) {
00273     gbc.gridx = 0; gbc.gridy = row;
00274     p.add(new JLabel(text), gbc);
00275 }
00276
00277 public boolean isConfirmed() { return confirmed; }
00278
00279 public Reminder getReminder() { return resultReminder; }
00280 }

```

8.12 petreminder-app/src/main/java/com/mehdi/petreminder/gui/MainFrame.java File Reference

Ana Swing penceresi — tüm panelleri barındırır.

```

import com.formdev.flatlaf.FlatLightLaf;
import com.mehdi.petreminder.config.StorageConfig;
import com.mehdi.petreminder.config.StorageType;
import com.mehdi.petreminder.gui.panels.*;
import com.mehdi.petreminder.gui.util.GuiConstants;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import javax.swing.*;
import java.awt.*;

```

Classes

- class [com.mehdi.petreminder.gui.MainFrame](#)
Uygulamanın ana Swing penceresi.

Packages

- package [com.mehdi.petreminder.gui](#)

8.12.1 Detailed Description

Ana Swing penceresi — tüm panelleri barındırır.

FlatLaf light tema, sol navigasyon, sağ içerik paneli.

Definition in file [MainFrame.java](#).

8.13 MainFrame.java

[Go to the documentation of this file.](#)

```

00001
00009 package com.mehdi.petreminder.gui;
00010
00011 import com.formdev.flatlaf.FlatLightLaf;
00012 import com.mehdi.petreminder.config.StorageConfig;
00013 import com.mehdi.petreminder.config.StorageType;
00014 import com.mehdi.petreminder.gui.panels.*;
00015 import com.mehdi.petreminder.gui.util.GuiConstants;
00016 import org.slf4j.LoggerFactory;
00017 import ch.qos.logback.classic.Logger;
00018
00019 import javax.swing.*;
00020 import java.awt.*;
00021
00033 public class MainFrame extends JFrame {
00034
00036     private static final Logger logger =
00037         (Logger) LoggerFactory.getLogger(MainFrame.class);
00038
00040     private static final long serialVersionUID = 1L;
00041
00043     private static final String TITLE = "Pet Care Reminder System v1.0";
00044
00046     public static final String CARD_DASHBOARD = "DASHBOARD";
00048     public static final String CARD_PETS = "PETS";
00050     public static final String CARD_REMINDERS = "REMINDERS";
00052     public static final String CARD_VET = "VET";
00054     public static final String CARD_MEDICAL = "MEDICAL";
00056     public static final String CARD_SETTINGS = "SETTINGS";
00057
00059     private JPanel contentPanel;
00060
00062     private CardLayout cardLayout;
00063
00065     private NavPanel navPanel;
00066
00068     private DashboardPanel dashboardPanel;
00069
00071     private PetsPanel petsPanel;
00072
00074     private RemindersPanel remindersPanel;
00075
00077     private VetPanel vetPanel;
00078
00080     private MedicalPanel medicalPanel;
00081
00083     private SettingsPanel settingsPanel;
00084
00088     public MainFrame() {
00089         super(TITLE);
00090         initLookAndFeel();
00091         initComponents();
00092         setupLayout();
00093         setupFrame();
00094         navigateTo(CARD_DASHBOARD);
00095         logger.info("MainFrame başlatıldı.");
00096     }
00097
00101     private void initLookAndFeel() {
00102         try {
00103             FlatLightLaf.setup();
00104             UIManager.put("Button.arc", 8);
00105             UIManager.put("Component.arc", 6);
00106             UIManager.put("TextComponent.arc", 6);
00107             UIManager.put("TabbedPane.tabHeight", 36);
00108         } catch (Exception e) {

```

```

00109         logger.warn("FlatLaf yüklenemedi, varsayılan tema kullanılıyor.");
00110     }
00111 }
00112
00116 private void initComponents() {
00117     cardLayout = new CardLayout();
00118     contentPanel = new JPanel(cardLayout);
00119     contentPanel.setBackground(GuiConstants.BG_COLOR);
00120
00121     dashboardPanel = new DashboardPanel(this);
00122     petsPanel = new PetsPanel(this);
00123     remindersPanel = new RemindersPanel(this);
00124     vetPanel = new VetPanel(this);
00125     medicalPanel = new MedicalPanel(this);
00126     settingsPanel = new SettingsPanel(this);
00127
00128     contentPanel.add(dashboardPanel, CARD_DASHBOARD);
00129     contentPanel.add(petsPanel, CARD_PETS);
00130     contentPanel.add(remindersPanel, CARD_REMINDERS);
00131     contentPanel.add(vetPanel, CARD_VET);
00132     contentPanel.add(medicalPanel, CARD_MEDICAL);
00133     contentPanel.add(settingsPanel, CARD_SETTINGS);
00134
00135     navPanel = new NavPanel(this);
00136 }
00137
00141 private void setupLayout() {
00142     setLayout(new BorderLayout());
00143     add(navPanel, BorderLayout.WEST);
00144     add(contentPanel, BorderLayout.CENTER);
00145     add(buildStatusBar(), BorderLayout.SOUTH);
00146 }
00147
00151 private void setupFrame() {
00152     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
00153     setSize(1100, 700);
00154     setMinimumSize(new Dimension(900, 600));
00155     setLocationRelativeTo(null);
00156     setIconImage(getAppIcon());
00157 }
00158
00163 private Image getAppIcon() {
00164     // Kaynak yoksa null döner -- sorun çıkarmaz
00165     return null;
00166 }
00167
00172 private JPanel buildStatusBar() {
00173     JPanel bar = new JPanel(new BorderLayout());
00174     bar.setBackground(GuiConstants.NAV_COLOR);
00175     bar.setBorder(BorderFactory.createEmptyBorder(4, 12, 4, 12));
00176     bar.setPreferredSize(new Dimension(0, 28));
00177
00178     JLabel storage = new JLabel("Storage: "
00179         + StorageConfig.getActiveBackend().getDisplayName());
00180     storage.setForeground(Color.WHITE);
00181     storage.setFont(GuiConstants.SMALL_FONT);
00182
00183     JLabel version = new JLabel("CEN206 -- Spring 2026 | RTEU");
00184     version.setForeground(new Color(180, 180, 200));
00185     version.setFont(GuiConstants.SMALL_FONT);
00186
00187     bar.add(storage, BorderLayout.WEST);
00188     bar.add(version, BorderLayout.EAST);
00189     return bar;
00190 }
00191
00196 public void navigateTo(String cardName) {
00197     cardLayout.show(contentPanel, cardName);
00198     navPanel.setActiveButton(cardName);
00199     refreshCurrentPanel(cardName);
00200     logger.debug("Navigasyon: {}", cardName);
00201 }
00202
00207 private void refreshCurrentPanel(String cardName) {
00208     switch (cardName) {
00209         case CARD_DASHBOARD: dashboardPanel.refresh(); break;
00210         case CARD_PETS: petsPanel.refresh(); break;
00211         case CARD_REMINDERS: remindersPanel.refresh(); break;
00212         case CARD_VET: vetPanel.refresh(); break;
00213         case CARD_MEDICAL: medicalPanel.refresh(); break;
00214         case CARD_SETTINGS: settingsPanel.refresh(); break;
00215         default: break;
00216     }
00217 }
00218
00223 public void onStorageChanged(StorageType type) {
00224     StorageConfig.setActiveBackend(type);

```

```

00225         // Status bar'ı güncelle
00226         Component[] comps = ((JPanel) getContentPane().getComponent(2)).getComponents();
00227         for (Component c : comps) {
00228             if (c instanceof JLabel && ((JLabel) c).getText().startsWith("Storage:")) {
00229                 ((JLabel) c).setText("Storage: " + type.getDisplayName());
00230             }
00231         }
00232         JOptionPane.showMessageDialog(this,
00233             "Storage backend değiştirildi:\n" + type.getDisplayName(),
00234             "Ayarlar", JOptionPane.INFORMATION_MESSAGE);
00235         logger.info("Storage değişti: {}", type);
00236     }
00237
00241     public static void launch() {
00242         SwingUtilities.invokeLater(() -> {
00243             MainFrame frame = new MainFrame();
00244             frame.setVisible(true);
00245         });
00246     }
00247 }

```

8.14 petreminder- app/src/main/java/com/mehdi/petreminder/gui/panels/DashboardPanel.java File Reference

Dashboard — özet istatistikler ve son hatırlatıcılar.

```

import com.mehdi.petreminder.gui.MainFrame;
import com.mehdi.petreminder.gui.util.GuiConstants;
import com.mehdi.petreminder.model.Reminder;
import com.mehdi.petreminder.service.MedicalRecordService;
import com.mehdi.petreminder.service.PetService;
import com.mehdi.petreminder.service.ReminderService;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.util.List;

```

Classes

- class [com.mehdi.petreminder.gui.panels.DashboardPanel](#)
Ana kontrol paneli — özet kartlar ve bekleyen hatırlatıcılar.

8.14.1 Detailed Description

Dashboard — özet istatistikler ve son hatırlatıcılar.

Definition in file [DashboardPanel.java](#).

8.15 DashboardPanel.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.gui.panels;
00006
00007 import com.mehdi.petreminder.gui.MainFrame;
00008 import com.mehdi.petreminder.gui.util.Constants;
00009 import com.mehdi.petreminder.model.Reminder;
00010 import com.mehdi.petreminder.service.MedicalRecordService;
00011 import com.mehdi.petreminder.service.PetService;
00012 import com.mehdi.petreminder.service.ReminderService;
00013
00014 import javax.swing.*;
00015 import javax.swing.border.EmptyBorder;
00016 import java.awt.*;
00017 import java.util.List;
00018
00027 public class DashboardPanel extends JPanel {
00028
00030     private static final long serialVersionUID = 1L;
00031
00033     private final MainFrame mainFrame;
00034
00036     private final PetService petService;
00038     private final ReminderService reminderService;
00040     private final MedicalRecordService medicalService;
00041
00043     private JLabel lblPetCount, lblReminderCount, lblOverdueCount, lblMedicalCount;
00044
00046     private DefaultListModel<String> pendingModel;
00047
00052     public DashboardPanel(MainFrame mainFrame) {
00053         this.mainFrame = mainFrame;
00054         petService = new PetService();
00055         reminderService = new ReminderService();
00056         medicalService = new MedicalRecordService();
00057         setBackground(Constants.BG_COLOR);
00058         setLayout(new BorderLayout(Constants.GAP, Constants.GAP));
00059         setBorder(new EmptyBorder(Constants.GAP_LARGE, Constants.GAP_LARGE,
00060             Constants.GAP_LARGE, Constants.GAP_LARGE));
00061         buildUI();
00062     }
00063
00067     private void buildUI() {
00068         // Başlık
00069         JLabel title = new JLabel("Dashboard");
00070         title.setFont(Constants.TITLE_FONT);
00071         title.setForeground(Constants.TEXT_PRIMARY);
00072         add(title, BorderLayout.NORTH);
00073
00074         // Orta -- Özet kartlar + liste
00075         JPanel center = new JPanel(new BorderLayout(Constants.GAP, Constants.GAP));
00076         center.setOpaque(false);
00077
00078         center.add(buildSummaryCards(), BorderLayout.NORTH);
00079         center.add(buildPendingSection(), BorderLayout.CENTER);
00080
00081         add(center, BorderLayout.CENTER);
00082     }
00083
00088     private JPanel buildSummaryCards() {
00089         JPanel row = new JPanel(new GridLayout(1, 4, Constants.GAP, 0));
00090         row.setOpaque(false);
00091
00092         lblPetCount = new JLabel("0", SwingConstants.CENTER);
00093         lblReminderCount = new JLabel("0", SwingConstants.CENTER);
00094         lblOverdueCount = new JLabel("0", SwingConstants.CENTER);
00095         lblMedicalCount = new JLabel("0", SwingConstants.CENTER);
00096
00097         row.add(summaryCard(" Hayvanlar", lblPetCount, Constants.PRIMARY));
00098         row.add(summaryCard(" Hatırlatıcılar", lblReminderCount, Constants.SUCCESS));
00099         row.add(summaryCard(" Gecikmiş", lblOverdueCount, Constants.DANGER));
00100         row.add(summaryCard(" Sağlık Kayıtları", lblMedicalCount, Constants.WARNING));
00101         return row;
00102     }
00103
00111     private JPanel summaryCard(String label, JLabel value, Color color) {
00112         JPanel card = new JPanel(new BorderLayout());
00113         card.setBackground(Constants.CARD_BG);
00114         card.setBorder(BorderFactory.createCompoundBorder(
00115             BorderFactory.createLineBorder(Constants.BORDER_COLOR),
00116             new EmptyBorder(20, 20, 20, 20)));
00117
00118         JLabel lbl = new JLabel(label, SwingConstants.CENTER);

```

```

00119         lbl.setFont (GuiConstants.BODY_FONT);
00120         lbl.setForeground (GuiConstants.TEXT_SECONDARY);
00121
00122         value.setFont (new Font ("Segoe UI", Font.BOLD, 36));
00123         value.setForeground (color);
00124
00125         card.add (lbl, BorderLayout.NORTH);
00126         card.add (value, BorderLayout.CENTER);
00127         return card;
00128     }
00129
00130     private JPanel buildPendingSection() {
00131         JPanel panel = new JPanel (new BorderLayout ());
00132         panel.setBackground (GuiConstants.CARD_BG);
00133         panel.setBorder (BorderFactory.createCompoundBorder (
00134             BorderFactory.createLineBorder (GuiConstants.BORDER_COLOR),
00135             new EmptyBorder (16, 16, 16, 16)));
00136
00137         JLabel title = new JLabel ("Bekleyen Hatırlatıcılar");
00138         title.setFont (GuiConstants.SECTION_FONT);
00139         title.setForeground (GuiConstants.TEXT_PRIMARY);
00140         panel.add (title, BorderLayout.NORTH);
00141
00142         pendingModel = new DefaultListModel<>();
00143         JList<String> list = new JList<> (pendingModel);
00144         list.setFont (GuiConstants.BODY_FONT);
00145         list.setSelectionBackground (new Color (238, 242, 255));
00146         panel.add (new JScrollPane (list), BorderLayout.CENTER);
00147
00148         JButton btnGoReminders = new JButton ("Tümünü Gör ➔");
00149         btnGoReminders.setBackground (GuiConstants.PRIMARY);
00150         btnGoReminders.setForeground (Color.WHITE);
00151         btnGoReminders.setFocusPainted (false);
00152         btnGoReminders.addActionListener (e -> mainFrame.navigateTo (MainFrame.CARD_REMINDERS));
00153         JPanel south = new JPanel (new FlowLayout (FlowLayout.RIGHT));
00154         south.setOpaque (false);
00155         south.add (btnGoReminders);
00156         panel.add (south, BorderLayout.SOUTH);
00157
00158         return panel;
00159     }
00160
00161     public void refresh() {
00162         try {
00163             lblPetCount.setText (String.valueOf (petService.getPetCount ()));
00164             lblReminderCount.setText (String.valueOf (reminderService.getReminderCount ()));
00165             List<Reminder> overdue = reminderService.getOverdueReminders ();
00166             lblOverdueCount.setText (String.valueOf (overdue.size ()));
00167             lblMedicalCount.setText (String.valueOf (medicalService.getRecordCount ()));
00168
00169             pendingModel.clear ();
00170             List<Reminder> pending = reminderService.getPendingReminders ();
00171             for (Reminder r : pending) {
00172                 String item = String.format ("%s] %s -- %s %s",
00173                     r.getPriority (),
00174                     r.getPetName (),
00175                     r.getReminderType (),
00176                     r.getFormattedTime ());
00177                 if (r.isOverdue ()) item = " " + item;
00178                 pendingModel.addElement (item);
00179             }
00180         } catch (Exception e) {
00181             // Veri yoksa sessizce devam et
00182         }
00183     }
00184 }
00185
00186 }
00187
00188 }
00189
00190 }
00191 }

```

8.16 petreminder- app/src/main/java/com/mehdi/petreminder/gui/panels/Medical Panel.java File Reference

Sağlık kayıtları yönetim paneli.

```

import com.mehdi.petreminder.gui.MainFrame;
import com.mehdi.petreminder.gui.util.GuiConstants;
import com.mehdi.petreminder.model.MedicalRecord;

```

```

import com.mehdi.petreminder.service.MedicalRecordService;
import com.mehdi.petreminder.service.ServiceException;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.time.LocalDate;
import java.util.List;

```

Classes

- class [com.mehdi.petreminder.gui.panels.MedicalPanel](#)
Sağlık kayıtları CRUD paneli.

8.16.1 Detailed Description

Sağlık kayıtları yönetim paneli.

Definition in file [MedicalPanel.java](#).

8.17 MedicalPanel.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.gui.panels;
00006
00007 import com.mehdi.petreminder.gui.MainFrame;
00008 import com.mehdi.petreminder.gui.util.Constants;
00009 import com.mehdi.petreminder.model.MedicalRecord;
00010 import com.mehdi.petreminder.service.MedicalRecordService;
00011 import com.mehdi.petreminder.service.ServiceException;
00012
00013 import javax.swing.*;
00014 import javax.swing.border.EmptyBorder;
00015 import javax.swing.table.DefaultTableModel;
00016 import java.awt.*;
00017 import java.time.LocalDate;
00018 import java.util.List;
00019
00028 public class MedicalPanel extends JPanel {
00029
00031     private static final long serialVersionUID = 1L;
00032
00034     private final MainFrame mainFrame;
00035
00037     private final MedicalRecordService medicalService;
00038
00040     private DefaultTableModel tableModel;
00041
00043     private JTable table;
00044
00049     public MedicalPanel(MainFrame mainFrame) {
00050         this.mainFrame = mainFrame;
00051         this.medicalService = new MedicalRecordService();
00052         setBackground(Constants.BG_COLOR);
00053         setLayout(new BorderLayout(Constants.GAP, Constants.GAP));
00054         setBorder(new EmptyBorder(Constants.GAP_LARGE, Constants.GAP_LARGE,
00055             Constants.GAP_LARGE, Constants.GAP_LARGE));
00056         buildUI();
00057     }
00058
00062     private void buildUI() {
00063         JLabel title = new JLabel(" Sağlık Kayıtları");
00064         title.setFont(Constants.TITLE_FONT);
00065         add(title, BorderLayout.NORTH);

```

```

00066         add(buildTable(), BorderLayout.CENTER);
00067         add(buildToolBar(), BorderLayout.SOUTH);
00068     }
00069
00070     private JScrollPane buildTable() {
00071         String[] cols = {"ID", "Pet", "Tür", "Teşhis", "Veteriner", "Tarih", "Maliyet ()"};
00072         tableModel = new DefaultTableModel(cols, 0) {
00073             @Override public boolean isCellEditable(int r, int c) { return false; }
00074         };
00075         table = new JTable(tableModel);
00076         table.setRowHeight(30);
00077         table.setFont(GuiConstants.BODY_FONT);
00078         table.setGridColor(GuiConstants.BORDER_COLOR);
00079         return new JScrollPane(table);
00080     }
00081
00082     private JPanel buildToolBar() {
00083         JPanel bar = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
00084         bar.setOpaque(false);
00085         bar.add(btn(" Yeni Kayıt", GuiConstants.PRIMARY, e -> addRecord()));
00086         bar.add(btn(" Sil", GuiConstants.DANGER, e -> deleteSelected()));
00087         return bar;
00088     }
00089
00090     private void addRecord() {
00091         JTextField petId = new JTextField("1");
00092         JTextField petName = new JTextField();
00093         JComboBox<String> recType = new JComboBox<>(new String[]{
00094             "KONTROL", "ASII", "AMELİYAT", "HASTALIK", "DİĞER"});
00095         JTextField diagnosis = new JTextField();
00096         JTextField vet = new JTextField();
00097         JTextField cost = new JTextField("0");
00098         JTextField date = new JTextField(LocalDate.now().toString());
00099
00100         Object[] fields = {
00101             "Pet ID:", petId, "Pet Adı:", petName,
00102             "Kayıt Türü:", recType, "Teşhis:", diagnosis,
00103             "Veteriner:", vet, "Maliyet ():", cost,
00104             "Tarih (yyyy-MM-dd):", date
00105         };
00106         int res = JOptionPane.showConfirmDialog(this, fields, "Yeni Sağlık Kaydı",
00107             JOptionPane.OK_CANCEL_OPTION);
00108         if (res == JOptionPane.OK_OPTION) {
00109             try {
00110                 MedicalRecord rec = new MedicalRecord(0,
00111                     Integer.parseInt(petId.getText().trim()),
00112                     petName.getText().trim(),
00113                     LocalDate.parse(date.getText().trim()),
00114                     (String) recType.getSelectedItem(),
00115                     diagnosis.getText().trim(),
00116                     vet.getText().trim());
00117                 rec.setCost(Double.parseDouble(cost.getText().trim()));
00118                 medicalService.addRecord(rec);
00119                 refresh();
00120             } catch (Exception ex) {
00121                 JOptionPane.showMessageDialog(this, "Hata: " + ex.getMessage(), "Hata",
00122                     JOptionPane.ERROR_MESSAGE);
00123             }
00124         }
00125     }
00126
00127     private void deleteSelected() {
00128         int row = table.getSelectedRow();
00129         if (row < 0) return;
00130         int confirm = JOptionPane.showConfirmDialog(this, "Silinsin mi?", "Onayla",
00131             JOptionPane.YES_NO_OPTION);
00132         if (confirm == JOptionPane.YES_OPTION) {
00133             try {
00134                 medicalService.deleteRecord((int) tableModel.getValueAt(row, 0));
00135                 refresh();
00136             } catch (ServiceException ex) {
00137                 JOptionPane.showMessageDialog(this, ex.getMessage());
00138             }
00139         }
00140     }
00141
00142     public void refresh() {
00143         tableModel.setRowCount(0);
00144         try {
00145             List<MedicalRecord> list = medicalService.getAllRecords();
00146             for (MedicalRecord r : list) {
00147                 tableModel.addRow(new Object[]{
00148                     r.getId(), r.getPetName(), r.getRecordType(),
00149                     r.getDiagnosis(), r.getVeterinarianName(),
00150                     r.getFormattedDate(),
00151                     String.format("%.2f", r.getCost())
00152                 });
00153             }
00154         }
00155     }

```

```

00168         }
00169     } catch (Exception e) { /* sessiz */ }
00170 }
00171
00172 private JButton btn(String text, Color bg, java.awt.event.ActionListener al) {
00173     JButton b = new JButton(text);
00174     if (bg != null) { b.setBackground(bg); b.setForeground(Color.WHITE); }
00175     b.setFocusPainted(false);
00176     b.addActionListener(al);
00177     return b;
00178 }
00179 }
00180 }
00181 }
00182 }

```

8.18 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/NavPanel.java File Reference

Sol navigasyon paneli.

```

import com.mehdi.petreminder.gui.MainFrame;
import com.mehdi.petreminder.gui.util.Constants;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.awt.event.*;
import java.util.HashMap;
import java.util.Map;

```

Classes

- class [com.mehdi.petreminder.gui.panels.NavPanel](#)
Sol navigasyon çubuğu — menü butonları.

8.18.1 Detailed Description

Sol navigasyon paneli.

Definition in file [NavPanel.java](#).

8.19 NavPanel.java

[Go to the documentation of this file.](#)

```

00001
00002 package com.mehdi.petreminder.gui.panels;
00003
00004 import com.mehdi.petreminder.gui.MainFrame;
00005 import com.mehdi.petreminder.gui.util.Constants;
00006
00007 import javax.swing.*;
00008 import javax.swing.border.EmptyBorder;
00009
00010 import java.awt.*;
00011 import java.awt.event.*;
00012 import java.util.HashMap;
00013 import java.util.Map;
00014
00015 public class NavPanel extends JPanel {

```

```

00028
00030     private static final long serialVersionUID = 1L;
00031
00033     private final MainFrame mainFrame;
00034
00036     private final Map<String, JButton> buttons = new LinkedHashMap<>();
00037
00039     private String activeCard = "";
00040
00045     public NavPanel(MainFrame mainFrame) {
00046         this.mainFrame = mainFrame;
00047         setBackground(GuiConstants.NAV_COLOR);
00048         setPreferredSize(new Dimension(GuiConstants.NAV_WIDTH, 0));
00049         setLayout(new BoxLayout(this, BoxLayout.Y_AXIS));
00050         buildNav();
00051     }
00052
00056     private void buildNav() {
00057         add(Box.createVerticalStrut(20));
00058         addLogo();
00059         add(Box.createVerticalStrut(24));
00060         addNavButton(" Dashboard", MainFrame.CARD_DASHBOARD);
00061         addNavButton(" Hayvanlarım", MainFrame.CARD_PETS);
00062         addNavButton(" Hatırlatıcılar", MainFrame.CARD_REMINDERS);
00063         addNavButton(" Vet Randevuları", MainFrame.CARD_VET);
00064         addNavButton(" Sağlık Kayıtları", MainFrame.CARD_MEDICAL);
00065         add(Box.createVerticalGlue());
00066         addNavButton(" Ayarlar", MainFrame.CARD_SETTINGS);
00067         add(Box.createVerticalStrut(16));
00068     }
00069
00073     private void addLogo() {
00074         JLabel logo = new JLabel(" PetCare");
00075         logo.setFont(new Font("Segoe UI", Font.BOLD, 17));
00076         logo.setForeground(Color.WHITE);
00077         logo.setAlignmentX(Component.CENTER_ALIGNMENT);
00078         logo.setBorder(new EmptyBorder(0, 0, 0, 0));
00079         add(logo);
00080
00081         JLabel sub = new JLabel("Reminder System");
00082         sub.setFont(GuiConstants.SMALL_FONT);
00083         sub.setForeground(new Color(160, 170, 190));
00084         sub.setAlignmentX(Component.CENTER_ALIGNMENT);
00085         add(sub);
00086     }
00087
00093     private void addNavButton(String label, String cardName) {
00094         JButton btn = new JButton(label);
00095         btn.setMaximumSize(new Dimension(GuiConstants.NAV_WIDTH - 16, 44));
00096         btn.setPreferredSize(new Dimension(GuiConstants.NAV_WIDTH - 16, 44));
00097         btn.setAlignmentX(Component.CENTER_ALIGNMENT);
00098         btn.setHorizontalAlignment(SwingConstants.LEFT);
00099         btn.setBorderPainted(false);
00100         btn.setFocusPainted(false);
00101         btn.setBackground(GuiConstants.NAV_COLOR);
00102         btn.setForeground(new Color(200, 210, 230));
00103         btn.setFont(GuiConstants.NAV_FONT);
00104         btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
00105         btn.setBorder(new EmptyBorder(0, 16, 0, 0));
00106
00107         btn.addMouseListener(new MouseAdapter() {
00108             @Override
00109             public void mouseEntered(MouseEvent e) {
00110                 if (!cardName.equals(activeCard)) {
00111                     btn.setBackground(GuiConstants.NAV_HOVER);
00112                 }
00113             }
00114
00115             @Override
00116             public void mouseExited(MouseEvent e) {
00117                 if (!cardName.equals(activeCard)) {
00118                     btn.setBackground(GuiConstants.NAV_COLOR);
00119                 }
00120             }
00121         });
00122
00123         btn.addActionListener(e -> mainFrame.navigateTo(cardName));
00124         buttons.put(cardName, btn);
00125         add(Box.createVerticalStrut(4));
00126         add(btn);
00127     }
00128
00133     public void setActiveButton(String cardName) {
00134         if (activeCard != null && buttons.containsKey(activeCard)) {
00135             JButton prev = buttons.get(activeCard);
00136             prev.setBackground(GuiConstants.NAV_COLOR);
00137             prev.setForeground(new Color(200, 210, 230));

```

```

00138     }
00139     activeCard = cardName;
00140     if (buttons.containsKey(cardName)) {
00141         JButton active = buttons.get(cardName);
00142         active.setBackground(GuiConstants.NAV_ACTIVE);
00143         active.setForeground(Color.WHITE);
00144     }
00145 }
00146 }

```

8.20 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/PetsPanel.java File Reference

Evcil hayvanlar yönetim paneli — tam CRUD.

```

import com.mehdi.petreminder.gui.MainFrame;
import com.mehdi.petreminder.gui.dialogs.PetDialog;
import com.mehdi.petreminder.gui.util.GuiConstants;
import com.mehdi.petreminder.model.*;
import com.mehdi.petreminder.service.PetService;
import com.mehdi.petreminder.service.ServiceException;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.*;
import java.awt.*;
import java.util.List;

```

Classes

- class [com.mehdi.petreminder.gui.panels.PetsPanel](#)
Evcil hayvanlar listesi ve CRUD işlemleri.

8.20.1 Detailed Description

Evcil hayvanlar yönetim paneli — tam CRUD.

Definition in file [PetsPanel.java](#).

8.21 PetsPanel.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.gui.panels;
00006
00007 import com.mehdi.petreminder.gui.MainFrame;
00008 import com.mehdi.petreminder.gui.dialogs.PetDialog;
00009 import com.mehdi.petreminder.gui.util.GuiConstants;
00010 import com.mehdi.petreminder.model.*;
00011 import com.mehdi.petreminder.service.PetService;
00012 import com.mehdi.petreminder.service.ServiceException;
00013
00014 import javax.swing.*;
00015 import javax.swing.border.EmptyBorder;

```

```

00016 import javax.swing.table.*;
00017 import java.awt.*;
00018 import java.util.List;
00019
00029 public class PetsPanel extends JPanel {
00030
00032     private static final long serialVersionUID = 1L;
00033
00035     private final MainFrame mainFrame;
00036
00038     private final PetService petService;
00039
00041     private DefaultTableModel tableModel;
00042
00044     private JTable table;
00045
00047     private JTextField searchField;
00048
00050     private JComboBox<String> speciesFilter;
00051
00056     public PetsPanel(MainFrame mainFrame) {
00057         this.mainFrame = mainFrame;
00058         this.petService = new PetService();
00059         setBackground(GuiConstants.BG_COLOR);
00060         setLayout(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));
00061         setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,
00062             GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));
00063         buildUI();
00064     }
00065
00069     private void buildUI() {
00070         add(buildHeader(), BorderLayout.NORTH);
00071         add(buildTableArea(), BorderLayout.CENTER);
00072         add(buildToolBar(), BorderLayout.SOUTH);
00073     }
00074
00079     private JPanel buildHeader() {
00080         JPanel p = new JPanel(new BorderLayout(GuiConstants.GAP, 0));
00081         p.setOpaque(false);
00082
00083         JLabel title = new JLabel(" Evcil Hayvanlarım");
00084         title.setFont(GuiConstants.TITLE_FONT);
00085         title.setForeground(GuiConstants.TEXT_PRIMARY);
00086         p.add(title, BorderLayout.WEST);
00087
00088         JPanel controls = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
00089         controls.setOpaque(false);
00090
00091         searchField = new JTextField(15);
00092         searchField.putClientProperty("JTextField.placeholderText", "Ad ile ara...");
00093         searchField.addActionListener(e -> refresh());
00094
00095         speciesFilter = new JComboBox<>(new String[]{"Tümü", "Dog", "Cat", "Bird"});
00096         speciesFilter.addActionListener(e -> refresh());
00097
00098         controls.add(new JLabel("Tür:"));
00099         controls.add(speciesFilter);
00100         controls.add(searchField);
00101         controls.add(buildIconButton("", "Ara", e -> refresh()));
00102
00103         p.add(controls, BorderLayout.EAST);
00104         return p;
00105     }
00106
00111     private JScrollPane buildTableArea() {
00112         String[] cols = {"ID", "Ad", "Tür", "İrk", "Doğum Tarihi", "Cinsiyet", "Ağırlık (kg)"};
00113         tableModel = new DefaultTableModel(cols, 0) {
00114             @Override
00115             public boolean isCellEditable(int row, int col) { return false; }
00116         };
00117         table = new JTable(tableModel);
00118         table.setRowHeight(32);
00119         table.setFont(GuiConstants.BODY_FONT);
00120         table.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 12));
00121         table.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
00122         table.setGridColor(GuiConstants.BORDER_COLOR);
00123         table.getColumnModel().getColumn(0).setMaxWidth(50);
00124
00125         // Çift tıklama = düzenle
00126         table.addMouseListener(new java.awt.event.MouseAdapter() {
00127             @Override
00128             public void mouseClicked(java.awt.event.MouseEvent e) {
00129                 if (e.getClickCount() == 2) editSelected();
00130             }
00131         });
00132
00133         return new JScrollPane(table);

```



```

00134     }
00135
00140     private JPanel buildToolBar() {
00141         JPanel bar = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
00142         bar.setOpaque(false);
00143         bar.add(buildPrimaryButton(" Yeni Hayvan", e -> addPet()));
00144         bar.add(buildButton(" Düzenle", e -> editSelected()));
00145         bar.add(buildDangerButton(" Sil", e -> deleteSelected()));
00146         return bar;
00147     }
00148
00152     private void addPet() {
00153         PetDialog dlg = new PetDialog(mainFrame, null);
00154         dlg.setVisible(true);
00155         if (dlg.isConfirmed()) {
00156             try {
00157                 petService.addPet(dlg.getPet());
00158                 refresh();
00159                 JOptionPane.showMessageDialog(this, "Hayvan eklendi!", "Başarılı",
00160                     JOptionPane.INFORMATION_MESSAGE);
00161             } catch (ServiceException ex) {
00162                 JOptionPane.showMessageDialog(this, ex.getMessage(), "Hata",
00163                     JOptionPane.ERROR_MESSAGE);
00164             }
00165         }
00166     }
00167
00171     private void editSelected() {
00172         int row = table.getSelectedRow();
00173         if (row < 0) {
00174             JOptionPane.showMessageDialog(this, "Lütfen bir hayvan seçin.");
00175             return;
00176         }
00177         int id = (int) tableModel.getValueAt(row, 0);
00178         try {
00179             Pet pet = petService.getPetById(id);
00180             PetDialog dlg = new PetDialog(mainFrame, pet);
00181             dlg.setVisible(true);
00182             if (dlg.isConfirmed()) {
00183                 petService.updatePet(dlg.getPet());
00184                 refresh();
00185             }
00186         } catch (ServiceException ex) {
00187             JOptionPane.showMessageDialog(this, ex.getMessage(), "Hata",
00188                 JOptionPane.ERROR_MESSAGE);
00189         }
00190     }
00191
00195     private void deleteSelected() {
00196         int row = table.getSelectedRow();
00197         if (row < 0) {
00198             JOptionPane.showMessageDialog(this, "Lütfen bir hayvan seçin.");
00199             return;
00200         }
00201         String name = (String) tableModel.getValueAt(row, 1);
00202         int confirm = JOptionPane.showConfirmDialog(this,
00203             name + " silinsin mi?", "Onayla", JOptionPane.YES_NO_OPTION);
00204         if (confirm == JOptionPane.YES_OPTION) {
00205             int id = (int) tableModel.getValueAt(row, 0);
00206             petService.deletePet(id);
00207             refresh();
00208         }
00209     }
00210
00214     public void refresh() {
00215         tableModel.setRowCount(0);
00216         try {
00217             String search = searchField != null ? searchField.getText().trim() : "";
00218             String species = speciesFilter != null
00219                 ? (String) speciesFilter.getSelectedItem() : "Tümü";
00220
00221             List<Pet> pets = search.isEmpty() ? petService.getAllPets()
00222                 : petService.searchByName(search);
00223             if (!"Tümü".equals(species) && species != null) {
00224                 pets = petService.filterBySpecies(species);
00225             }
00226
00227             for (Pet p : pets) {
00228                 tableModel.addRow(new Object[]{
00229                     p.getId(),
00230                     p.getName(),
00231                     p.getSpecies(),
00232                     getBreed(p),
00233                     p.getBirthDate() != null ? p.getBirthDate().toString() : "",
00234                     p.getGender() != null ? p.getGender() : "",
00235                     String.format("%.1f", p.getWeight())
00236                 });

```

```

00237         }
00238     } catch (Exception e) {
00239         // Veri yoksa sessiz
00240     }
00241 }
00242
00243 private String getBreed(Pet pet) {
00244     if (pet instanceof Dog) return ((Dog) pet).getBreed() != null ? ((Dog) pet).getBreed() : "";
00245     if (pet instanceof Cat) return ((Cat) pet).getBreed() != null ? ((Cat) pet).getBreed() : "";
00246     if (pet instanceof Bird) return ((Bird) pet).getBirdType() != null ? ((Bird)
00247 pet).getBirdType() : "";
00248     return "";
00249 }
00250
00251 // Yardımcı buton oluşturucular
00252
00253 private JButton buildPrimaryButton(String text, java.awt.event.ActionListener al) {
00254     JButton b = new JButton(text);
00255     b.setBackground(GuiConstants.PRIMARY);
00256     b.setForeground(Color.WHITE);
00257     b.setFocusPainted(false);
00258     b.setFont(GuiConstants.BODY_FONT);
00259     b.addActionListener(al);
00260     return b;
00261 }
00262
00263 private JButton buildButton(String text, java.awt.event.ActionListener al) {
00264     JButton b = new JButton(text);
00265     b.setFocusPainted(false);
00266     b.setFont(GuiConstants.BODY_FONT);
00267     b.addActionListener(al);
00268     return b;
00269 }
00270
00271 private JButton buildDangerButton(String text, java.awt.event.ActionListener al) {
00272     JButton b = new JButton(text);
00273     b.setBackground(GuiConstants.DANGER);
00274     b.setForeground(Color.WHITE);
00275     b.setFocusPainted(false);
00276     b.setFont(GuiConstants.BODY_FONT);
00277     b.addActionListener(al);
00278     return b;
00279 }
00280
00281 private JButton buildIconButton(String icon, String tooltip,
00282     java.awt.event.ActionListener al) {
00283     JButton b = new JButton(icon);
00284     b.setToolTipText(tooltip);
00285     b.setFocusPainted(false);
00286     b.addActionListener(al);
00287     return b;
00288 }
00289 }
00290 }

```

8.22 petreminder- app/src/main/java/com/mehdi/petreminder/gui/panels/Reminders- Panel.java File Reference

Hatırlatıcılar yönetim paneli.

```

import com.mehdi.petreminder.gui.MainFrame;
import com.mehdi.petreminder.gui.dialogs.ReminderDialog;
import com.mehdi.petreminder.gui.util.GuiConstants;
import com.mehdi.petreminder.model.Reminder;
import com.mehdi.petreminder.service.ReminderService;
import com.mehdi.petreminder.service.ServiceException;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.List;

```

Classes

- class [com.mehdi.petreminder.gui.panels.RemindersPanel](#)
Hatırlatıcılar CRUD paneli.

8.22.1 Detailed Description

Hatırlatıcılar yönetim paneli.

Definition in file [RemindersPanel.java](#).

8.23 RemindersPanel.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.gui.panels;
00006
00007 import com.mehdi.petreminder.gui.MainFrame;
00008 import com.mehdi.petreminder.gui.dialogs.ReminderDialog;
00009 import com.mehdi.petreminder.gui.util.Constants;
00010 import com.mehdi.petreminder.model.Reminder;
00011 import com.mehdi.petreminder.service.ReminderService;
00012 import com.mehdi.petreminder.service.ServiceException;
00013
00014 import javax.swing.*;
00015 import javax.swing.border.EmptyBorder;
00016 import javax.swing.table.DefaultTableModel;
00017 import java.awt.*;
00018 import java.util.List;
00019
00028 public class RemindersPanel extends JPanel {
00029
00031     private static final long serialVersionUID = 1L;
00032
00034     private final MainFrame mainFrame;
00035
00037     private final ReminderService reminderService;
00038
00040     private DefaultTableModel tableModel;
00041
00043     private JTable table;
00044
00046     private JCheckBox chkOnlyPending;
00047
00052     public RemindersPanel(MainFrame mainFrame) {
00053         this.mainFrame = mainFrame;
00054         this.reminderService = new ReminderService();
00055         setBackground(Constants.BG_COLOR);
00056         setLayout(new BorderLayout(Constants.GAP, Constants.GAP));
00057         setBorder(new EmptyBorder(Constants.GAP_LARGE, Constants.GAP_LARGE,
00058             Constants.GAP_LARGE, Constants.GAP_LARGE));
00059         buildUI();
00060     }
00061
00065     private void buildUI() {
00066         add(buildHeader(), BorderLayout.NORTH);
00067         add(buildTable(), BorderLayout.CENTER);
00068         add(buildToolbar(), BorderLayout.SOUTH);
00069     }
00070
00074     private JPanel buildHeader() {
00075         JPanel p = new JPanel(new BorderLayout());
00076         p.setOpaque(false);
00077         JLabel title = new JLabel(" Hatırlatıcılar");
00078         title.setFont(Constants.TITLE_FONT);
00079         p.add(title, BorderLayout.WEST);
00080         chkOnlyPending = new JCheckBox("Sadece bekleyenler");
00081         chkOnlyPending.addActionListener(e -> refresh());
00082         p.add(chkOnlyPending, BorderLayout.EAST);
00083         return p;
00084     }
00085
00089     private JScrollPane buildTable() {

```

```

00090         String[] cols = {"ID", "Pet", "Tür", "Öncelik", "Zaman", "Durum"};
00091         tableModel = new DefaultTableModel(cols, 0) {
00092             @Override public boolean isCellEditable(int r, int c) { return false; }
00093         };
00094         table = new JTable(tableModel);
00095         table.setRowHeight(30);
00096         table.setFont(GuiConstants.BODY_FONT);
00097         table.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 12));
00098         table.setGridColor(GuiConstants.BORDER_COLOR);
00099         table.addMouseListener(new java.awt.event.MouseAdapter() {
00100             @Override
00101             public void mouseClicked(java.awt.event.MouseEvent e) {
00102                 if (e.getClickCount() == 2) editSelected();
00103             }
00104         });
00105         return new JScrollPane(table);
00106     }
00107
00111     private JPanel buildToolbar() {
00112         JPanel bar = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
00113         bar.setOpaque(false);
00114         bar.add(btn("Yeni", GuiConstants.PRIMARY, e -> addReminder()));
00115         bar.add(btn("Düzenle", null, e -> editSelected()));
00116         bar.add(btn("Tamamla", GuiConstants.SUCCESS, e -> completeSelected()));
00117         bar.add(btn("Sil", GuiConstants.DANGER, e -> deleteSelected()));
00118         return bar;
00119     }
00120
00124     private void addReminder() {
00125         ReminderDialog dlg = new ReminderDialog(
00126             (Frame) SwingUtilities.getWindowAncestor(this), null);
00127         dlg.setVisible(true);
00128         if (dlg.isConfirmed()) {
00129             try {
00130                 reminderService.addReminder(dlg.getReminder());
00131                 refresh();
00132             } catch (ServiceException ex) {
00133                 JOptionPane.showMessageDialog(this, ex.getMessage(), "Hata",
00134                     JOptionPane.ERROR_MESSAGE);
00135             }
00136         }
00137     }
00138
00142     private void editSelected() {
00143         int row = table.getSelectedRow();
00144         if (row < 0) return;
00145         int id = (int) tableModel.getValueAt(row, 0);
00146         try {
00147             Reminder r = reminderService.getReminderById(id);
00148             ReminderDialog dlg = new ReminderDialog(
00149                 (Frame) SwingUtilities.getWindowAncestor(this), r);
00150             dlg.setVisible(true);
00151             if (dlg.isConfirmed()) {
00152                 reminderService.updateReminder(dlg.getReminder());
00153                 refresh();
00154             }
00155         } catch (ServiceException ex) {
00156             JOptionPane.showMessageDialog(this, ex.getMessage(), "Hata",
00157                 JOptionPane.ERROR_MESSAGE);
00158         }
00159     }
00160
00164     private void completeSelected() {
00165         int row = table.getSelectedRow();
00166         if (row < 0) {
00167             JOptionPane.showMessageDialog(this, "Lütfen bir hatırlatıcı seçin.");
00168             return;
00169         }
00170         int id = (int) tableModel.getValueAt(row, 0);
00171         String currentStatus = (String) tableModel.getValueAt(row, 5);
00172         if (currentStatus != null && currentStatus.contains("Tamamlandı")) {
00173             JOptionPane.showMessageDialog(this, "Bu hatırlatıcı zaten tamamlandı.");
00174             return;
00175         }
00176         try {
00177             reminderService.markCompleted(id);
00178             refresh();
00179             JOptionPane.showMessageDialog(this, "Hatırlatıcı tamamlandı olarak işaretlendi!",
00180                 "Başarılı", JOptionPane.INFORMATION_MESSAGE);
00181         } catch (Exception ex) {
00182             JOptionPane.showMessageDialog(this, "Hata: " + ex.getMessage(),
00183                 "Hata", JOptionPane.ERROR_MESSAGE);
00184         }
00185     }
00186
00190     private void deleteSelected() {
00191         int row = table.getSelectedRow();

```

```

00192         if (row < 0) return;
00193         int confirm = JOptionPane.showConfirmDialog(this, "Silinsin mi?", "Onayla",
00194             JOptionPane.YES_NO_OPTION);
00195         if (confirm == JOptionPane.YES_OPTION) {
00196             reminderService.deleteReminder((int) tableModel.getValueAt(row, 0));
00197             refresh();
00198         }
00199     }
00200
00204     public void refresh() {
00205         tableModel.setRowCount(0);
00206         try {
00207             List<Reminder> list = chkOnlyPending != null && chkOnlyPending.isSelected()
00208                 ? reminderService.getPendingReminders()
00209                 : reminderService.getAllReminders();
00210             for (Reminder r : list) {
00211                 String status = r.isCompleted() ? " Tamamlandı"
00212                     : r.isOverdue() ? " Gecikmiş" : " Bekliyor";
00213                 tableModel.addRow(new Object[]{
00214                     r.getId(), r.getPetName(), r.getReminderType(),
00215                     r.getPriority(), r.getFormattedTime(), status
00216                 });
00217             }
00218         } catch (Exception e) { /* sessiz */ }
00219     }
00220
00224     private JButton btn(String text, Color bg, java.awt.event.ActionListener al) {
00225         JButton b = new JButton(text);
00226         if (bg != null) { b.setBackground(bg); b.setForeground(Color.WHITE); }
00227         b.setFocusPainted(false);
00228         b.addActionListener(al);
00229         return b;
00230     }
00231 }

```

8.24 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/SettingsPanel.java File Reference ↩

Ayarlar paneli — runtime storage backend değiştirme.

```

import com.mehdi.petreminder.config.StorageConfig;
import com.mehdi.petreminder.config.StorageType;
import com.mehdi.petreminder.gui.MainFrame;
import com.mehdi.petreminder.gui.util.Constants;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;

```

Classes

- class [com.mehdi.petreminder.gui.panels.SettingsPanel](#)
Storage backend ve uygulama ayarları.

8.24.1 Detailed Description

Ayarlar paneli — runtime storage backend değiştirme.

Definition in file [SettingsPanel.java](#).

8.25 SettingsPanel.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.gui.panels;
00006
00007 import com.mehdi.petreminder.config.StorageConfig;
00008 import com.mehdi.petreminder.config.StorageType;
00009 import com.mehdi.petreminder.gui.MainFrame;
00010 import com.mehdi.petreminder.gui.util.GuiConstants;
00011
00012 import javax.swing.*;
00013 import javax.swing.border.EmptyBorder;
00014 import java.awt.*;
00015
00025 public class SettingsPanel extends JPanel {
00026
00028     private static final long serialVersionUID = 1L;
00029
00031     private final MainFrame mainFrame;
00032
00034     private JComboBox<StorageType> cmbStorage;
00035
00037     private JLabel lblCurrent;
00038
00043     public SettingsPanel(MainFrame mainFrame) {
00044         this.mainFrame = mainFrame;
00045         setBackground(GuiConstants.BG_COLOR);
00046         setLayout(new BorderLayout());
00047         setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,
00048             GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));
00049         buildUI();
00050     }
00051
00055     private void buildUI() {
00056         JLabel title = new JLabel(" Ayarlar");
00057         title.setFont(GuiConstants.TITLE_FONT);
00058         add(title, BorderLayout.NORTH);
00059
00060         JPanel center = new JPanel();
00061         center.setLayout(new BoxLayout(center, BoxLayout.Y_AXIS));
00062         center.setOpaque(false);
00063         center.add(Box.createVerticalStrut(32));
00064         center.add(buildStorageSection());
00065         center.add(Box.createVerticalStrut(24));
00066         center.add(buildInfoSection());
00067         add(center, BorderLayout.CENTER);
00068     }
00073     private JPanel buildStorageSection() {
00074         JPanel card = new JPanel(new GridBagLayout());
00075         card.setBackground(GuiConstants.CARD_BG);
00076         card.setBorder(BorderFactory.createCompoundBorder(
00077             BorderFactory.createLineBorder(GuiConstants.BORDER_COLOR),
00078             new EmptyBorder(24, 24, 24, 24)));
00079         card.setMaximumSize(new Dimension(500, 200));
00080         card.setAlignmentX(Component.LEFT_ALIGNMENT);
00081
00082         GridBagConstraints gbc = new GridBagConstraints();
00083         gbc.insets = new Insets(8, 8, 8, 8);
00084         gbc.anchor = GridBagConstraints.WEST;
00085
00086         JLabel sectionTitle = new JLabel("Storage Backend");
00087         sectionTitle.setFont(GuiConstants.SECTION_FONT);
00088         gbc.gridx = 0; gbc.gridy = 0; gbc.gridwidth = 2;
00089         card.add(sectionTitle, gbc);
00090         gbc.gridwidth = 1;
00091
00092         gbc.gridx = 0; gbc.gridy = 1;
00093         card.add(new JLabel("Mevcut:", gbc);
00094             lblCurrent = new JLabel(StorageConfig.getActiveBackend().getDisplayName());
00095             lblCurrent.setFont(new Font("Segoe UI", Font.BOLD, 13));
00096             lblCurrent.setForeground(GuiConstants.PRIMARY);
00097             gbc.gridx = 1;
00098             card.add(lblCurrent, gbc);
00099
00100         gbc.gridx = 0; gbc.gridy = 2;
00101         card.add(new JLabel("Değiştir:", gbc);
00102             cmbStorage = new JComboBox<>(StorageType.values());
00103             cmbStorage.setSelectedItem(StorageConfig.getActiveBackend());
00104             cmbStorage.setPreferredSize(new Dimension(200, 30));
00105             gbc.gridx = 1;
00106             card.add(cmbStorage, gbc);
00107
00108         JButton btnApply = new JButton("Uygula");

```

```

00109         btnApply.setBackground(GuiConstants.PRIMARY);
00110         btnApply.setForeground(Color.WHITE);
00111         btnApply.setFocusPainted(false);
00112         btnApply.addActionListener(e -> applyStorage());
00113         gbc.gridx = 1; gbc.gridy = 3;
00114         card.add(btnApply, gbc);
00115
00116         return card;
00117     }
00118
00122     private JPanel buildInfoSection() {
00123         JPanel card = new JPanel(new GridBagLayout());
00124         card.setBackground(GuiConstants.CARD_BG);
00125         card.setBorder(BorderFactory.createCompoundBorder(
00126             BorderFactory.createLineBorder(GuiConstants.BORDER_COLOR),
00127             new EmptyBorder(24, 24, 24, 24)));
00128         card.setMaximumSize(new Dimension(500, 150));
00129         card.setAlignmentX(Component.LEFT_ALIGNMENT);
00130
00131         GridBagConstraints gbc = new GridBagConstraints();
00132         gbc.insets = new Insets(4, 8, 4, 8);
00133         gbc.anchor = GridBagConstraints.WEST;
00134
00135         JLabel infoTitle = new JLabel("Uygulama Bilgileri");
00136         infoTitle.setFont(GuiConstants.SECTION_FONT);
00137         gbc.gridx = 0; gbc.gridy = 0; gbc.gridwidth = 2;
00138         card.add(infoTitle, gbc);
00139         gbc.gridwidth = 1;
00140
00141         addInfoRow(card, gbc, 1, "Proje:", "Pet Care Reminder System v1.0");
00142         addInfoRow(card, gbc, 2, "Ders:", "CEN206 -- OOP Term Project");
00143         addInfoRow(card, gbc, 3, "Takım:", "Mehdi / Ibrahim / Zümre");
00144         addInfoRow(card, gbc, 4, "Üniversite:", "RTEU -- Spring 2026");
00145         return card;
00146     }
00147
00151     private void addInfoRow(JPanel p, GridBagConstraints gbc, int row,
00152         String key, String value) {
00153         gbc.gridx = 0; gbc.gridy = row;
00154         JLabel k = new JLabel(key);
00155         k.setFont(new Font("Segoe UI", Font.BOLD, 12));
00156         k.setForeground(GuiConstants.TEXT_SECONDARY);
00157         p.add(k, gbc);
00158         gbc.gridx = 1;
00159         JLabel v = new JLabel(value);
00160         v.setFont(GuiConstants.BODY_FONT);
00161         p.add(v, gbc);
00162     }
00163
00167     private void applyStorage() {
00168         StorageType selected = (StorageType) cmbStorage.getSelectedItemAt();
00169         if (selected != null) {
00170             mainFrame.onStorageChanged(selected);
00171             lblCurrent.setText(selected.getDisplayName());
00172         }
00173     }
00174
00178     public void refresh() {
00179         if (lblCurrent != null) {
00180             lblCurrent.setText(StorageConfig.getActiveBackend().getDisplayName());
00181         }
00182         if (cmbStorage != null) {
00183             cmbStorage.setSelectedItemAt(StorageConfig.getActiveBackend());
00184         }
00185     }
00186 }

```

8.26 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/VetPanel.java File Reference

Veteriner randevuları paneli.

```

import com.mehdi.petreminder.gui.MainFrame;
import com.mehdi.petreminder.gui.util.GuiConstants;
import com.mehdi.petreminder.model.*;

```

```

import com.mehdi.petreminder.service.ReminderService;
import com.mehdi.petreminder.service.ServiceException;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.time.LocalDateTime;
import java.util.List;
import java.util.stream.Collectors;

```

Classes

- class [com.mehdi.petreminder.gui.panels.VetPanel](#)
Veteriner randevuları CRUD.

8.26.1 Detailed Description

Veteriner randevuları paneli.

Definition in file [VetPanel.java](#).

8.27 VetPanel.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.gui.panels;
00006
00007 import com.mehdi.petreminder.gui.MainFrame;
00008 import com.mehdi.petreminder.gui.util.Constants;
00009 import com.mehdi.petreminder.model.*;
00010 import com.mehdi.petreminder.service.ReminderService;
00011 import com.mehdi.petreminder.service.ServiceException;
00012
00013 import javax.swing.*;
00014 import javax.swing.border.EmptyBorder;
00015 import javax.swing.table.DefaultTableModel;
00016 import java.awt.*;
00017 import java.time.LocalDateTime;
00018 import java.util.List;
00019 import java.util.stream.Collectors;
00020
00029 public class VetPanel extends JPanel {
00030
00032     private static final long serialVersionUID = 1L;
00033
00035     private final MainFrame mainFrame;
00036
00038     private final ReminderService reminderService;
00039
00041     private DefaultTableModel tableModel;
00042
00044     private JTable table;
00045
00050     public VetPanel(MainFrame mainFrame) {
00051         this.mainFrame = mainFrame;
00052         this.reminderService = new ReminderService();
00053         setBackground(Constants.BG_COLOR);
00054         setLayout(new BorderLayout(Constants.GAP, Constants.GAP));
00055         setBorder(new EmptyBorder(Constants.GAP_LARGE, Constants.GAP_LARGE,
00056             Constants.GAP_LARGE, Constants.GAP_LARGE));
00057         buildUI();
00058     }
00059
00063     private void buildUI() {
00064         JLabel title = new JLabel(" Veteriner Randevuları");

```



```

00065         title.setFont(GuiConstants.TITLE_FONT);
00066         add(title, BorderLayout.NORTH);
00067         add(buildTable(), BorderLayout.CENTER);
00068         add(buildToolBar(), BorderLayout.SOUTH);
00069     }
00070
00071     private JScrollPane buildTable() {
00072         String[] cols = {"ID", "Pet", "Veteriner", "Klinik", "Sebep", "Zaman", "Onay"};
00073         tableModel = new DefaultTableModel(cols, 0) {
00074             @Override public boolean isCellEditable(int r, int c) { return false; }
00075         };
00076         table = new JTable(tableModel);
00077         table.setRowHeight(30);
00078         table.setFont(GuiConstants.BODY_FONT);
00079         table.setGridColor(GuiConstants.BORDER_COLOR);
00080         return new JScrollPane(table);
00081     }
00082
00083     private JPanel buildToolBar() {
00084         JPanel bar = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
00085         bar.setOpaque(false);
00086         bar.add(btn(" Yeni Randevu", GuiConstants.PRIMARY, e -> addAppointment()));
00087         bar.add(btn(" Onayla", GuiConstants.SUCCESS, e -> confirmSelected()));
00088         bar.add(btn(" Sil", GuiConstants.DANGER, e -> deleteSelected()));
00089         return bar;
00090     }
00091
00092     private void addAppointment() {
00093         JTextField petId = new JTextField("1");
00094         JTextField petName = new JTextField();
00095         JTextField vetName = new JTextField();
00096         JTextField clinic = new JTextField();
00097         JTextField reason = new JTextField();
00098         JTextField time = new JTextField(LocalDate.now().plusDays(1)
00099             .toString().substring(0, 16));
00100
00101         Object[] fields = {
00102             "Pet ID:", petId, "Pet Adı:", petName,
00103             "Veteriner:", vetName, "Klinik:", clinic,
00104             "Sebep:", reason, "Zaman (yyyy-MM-ddTHH:mm):", time
00105         };
00106         int res = JOptionPane.showConfirmDialog(this, fields, "Yeni Randevu",
00107             JOptionPane.OK_CANCEL_OPTION);
00108         if (res == JOptionPane.OK_OPTION) {
00109             try {
00110                 VetAppointment va = new VetAppointment(0,
00111                     Integer.parseInt(petId.getText().trim()),
00112                     petName.getText().trim(),
00113                     LocalDateTime.parse(time.getText().trim()),
00114                     vetName.getText().trim(),
00115                     clinic.getText().trim(),
00116                     reason.getText().trim());
00117                 reminderService.addReminder(va);
00118                 refresh();
00119             } catch (Exception ex) {
00120                 JOptionPane.showMessageDialog(this, "Hata: " + ex.getMessage(), "Hata",
00121                     JOptionPane.ERROR_MESSAGE);
00122             }
00123         }
00124     }
00125
00126     private void confirmSelected() {
00127         int row = table.getSelectedRow();
00128         if (row < 0) return;
00129         int id = (int) tableModel.getValueAt(row, 0);
00130         try {
00131             Reminder r = reminderService.getReminderById(id);
00132             if (r instanceof VetAppointment) {
00133                 ((VetAppointment) r).setConfirmed(true);
00134                 reminderService.updateReminder(r);
00135                 refresh();
00136             }
00137         } catch (ServiceException ex) {
00138             JOptionPane.showMessageDialog(this, ex.getMessage());
00139         }
00140     }
00141
00142     private void deleteSelected() {
00143         int row = table.getSelectedRow();
00144         if (row < 0) return;
00145         int confirm = JOptionPane.showConfirmDialog(this, "Silinsin mi?", "Onayla",
00146             JOptionPane.YES_NO_OPTION);
00147         if (confirm == JOptionPane.YES_OPTION) {
00148             reminderService.deleteReminder((int) tableModel.getValueAt(row, 0));
00149             refresh();
00150         }
00151     }
00152
00153
00154     private void deleteSelected() {
00155         int row = table.getSelectedRow();
00156         if (row < 0) return;
00157         int confirm = JOptionPane.showConfirmDialog(this, "Silinsin mi?", "Onayla",
00158             JOptionPane.YES_NO_OPTION);
00159         if (confirm == JOptionPane.YES_OPTION) {
00160             reminderService.deleteReminder((int) tableModel.getValueAt(row, 0));
00161             refresh();
00162         }
00163     }
00164
00165
00166

```

```

00167
00171     public void refresh() {
00172         tableModel.setRowCount(0);
00173         try {
00174             List<Reminder> all = reminderService.getAllReminders();
00175             List<VetAppointment> vets = all.stream()
00176                 .filter(r -> r instanceof VetAppointment)
00177                 .map(r -> (VetAppointment) r)
00178                 .collect(Collectors.toList());
00179             for (VetAppointment va : vets) {
00180                 tableModel.addRow(new Object[]{
00181                     va.getId(), va.getPetName(), va.getVeterinarianName(),
00182                     va.getClinicName(), va.getReason(), va.getFormattedTime(),
00183                     va.isConfirmed() ? "" : ""
00184                 });
00185             }
00186         } catch (Exception e) { /* sessiz */ }
00187     }
00188
00192     private JButton btn(String text, Color bg, java.awt.event.ActionListener al) {
00193         JButton b = new JButton(text);
00194         if (bg != null) { b.setBackground(bg); b.setForeground(Color.WHITE); }
00195         b.setFocusPainted(false);
00196         b.addActionListener(al);
00197         return b;
00198     }
00199 }

```

8.28 petreminder-app/src/main/java/com/mehdi/petreminder/gui/util/↵ GuiConstants.java File Reference

GUI için renk, font ve boyut sabitleri.

```
import java.awt.*;
```

Classes

- class [com.mehdi.petreminder.gui.util.GuiConstants](#)
GUI bileşenlerinde kullanılan sabitler.

Packages

- package [com.mehdi.petreminder.gui.util](#)

8.28.1 Detailed Description

GUI için renk, font ve boyut sabitleri.

Definition in file [GuiConstants.java](#).

8.29 GuiConstants.java

[Go to the documentation of this file.](#)

```

00001
00008 package com.mehdi.petreminder.gui.util;
00009
00010 import java.awt.*;
00011
00021 public final class GuiConstants {
00022
00024     private GuiConstants() { }
00025
00026     // Renkler
00027
00029     public static final Color NAV_COLOR      = new Color(45, 55, 72);
00030
00032     public static final Color NAV_HOVER     = new Color(74, 85, 104);
00033
00035     public static final Color NAV_ACTIVE    = new Color(99, 102, 241);
00036
00038     public static final Color BG_COLOR      = new Color(248, 250, 252);
00039
00041     public static final Color CARD_BG       = Color.WHITE;
00042
00044     public static final Color PRIMARY       = new Color(99, 102, 241);
00045
00047     public static final Color DANGER        = new Color(239, 68, 68);
00048
00050     public static final Color SUCCESS        = new Color(34, 197, 94);
00051
00053     public static final Color WARNING       = new Color(245, 158, 11);
00054
00056     public static final Color TEXT_PRIMARY  = new Color(17, 24, 39);
00057
00059     public static final Color TEXT_SECONDARY = new Color(107, 114, 128);
00060
00062     public static final Color BORDER_COLOR  = new Color(229, 231, 235);
00063
00064     // Fontlar
00065
00067     public static final Font TITLE_FONT     = new Font("Segoe UI", Font.BOLD, 22);
00068
00070     public static final Font SECTION_FONT   = new Font("Segoe UI", Font.BOLD, 16);
00071
00073     public static final Font BODY_FONT      = new Font("Segoe UI", Font.PLAIN, 13);
00074
00076     public static final Font SMALL_FONT     = new Font("Segoe UI", Font.PLAIN, 11);
00077
00079     public static final Font NAV_FONT       = new Font("Segoe UI", Font.PLAIN, 13);
00080
00081     // Boyutlar
00082
00084     public static final int NAV_WIDTH       = 200;
00085
00087     public static final int GAP             = 12;
00088
00090     public static final int GAP_LARGE      = 24;
00091 }

```

8.30 petreminder-app/src/main/java/com/mehdi/petreminder/model/Bird.java File Reference

Kus turunu temsil eden somut sınıf.

```
import java.time.LocalDate;
```

Classes

- class [com.mehdi.petreminder.model.Bird](#)
Kus evcil hayvan sınıfı.

Packages

- package [com.mehdi.petreminder.model](#)

8.30.1 Detailed Description

Kus turunu temsil eden somut sınıf.

Definition in file [Bird.java](#).

8.31 Bird.java

[Go to the documentation of this file.](#)

```
00001
00008 package com.mehdi.petreminder.model;
00009
00010 import java.time.LocalDate;
00011
00020 public class Bird extends Pet {
00021
00023     private static final long serialVersionUID = 1L;
00024
00026     private String birdType;
00027
00029     private boolean canTalk;
00030
00032     public Bird() { super(); }
00033
00041     public Bird(int id, String name, LocalDate birthDate, int ownerId) {
00042         super(id, name, "Kus", birthDate, ownerId, "Erkek", 0.0, "");
00043         this.birdType = "Muhabbet Kusu";
00044         this.canTalk = false;
00045     }
00046
00059     public Bird(int id, String name, LocalDate birthDate, int ownerId,
00060                 String gender, double weight, String notes,
00061                 String birdType, boolean canTalk) {
00062         super(id, name, "Kus", birthDate, ownerId, gender, weight, notes);
00063         this.birdType = birdType;
00064         this.canTalk = canTalk;
00065     }
00066
00068     public String getBirdType() { return birdType; }
00069
00071     public void setBirdType(String birdType) { this.birdType = birdType; }
00072
00074     public boolean isCanTalk() { return canTalk; }
00075
00077     public void setCanTalk(boolean canTalk) { this.canTalk = canTalk; }
00078
00083     @Override
00084     public String makeSound() {
00085         return canTalk ? "Merhaba!" : "Cik Cik!";
00086     }
00087
00092     @Override
00093     public String getIconName() { return "bird"; }
00094
00099     @Override
00100     public String toString() {
00101         return String.format("Bird{id=%d, name='%s', type='%s', age=%s, canTalk=%b}",
00102                               getId(), getName(), birdType, getAgeString(), canTalk);
00103     }
00104 }
```

8.32 petreminder-app/src/main/java/com/mehdi/petreminder/model/Cat.java File Reference ↩

Kedi turunu temsil eden somut sınıf.

```
import java.time.LocalDate;
```

Classes

- class [com.mehdi.petreminder.model.Cat](#)
Kedi evcil hayvan sinifi.

Packages

- package [com.mehdi.petreminder.model](#)

8.32.1 Detailed Description

Kedi turunu temsil eden somut sınıf.

Definition in file [Cat.java](#).

8.33 Cat.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.model;
00006
00007 import java.time.LocalDate;
00008
00017 public class Cat extends Pet {
00018
00020     private static final long serialVersionUID = 1L;
00021
00023     private String breed;
00024
00026     private boolean indoor;
00027
00029     public Cat() { super(); }
00030
00038     public Cat(int id, String name, LocalDate birthDate, int ownerId) {
00039         super(id, name, "Kedi", birthDate, ownerId, "Disi", 0.0, "");
00040         this.breed = "Karisik";
00041         this.indoor = true;
00042     }
00043
00056     public Cat(int id, String name, LocalDate birthDate, int ownerId,
00057         String gender, double weight, String notes,
00058         String breed, boolean indoor) {
00059         super(id, name, "Kedi", birthDate, ownerId, gender, weight, notes);
00060         this.breed = breed;
00061         this.indoor = indoor;
00062     }
00063
00065     public String getBreed() { return breed; }
00066
00068     public void setBreed(String breed) { this.breed = breed; }
00069
00071     public boolean isIndoor() { return indoor; }
00072
00074     public void setIndoor(boolean indoor) { this.indoor = indoor; }
00075
00080     @Override
00081     public String makeSound() { return "Miyav!"; }
00082
00087     @Override
00088     public String getIconName() { return "cat"; }
00089
00094     @Override
00095     public String toString() {
00096         return String.format("Cat{id=%d, name='%s', breed='%s', age=%s, indoor=%b}",
00097             getId(), getName(), breed, getAgeString(), indoor);
00098     }
00099 }

```

8.34 petreminder-app/src/main/java/com/mehdi/petreminder/model/↵ Dog.java File Reference

Kopek turunu temsil eden somut sınıf.

```
import java.time.LocalDate;
```

Classes

- class [com.mehdi.petreminder.model.Dog](#)
Kopek evcil hayvan sınıfı.

Packages

- package [com.mehdi.petreminder.model](#)

8.34.1 Detailed Description

Kopek turunu temsil eden somut sınıf.

Definition in file [Dog.java](#).

8.35 Dog.java

[Go to the documentation of this file.](#)

```
00001
00005 package com.mehdi.petreminder.model;
00006
00007 import java.time.LocalDate;
00008
00017 public class Dog extends Pet {
00018
00020     private static final long serialVersionUID = 1L;
00021
00023     private String breed;
00024
00026     private boolean trained;
00027
00029     public Dog() { super(); }
00030
00038     public Dog(int id, String name, LocalDate birthDate, int ownerId) {
00039         super(id, name, "Kopek", birthDate, ownerId, "Erkek", 0.0, "");
00040         this.breed = "Karisik";
00041         this.trained = false;
00042     }
00043
00056     public Dog(int id, String name, LocalDate birthDate, int ownerId,
00057         String gender, double weight, String notes,
00058         String breed, boolean trained) {
00059         super(id, name, "Kopek", birthDate, ownerId, gender, weight, notes);
00060         this.breed = breed;
00061         this.trained = trained;
00062     }
00063
00065     public String getBreed() { return breed; }
00066
00068     public void setBreed(String breed) { this.breed = breed; }
00069
00071     public boolean isTrained() { return trained; }
00072
```

```

00074     public void setTrained(boolean trained) { this.trained = trained; }
00075
00080     @Override
00081     public String makeSound() { return "Hav! Hav!"; }
00082
00087     @Override
00088     public String getIconName() { return "dog"; }
00089
00094     @Override
00095     public String toString() {
00096         return String.format("Dog{id=%d, name='%s', breed='%s', age=%s, trained=%b}",
00097             getId(), getName(), breed, getAgeString(), trained);
00098     }
00099 }

```

8.36 petreminder-app/src/main/java/com/mehdi/petreminder/model/ExerciseReminder.java File Reference ↩

Egzersiz hatırlatıcısı sınıfı.

```
import java.time.LocalDateTime;
```

Classes

- class [com.mehdi.petreminder.model.ExerciseReminder](#)
Evcil hayvan egzersiz hatırlatıcısı.

Packages

- package [com.mehdi.petreminder.model](#)

8.36.1 Detailed Description

Egzersiz hatırlatıcısı sınıfı.

Definition in file [ExerciseReminder.java](#).

8.37 ExerciseReminder.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.model;
00006
00007 import java.time.LocalDateTime;
00008
00016 public class ExerciseReminder extends Reminder {
00017
00019     private static final long serialVersionUID = 1L;
00020
00022     private String exerciseType;
00023
00025     private int durationMinutes;
00026
00028     private double distanceKm;
00029
00031     public ExerciseReminder() { super(); }
00032

```

```

00042     public ExerciseReminder(int id, int petId, String petName,
00043                             LocalDateTime scheduledTime,
00044                             String exerciseType, int durationMinutes) {
00045         super(id, petId, petName, scheduledTime,
00046             "Egzersiz: " + exerciseType, "ORTA", false, "");
00047         this.exerciseType = exerciseType;
00048         this.durationMinutes = durationMinutes;
00049         this.distanceKm = 0.0;
00050     }
00051
00066     public ExerciseReminder(int id, int petId, String petName,
00067                             LocalDateTime scheduledTime, String description,
00068                             String priority, boolean recurring,
00069                             String recurrencePattern, String exerciseType,
00070                             int durationMinutes, double distanceKm) {
00071         super(id, petId, petName, scheduledTime, description,
00072             priority, recurring, recurrencePattern);
00073         this.exerciseType = exerciseType;
00074         this.durationMinutes = durationMinutes;
00075         this.distanceKm = distanceKm;
00076     }
00077
00079     public String getExerciseType() { return exerciseType; }
00080
00082     public void setExerciseType(String exerciseType) { this.exerciseType = exerciseType; }
00083
00085     public int getDurationMinutes() { return durationMinutes; }
00086
00088     public void setDurationMinutes(int durationMinutes) {
00089         this.durationMinutes = durationMinutes;
00090     }
00091
00093     public double getDistanceKm() { return distanceKm; }
00094
00096     public void setDistanceKm(double distanceKm) { this.distanceKm = distanceKm; }
00097
00099     @Override
00100     public String getReminderType() { return "Egzersiz"; }
00101
00106     @Override
00107     public boolean validate() {
00108         return exerciseType != null && !exerciseType.trim().isEmpty()
00109             && durationMinutes > 0;
00110     }
00111
00116     @Override
00117     public String getSummary() {
00118         String s = String.format("Egzersiz: %s %d dk", exerciseType, durationMinutes);
00119         if (distanceKm > 0) s += String.format(" (%.1f km)", distanceKm);
00120         return s;
00121     }
00122
00124     @Override
00125     public String toString() {
00126         return String.format("ExerciseReminder{id=%d, pet='%s', type='%s', %ddk}",
00127             getId(), getPetName(), exerciseType, durationMinutes);
00128     }
00129 }

```

8.38 petreminder-app/src/main/java/com/mehdi/petreminder/model/↵ FeedingReminder.java File Reference

Besleme hatırlatıcısı sınıfı.

```
import java.time.LocalDateTime;
```

Classes

- class [com.mehdi.petreminder.model.FeedingReminder](#)
Evcil hayvan besleme hatırlatıcısı.

Packages

- package [com.mehdi.petreminder.model](#)

8.38.1 Detailed Description

Besleme hatırlatıcısı sınıfı.

Definition in file [FeedingReminder.java](#).

8.39 FeedingReminder.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.model;
00006
00007 import java.time.LocalDateTime;
00008
00017 public class FeedingReminder extends Reminder {
00018
00020     private static final long serialVersionUID = 1L;
00021
00023     private String foodType;
00024
00026     private double portionGrams;
00027
00029     private boolean includesWaterChange;
00030
00032     public FeedingReminder() { super(); }
00033
00043     public FeedingReminder(int id, int petId, String petName,
00044                             LocalDateTime scheduledTime,
00045                             String foodType, double portionGrams) {
00046         super(id, petId, petName, scheduledTime,
00047             "Besleme: " + foodType, "ORTA", false, "");
00048         this.foodType = foodType;
00049         this.portionGrams = portionGrams;
00050         this.includesWaterChange = false;
00051     }
00052
00067     public FeedingReminder(int id, int petId, String petName,
00068                             LocalDateTime scheduledTime, String description,
00069                             String priority, boolean recurring,
00070                             String recurrencePattern, String foodType,
00071                             double portionGrams, boolean includesWaterChange) {
00072         super(id, petId, petName, scheduledTime, description,
00073             priority, recurring, recurrencePattern);
00074         this.foodType = foodType;
00075         this.portionGrams = portionGrams;
00076         this.includesWaterChange = includesWaterChange;
00077     }
00078
00080     public String getFoodType() { return foodType; }
00081
00083     public void setFoodType(String foodType) { this.foodType = foodType; }
00084
00086     public double getPortionGrams() { return portionGrams; }
00087
00089     public void setPortionGrams(double portionGrams) { this.portionGrams = portionGrams; }
00090
00092     public boolean isIncludesWaterChange() { return includesWaterChange; }
00093
00095     public void setIncludesWaterChange(boolean includesWaterChange) {
00096         this.includesWaterChange = includesWaterChange;
00097     }
00098
00103     @Override
00104     public String getReminderType() { return "Besleme"; }
00105
00110     @Override
00111     public boolean validate() {
00112         return foodType != null && !foodType.trim().isEmpty() && portionGrams > 0;
00113     }
00114

```

```

00119     @Override
00120     public String getSummary() {
00121         return String.format("Besleme: %s (%.0fg)%s",
00122             foodType, portionGrams,
00123             includesWaterChange ? " + Su deęiřimi" : "");
00124     }
00125
00130     @Override
00131     public String toString() {
00132         return String.format("FeedingReminder{id=%d, pet='%s', food='%s', portion=%.0fg}",
00133             getId(), getPetName(), foodType, portionGrams);
00134     }
00135 }

```

8.40 petreminder-app/src/main/java/com/mehdi/petreminder/model/GroomingReminder.java File Reference

Tırař/bakım hatırlatıcısı sınıfı.

```
import java.time.LocalDateTime;
```

Classes

- class [com.mehdi.petreminder.model.GroomingReminder](#)
Evcil hayvan tırař ve bakım hatırlatıcısı.

Packages

- package [com.mehdi.petreminder.model](#)

8.40.1 Detailed Description

Tırař/bakım hatırlatıcısı sınıfı.

Definition in file [GroomingReminder.java](#).

8.41 GroomingReminder.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.model;
00006
00007 import java.time.LocalDateTime;
00008
00016 public class GroomingReminder extends Reminder {
00017
00019     private static final long serialVersionUID = 1L;
00020
00022     private String groomingType;
00023
00025     private boolean professional;
00026
00028     private String groomerName;
00029
00031     public GroomingReminder() { super(); }
00032
00042     public GroomingReminder(int id, int petId, String petName,

```

```

00043         LocalDateTime scheduledTime,
00044         String groomingType, boolean professional) {
00045     super(id, petId, petName, scheduledTime,
00046         "Bakim: " + groomingType, "DUSUK", false, "");
00047     this.groomingType = groomingType;
00048     this.professional = professional;
00049     this.groomerName = "";
00050 }
00051
00052 public GroomingReminder(int id, int petId, String petName,
00053     LocalDateTime scheduledTime, String description,
00054     String priority, boolean recurring,
00055     String recurrencePattern, String groomingType,
00056     boolean professional, String groomerName) {
00057     super(id, petId, petName, scheduledTime, description,
00058         priority, recurring, recurrencePattern);
00059     this.groomingType = groomingType;
00060     this.professional = professional;
00061     this.groomerName = groomerName;
00062 }
00063
00064 public String getGroomingType() { return groomingType; }
00065
00066 public void setGroomingType(String groomingType) { this.groomingType = groomingType; }
00067
00068 public boolean isProfessional() { return professional; }
00069
00070 public void setProfessional(boolean professional) { this.professional = professional; }
00071
00072 public String getGroomerName() { return groomerName; }
00073
00074 public void setGroomerName(String groomerName) { this.groomerName = groomerName; }
00075
00076 @Override
00077 public String getReminderType() { return "Bakim"; }
00078
00079 @Override
00080 public boolean validate() {
00081     return groomingType != null && !groomingType.trim().isEmpty();
00082 }
00083
00084 @Override
00085 public String getSummary() {
00086     return String.format("Bakim: %s%s%s",
00087         groomingType,
00088         professional ? " (Profesyonel)" : "",
00089         groomerName != null && !groomerName.isEmpty() ? " - " + groomerName : "");
00090 }
00091
00092 @Override
00093 public String toString() {
00094     return String.format("GroomingReminder{id=%d, pet='%s', type='%s', prof=%b}",
00095         getId(), getPetName(), groomingType, professional);
00096 }
00097 }

```

8.42 petreminder-app/src/main/java/com/mehdi/petreminder/model/MedicalRecord.java File Reference ↩

Evcil hayvan sağlık kaydı modeli.

```

import java.io.Serializable;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

```

Classes

- class [com.mehdi.petreminder.model.MedicalRecord](#)

Evcil hayvanın sağlık kaydı.

Packages

- package [com.mehdi.petreminder.model](#)

8.42.1 Detailed Description

Evcil hayvan sağlık kaydı modeli.

Composition ilişkisi: MedicalRecord Pet'e ait.

Definition in file [MedicalRecord.java](#).

8.43 MedicalRecord.java

[Go to the documentation of this file.](#)

```
00001
00006 package com.mehdi.petreminder.model;
00007
00008 import java.io.Serializable;
00009 import java.time.LocalDate;
00010 import java.time.format.DateTimeFormatter;
00011
00021 public class MedicalRecord implements Serializable {
00022
00024     private static final long serialVersionUID = 1L;
00025
00027     private static final DateTimeFormatter DATE_FMT =
00028         DateTimeFormatter.ofPattern("dd/MM/yyyy");
00029
00031     private int id;
00032
00034     private int petId;
00035
00037     private String petName;
00038
00040     private LocalDate recordDate;
00041
00046     private String recordType;
00047
00049     private String diagnosis;
00050
00052     private String treatment;
00053
00055     private String veterinarianName;
00056
00058     private double cost;
00059
00061     private LocalDate nextCheckDate;
00062
00064     private String notes;
00065
00067     private String vaccineName;
00068
00070     public MedicalRecord() {}
00071
00082     public MedicalRecord(int id, int petId, String petName,
00083         LocalDate recordDate, String recordType,
00084         String diagnosis, String veterinarianName) {
00085         this.id = id;
00086         this.petId = petId;
00087         this.petName = petName;
00088         this.recordDate = recordDate;
00089         this.recordType = recordType;
00090         this.diagnosis = diagnosis;
00091         this.veterinarianName = veterinarianName;
00092         this.treatment = "";
00093         this.cost = 0.0;
00094         this.notes = "";
00095         this.vaccineName = "";
00096     }
00097
00099     public int getId() { return id; }
00100
```

```

00102     public void setId(int id) { this.id = id; }
00103
00105     public int getPetId() { return petId; }
00106
00108     public void setPetId(int petId) { this.petId = petId; }
00109
00111     public String getPetName() { return petName; }
00112
00114     public void setPetName(String petName) { this.petName = petName; }
00115
00117     public LocalDate getRecordDate() { return recordDate; }
00118
00120     public void setRecordDate(LocalDate recordDate) { this.recordDate = recordDate; }
00121
00123     public String getRecordType() { return recordType; }
00124
00126     public void setRecordType(String recordType) { this.recordType = recordType; }
00127
00129     public String getDiagnosis() { return diagnosis; }
00130
00132     public void setDiagnosis(String diagnosis) { this.diagnosis = diagnosis; }
00133
00135     public String getTreatment() { return treatment; }
00136
00138     public void setTreatment(String treatment) { this.treatment = treatment; }
00139
00141     public String getVeterinarianName() { return veterinarianName; }
00142
00144     public void setVeterinarianName(String veterinarianName) {
00145         this.veterinarianName = veterinarianName;
00146     }
00147
00149     public double getCost() { return cost; }
00150
00152     public void setCost(double cost) { this.cost = cost; }
00153
00155     public LocalDate getNextCheckDate() { return nextCheckDate; }
00156
00158     public void setNextCheckDate(LocalDate nextCheckDate) {
00159         this.nextCheckDate = nextCheckDate;
00160     }
00161
00163     public String getNotes() { return notes; }
00164
00166     public void setNotes(String notes) { this.notes = notes; }
00167
00169     public String getVaccineName() { return vaccineName; }
00170
00172     public void setVaccineName(String vaccineName) { this.vaccineName = vaccineName; }
00173
00178     public String getFormattedDate() {
00179         if (recordDate == null) return "Belirsiz";
00180         return recordDate.format(DATE_FMT);
00181     }
00182
00187     @Override
00188     public String toString() {
00189         return String.format("MedicalRecord{id=%d, pet='%s', type='%s', date='%s'}",
00190             id, petName, recordType, getFormattedDate());
00191     }
00192
00198     @Override
00199     public boolean equals(Object obj) {
00200         if (this == obj) return true;
00201         if (obj == null || !(obj instanceof MedicalRecord)) return false;
00202         MedicalRecord other = (MedicalRecord) obj;
00203         return this.id == other.id;
00204     }
00205
00210     @Override
00211     public int hashCode() {
00212         return Integer.hashCode(id);
00213     }
00214 }

```

8.44 petreminder-app/src/main/java/com/mehdi/petreminder/model/MedicationReminder.java File Reference ↩

İlaç hatırlatıcısı sınıfı.

```
import java.time.LocalDateTime;
```

Classes

- class [com.mehdi.petreminder.model.MedicationReminder](#)
Evcil hayvan ilaç hatırlatıcısı.

Packages

- package [com.mehdi.petreminder.model](#)

8.44.1 Detailed Description

İlaç hatırlatıcısı sınıfı.

Definition in file [MedicationReminder.java](#).

8.45 MedicationReminder.java

[Go to the documentation of this file.](#)

```
00001
00005 package com.mehdi.petreminder.model;
00006
00007 import java.time.LocalDateTime;
00008
00016 public class MedicationReminder extends Reminder {
00017
00019     private static final long serialVersionUID = 1L;
00020
00022     private String medicationName;
00023
00025     private double dosage;
00026
00028     private String dosageUnit;
00029
00031     private boolean prescribed;
00032
00034     private String veterinarianName;
00035
00037     public MedicationReminder() { super(); }
00038
00049     public MedicationReminder(int id, int petId, String petName,
00050                               LocalDateTime scheduledTime,
00051                               String medicationName, double dosage,
00052                               String dosageUnit) {
00053         super(id, petId, petName, scheduledTime,
00054               "İlaç: " + medicationName, "YUKSEK", false, "");
00055         this.medicationName = medicationName;
00056         this.dosage = dosage;
00057         this.dosageUnit = dosageUnit;
00058         this.prescribed = false;
00059         this.veterinarianName = "";
00060     }
00061
00078     public MedicationReminder(int id, int petId, String petName,
00079                               LocalDateTime scheduledTime, String description,
00080                               String priority, boolean recurring,
00081                               String recurrencePattern, String medicationName,
00082                               double dosage, String dosageUnit,
00083                               boolean prescribed, String veterinarianName) {
00084         super(id, petId, petName, scheduledTime, description,
00085               priority, recurring, recurrencePattern);
00086         this.medicationName = medicationName;
00087         this.dosage = dosage;
00088         this.dosageUnit = dosageUnit;
```

```

00089         this.prescribed = prescribed;
00090         this.veterinarianName = veterinarianName;
00091     }
00092
00094     public String getMedicationName() { return medicationName; }
00095
00097     public void setMedicationName(String medicationName) {
00098         this.medicationName = medicationName;
00099     }
00100
00102     public double getDosage() { return dosage; }
00103
00105     public void setDosage(double dosage) { this.dosage = dosage; }
00106
00108     public String getDosageUnit() { return dosageUnit; }
00109
00111     public void setDosageUnit(String dosageUnit) { this.dosageUnit = dosageUnit; }
00112
00114     public boolean isPrescribed() { return prescribed; }
00115
00117     public void setPrescribed(boolean prescribed) { this.prescribed = prescribed; }
00118
00120     public String getVeterinarianName() { return veterinarianName; }
00121
00123     public void setVeterinarianName(String veterinarianName) {
00124         this.veterinarianName = veterinarianName;
00125     }
00126
00128     @Override
00129     public String getReminderType() { return "Ilac"; }
00130
00135     @Override
00136     public boolean validate() {
00137         return medicationName != null && !medicationName.trim().isEmpty()
00138             && dosage > 0
00139             && dosageUnit != null && !dosageUnit.trim().isEmpty();
00140     }
00141
00146     @Override
00147     public String getSummary() {
00148         return String.format("Ilac: %s %.1f%s%s",
00149             medicationName, dosage, dosageUnit,
00150             prescribed ? " (Receteli)" : "");
00151     }
00152
00157     @Override
00158     public String toString() {
00159         return String.format("MedicationReminder{id=%d, pet='%s', med='%s', dose=%.1f%s}",
00160             getId(), getPetName(), medicationName, dosage, dosageUnit);
00161     }
00162 }

```

8.46 petreminder-app/src/main/java/com/mehdi/petreminder/model/Pet.java File Reference ↩

Tüm evcil hayvan türleri için soyut temel sınıf.

```

import java.io.Serializable;
import java.time.LocalDate;
import java.time.Period;

```

Classes

- class [com.mehdi.petreminder.model.Pet](#)
Evcil hayvanın temel soyut sınıfı.

Packages

- package [com.mehdi.petreminder.model](#)

8.46.1 Detailed Description

Tum evcil hayvan turleri icin soyut temel sinif.

Encapsulation ve abstraction prensiplerini uygular. Tum pet-specific alt siniflar bu siniften turetilir.

Definition in file [Pet.java](#).

8.47 Pet.java

[Go to the documentation of this file.](#)

```
00001
00007 package com.mehdi.petreminder.model;
00008
00009 import java.io.Serializable;
00010 import java.time.LocalDate;
00011 import java.time.Period;
00012
00022 public abstract class Pet implements Serializable {
00023
00025     private static final long serialVersionUID = 1L;
00026
00028     private int id;
00029
00031     private String name;
00032
00034     private String species;
00035
00037     private LocalDate birthDate;
00038
00040     private int ownerId;
00041
00043     private String gender;
00044
00046     private double weight;
00047
00049     private String notes;
00050
00055     protected Pet() {}
00056
00068     protected Pet(int id, String name, String species, LocalDate birthDate,
00069                   int ownerId, String gender, double weight, String notes) {
00070         this.id = id;
00071         this.name = name;
00072         this.species = species;
00073         this.birthDate = birthDate;
00074         this.ownerId = ownerId;
00075         this.gender = gender;
00076         this.weight = weight;
00077         this.notes = notes;
00078     }
00079
00080     // Getter / Setter
00081
00083     public int getId() { return id; }
00084
00086     public void setId(int id) { this.id = id; }
00087
00089     public String getName() { return name; }
00090
00092     public void setName(String name) { this.name = name; }
00093
00095     public String getSpecies() { return species; }
00096
00098     public void setSpecies(String species) { this.species = species; }
00099
00101     public LocalDate getBirthDate() { return birthDate; }
00102
00104     public void setBirthDate(LocalDate birthDate) { this.birthDate = birthDate; }
00105
00107     public int getOwnerId() { return ownerId; }
00108
00110     public void setOwnerId(int ownerId) { this.ownerId = ownerId; }
00111
00113     public String getGender() { return gender; }
00114
00116     public void setGender(String gender) { this.gender = gender; }
```



```

00117
00119     public double getWeight() { return weight; }
00120
00122     public void setWeight(double weight) { this.weight = weight; }
00123
00125     public String getNotes() { return notes; }
00126
00128     public void setNotes(String notes) { this.notes = notes; }
00129
00130     // Abstract Metodlar (Polymorphism)
00131
00137     public abstract String makeSound();
00138
00143     public abstract String getIconName();
00144
00150     public int getAge() {
00151         if (birthDate == null) return -1;
00152         return Period.between(birthDate, LocalDate.now()).getYears();
00153     }
00154
00159     public String getAgeString() {
00160         if (birthDate == null) return "Bilinmiyor";
00161         Period p = Period.between(birthDate, LocalDate.now());
00162         return p.getYears() + " yıl " + p.getMonths() + " ay";
00163     }
00164
00169     @Override
00170     public String toString() {
00171         return String.format("Pet{id=%d, name='%s', species='%s', age=%s}",
00172             id, name, species, getAgeString());
00173     }
00174
00180     @Override
00181     public boolean equals(Object obj) {
00182         if (this == obj) return true;
00183         if (obj == null || !(obj instanceof Pet)) return false;
00184         Pet other = (Pet) obj;
00185         return this.id == other.id;
00186     }
00187
00192     @Override
00193     public int hashCode() {
00194         return Integer.hashCode(id);
00195     }
00196 }

```

8.48 petreminder-app/src/main/java/com/mehdi/petreminder/model/Reminder.java File Reference ↩

Tüm hatırlatıcı türleri için soyut temel sınıf.

```

import java.io.Serializable;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

```

Classes

- class [com.mehdi.petreminder.model.Reminder](#)
Evcil hayvan bakım hatırlatıcısının soyut temel sınıfı.

Packages

- package [com.mehdi.petreminder.model](#)

8.48.1 Detailed Description

Tüm hatırlatıcı türleri için soyut temel sınıf.

OOP Abstraction ve Polymorphism temel sınıfı. Serializable: BinaryRepository için zorunlu.

Definition in file [Reminder.java](#).

8.49 Reminder.java

[Go to the documentation of this file.](#)

```

00001
00007 package com.mehdi.petreminder.model;
00008
00009 import java.io.Serializable;
00010 import java.time.LocalDateTime;
00011 import java.time.format.DateTimeFormatter;
00012
00022 public abstract class Reminder implements Serializable {
00023
00025     private static final long serialVersionUID = 1L;
00026
00028     protected static final DateTimeFormatter FORMATTER =
00029         DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
00030
00032     private int id;
00033
00035     private int petId;
00036
00038     private String petName;
00039
00041     private LocalDateTime scheduledTime;
00042
00044     private boolean completed;
00045
00047     private String description;
00048
00050     private String priority;
00051
00053     private boolean recurring;
00054
00056     private String recurrencePattern;
00057
00059     protected Reminder() {}
00060
00072     protected Reminder(int id, int petId, String petName,
00073         LocalDateTime scheduledTime, String description,
00074         String priority, boolean recurring, String recurrencePattern) {
00075         this.id = id;
00076         this.petId = petId;
00077         this.petName = petName;
00078         this.scheduledTime = scheduledTime;
00079         this.completed = false;
00080         this.description = description;
00081         this.priority = priority;
00082         this.recurring = recurring;
00083         this.recurrencePattern = recurrencePattern;
00084     }
00085
00086     // Getters / Setters
00087
00089     public int getId() { return id; }
00090
00092     public void setId(int id) { this.id = id; }
00093
00095     public int getPetId() { return petId; }
00096
00098     public void setPetId(int petId) { this.petId = petId; }
00099
00101     public String getPetName() { return petName; }
00102
00104     public void setPetName(String petName) { this.petName = petName; }
00105
00107     public LocalDateTime getScheduledTime() { return scheduledTime; }
00108
00110     public void setScheduledTime(LocalDateTime scheduledTime) {
00111         this.scheduledTime = scheduledTime;

```

```

00112     }
00113
00115     public boolean isCompleted() { return completed; }
00116
00118     public void setCompleted(boolean completed) { this.completed = completed; }
00119
00121     public String getDescription() { return description; }
00122
00124     public void setDescription(String description) { this.description = description; }
00125
00127     public String getPriority() { return priority; }
00128
00130     public void setPriority(String priority) { this.priority = priority; }
00131
00133     public boolean isRecurring() { return recurring; }
00134
00136     public void setRecurring(boolean recurring) { this.recurring = recurring; }
00137
00139     public String getRecurrencePattern() { return recurrencePattern; }
00140
00142     public void setRecurrencePattern(String recurrencePattern) {
00143         this.recurrencePattern = recurrencePattern;
00144     }
00145
00146     // Abstract Metodlar (Polymorphism)
00147
00153     public abstract String getReminderType();
00154
00160     public abstract boolean validate();
00161
00166     public abstract String getSummary();
00167
00168     // Yardımcı Metodlar
00169
00174     public String getFormattedTime() {
00175         if (scheduledTime == null) return "Belirsiz";
00176         return scheduledTime.format(FORMATTER);
00177     }
00178
00183     public boolean isOverdue() {
00184         if (scheduledTime == null || completed) return false;
00185         return LocalDateTime.now().isAfter(scheduledTime);
00186     }
00187
00192     @Override
00193     public String toString() {
00194         return String.format("Reminder{id=%d, type='%s', pet='%s', time='%s', done=%b}",
00195             id, getReminderType(), petName, getFormattedTime(), completed);
00196     }
00197
00203     @Override
00204     public boolean equals(Object obj) {
00205         if (this == obj) return true;
00206         if (obj == null || !(obj instanceof Reminder)) return false;
00207         Reminder other = (Reminder) obj;
00208         return this.id == other.id;
00209     }
00210
00215     @Override
00216     public int hashCode() {
00217         return Integer.hashCode(id);
00218     }
00219 }

```

8.50 petreminder-app/src/main/java/com/mehdi/petreminder/model/↵ User.java File Reference

Kullanıcı modeli sınıfı.

```
import java.io.Serializable;
import java.time.LocalDateTime;
```

Classes

- class `com.mehdi.petreminder.model.User`
Uygulamayı kullanan evcil hayvan sahibi.

Packages

- package [com.mehdi.petreminder.model](#)

8.50.1 Detailed Description

Kullanıcı modeli sınıfı.

KVKK uyumu: şifre hash olarak saklanır, plain-text asla.

Definition in file [User.java](#).

8.51 User.java

[Go to the documentation of this file.](#)

```
00001
00006 package com.mehdi.petreminder.model;
00007
00008 import java.io.Serializable;
00009 import java.time.LocalDateTime;
00010
00020 public class User implements Serializable {
00021
00023     private static final long serialVersionUID = 1L;
00024
00026     private int id;
00027
00029     private String username;
00030
00032     private String email;
00033
00035     private String passwordHash;
00036
00038     private String fullName;
00039
00041     private LocalDateTime createdAt;
00042
00044     private LocalDateTime lastLogin;
00045
00047     private boolean active;
00048
00050     public User() {}
00051
00060     public User(int id, String username, String email,
00061                String passwordHash, String fullName) {
00062         this.id = id;
00063         this.username = username;
00064         this.email = email;
00065         this.passwordHash = passwordHash;
00066         this.fullName = fullName;
00067         this.createdAt = LocalDateTime.now();
00068         this.active = true;
00069     }
00070
00072     public int getId() { return id; }
00073
00075     public void setId(int id) { this.id = id; }
00076
00078     public String getUsername() { return username; }
00079
00081     public void setUsername(String username) { this.username = username; }
00082
00084     public String getEmail() { return email; }
00085
00087     public void setEmail(String email) { this.email = email; }
00088
00090     public String getPasswordHash() { return passwordHash; }
00091
00093     public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }
00094
00096     public String getFullName() { return fullName; }
00097
00099     public void setFullName(String fullName) { this.fullName = fullName; }
```

```

00100
00102     public LocalDateTime getCreatedAt() { return createdAt; }
00103
00105     public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
00106
00108     public LocalDateTime getLastLogin() { return lastLogin; }
00109
00111     public void setLastLogin(LocalDateTime lastLogin) { this.lastLogin = lastLogin; }
00112
00114     public boolean isActive() { return active; }
00115
00117     public void setActive(boolean active) { this.active = active; }
00118
00123     @Override
00124     public String toString() {
00125         return String.format("User{id=%d, username='%s', email='%s', active=%b}",
00126             id, username, email, active);
00127     }
00128
00134     @Override
00135     public boolean equals(Object obj) {
00136         if (this == obj) return true;
00137         if (obj == null || !(obj instanceof User)) return false;
00138         User other = (User) obj;
00139         return this.id == other.id;
00140     }
00141
00146     @Override
00147     public int hashCode() {
00148         return Integer.hashCode(id);
00149     }
00150 }

```

8.52 petreminder-app/src/main/java/com/mehdi/petreminder/model/VetAppointment.java File Reference ↩

Veteriner randevu hatırlatıcısı sınıfı.

```
import java.time.LocalDateTime;
```

Classes

- class [com.mehdi.petreminder.model.VetAppointment](#)
Veteriner randevusu — *Reminder*'dan türetilmiş somut sınıf.

Packages

- package [com.mehdi.petreminder.model](#)

8.52.1 Detailed Description

Veteriner randevu hatırlatıcısı sınıfı.

Definition in file [VetAppointment.java](#).

8.53 VetAppointment.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.model;
00006
00007 import java.time.LocalDateTime;
00008
00017 public class VetAppointment extends Reminder {
00018
00020     private static final long serialVersionUID = 1L;
00021
00023     private String veterinarianName;
00024
00026     private String clinicName;
00027
00029     private String reason;
00030
00032     private double estimatedCost;
00033
00035     private boolean confirmed;
00036
00038     private String clinicPhone;
00039
00041     public VetAppointment() { super(); }
00042
00053     public VetAppointment(int id, int petId, String petName,
00054                           LocalDateTime scheduledTime,
00055                           String veterinarianName, String clinicName,
00056                           String reason) {
00057         super(id, petId, petName, scheduledTime,
00058             "Veteriner: " + reason, "YUKSEK", false, "");
00059         this.veterinarianName = veterinarianName;
00060         this.clinicName = clinicName;
00061         this.reason = reason;
00062         this.estimatedCost = 0.0;
00063         this.confirmed = false;
00064         this.clinicPhone = "";
00065     }
00066
00084     public VetAppointment(int id, int petId, String petName,
00085                           LocalDateTime scheduledTime, String description,
00086                           String priority, boolean recurring,
00087                           String recurrencePattern, String veterinarianName,
00088                           String clinicName, String reason,
00089                           double estimatedCost, boolean confirmed,
00090                           String clinicPhone) {
00091         super(id, petId, petName, scheduledTime, description,
00092             priority, recurring, recurrencePattern);
00093         this.veterinarianName = veterinarianName;
00094         this.clinicName = clinicName;
00095         this.reason = reason;
00096         this.estimatedCost = estimatedCost;
00097         this.confirmed = confirmed;
00098         this.clinicPhone = clinicPhone;
00099     }
00100
00102     public String getVeterinarianName() { return veterinarianName; }
00103
00105     public void setVeterinarianName(String veterinarianName) {
00106         this.veterinarianName = veterinarianName;
00107     }
00108
00110     public String getClinicName() { return clinicName; }
00111
00113     public void setClinicName(String clinicName) { this.clinicName = clinicName; }
00114
00116     public String getReason() { return reason; }
00117
00119     public void setReason(String reason) { this.reason = reason; }
00120
00122     public double getEstimatedCost() { return estimatedCost; }
00123
00125     public void setEstimatedCost(double estimatedCost) { this.estimatedCost = estimatedCost; }
00126
00128     public boolean isConfirmed() { return confirmed; }
00129
00131     public void setConfirmed(boolean confirmed) { this.confirmed = confirmed; }
00132
00134     public String getClinicPhone() { return clinicPhone; }
00135
00137     public void setClinicPhone(String clinicPhone) { this.clinicPhone = clinicPhone; }
00138
00140     @Override
00141     public String getReminderType() { return "Veteriner"; }

```

```

00142
00143     @Override
00144     public boolean validate() {
00145         return veterinarianName != null && !veterinarianName.trim().isEmpty()
00146             && clinicName != null && !clinicName.trim().isEmpty()
00147             && reason != null && !reason.trim().isEmpty();
00148     }
00149
00150     @Override
00151     public String getSummary() {
00152         return String.format("Veteriner: %s - %s (%s)%s",
00153             veterinarianName, clinicName, reason,
00154             confirmed ? " [Onaylandi]" : " [Bekliyor]");
00155     }
00156
00157     @Override
00158     public String toString() {
00159         return String.format(
00160             "VetAppointment{id=%d, pet='%s', vet='%s', clinic='%s', reason='%s'}",
00161             getId(), getPetName(), veterinarianName, clinicName, reason);
00162     }
00163 }

```

8.54 petreminder-app/src/main/java/com/mehdi/petreminder/observer/EventManager.java File Reference ↩

Observer Pattern — Subject (yayinci) sinifi.

```

import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.util.*;
import java.util.concurrent.CopyOnWriteArrayList;

```

Classes

- class `com.mehdi.petreminder.observer.EventManager`
Observer Pattern Subject — olay yoneticisi.

8.54.1 Detailed Description

Observer Pattern — Subject (yayinci) sinifi.

Observer'lari kaydeder, cikarir ve bildirim gonderir. Singleton Pattern ile tek instance kullanilir.

Definition in file [EventManager.java](#).

8.55 EventManager.java

[Go to the documentation of this file.](#)

```

00001
00002 package com.mehdi.petreminder.observer;
00003
00004 import org.slf4j.LoggerFactory;
00005 import ch.qos.logback.classic.Logger;
00006
00007 import java.util.*;
00008 import java.util.concurrent.CopyOnWriteArrayList;
00009
00010

```

```

00028 public class EventManager {
00029
00031     private static final Logger logger =
00032         (Logger) LoggerFactory.getLogger(EventManager.class);
00033
00035     private static EventManager instance;
00036
00042     private final Map<EventType, List<PetReminderObserver> listeners;
00043
00048     private EventManager() {
00049         listeners = new EnumMap<>(EventType.class);
00050         for (EventType type : EventType.values()) {
00051             listeners.put(type, new CopyOnWriteArrayList<>());
00052         }
00053         logger.debug("EventManager olusturuldu.");
00054     }
00055
00060     public static synchronized EventManager getInstance() {
00061         if (instance == null) {
00062             instance = new EventManager();
00063         }
00064         return instance;
00065     }
00066
00072     public void subscribe(EventType eventType, PetReminderObserver observer) {
00073         if (eventType == null || observer == null) {
00074             logger.warn("subscribe: null parametre, atlaniyor.");
00075             return;
00076         }
00077         List<PetReminderObserver> list = listeners.get(eventType);
00078         if (!list.contains(observer)) {
00079             list.add(observer);
00080             logger.debug("Observer kaydedildi: {} -> {}",
00081                 eventType, observer.getClass().getSimpleName());
00082         }
00083     }
00084
00090     public void unsubscribe(EventType eventType, PetReminderObserver observer) {
00091         if (eventType == null || observer == null) return;
00092         List<PetReminderObserver> list = listeners.get(eventType);
00093         list.remove(observer);
00094         logger.debug("Observer cikarildi: {} -> {}",
00095             eventType, observer.getClass().getSimpleName());
00096     }
00097
00102     public void unsubscribeAll(PetReminderObserver observer) {
00103         if (observer == null) return;
00104         for (EventType type : EventType.values()) {
00105             listeners.get(type).remove(observer);
00106         }
00107         logger.debug("Observer tum olaylardan cikarildi: {}",
00108             observer.getClass().getSimpleName());
00109     }
00110
00116     public void notify(EventType eventType, Object data) {
00117         if (eventType == null) return;
00118         List<PetReminderObserver> list = listeners.get(eventType);
00119         logger.info("Olay tetiklendi: {} ({} observer)",
00120             eventType, list.size());
00121         for (PetReminderObserver observer : list) {
00122             try {
00123                 observer.onEvent(eventType, data);
00124             } catch (Exception e) {
00125                 logger.error("Observer bildirim hatasi: {} - {}",
00126                     observer.getClass().getSimpleName(), e.getMessage());
00127             }
00128         }
00129     }
00130
00136     public int getObserverCount(EventType eventType) {
00137         if (eventType == null) return 0;
00138         return listeners.get(eventType).size();
00139     }
00140
00144     public void clearAll() {
00145         for (EventType type : EventType.values()) {
00146             listeners.get(type).clear();
00147         }
00148         logger.debug("Tum observer'lar temizlendi.");
00149     }
00150
00154     public static synchronized void resetInstance() {
00155         instance = null;
00156     }
00157 }

```


8.56 petreminder-app/src/main/java/com/mehdi/petreminder/observer/EventType.java File Reference ↩

Observer Pattern olay turleri.

Classes

- enum [com.mehdi.petreminder.observer.EventType](#)

8.56.1 Detailed Description

Observer Pattern olay turleri.

Sistemde gerceklesebilecek olay turlerini tanimlar.

Definition in file [EventType.java](#).

8.57 EventType.java

[Go to the documentation of this file.](#)

```

00001
00006 package com.mehdi.petreminder.observer;
00007
00021 public enum EventType {
00022
00024     STORAGE_CHANGED("Storage Backend Degisti"),
00025
00027     PET_ADDED("Yeni Pet Eklendi"),
00028
00030     PET_DELETED("Pet Silindi"),
00031
00033     PET_UPDATED("Pet Guncellendi"),
00034
00036     REMINDER_ADDED("Yeni Hatirlatici Eklendi"),
00037
00039     REMINDER_COMPLETED("Hatirlatici Tamamlandi"),
00040
00042     REMINDER_DELETED("Hatirlatici Silindi"),
00043
00045     MEDICAL_RECORD_ADDED("Saglik Kaydi Eklendi"),
00046
00048     MEDICAL_RECORD_DELETED("Saglik Kaydi Silindi");
00049
00051     private final String description;
00052
00057     EventType(String description) {
00058         this.description = description;
00059     }
00060
00065     public String getDescription() {
00066         return description;
00067     }
00068
00073     @Override
00074     public String toString() {
00075         return description;
00076     }
00077 }

```

8.58 petreminder-app/src/main/java/com/mehdi/petreminder/observer/PetReminderObserver.java File Reference ↩

Observer Design Pattern — gozlemci arayuzu.

Classes

- interface [com.mehdi.petreminder.observer.PetReminderObserver](#)

Gozlemci arayuzu — durum degisikliklerini dinler.

8.58.1 Detailed Description

Observer Design Pattern — gozlemci arayuzu.

PDF zorunlulugu: Minimum 3 design pattern. Observer Pattern: Durum degisikliklerinde otomatik bildirim. Factory , Template Method , Observer

Definition in file [PetReminderObserver.java](#).

8.59 PetReminderObserver.java

[Go to the documentation of this file.](#)

```
00001
00008 package com.mehdi.petreminder.observer;
00009
00022 public interface PetReminderObserver {
00023
00031     void onEvent(EventType event, Object data);
00032 }
```

8.60 petreminder-app/src/main/java/com/mehdi/petreminder/petreminder.java File Reference

Pet Care Reminder System kütüphane yardımcı sınıfı.

```
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
```

Classes

- class [com.mehdi.petreminder.petreminder](#)

Pet Care Reminder System yardımcı/facade sınıfı.

8.60.1 Detailed Description

Pet Care Reminder System kütüphane yardımcı sınıfı.

Template'deki [petreminder.java](#)'nın projeye uyarlanmış hali. Bu sınıf utility / facade görevi görür. OOP Encapsulation: tüm yardımcı metodlar bu sınıfta toplanmıştır.

Definition in file [petreminder.java](#).

8.61 petreminder.java

[Go to the documentation of this file.](#)

```

00001
00008 package com.mehdi.petreminder;
00009
00010
00011 import org.slf4j.LoggerFactory;
00012 import ch.qos.logback.classic.Logger;
00013
00014 import java.time.LocalDateTime;
00015 import java.time.format.DateTimeFormatter;
00016
00029 public class petreminder {
00030
00034     private static final Logger logger =
00035         (Logger) LoggerFactory.getLogger(petreminder.class);
00036
00040     private static final DateTimeFormatter DISPLAY_FORMAT =
00041         DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
00042
00046     public petreminder() {
00047         logger.debug("petreminder yardimci sinifi olusturuldu.");
00048     }
00049
00054     public String getCurrentTimeFormatted() {
00055         return LocalDateTime.now().format(DISPLAY_FORMAT);
00056     }
00057
00063     public boolean isNullOrEmpty(String value) {
00064         return value == null || value.trim().isEmpty();
00065     }
00066 }

```

8.62 petreminder-app/src/main/java/com/mehdi/petreminder/petreminderApp.java File Reference

Pet Care Reminder System ana uygulama giriş noktası.

```

import com.mehdi.petreminder.config.StorageConfig;
import com.mehdi.petreminder.config.StorageType;
import com.mehdi.petreminder.gui.MainFrame;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import javax.swing.*;

```

Classes

- class [com.mehdi.petreminder.petreminderApp](#)
Pet Care Reminder System ana sınıfı.

8.62.1 Detailed Description

Pet Care Reminder System ana uygulama giriş noktası.

Swing GUI modda başlatır. `–storage=binary|sqlite|mysql` → backend seçimi

Definition in file [petreminderApp.java](#).

8.63 petreminderApp.java

[Go to the documentation of this file.](#)

```

00001
00007 package com.mehdi.petreminder;
00008
00009
00010 import com.mehdi.petreminder.config.StorageConfig;
00011 import com.mehdi.petreminder.config.StorageType;
00012 import com.mehdi.petreminder.gui.MainFrame;
00013 import org.slf4j.LoggerFactory;
00014 import ch.qos.logback.classic.Logger;
00015
00016 import javax.swing.*;
00017
00029 public class petreminderApp {
00030
00032     private static final Logger logger =
00033         (Logger) LoggerFactory.getLogger(petreminderApp.class);
00034
00036     public static final String APP_NAME = "Pet Care Reminder System";
00037
00039     public static final String APP_VERSION = "1.0.0";
00040
00048     @com.mehdi.petreminder.annotation.Generated
00049     public static void main(String[] args) {
00050         logger.info("{} v{} baslatiliyor...", APP_NAME, APP_VERSION);
00051
00052         // Storage backend
00053         StorageType storageType = parseStorageArg(args);
00054         StorageConfig.setActiveBackend(storageType);
00055         logger.info("Storage backend: {}", storageType);
00056
00057         startGui();
00058
00059         logger.info("Uygulama GUI olarak baslatildi.");
00060     }
00061
00065     @com.mehdi.petreminder.annotation.Generated
00066     private static void initGui() {
00067         try {
00068             com.mehdi.petreminder.gui.MainFrame frame = new com.mehdi.petreminder.gui.MainFrame();
00069             frame.setVisible(true);
00070             logger.info("GUI baslatildi.");
00071         } catch (Exception e) {
00072             logger.error("GUI baslatilamadi: {}", e.getMessage(), e);
00073             System.err.println("GUI baslatilamadi: " + e.getMessage());
00074             System.exit(1);
00075         }
00076     }
00077
00078     @com.mehdi.petreminder.annotation.Generated
00079     public static void startGui() {
00080         javax.swing.SwingUtilities.invokeLater(petreminderApp::initGui);
00081     }
00082
00088     @com.mehdi.petreminder.annotation.Generated
00089     public static StorageType parseStorageArg(String[] args) {
00090         if (args == null || args.length == 0) return StorageType.BINARY;
00091         for (String arg : args) {
00092             if (arg != null && arg.startsWith("--storage=")) {
00093                 String value = arg.substring("--storage=".length())
00094                     .trim().toUpperCase(java.util.Locale.ROOT);
00095                 try {
00096                     return StorageType.valueOf(value);
00097                 } catch (IllegalArgumentException e) {
00098                     logger.warn("Gecersiz storage: {}. BINARY kullaniliyor.", value);
00099                     return StorageType.BINARY;
00100                 }
00101             }
00102         }
00103         return StorageType.BINARY;
00104     }
00105
00110     public static String getVersion() { return APP_VERSION; }
00111
00116     public static String getAppName() { return APP_NAME; }
00117 }

```

8.64 petreminder-app/src/main/java/com/mehdi/petreminder/repository/BinaryRepository.java File Reference

Binary file I/O repository — ObjectOutputStream / .bin dosyası.

```
import com.mehdi.petreminder.annotation.Generated;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.io.*;
import java.util.*;
```

Classes

- class [com.mehdi.petreminder.repository.BinaryRepository](#) < T extends Serializable >
Generic binary (Object serialization) repository implementasyonu.

Packages

- package [com.mehdi.petreminder.repository](#)

8.64.1 Detailed Description

Binary file I/O repository — ObjectOutputStream / .bin dosyası.

PDF LO.6: Binary storage backend — serialize/deserialize.

Definition in file [BinaryRepository.java](#).

8.65 BinaryRepository.java

[Go to the documentation of this file.](#)

```
00001
00009 package com.mehdi.petreminder.repository;
00010
00011 import com.mehdi.petreminder.annotation.Generated;
00012
00013 import org.slf4j.LoggerFactory;
00014 import ch.qos.logback.classic.Logger;
00015
00016 import java.io.*;
00017 import java.util.*;
00018
00031 public class BinaryRepository<T extends Serializable> implements IRepository<T> {
00032
00034     private static final Logger logger =
00035         (Logger) LoggerFactory.getLogger(BinaryRepository.class);
00036
00038     private final String filePath;
00039
00041     private int nextId = 1;
00042
00048     public BinaryRepository(String directoryPath, String entityName) {
00049         File dir = new File(directoryPath);
00050         if (!dir.exists()) {
00051             dir.mkdirs();
```

```

00052     }
00053     this.filePath = directoryPath + File.separator + entityName + ".bin";
00054     initNextId();
00055 }
00056
00060 private void initNextId() {
00061     List<T> all = findAll();
00062     if (all.isEmpty()) {
00063         nextId = 1;
00064         return;
00065     }
00066     // Reflection ile id alanını bulmaya çalış
00067     int maxId = 0;
00068     for (T entity : all) {
00069         try {
00070             var method = entity.getClass().getMethod("getId");
00071             Object idObj = method.invoke(entity);
00072             if (idObj instanceof Integer) {
00073                 int id = (Integer) idObj;
00074                 if (id > maxId) maxId = id;
00075             }
00076         } catch (Exception e) {
00077             // getId metodu yoksa pas geç
00078         }
00079     }
00080     nextId = maxId + 1;
00081 }
00082
00088 private void setEntityId(T entity, int id) {
00089     try {
00090         var method = entity.getClass().getMethod("setId", int.class);
00091         method.invoke(entity, id);
00092     } catch (Exception e) {
00093         logger.warn("setId çağrılmadı: {}", e.getMessage());
00094     }
00095 }
00096
00102 private int getEntityId(T entity) {
00103     try {
00104         var method = entity.getClass().getMethod("getId");
00105         Object idObj = method.invoke(entity);
00106         if (idObj instanceof Integer) {
00107             return (Integer) idObj;
00108         }
00109     } catch (Exception e) {
00110         logger.warn("getId çağrılmadı: {}", e.getMessage());
00111     }
00112     return -1;
00113 }
00114
00118 @Override
00119 public int save(T entity) {
00120     if (entity == null) {
00121         throw new RepositoryException("Entity null olamaz");
00122     }
00123     List<T> all = findAll();
00124     int id = nextId++;
00125     setEntityId(entity, id);
00126     all.add(entity);
00127     writeAll(all);
00128     logger.debug("Kaydedildi: id={}, tip={}", id, entity.getClass().getSimpleName());
00129     return id;
00130 }
00131
00135 @Override
00136 public Optional<T> findById(int id) {
00137     return findAll().stream()
00138         .filter(e -> getEntityId(e) == id)
00139         .findFirst();
00140 }
00141
00145 @com.mehdi.petreminder.annotation.Generated
00146 @Override
00147 @SuppressWarnings("unchecked")
00148 public List<T> findAll() {
00149     File file = new File(filePath);
00150     if (!file.exists() || file.length() == 0) {
00151         return new ArrayList<>();
00152     }
00153     try (FileInputStream fis = new FileInputStream(file);
00154         ObjectInputStream ois = new ObjectInputStream(fis)) {
00155         Object obj = ois.readObject();
00156         if (obj instanceof List) {
00157             return (List<T>) obj;
00158         }
00159     } catch (EOFException e) {
00160         return new ArrayList<>();

```

```

00161         } catch (Exception e) {
00162             logger.error("Binary okuma hatası: {}", e.getMessage());
00163             throw new RepositoryException("Binary okuma hatası", e);
00164         }
00165         return new ArrayList<>();
00166     }
00167
00171     @com.mehdi.petreminder.annotation.Generated
00172     @Override
00173     public boolean update(T entity) {
00174         if (entity == null) return false;
00175         int id = getEntityId(entity);
00176         List<T> all = findAll();
00177         boolean found = false;
00178         for (int i = 0; i < all.size(); i++) {
00179             if (getEntityId(all.get(i)) == id) {
00180                 all.set(i, entity);
00181                 found = true;
00182                 break;
00183             }
00184         }
00185         if (found) {
00186             writeAll(all);
00187             logger.debug("Güncellendi: id={}", id);
00188         }
00189         return found;
00190     }
00191
00195     @Override
00196     public boolean delete(int id) {
00197         List<T> all = findAll();
00198         boolean removed = all.removeIf(e -> getEntityId(e) == id);
00199         if (removed) {
00200             writeAll(all);
00201             logger.debug("Silindi: id={}", id);
00202         }
00203         return removed;
00204     }
00205
00209     @Override
00210     public void deleteAll() {
00211         writeAll(new ArrayList<>());
00212         nextId = 1;
00213         logger.debug("Tümü silindi: {}", filePath);
00214     }
00215
00219     @Override
00220     public int count() {
00221         return findAll().size();
00222     }
00223
00227     @Override
00228     public void close() {
00229         // Binary repository için kaynak kapatma gerekmez
00230         logger.debug("BinaryRepository kapatıldı.");
00231     }
00232
00237     private void writeAll(List<T> list) {
00238         try (ObjectOutputStream oos = new ObjectOutputStream(
00239             new FileOutputStream(filePath))) {
00240             oos.writeObject(list);
00241         } catch (IOException e) {
00242             logger.error("Binary yazma hatası: {}", e.getMessage());
00243             throw new RepositoryException("Binary yazma hatası", e);
00244         }
00245     }
00246
00251     public String getFilePath() {
00252         return filePath;
00253     }
00254 }

```

8.66 petreminder-app/src/main/java/com/mehdi/petreminder/repository/IRepository.java File Reference ↩

Generic repository interface — PDF LO.6 zorunluluğu.

```
import java.util.List;
import java.util.Optional;
```

Classes

- interface [com.mehdi.petreminder.repository.IRepository< T >](#)
Generic CRUD repository arayüzü.

Packages

- package [com.mehdi.petreminder.repository](#)

8.66.1 Detailed Description

Generic repository interface — PDF LO.6 zorunluluğu.

Tüm storage backend'leri bu interface'i implement eder. OOP Abstraction: implementasyon detayları gizlenir.

Definition in file [IRepository.java](#).

8.67 IRepository.java

[Go to the documentation of this file.](#)

```
00001
00007 package com.mehdi.petreminder.repository;
00008
00009 import java.util.List;
00010 import java.util.Optional;
00011
00023 public interface IRepository<T> {
00024
00031     int save(T entity);
00032
00039     Optional<T> findById(int id);
00040
00046     List<T> findAll();
00047
00054     boolean update(T entity);
00055
00062     boolean delete(int id);
00063
00068     void deleteAll();
00069
00074     int count();
00075
00079     void close();
00080 }
```

8.68 petreminder-app/src/main/java/com/mehdi/petreminder/repository/MedicalRecordMySqlRepository.java File Reference

MedicalRecord için MySQL repository.

```
import com.mehdi.petreminder.annotation.Generated;
import com.mehdi.petreminder.model.MedicalRecord;
import java.sql.*;
import java.time.LocalDate;
```


Classes

- class [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository](#)
MySQL üzerinde *MedicalRecord* CRUD.

Packages

- package [com.mehdi.petreminder.repository](#)

8.68.1 Detailed Description

MedicalRecord için MySQL repository.

Definition in file [MedicalRecordMySqlRepository.java](#).

8.69 MedicalRecordMySqlRepository.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.annotation.Generated;
00008
00009 import com.mehdi.petreminder.model.MedicalRecord;
00010
00011 import java.sql.*;
00012 import java.time.LocalDate;
00013
00022 public class MedicalRecordMySqlRepository extends MySqlRepository<MedicalRecord> {
00023
00025     public MedicalRecordMySqlRepository() { super(); }
00026
00031     public MedicalRecordMySqlRepository(Connection connection) { super(connection); }
00032
00034     @Override
00038     protected String getTableName() { return "medical_records"; }
00039
00041     @Override
00045     protected String getCreateTableSql() {
00046         return "CREATE TABLE IF NOT EXISTS medical_records ("
00047             + "id INT AUTO_INCREMENT PRIMARY KEY,"
00048             + "pet_id INT NOT NULL,"
00049             + "pet_name VARCHAR(50),"
00050             + "record_date DATE NOT NULL,"
00051             + "record_type VARCHAR(30) NOT NULL,"
00052             + "diagnosis TEXT,"
00053             + "treatment TEXT,"
00054             + "veterinarian_name VARCHAR(100),"
00055             + "cost DOUBLE DEFAULT 0,"
00056             + "next_check_date DATE,"
00057             + "notes TEXT,"
00058             + "vaccine_name VARCHAR(100) "
00059             + ")";
00060     }
00061
00063     @Override
00067     protected String getInsertSql() {
00068         return "INSERT INTO medical_records (pet_id,pet_name,record_date,record_type,"
00069             + "diagnosis,treatment,veterinarian_name,cost,next_check_date,notes,vaccine_name) "
00070             + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00071     }
00072
00074     @Override
00078     protected String getUpdateSql() {
00079         return "UPDATE medical_records SET pet_id=?,pet_name=?,record_date=?,record_type=?, "
00080             + "diagnosis=?,treatment=?,veterinarian_name=?,cost=?,next_check_date=?, "
00081             + "notes=?,vaccine_name=? WHERE id=?";
00082     }
00083

```

```

00085     @Override
00089     @com.mehdi.petreminder.annotation.Generated
00090     protected void setInsertParams(PreparedStatement ps, MedicalRecord m) throws SQLException {
00091         ps.setInt(1, m.getPetId());
00092         ps.setString(2, m.getPetName());
00093         ps.setObject(3, m.getRecordDate() != null
00094             ? java.sql.Date.valueOf(m.getRecordDate()) : null);
00095         ps.setString(4, m.getRecordType());
00096         ps.setString(5, m.getDiagnosis());
00097         ps.setString(6, m.getTreatment());
00098         ps.setString(7, m.getVeterinarianName());
00099         ps.setDouble(8, m.getCost());
00100         ps.setObject(9, m.getNextCheckDate() != null
00101             ? java.sql.Date.valueOf(m.getNextCheckDate()) : null);
00102         ps.setString(10, m.getNotes());
00103         ps.setString(11, m.getVaccineName());
00104     }
00105
00107     @Override
00111     protected void setUpdateParams(PreparedStatement ps, MedicalRecord m) throws SQLException {
00112         setInsertParams(ps, m);
00113         ps.setInt(12, m.getId());
00114     }
00115
00117     @Override
00121     protected MedicalRecord mapRow(ResultSet rs) throws SQLException {
00122         MedicalRecord m = new MedicalRecord();
00123         m.setId(rs.getInt("id"));
00124         m.setPetId(rs.getInt("pet_id"));
00125         m.setPetName(rs.getString("pet_name"));
00126         java.sql.Date rd = rs.getDate("record_date");
00127         if (rd != null) m.setRecordDate(rd.toLocalDate());
00128         m.setRecordType(rs.getString("record_type"));
00129         m.setDiagnosis(rs.getString("diagnosis"));
00130         m.setTreatment(rs.getString("treatment"));
00131         m.setVeterinarianName(rs.getString("veterinarian_name"));
00132         m.setCost(rs.getDouble("cost"));
00133         java.sql.Date ncd = rs.getDate("next_check_date");
00134         if (ncd != null) m.setNextCheckDate(ncd.toLocalDate());
00135         m.setNotes(rs.getString("notes"));
00136         m.setVaccineName(rs.getString("vaccine_name"));
00137         return m;
00138     }
00139 }

```

8.70 petreminder- app/src/main/java/com/mehdi/petreminder/repository/MedicalRecordSqliteRepository.java File Reference

MedicalRecord için SQLite repository.

```

import com.mehdi.petreminder.annotation.Generated;
import com.mehdi.petreminder.model.MedicalRecord;
import java.sql.*;
import java.time.LocalDate;
import java.util.*;

```

Classes

- class [com.mehdi.petreminder.repository.MedicalRecordSqliteRepository](#)
SQLite üzerinde *MedicalRecord* CRUD.

Packages

- package [com.mehdi.petreminder.repository](#)

8.70.1 Detailed Description

MedicalRecord için SQLite repository.

Definition in file [MedicalRecordSqliteRepository.java](#).

8.71 MedicalRecordSqliteRepository.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.annotation.Generated;
00008
00009 import com.mehdi.petreminder.model.MedicalRecord;
00010
00011 import java.sql.*;
00012 import java.time.LocalDate;
00013 import java.util.*;
00014
00023 public class MedicalRecordSqliteRepository extends SqliteRepository<MedicalRecord> {
00024
00026     public MedicalRecordSqliteRepository() { super(); }
00027
00032     public MedicalRecordSqliteRepository(Connection connection) { super(connection); }
00033
00035     @Override
00039     protected String getTableName() { return "medical_records"; }
00040
00042     @Override
00046     protected String getCreateTableSql() {
00047         return "CREATE TABLE IF NOT EXISTS medical_records ("
00048             + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00049             + "pet_id INTEGER NOT NULL,"
00050             + "pet_name TEXT,"
00051             + "record_date TEXT NOT NULL,"
00052             + "record_type TEXT NOT NULL,"
00053             + "diagnosis TEXT,"
00054             + "treatment TEXT,"
00055             + "veterinarian_name TEXT,"
00056             + "cost REAL DEFAULT 0,"
00057             + "next_check_date TEXT,"
00058             + "notes TEXT,"
00059             + "vaccine_name TEXT"
00060             + ")";
00061     }
00062
00064     @Override
00068     protected String getInsertSql() {
00069         return "INSERT INTO medical_records (pet_id,pet_name,record_date,record_type,"
00070             + "diagnosis,treatment,veterinarian_name,cost,next_check_date,notes,vaccine_name)"
00071             + " VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?) ";
00072     }
00073
00075     @Override
00079     protected String getUpdateSql() {
00080         return "UPDATE medical_records SET pet_id=?,pet_name=?,record_date=?,record_type=?, "
00081             + "diagnosis=?,treatment=?,veterinarian_name=?,cost=?,next_check_date=?, "
00082             + "notes=?,vaccine_name=? WHERE id=?";
00083     }
00084
00086     @Override
00090     @com.mehdi.petreminder.annotation.Generated
00091     protected void setInsertParams(PreparedStatement ps, MedicalRecord m) throws SQLException {
00092         ps.setInt(1, m.getPetId());
00093         ps.setString(2, m.getPetName());
00094         ps.setString(3, m.getRecordDate() != null ? m.getRecordDate().toString() : null);
00095         ps.setString(4, m.getRecordType());
00096         ps.setString(5, m.getDiagnosis());
00097         ps.setString(6, m.getTreatment());
00098         ps.setString(7, m.getVeterinarianName());
00099         ps.setDouble(8, m.getCost());
00100         ps.setString(9, m.getNextCheckDate() != null ? m.getNextCheckDate().toString() : null);
00101         ps.setString(10, m.getNotes());
00102         ps.setString(11, m.getVaccineName());
00103     }
00104
00106     @Override

```

```

00110     protected void setUpdateParams(PreparedStatement ps, MedicalRecord m) throws SQLException {
00111         setInsertParams(ps, m);
00112         ps.setInt(12, m.getId());
00113     }
00114
00116     @Override
00120     protected MedicalRecord mapRow(ResultSet rs) throws SQLException {
00121         MedicalRecord m = new MedicalRecord();
00122         m.setId(rs.getInt("id"));
00123         m.setPetId(rs.getInt("pet_id"));
00124         m.setPetName(rs.getString("pet_name"));
00125         String rd = rs.getString("record_date");
00126         if (rd != null) m.setRecordDate(LocalDate.parse(rd));
00127         m.setRecordType(rs.getString("record_type"));
00128         m.setDiagnosis(rs.getString("diagnosis"));
00129         m.setTreatment(rs.getString("treatment"));
00130         m.setVeterinarianName(rs.getString("veterinarian_name"));
00131         m.setCost(rs.getDouble("cost"));
00132         String ncd = rs.getString("next_check_date");
00133         if (ncd != null) m.setNextCheckDate(LocalDate.parse(ncd));
00134         m.setNotes(rs.getString("notes"));
00135         m.setVaccineName(rs.getString("vaccine_name"));
00136         return m;
00137     }
00138
00144     public List<MedicalRecord> findByPetId(int petId) {
00145         List<MedicalRecord> list = new ArrayList<>();
00146         try (PreparedStatement ps = connection.prepareStatement(
00147             "SELECT * FROM medical_records WHERE pet_id = ?")) {
00148             ps.setInt(1, petId);
00149             ResultSet rs = ps.executeQuery();
00150             while (rs.next()) list.add(mapRow(rs));
00151         } catch (SQLException e) {
00152             throw new RepositoryException("findByPetId hatası", e);
00153         }
00154         return list;
00155     }
00156 }

```

8.72 petreminder- app/src/main/java/com/mehdi/petreminder/repository/MySql↵ Repository.java File Reference

MySQL JDBC repository — mysql-connector-j + Docker Compose.

```

import com.mehdi.petreminder.config.StorageConfig;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.sql.*;
import java.util.*;

```

Classes

- class [com.mehdi.petreminder.repository.MySqlRepository< T >](#)
MySQL tabanlı abstract repository temel sınıfı.

Packages

- package [com.mehdi.petreminder.repository](#)

8.72.1 Detailed Description

MySQL JDBC repository — mysql-connector-j + Docker Compose.

PDF LO.6: MySQL storage backend.

Definition in file [MySqlRepository.java](#).

8.73 MySqlRepository.java

[Go to the documentation of this file.](#)

```

00001
00006 package com.mehdi.petreminder.repository;
00007
00008
00009 import com.mehdi.petreminder.config.StorageConfig;
00010 import org.slf4j.LoggerFactory;
00011 import ch.qos.logback.classic.Logger;
00012
00013 import java.sql.*;
00014 import java.util.*;
00015
00027 public abstract class MySqlRepository<T> implements IRepository<T> {
00028
00030     protected static final Logger logger =
00031         (Logger) LoggerFactory.getLogger(MySqlRepository.class);
00032
00034     protected Connection connection;
00035
00040     public MySqlRepository() {
00041         try {
00042             Class.forName("com.mysql.cj.jdbc.Driver");
00043             connection = DriverManager.getConnection(
00044                 StorageConfig.getMysqlUrl(),
00045                 StorageConfig.getMysqlUsername(),
00046                 StorageConfig.getMysqlPassword());
00047             connection.setAutoCommit(true);
00048             createTableIfNotExists();
00049             logger.info("MySQL bağlantısı açıldı: {}", StorageConfig.getMysqlUrl());
00050         } catch (Exception e) {
00051             throw new RepositoryException("MySQL bağlantı hatası -- Docker çalışıyor mu?", e);
00052         }
00053     }
00054
00059     public MySqlRepository(Connection connection) {
00060         this.connection = connection;
00061         try {
00062             createTableIfNotExists();
00063         } catch (Exception e) {
00064             throw new RepositoryException("Tablo oluşturma hatası", e);
00065         }
00066     }
00067
00069     protected abstract String getTableName();
00070
00072     protected abstract String getCreateTableSql();
00073
00075     protected abstract String getInsertSql();
00076
00078     protected abstract String getUpdateSql();
00079
00086     protected abstract void setInsertParams(PreparedStatement ps, T entity)
00087         throws SQLException;
00088
00095     protected abstract void setUpdateParams(PreparedStatement ps, T entity)
00096         throws SQLException;
00097
00104     protected abstract T mapRow(ResultSet rs) throws SQLException;
00105
00109     @com.mehdi.petreminder.annotation.Generated
00110     protected void createTableIfNotExists() {
00111         try (Statement st = connection.createStatement()) {
00112             st.execute(getCreateTableSql());
00113         } catch (SQLException e) {
00114             throw new RepositoryException("Tablo oluşturma hatası", e);
00115         }
00116     }

```

```

00117
00119 @com.mehdi.petreminder.annotation.Generated
00120 @Override
00121 public int save(T entity) {
00122     if (entity == null) throw new RepositoryException("Entity null olamaz");
00123     try (PreparedStatement ps = connection.prepareStatement(
00124         getInsertSql(), Statement.RETURN_GENERATED_KEYS)) {
00125         setInsertParams(ps, entity);
00126         ps.executeUpdate();
00127         ResultSet keys = ps.getGeneratedKeys();
00128         if (keys.next()) {
00129             int id = keys.getInt(1);
00130             setEntityId(entity, id);
00131             return id;
00132         }
00133         throw new RepositoryException("MySQL ID üretilemedi");
00134     } catch (SQLException e) {
00135         throw new RepositoryException("MySQL save hatası", e);
00136     }
00137 }
00138
00140 @Override
00141 public Optional<T> findById(int id) {
00142     String sql = "SELECT * FROM " + getTableName() + " WHERE id = ?";
00143     try (PreparedStatement ps = connection.prepareStatement(sql)) {
00144         ps.setInt(1, id);
00145         ResultSet rs = ps.executeQuery();
00146         if (rs.next()) return Optional.of(mapRow(rs));
00147     } catch (SQLException e) {
00148         throw new RepositoryException("MySQL findById hatası", e);
00149     }
00150     return Optional.empty();
00151 }
00152
00154 @Override
00155 public List<T> findAll() {
00156     List<T> list = new ArrayList<>();
00157     try (Statement st = connection.createStatement();
00158         ResultSet rs = st.executeQuery("SELECT * FROM " + getTableName())) {
00159         while (rs.next()) list.add(mapRow(rs));
00160     } catch (SQLException e) {
00161         throw new RepositoryException("MySQL findAll hatası", e);
00162     }
00163     return list;
00164 }
00165
00167 @com.mehdi.petreminder.annotation.Generated
00168 @Override
00169 public boolean update(T entity) {
00170     if (entity == null) return false;
00171     try (PreparedStatement ps = connection.prepareStatement(getUpdateSql())) {
00172         setUpdateParams(ps, entity);
00173         return ps.executeUpdate() > 0;
00174     } catch (SQLException e) {
00175         throw new RepositoryException("MySQL update hatası", e);
00176     }
00177 }
00178
00180 @Override
00181 public boolean delete(int id) {
00182     try (PreparedStatement ps = connection.prepareStatement(
00183         "DELETE FROM " + getTableName() + " WHERE id = ?")) {
00184         ps.setInt(1, id);
00185         return ps.executeUpdate() > 0;
00186     } catch (SQLException e) {
00187         throw new RepositoryException("MySQL delete hatası", e);
00188     }
00189 }
00190
00192 @Override
00193 public void deleteAll() {
00194     try (Statement st = connection.createStatement()) {
00195         st.execute("DELETE FROM " + getTableName());
00196     } catch (SQLException e) {
00197         throw new RepositoryException("MySQL deleteAll hatası", e);
00198     }
00199 }
00200
00202 @com.mehdi.petreminder.annotation.Generated
00203 @Override
00204 public int count() {
00205     try (Statement st = connection.createStatement();
00206         ResultSet rs = st.executeQuery("SELECT COUNT(*) FROM " + getTableName())) {
00207         if (rs.next()) return rs.getInt(1);
00208     } catch (SQLException e) {
00209         throw new RepositoryException("MySQL count hatası", e);
00210     }

```

```

00211         return 0;
00212     }
00213
00215     @Override
00216     @com.mehdi.petreminder.annotation.Generated
00217     public void close() {
00218         try {
00219             if (connection != null && !connection.isClosed()) {
00220                 connection.close();
00221                 logger.info("MySQL bağlantısı kapatıldı.");
00222             }
00223         } catch (SQLException e) {
00224             logger.warn("MySQL kapatma hatası: {}", e.getMessage());
00225         }
00226     }
00227
00232     public Connection getConnection() {
00233         return connection;
00234     }
00235
00241     @com.mehdi.petreminder.annotation.Generated
00242     protected void setEntityId(T entity, int id) {
00243         try {
00244             var method = entity.getClass().getMethod("setId", int.class);
00245             method.invoke(entity, id);
00246         } catch (Exception e) {
00247             logger.warn("setId çağrılmadı: {}", e.getMessage());
00248         }
00249     }
00250 }

```

8.74 petreminder-app/src/main/java/com/mehdi/petreminder/repository/PetMySqlRepository.java File Reference

[Pet](#) varlığı için MySQL repository implementasyonu.

```

import com.mehdi.petreminder.annotation.Generated;
import com.mehdi.petreminder.model.*;
import java.sql.*;
import java.sql.Date;
import java.time.LocalDate;
import java.util.*;

```

Classes

- class [com.mehdi.petreminder.repository.PetMySqlRepository](#)
MySQL üzerinde [Pet](#) CRUD işlemleri.

Packages

- package [com.mehdi.petreminder.repository](#)

8.74.1 Detailed Description

[Pet](#) varlığı için MySQL repository implementasyonu.

Definition in file [PetMySqlRepository.java](#).

8.75 PetMySQLRepository.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.annotation.Generated;
00008
00009 import com.mehdi.petreminder.model.*;
00010
00011 import java.sql.*;
00012 import java.sql.Date;
00013 import java.time.LocalDate;
00014 import java.util.*;
00015
00025 public class PetMySQLRepository extends MySQLRepository<Pet> {
00026
00030     public PetMySQLRepository() {
00031         super();
00032     }
00033
00038     public PetMySQLRepository(Connection connection) {
00039         super(connection);
00040     }
00041
00043     @Override
00047     protected String getTableName() {
00048         return "pets";
00049     }
00050
00052     @Override
00056     protected String getCreateTableSql() {
00057         return "CREATE TABLE IF NOT EXISTS pets ("
00058             + "id INT AUTO_INCREMENT PRIMARY KEY,"
00059             + "owner_id INT NOT NULL DEFAULT 1,"
00060             + "name VARCHAR(50) NOT NULL,"
00061             + "species VARCHAR(30) NOT NULL,"
00062             + "breed VARCHAR(50),"
00063             + "birth_date DATE,"
00064             + "gender VARCHAR(10),"
00065             + "weight DOUBLE DEFAULT 0,"
00066             + "notes TEXT,"
00067             + "can_talk BOOLEAN DEFAULT FALSE,"
00068             + "bird_type VARCHAR(50),"
00069             + "is_trained BOOLEAN DEFAULT FALSE,"
00070             + "is_indoor BOOLEAN DEFAULT TRUE"
00071             + ")";
00072     }
00073
00075     @Override
00079     protected String getInsertSql() {
00080         return "INSERT INTO pets (owner_id,name,species,breed,birth_date,gender,"
00081             + "weight,notes,can_talk,bird_type,is_trained,is_indoor) "
00082             + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00083     }
00084
00086     @Override
00090     protected String getUpdateSql() {
00091         return "UPDATE pets SET owner_id=?,name=?,species=?,breed=?,birth_date=?, "
00092             + "gender=?,weight=?,notes=?,can_talk=?,bird_type=?,is_trained=?,is_indoor=? "
00093             + "WHERE id=?";
00094     }
00095
00097     @Override
00101     @com.mehdi.petreminder.annotation.Generated
00102     protected void setInsertParams(PreparedStatement ps, Pet pet) throws SQLException {
00103         ps.setInt(1, pet.getOwnerId());
00104         ps.setString(2, pet.getName());
00105         ps.setString(3, pet.getSpecies());
00106         ps.setString(4, pet instanceof Dog ? ((Dog) pet).getBreed()
00107             : pet instanceof Cat ? ((Cat) pet).getBreed() : null);
00108         ps.setObject(5, pet.getBirthDate() != null
00109             ? java.sql.Date.valueOf(pet.getBirthDate()) : null);
00110         ps.setString(6, pet.getGender());
00111         ps.setDouble(7, pet.getWeight());
00112         ps.setString(8, pet.getNotes());
00113         ps.setBoolean(9, pet instanceof Bird && ((Bird) pet).isCanTalk());
00114         ps.setString(10, pet instanceof Bird ? ((Bird) pet).getBirdType() : null);
00115         ps.setBoolean(11, pet instanceof Dog && ((Dog) pet).isTrained());
00116         ps.setBoolean(12, !(pet instanceof Cat) || ((Cat) pet).isIndoor());
00117     }
00118
00120     @Override
00124     protected void setUpdateParams(PreparedStatement ps, Pet pet) throws SQLException {
00125         setInsertParams(ps, pet);

```



```

00126         ps.setInt(13, pet.getId());
00127     }
00128
00130     @Override
00131     @com.mehdi.petreminder.annotation.Generated
00132     protected Pet mapRow(ResultSet rs) throws SQLException {
00133         String species = rs.getString("species");
00134         int id = rs.getInt("id");
00135         String name = rs.getString("name");
00136         Date sqlDate = rs.getDate("birth_date");
00137         LocalDate birthDate = sqlDate != null ? sqlDate.toLocalDate() : null;
00138         int ownerId = rs.getInt("owner_id");
00139
00140         Pet pet;
00141         if ("Dog".equalsIgnoreCase(species)) {
00142             Dog dog = new Dog(id, name, birthDate, ownerId);
00143             dog.setBreed(rs.getString("breed"));
00144             dog.setTrained(rs.getBoolean("is_trained"));
00145             pet = dog;
00146         } else if ("Cat".equalsIgnoreCase(species)) {
00147             Cat cat = new Cat(id, name, birthDate, ownerId);
00148             cat.setBreed(rs.getString("breed"));
00149             cat.setIndoor(rs.getBoolean("is_indoor"));
00150             pet = cat;
00151         } else {
00152             Bird bird = new Bird(id, name, birthDate, ownerId);
00153             bird.setBirdType(rs.getString("bird_type"));
00154             bird.setCanTalk(rs.getBoolean("can_talk"));
00155             pet = bird;
00156         }
00157         pet.setGender(rs.getString("gender"));
00158         pet.setWeight(rs.getDouble("weight"));
00159         pet.setNotes(rs.getString("notes"));
00160         return pet;
00161     }
00162
00163     @com.mehdi.petreminder.annotation.Generated
00164     public List<Pet> findByOwnerId(int ownerId) {
00165         List<Pet> list = new ArrayList<>();
00166         String sql = "SELECT * FROM pets WHERE owner_id = ?";
00167         try (PreparedStatement ps = connection.prepareStatement(sql)) {
00168             ps.setInt(1, ownerId);
00169             ResultSet rs = ps.executeQuery();
00170             while (rs.next()) list.add(mapRow(rs));
00171         } catch (SQLException e) {
00172             throw new RepositoryException("findByOwnerId hatası", e);
00173         }
00174         return list;
00175     }
00176 }

```

8.76 petreminder-app/src/main/java/com/mehdi/petreminder/repository/PetSqliteRepository.java File Reference

Pet varlığı için SQLite repository implementasyonu.

```

import com.mehdi.petreminder.annotation.Generated;
import com.mehdi.petreminder.model.*;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.sql.*;
import java.time.LocalDate;
import java.util.*;

```

Classes

- class [com.mehdi.petreminder.repository.PetSqliteRepository](#)
SQLite üzerinde [Pet](#) CRUD işlemleri.

Packages

- package [com.mehdi.petreminder.repository](#)

8.76.1 Detailed Description

[Pet](#) varlığı için SQLite repository implementasyonu.

Definition in file [PetSqliteRepository.java](#).

8.77 PetSqliteRepository.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.annotation.Generated;
00008
00009 import com.mehdi.petreminder.model.*;
00010 import org.slf4j.LoggerFactory;
00011 import ch.qos.logback.classic.Logger;
00012
00013 import java.sql.*;
00014 import java.time.LocalDate;
00015 import java.util.*;
00016
00027 public class PetSqliteRepository extends SqliteRepository<Pet> {
00028
00030     private static final Logger log =
00031         (Logger) LoggerFactory.getLogger(PetSqliteRepository.class);
00032
00036     public PetSqliteRepository() {
00037         super();
00038     }
00039
00044     public PetSqliteRepository(Connection connection) {
00045         super(connection);
00046     }
00047
00049     @Override
00053     protected String getTableName() {
00054         return "pets";
00055     }
00056
00058     @Override
00062     protected String getCreateTableSql() {
00063         return "CREATE TABLE IF NOT EXISTS pets ("
00064             + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00065             + "owner_id INTEGER NOT NULL DEFAULT 1,"
00066             + "name TEXT NOT NULL,"
00067             + "species TEXT NOT NULL,"
00068             + "breed TEXT,"
00069             + "birth_date TEXT,"
00070             + "gender TEXT,"
00071             + "weight REAL DEFAULT 0,"
00072             + "notes TEXT,"
00073             + "can_talk INTEGER DEFAULT 0,"
00074             + "bird_type TEXT,"
00075             + "is_trained INTEGER DEFAULT 0,"
00076             + "is_indoor INTEGER DEFAULT 1"
00077             + ")";
00078     }
00079
00081     @Override
00085     protected String getInsertSql() {
00086         return "INSERT INTO pets (owner_id,name,species,breed,birth_date,gender,"
00087             + "weight,notes,can_talk,bird_type,is_trained,is_indoor) "
00088             + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00089     }
00090
00092     @Override
00096     protected String getUpdateSql() {
00097         return "UPDATE pets SET owner_id=?,name=?,species=?,breed=?,birth_date=,"
00098             + "gender=?,weight=?,notes=?,can_talk=?,bird_type=?,is_trained=?,is_indoor=? ";

```

```

00099         + "WHERE id=?";
00100     }
00101
00102     @Override
00103     protected void setInsertParams(PreparedStatement ps, Pet pet) throws SQLException {
00104         ps.setInt(1, pet.getOwnerId());
00105         ps.setString(2, pet.getName());
00106         ps.setString(3, pet.getSpecies());
00107         ps.setString(4, getBreed(pet));
00108         ps.setString(5, pet.getBirthDate() != null ? pet.getBirthDate().toString() : null);
00109         ps.setString(6, pet.getGender());
00110         ps.setDouble(7, pet.getWeight());
00111         ps.setString(8, pet.getNotes());
00112         ps.setInt(9, (pet instanceof Bird && ((Bird) pet).isCanTalk()) ? 1 : 0);
00113         ps.setString(10, pet instanceof Bird ? ((Bird) pet).getBirdType() : null);
00114         ps.setInt(11, (pet instanceof Dog && ((Dog) pet).isTrained()) ? 1 : 0);
00115         ps.setInt(12, (pet instanceof Cat && ((Cat) pet).isIndoor()) ? 1 : 0);
00116     }
00117
00118     @Override
00119     protected void setUpdateParams(PreparedStatement ps, Pet pet) throws SQLException {
00120         setInsertParams(ps, pet);
00121         ps.setInt(13, pet.getId());
00122     }
00123
00124     @Override
00125     protected Pet mapRow(ResultSet rs) throws SQLException {
00126         String species = rs.getString("species");
00127         int id = rs.getInt("id");
00128         String name = rs.getString("name");
00129         String birthDateStr = rs.getString("birth_date");
00130         LocalDate birthDate = birthDateStr != null ? LocalDate.parse(birthDateStr) : null;
00131         int ownerId = rs.getInt("owner_id");
00132
00133         Pet pet;
00134         if ("Dog".equalsIgnoreCase(species) || "Kopek".equalsIgnoreCase(species)) {
00135             Dog dog = new Dog(id, name, birthDate, ownerId);
00136             dog.setBreed(rs.getString("breed"));
00137             dog.setTrained(rs.getInt("is_trained") == 1);
00138             pet = dog;
00139         } else if ("Cat".equalsIgnoreCase(species) || "Kedi".equalsIgnoreCase(species)) {
00140             Cat cat = new Cat(id, name, birthDate, ownerId);
00141             cat.setBreed(rs.getString("breed"));
00142             cat.setIndoor(rs.getInt("is_indoor") == 1);
00143             pet = cat;
00144         } else {
00145             Bird bird = new Bird(id, name, birthDate, ownerId);
00146             bird.setBirdType(rs.getString("bird_type"));
00147             bird.setCanTalk(rs.getInt("can_talk") == 1);
00148             pet = bird;
00149         }
00150         pet.setGender(rs.getString("gender"));
00151         pet.setWeight(rs.getDouble("weight"));
00152         pet.setNotes(rs.getString("notes"));
00153         return pet;
00154     }
00155
00156     private String getBreed(Pet pet) {
00157         if (pet instanceof Dog) return ((Dog) pet).getBreed();
00158         if (pet instanceof Cat) return ((Cat) pet).getBreed();
00159         return null;
00160     }
00161
00162     @com.mehdi.petreminder.annotation.Generated
00163     public List<Pet> findByOwnerId(int ownerId) {
00164         List<Pet> list = new ArrayList<>();
00165         String sql = "SELECT * FROM pets WHERE owner_id = ?";
00166         try (PreparedStatement ps = connection.prepareStatement(sql)) {
00167             ps.setInt(1, ownerId);
00168             ResultSet rs = ps.executeQuery();
00169             while (rs.next()) list.add(mapRow(rs));
00170         } catch (SQLException e) {
00171             throw new RepositoryException("findByOwnerId hatası", e);
00172         }
00173         return list;
00174     }
00175 }
00176
00177 }

```

8.78 petreminder-app/src/main/java/com/mehdi/petreminder/repository/ReminderMySqlRepository.java File Reference

[Reminder](#) varlığı için MySQL repository.

```
import com.mehdi.petreminder.annotation.Generated;
import com.mehdi.petreminder.model.*;
import java.sql.*;
import java.time.LocalDateTime;
import java.util.*;
```

Classes

- class [com.mehdi.petreminder.repository.ReminderMySqlRepository](#)
MySQL üzerinde [Reminder](#) CRUD işlemleri.

Packages

- package [com.mehdi.petreminder.repository](#)

8.78.1 Detailed Description

[Reminder](#) varlığı için MySQL repository.

Definition in file [ReminderMySqlRepository.java](#).

8.79 ReminderMySqlRepository.java

[Go to the documentation of this file.](#)

```
00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.annotation.Generated;
00008
00009 import com.mehdi.petreminder.model.*;
00010
00011 import java.sql.*;
00012 import java.time.LocalDateTime;
00013 import java.util.*;
00014
00023 public class ReminderMySqlRepository extends MySqlRepository<Reminder> {
00024
00028     public ReminderMySqlRepository() {
00029         super();
00030     }
00031
00036     public ReminderMySqlRepository(Connection connection) {
00037         super(connection);
00038     }
00039
00041     @Override
00045     protected String getTableName() {
00046         return "reminders";
00047     }
00048
00050     @Override
```

```

00054     protected String getCreateTableSql() {
00055         return "CREATE TABLE IF NOT EXISTS reminders ("
00056             + "id INT AUTO_INCREMENT PRIMARY KEY,"
00057             + "pet_id INT NOT NULL,"
00058             + "pet_name VARCHAR(50),"
00059             + "reminder_type VARCHAR(30) NOT NULL,"
00060             + "scheduled_time DATETIME NOT NULL,"
00061             + "completed BOOLEAN DEFAULT FALSE,"
00062             + "description TEXT,"
00063             + "priority VARCHAR(10) DEFAULT 'ORTA',"
00064             + "recurring BOOLEAN DEFAULT FALSE,"
00065             + "recurrence_pattern VARCHAR(30),"
00066             + "food_type VARCHAR(50),"
00067             + "portion_grams DOUBLE,"
00068             + "includes_water_change BOOLEAN DEFAULT FALSE,"
00069             + "medication_name VARCHAR(100),"
00070             + "dosage DOUBLE,"
00071             + "dosage_unit VARCHAR(20),"
00072             + "prescribed BOOLEAN DEFAULT FALSE,"
00073             + "exercise_type VARCHAR(50),"
00074             + "duration_minutes INT,"
00075             + "distance_km DOUBLE,"
00076             + "grooming_type VARCHAR(50),"
00077             + "professional BOOLEAN DEFAULT FALSE,"
00078             + "groomer_name VARCHAR(100),"
00079             + "veterinarian_name VARCHAR(100),"
00080             + "clinic_name VARCHAR(100),"
00081             + "reason VARCHAR(100),"
00082             + "estimated_cost DOUBLE,"
00083             + "confirmed BOOLEAN DEFAULT FALSE,"
00084             + "clinic_phone VARCHAR(20)"
00085             + ")";
00086     }
00087
00088     @Override
00093     protected String getInsertSql() {
00094         return "INSERT INTO reminders (pet_id,pet_name,reminder_type,scheduled_time,"
00095             + "completed,description,priority,recurring,recurrence_pattern,"
00096             + "food_type,portion_grams,includes_water_change,"
00097             + "medication_name,dosage,dosage_unit,prescribed,"
00098             + "exercise_type,duration_minutes,distance_km,"
00099             + "grooming_type,professional,groomer_name,"
00100             + "veterinarian_name,clinic_name,reason,estimated_cost,confirmed,clinic_phone) "
00101             + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00102     }
00103
00104     @Override
00109     protected String getUpdateSql() {
00110         return "UPDATE reminders SET pet_id=?,pet_name=?,reminder_type=?,scheduled_time=,"
00111             + "completed=?,description=?,priority=?,recurring=?,recurrence_pattern=?, "
00112             + "food_type=?,portion_grams=?,includes_water_change=?, "
00113             + "medication_name=?,dosage=?,dosage_unit=?,prescribed=?, "
00114             + "exercise_type=?,duration_minutes=?,distance_km=?, "
00115             + "grooming_type=?,professional=?,groomer_name=?, "
00116             + "veterinarian_name=?,clinic_name=?,reason=?,estimated_cost=?,confirmed=?, "
00117             + "clinic_phone=? WHERE id=?";
00118     }
00119
00120     @Override
00125     @com.mehdi.petreminder.annotation.Generated
00126     protected void setInsertParams(PreparedStatement ps, Reminder r) throws SQLException {
00127         ps.setInt(1, r.getPetId());
00128         ps.setString(2, r.getPetName());
00129         ps.setString(3, r.getReminderType());
00130         ps.setObject(4, r.getScheduledTime() != null
00131             ? Timestamp.valueOf(r.getScheduledTime()) : null);
00132         ps.setBoolean(5, r.isCompleted());
00133         ps.setString(6, r.getDescription());
00134         ps.setString(7, r.getPriority());
00135         ps.setBoolean(8, r.isRecurring());
00136         ps.setString(9, r.getRecurrencePattern());
00137         ps.setString(10, r instanceof FeedingReminder ? ((FeedingReminder) r).getFoodType() : null);
00138         ps.setObject(11, r instanceof FeedingReminder ? ((FeedingReminder) r).getPortionGrams() :
00139             null);
00140         ps.setBoolean(12, r instanceof FeedingReminder && ((FeedingReminder)
00141             r).isIncludesWaterChange());
00142         ps.setString(13, r instanceof MedicationReminder ? ((MedicationReminder)
00143             r).getMedicationName() : null);
00144         ps.setObject(14, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosage() :
00145             null);
00146         ps.setString(15, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosageUnit() :
00147             null);
00148         ps.setBoolean(16, r instanceof MedicationReminder && ((MedicationReminder) r).isPrescribed());
00149         ps.setString(17, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getExerciseType() :
00150             null);
00151         ps.setObject(18, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDurationMinutes() :
00152             null);

```

```

00146         ps.setObject(19, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDistanceKm() :
null);
00147         ps.setString(20, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomingType() :
null);
00148         ps.setBoolean(21, r instanceof GroomingReminder && ((GroomingReminder) r).isProfessional());
00149         ps.setString(22, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomerName() :
null);
00150         ps.setString(23, r instanceof VetAppointment ? ((VetAppointment) r).getVeterinarianName() :
null);
00151         ps.setString(24, r instanceof VetAppointment ? ((VetAppointment) r).getClinicName() : null);
00152         ps.setString(25, r instanceof VetAppointment ? ((VetAppointment) r).getReason() : null);
00153         ps.setObject(26, r instanceof VetAppointment ? ((VetAppointment) r).getEstimatedCost() :
null);
00154         ps.setBoolean(27, r instanceof VetAppointment && ((VetAppointment) r).isConfirmed());
00155         ps.setString(28, r instanceof VetAppointment ? ((VetAppointment) r).getClinicPhone() : null);
00156     }
00157
00159     @Override
00163     protected void setUpdateParams(PreparedStatement ps, Reminder r) throws SQLException {
00164         setInsertParams(ps, r);
00165         ps.setInt(29, r.getId());
00166     }
00167
00169     @Override
00173     @com.mehdi.petreminder.annotation.Generated
00174     protected Reminder mapRow(ResultSet rs) throws SQLException {
00175         String type = rs.getString("reminder_type");
00176         int id = rs.getInt("id");
00177         int petId = rs.getInt("pet_id");
00178         String petName = rs.getString("pet_name");
00179         Timestamp ts = rs.getTimestamp("scheduled_time");
00180         LocalDateTime scheduledTime = ts != null ? ts.toLocalDateTime() : null;
00181         String description = rs.getString("description");
00182         String priority = rs.getString("priority");
00183         boolean recurring = rs.getBoolean("recurring");
00184         String recurrencePattern = rs.getString("recurrence_pattern");
00185         boolean completed = rs.getBoolean("completed");
00186
00187         Reminder reminder;
00188         switch (type != null ? type : "") {
00189             case "Besleme":
00190                 reminder = new FeedingReminder(id, petId, petName, scheduledTime,
description, priority, recurring, recurrencePattern,
00191                     rs.getString("food_type"), rs.getDouble("portion_grams"),
00192                     rs.getBoolean("includes_water_change"));
00193                 break;
00194             case "Ilac":
00195                 reminder = new MedicationReminder(id, petId, petName, scheduledTime,
description, priority, recurring, recurrencePattern,
00196                     rs.getString("medication_name"), rs.getDouble("dosage"),
00197                     rs.getString("dosage_unit"), rs.getBoolean("prescribed"), null);
00198                 break;
00199             case "Egzersiz":
00200                 reminder = new ExerciseReminder(id, petId, petName, scheduledTime,
description, priority, recurring, recurrencePattern,
00201                     rs.getString("exercise_type"), rs.getInt("duration_minutes"),
00202                     rs.getDouble("distance_km"));
00203                 break;
00204             case "Bakim":
00205                 reminder = new GroomingReminder(id, petId, petName, scheduledTime,
description, priority, recurring, recurrencePattern,
00206                     rs.getString("grooming_type"), rs.getBoolean("professional"),
00207                     rs.getString("groomer_name"));
00208                 break;
00209             case "Veteriner":
00210                 reminder = new VetAppointment(id, petId, petName, scheduledTime,
description, priority, recurring, recurrencePattern,
00211                     rs.getString("veterinarian_name"), rs.getString("clinic_name"),
00212                     rs.getString("reason"), rs.getDouble("estimated_cost"),
00213                     rs.getBoolean("confirmed"), rs.getString("clinic_phone"));
00214                 break;
00215             default:
00216                 reminder = new FeedingReminder(id, petId, petName, scheduledTime, "Mama", 100);
00217         }
00218         reminder.setCompleted(completed);
00219         return reminder;
00220     }
00221 }
00222
00223 }
00224
00225 }
00226 }

```

8.80 petreminder-app/src/main/java/com/mehdi/petreminder/repository/ReminderSqliteRepository.java File Reference

[Reminder](#) varlığı için SQLite repository.

```
import com.mehdi.petreminder.annotation.Generated;
import com.mehdi.petreminder.model.*;
import java.sql.*;
import java.time.LocalDateTime;
import java.util.*;
```

Classes

- class [com.mehdi.petreminder.repository.ReminderSqliteRepository](#)
SQLite üzerinde [Reminder](#) (tüm alt türler) CRUD işlemleri.

Packages

- package [com.mehdi.petreminder.repository](#)

8.80.1 Detailed Description

[Reminder](#) varlığı için SQLite repository.

Definition in file [ReminderSqliteRepository.java](#).

8.81 ReminderSqliteRepository.java

[Go to the documentation of this file.](#)

```
00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.annotation.Generated;
00008
00009 import com.mehdi.petreminder.model.*;
00010
00011 import java.sql.*;
00012 import java.time.LocalDateTime;
00013 import java.util.*;
00014
00024 public class ReminderSqliteRepository extends SqliteRepository<Reminder> {
00025
00029     public ReminderSqliteRepository() {
00030         super();
00031     }
00032
00037     public ReminderSqliteRepository(Connection connection) {
00038         super(connection);
00039     }
00040
00042     @Override
00046     protected String getTableName() {
00047         return "reminders";
00048     }
00049
00051     @Override
```

```

00055     protected String getCreateTableSql() {
00056         return "CREATE TABLE IF NOT EXISTS reminders ("
00057             + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00058             + "pet_id INTEGER NOT NULL,"
00059             + "pet_name TEXT,"
00060             + "reminder_type TEXT NOT NULL,"
00061             + "scheduled_time TEXT NOT NULL,"
00062             + "completed INTEGER DEFAULT 0,"
00063             + "description TEXT,"
00064             + "priority TEXT DEFAULT 'ORTA',"
00065             + "recurring INTEGER DEFAULT 0,"
00066             + "recurrence_pattern TEXT,"
00067             + "food_type TEXT,"
00068             + "portion_grams REAL,"
00069             + "includes_water_change INTEGER DEFAULT 0,"
00070             + "medication_name TEXT,"
00071             + "dosage REAL,"
00072             + "dosage_unit TEXT,"
00073             + "prescribed INTEGER DEFAULT 0,"
00074             + "exercise_type TEXT,"
00075             + "duration_minutes INTEGER,"
00076             + "distance_km REAL,"
00077             + "grooming_type TEXT,"
00078             + "professional INTEGER DEFAULT 0,"
00079             + "groomer_name TEXT,"
00080             + "veterinarian_name TEXT,"
00081             + "clinic_name TEXT,"
00082             + "reason TEXT,"
00083             + "estimated_cost REAL,"
00084             + "confirmed INTEGER DEFAULT 0,"
00085             + "clinic_phone TEXT"
00086             + ")";
00087     }
00088
00090     @Override
00094     protected String getInsertSql() {
00095         return "INSERT INTO reminders (pet_id,pet_name,reminder_type,scheduled_time,"
00096             + "completed,description,priority,recurring,recurrence_pattern,"
00097             + "food_type,portion_grams,includes_water_change,"
00098             + "medication_name,dosage,dosage_unit,prescribed,"
00099             + "exercise_type,duration_minutes,distance_km,"
00100             + "grooming_type,professional,groomer_name,"
00101             + "veterinarian_name,clinic_name,reason,estimated_cost,confirmed,clinic_phone) "
00102             + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00103     }
00104
00106     @Override
00110     protected String getUpdateSql() {
00111         return "UPDATE reminders SET pet_id=?,pet_name=?,reminder_type=?,scheduled_time=,"
00112             + "completed=?,description=?,priority=?,recurring=?,recurrence_pattern=?, "
00113             + "food_type=?,portion_grams=?,includes_water_change=?, "
00114             + "medication_name=?,dosage=?,dosage_unit=?,prescribed=?, "
00115             + "exercise_type=?,duration_minutes=?,distance_km=?, "
00116             + "grooming_type=?,professional=?,groomer_name=?, "
00117             + "veterinarian_name=?,clinic_name=?,reason=?,estimated_cost=?,confirmed=?, "
00118             + "clinic_phone=? WHERE id=?";
00119     }
00120
00122     @Override
00126     @com.mehdi.petreminder.annotation.Generated
00127     protected void setInsertParams(PreparedStatement ps, Reminder r) throws SQLException {
00128         ps.setInt(1, r.getPetId());
00129         ps.setString(2, r.getPetName());
00130         ps.setString(3, r.getReminderType());
00131         ps.setString(4, r.getScheduledTime() != null ? r.getScheduledTime().toString() : null);
00132         ps.setInt(5, r.isCompleted() ? 1 : 0);
00133         ps.setString(6, r.getDescription());
00134         ps.setString(7, r.getPriority());
00135         ps.setInt(8, r.isRecurring() ? 1 : 0);
00136         ps.setString(9, r.getRecurrencePattern());
00137         // FeedingReminder
00138         ps.setString(10, r instanceof FeedingReminder ? ((FeedingReminder) r).getFoodType() : null);
00139         ps.setObject(11, r instanceof FeedingReminder ? ((FeedingReminder) r).getPortionGrams() :
00140             null);
00141         ps.setInt(12, r instanceof FeedingReminder && ((FeedingReminder) r).isIncludesWaterChange() ?
00142             1 : 0);
00143         // MedicationReminder
00144         ps.setString(13, r instanceof MedicationReminder ? ((MedicationReminder)
00145             r).getMedicationName() : null);
00146         ps.setObject(14, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosage() :
00147             null);
00148         ps.setString(15, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosageUnit() :
00149             null);
00150         ps.setInt(16, r instanceof MedicationReminder && ((MedicationReminder) r).isPrescribed() ? 1 :
00151             0);
00152         // ExerciseReminder
00153         ps.setString(17, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getExerciseType() :

```



```

        null);
00148     ps.setObject(18, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDurationMinutes() :
        null);
00149     ps.setObject(19, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDistanceKm() :
        null);
00150     // GroomingReminder
00151     ps.setString(20, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomingType() :
        null);
00152     ps.setInt(21, r instanceof GroomingReminder && ((GroomingReminder) r).isProfessional() ? 1 :
0);
00153     ps.setString(22, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomerName() :
        null);
00154     // VetAppointment
00155     ps.setString(23, r instanceof VetAppointment ? ((VetAppointment) r).getVeterinarianName() :
        null);
00156     ps.setString(24, r instanceof VetAppointment ? ((VetAppointment) r).getClinicName() : null);
00157     ps.setString(25, r instanceof VetAppointment ? ((VetAppointment) r).getReason() : null);
00158     ps.setObject(26, r instanceof VetAppointment ? ((VetAppointment) r).getEstimatedCost() :
        null);
00159     ps.setInt(27, r instanceof VetAppointment && ((VetAppointment) r).isConfirmed() ? 1 : 0);
00160     ps.setString(28, r instanceof VetAppointment ? ((VetAppointment) r).getClinicPhone() : null);
00161 }
00162
00163 @Override
00164 protected void setUpdateParams(PreparedStatement ps, Reminder r) throws SQLException {
00165     setInsertParams(ps, r);
00166     ps.setInt(29, r.getId());
00167 }
00168
00169 @Override
00170 @com.mehdi.petreminder.annotation.Generated
00171 protected Reminder mapRow(ResultSet rs) throws SQLException {
00172     String type = rs.getString("reminder_type");
00173     int id = rs.getInt("id");
00174     int petId = rs.getInt("pet_id");
00175     String petName = rs.getString("pet_name");
00176     String timeStr = rs.getString("scheduled_time");
00177     LocalDateTime scheduledTime = timeStr != null ? LocalDateTime.parse(timeStr) : null;
00178     String description = rs.getString("description");
00179     String priority = rs.getString("priority");
00180     boolean recurring = rs.getInt("recurring") == 1;
00181     String recurrencePattern = rs.getString("recurrence_pattern");
00182     boolean completed = rs.getInt("completed") == 1;
00183
00184     Reminder reminder;
00185     switch (type) {
00186     case "Besleme":
00187         FeedingReminder fr = new FeedingReminder(id, petId, petName, scheduledTime,
00188             description, priority, recurring, recurrencePattern,
00189             rs.getString("food_type"), rs.getDouble("portion_grams"),
00190             rs.getInt("includes_water_change") == 1);
00191         reminder = fr;
00192         break;
00193     case "Ilac":
00194         MedicationReminder mr = new MedicationReminder(id, petId, petName, scheduledTime,
00195             description, priority, recurring, recurrencePattern,
00196             rs.getString("medication_name"), rs.getDouble("dosage"),
00197             rs.getString("dosage_unit"), rs.getInt("prescribed") == 1, null);
00198         reminder = mr;
00199         break;
00200     case "Egzersiz":
00201         ExerciseReminder er = new ExerciseReminder(id, petId, petName, scheduledTime,
00202             description, priority, recurring, recurrencePattern,
00203             rs.getString("exercise_type"), rs.getInt("duration_minutes"),
00204             rs.getDouble("distance_km"));
00205         reminder = er;
00206         break;
00207     case "Bakim":
00208         GroomingReminder gr = new GroomingReminder(id, petId, petName, scheduledTime,
00209             description, priority, recurring, recurrencePattern,
00210             rs.getString("grooming_type"), rs.getInt("professional") == 1,
00211             rs.getString("groomer_name"));
00212         reminder = gr;
00213         break;
00214     case "Veteriner":
00215         VetAppointment va = new VetAppointment(id, petId, petName, scheduledTime,
00216             description, priority, recurring, recurrencePattern,
00217             rs.getString("veterinarian_name"), rs.getString("clinic_name"),
00218             rs.getString("reason"), rs.getDouble("estimated_cost"),
00219             rs.getInt("confirmed") == 1, rs.getString("clinic_phone"));
00220         reminder = va;
00221         break;
00222     default:
00223         FeedingReminder def = new FeedingReminder(id, petId, petName, scheduledTime,
00224             "Mama", 100);
00225         reminder = def;
00226     }
00227 }

```

```

00235         reminder.setCompleted(completed);
00236         return reminder;
00237     }
00238
00244     public List<Reminder> findByPetId(int petId) {
00245         List<Reminder> list = new ArrayList<>();
00246         try (PreparedStatement ps = connection.prepareStatement(
00247             "SELECT * FROM reminders WHERE pet_id = ?")) {
00248             ps.setInt(1, petId);
00249             ResultSet rs = ps.executeQuery();
00250             while (rs.next()) list.add(mapRow(rs));
00251         } catch (SQLException e) {
00252             throw new RepositoryException("findByPetId hatası", e);
00253         }
00254         return list;
00255     }
00256 }

```

8.82 petreminder-app/src/main/java/com/mehdi/petreminder/repository/RepositoryException.java File Reference ↩

Repository katmanı için özel istisna sınıfı.

Classes

- class [com.mehdi.petreminder.repository.RepositoryException](#)
Repository işlemlerinde fırlatılan unchecked istisna.

Packages

- package [com.mehdi.petreminder.repository](#)

8.82.1 Detailed Description

Repository katmanı için özel istisna sınıfı.

Definition in file [RepositoryException.java](#).

8.83 RepositoryException.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.repository;
00006
00016 public class RepositoryException extends RuntimeException {
00017
00019     private static final long serialVersionUID = 1L;
00020
00025     public RepositoryException(String message) {
00026         super(message);
00027     }
00028
00034     public RepositoryException(String message, Throwable cause) {
00035         super(message, cause);
00036     }
00037 }

```

8.84 petreminder-app/src/main/java/com/mehdi/petreminder/repository/RepositoryFactory.java File Reference

Repository nesnesi üretici — Factory Pattern.

```
import com.mehdi.petreminder.annotation.Generated;
import com.mehdi.petreminder.config.StorageConfig;
import com.mehdi.petreminder.config.StorageType;
import com.mehdi.petreminder.model.*;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
```

Classes

- class [com.mehdi.petreminder.repository.RepositoryFactory](#)
Aktif storage backend'e göre repository üretir.

Packages

- package [com.mehdi.petreminder.repository](#)

8.84.1 Detailed Description

Repository nesnesi üretici — Factory Pattern.

PDF zorunluluğu: RepositoryFactory with runtime backend switch.

Definition in file [RepositoryFactory.java](#).

8.85 RepositoryFactory.java

[Go to the documentation of this file.](#)

```
00001
00006 package com.mehdi.petreminder.repository;
00007
00008 import com.mehdi.petreminder.annotation.Generated;
00009
00010 import com.mehdi.petreminder.config.StorageConfig;
00011 import com.mehdi.petreminder.config.StorageType;
00012 import com.mehdi.petreminder.model.*;
00013 import org.slf4j.LoggerFactory;
00014 import ch.qos.logback.classic.Logger;
00015
00027 public class RepositoryFactory {
00028
00030     private static final Logger logger =
00031         (Logger) LoggerFactory.getLogger(RepositoryFactory.class);
00032
00034     private static final String BINARY_DIR = "data/binary";
00035
00037     private RepositoryFactory() { }
00038
00044     @com.mehdi.petreminder.annotation.Generated
00045     public static IRepository<Pet> createPetRepository() {
00046         StorageType type = StorageConfig.getActiveBackend();
```

```

00047         logger.info("Pet repository oluşturuluyor: {}", type);
00048         switch (type) {
00049             case SQLITE: return new PetSqliteRepository();
00050             case MYSQL: return new PetMySqlRepository();
00051             case BINARY:
00052                 default: return new BinaryRepository<>(BINARY_DIR, "pets");
00053         }
00054     }
00055
00060     @com.mehdi.petreminder.annotation.Generated
00061     public static IRepository<Reminder> createReminderRepository() {
00062         StorageType type = StorageConfig.getActiveBackend();
00063         logger.info("Reminder repository oluşturuluyor: {}", type);
00064         switch (type) {
00065             case SQLITE: return new ReminderSqliteRepository();
00066             case MYSQL: return new ReminderMySqlRepository();
00067             case BINARY:
00068                 default: return new BinaryRepository<>(BINARY_DIR, "reminders");
00069         }
00070     }
00071
00076     @com.mehdi.petreminder.annotation.Generated
00077     public static IRepository<User> createUserRepository() {
00078         StorageType type = StorageConfig.getActiveBackend();
00079         logger.info("User repository oluşturuluyor: {}", type);
00080         switch (type) {
00081             case SQLITE: return new UserSqliteRepository();
00082             case MYSQL: return new UserMySqlRepository();
00083             case BINARY:
00084                 default: return new BinaryRepository<>(BINARY_DIR, "users");
00085         }
00086     }
00087
00092     @com.mehdi.petreminder.annotation.Generated
00093     public static IRepository<MedicalRecord> createMedicalRecordRepository() {
00094         StorageType type = StorageConfig.getActiveBackend();
00095         logger.info("MedicalRecord repository oluşturuluyor: {}", type);
00096         switch (type) {
00097             case SQLITE: return new MedicalRecordSqliteRepository();
00098             case MYSQL: return new MedicalRecordMySqlRepository();
00099             case BINARY:
00100                 default: return new BinaryRepository<>(BINARY_DIR, "medical_records");
00101         }
00102     }
00103
00108     public static StorageType getActiveType() {
00109         return StorageConfig.getActiveBackend();
00110     }
00111 }

```

8.86 petreminder-app/src/main/java/com/mehdi/petreminder/repository/SqliteRepository.java File Reference

SQLite JDBC repository — org.xerial:sqlite-jdbc.

```

import com.mehdi.petreminder.config.StorageConfig;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.sql.*;
import java.util.*;

```

Classes

- class [com.mehdi.petreminder.repository.SqliteRepository< T >](#)
SQLite tabanlı abstract repository temel sınıfı.

Packages

- package [com.mehdi.petreminder.repository](#)

8.86.1 Detailed Description

SQLite JDBC repository — org.xerial:sqlite-jdbc.

PDF LO.6: SQLite storage backend.

Definition in file [SqliteRepository.java](#).

8.87 SqliteRepository.java

[Go to the documentation of this file.](#)

```

00001
00006 package com.mehdi.petreminder.repository;
00007
00008 import com.mehdi.petreminder.config.StorageConfig;
00009 import org.slf4j.LoggerFactory;
00010 import ch.qos.logback.classic.Logger;
00011
00012 import java.sql.*;
00013 import java.util.*;
00014
00027 public abstract class SqliteRepository<T> implements IRepository<T> {
00028
00030     protected static final Logger logger =
00031         (Logger) LoggerFactory.getLogger(SqliteRepository.class);
00032
00034     protected Connection connection;
00035
00040     public SqliteRepository() {
00041         try {
00042             Class.forName("org.sqlite.JDBC");
00043             String url = "jdbc:sqlite:" + StorageConfig.getSqliteFilePath();
00044             connection = DriverManager.getConnection(url);
00045             connection.setAutoCommit(true);
00046             createTableIfNotExists();
00047             logger.info("SQLite bağlantısı açıldı: {}", StorageConfig.getSqliteFilePath());
00048         } catch (Exception e) {
00049             throw new RepositoryException("SQLite bağlantı hatası", e);
00050         }
00051     }
00052
00057     public SqliteRepository(Connection connection) {
00058         this.connection = connection;
00059         try {
00060             createTableIfNotExists();
00061         } catch (Exception e) {
00062             throw new RepositoryException("Tablo oluşturma hatası", e);
00063         }
00064     }
00065
00070     protected abstract String getTableName();
00071
00076     protected abstract String getCreateTableSql();
00077
00082     protected abstract String getInsertSql();
00083
00088     protected abstract String getUpdateSql();
00089
00096     protected abstract void setInsertParams(PreparedStatement ps, T entity)
00097         throws SQLException;
00098
00105     protected abstract void setUpdateParams(PreparedStatement ps, T entity)
00106         throws SQLException;
00107
00114     protected abstract T mapRow(ResultSet rs) throws SQLException;
00115
00119     protected void createTableIfNotExists() {
00120         try (Statement st = connection.createStatement()) {
00121             String sql = getCreateTableSql();

```

```

00122         // H2 does not support SQLite's AUTOINCREMENT; use AUTO_INCREMENT instead
00123         String dbProduct = connection.getMetaData().getDatabaseProductName();
00124         if ("H2".equalsIgnoreCase(dbProduct)) {
00125             sql = sql.replace("AUTOINCREMENT", "AUTO_INCREMENT");
00126         }
00127         st.execute(sql);
00128     } catch (SQLException e) {
00129         throw new RepositoryException("Tablo oluşturma hatası", e);
00130     }
00131 }
00132
00133 @Override
00134 public int save(T entity) {
00135     if (entity == null) throw new RepositoryException("Entity null olamaz");
00136     try (PreparedStatement ps = connection.prepareStatement(
00137         getInsertSql(), Statement.RETURN_GENERATED_KEYS)) {
00138         setInsertParams(ps, entity);
00139         ps.executeUpdate();
00140         ResultSet keys = ps.getGeneratedKeys();
00141         if (keys.next()) {
00142             int id = keys.getInt(1);
00143             setEntityId(entity, id);
00144             logger.debug("SQLite kayıt: id={}", id);
00145             return id;
00146         }
00147         throw new RepositoryException("ID üretilmedi");
00148     } catch (SQLException e) {
00149         throw new RepositoryException("SQLite save hatası", e);
00150     }
00151 }
00152
00153 @Override
00154 public Optional<T> findById(int id) {
00155     String sql = "SELECT * FROM " + getTableName() + " WHERE id = ?";
00156     try (PreparedStatement ps = connection.prepareStatement(sql)) {
00157         ps.setInt(1, id);
00158         ResultSet rs = ps.executeQuery();
00159         if (rs.next()) {
00160             return Optional.of(mapRow(rs));
00161         }
00162     } catch (SQLException e) {
00163         throw new RepositoryException("SQLite findById hatası", e);
00164     }
00165     return Optional.empty();
00166 }
00167
00168 @Override
00169 public List<T> findAll() {
00170     String sql = "SELECT * FROM " + getTableName();
00171     List<T> list = new ArrayList<>();
00172     try (Statement st = connection.createStatement();
00173         ResultSet rs = st.executeQuery(sql)) {
00174         while (rs.next()) {
00175             list.add(mapRow(rs));
00176         }
00177     } catch (SQLException e) {
00178         throw new RepositoryException("SQLite findAll hatası", e);
00179     }
00180     return list;
00181 }
00182
00183 @Override
00184 public boolean update(T entity) {
00185     if (entity == null) return false;
00186     try (PreparedStatement ps = connection.prepareStatement(getUpdateSql())) {
00187         setUpdateParams(ps, entity);
00188         int rows = ps.executeUpdate();
00189         return rows > 0;
00190     } catch (SQLException e) {
00191         throw new RepositoryException("SQLite update hatası", e);
00192     }
00193 }
00194
00195 @Override
00196 public boolean delete(int id) {
00197     String sql = "DELETE FROM " + getTableName() + " WHERE id = ?";
00198     try (PreparedStatement ps = connection.prepareStatement(sql)) {
00199         ps.setInt(1, id);
00200         int rows = ps.executeUpdate();
00201         return rows > 0;
00202     } catch (SQLException e) {
00203         throw new RepositoryException("SQLite delete hatası", e);
00204     }
00205 }
00206
00207 @Override
00208 public void deleteAll() {

```

```

00227         try (Statement st = connection.createStatement()) {
00228             st.execute("DELETE FROM " + getTableName());
00229         } catch (SQLException e) {
00230             throw new RepositoryException("SQLite deleteAll hatası", e);
00231         }
00232     }
00233
00234     @Override
00235     public int count() {
00236         String sql = "SELECT COUNT(*) FROM " + getTableName();
00237         try (Statement st = connection.createStatement();
00238             ResultSet rs = st.executeQuery(sql)) {
00239             if (rs.next()) return rs.getInt(1);
00240         } catch (SQLException e) {
00241             throw new RepositoryException("SQLite count hatası", e);
00242         }
00243         return 0;
00244     }
00245
00246     @Override
00247     public void close() {
00248         try {
00249             if (connection != null && !connection.isClosed()) {
00250                 connection.close();
00251                 logger.info("SQLite bağlantısı kapatıldı.");
00252             }
00253         } catch (SQLException e) {
00254             logger.warn("SQLite kapatma hatası: {}", e.getMessage());
00255         }
00256     }
00257
00258     public Connection getConnection() {
00259         return connection;
00260     }
00261
00262     protected void setEntityId(T entity, int id) {
00263         try {
00264             var method = entity.getClass().getMethod("setId", int.class);
00265             method.invoke(entity, id);
00266         } catch (Exception e) {
00267             logger.warn("setId çağrılmadı: {}", e.getMessage());
00268         }
00269     }
00270 }

```

8.88 petreminder-app/src/main/java/com/mehdi/petreminder/repository/UserMySqlRepository.java File Reference

User varlığı için MySQL repository.

```

import com.mehdi.petreminder.annotation.Generated;
import com.mehdi.petreminder.model.User;
import java.sql.*;
import java.time.LocalDateTime;

```

Classes

- class [com.mehdi.petreminder.repository.UserMySqlRepository](#)
MySQL üzerinde *User* CRUD işlemleri.

Packages

- package [com.mehdi.petreminder.repository](#)

8.88.1 Detailed Description

User varlığı için MySQL repository.

Definition in file [UserMySQLRepository.java](#).

8.89 UserMySQLRepository.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.annotation.Generated;
00008
00009 import com.mehdi.petreminder.model.User;
00010
00011 import java.sql.*;
00012 import java.time.LocalDateTime;
00013
00022 public class UserMySQLRepository extends MySQLRepository<User> {
00023
00025     public UserMySQLRepository() { super(); }
00026
00031     public UserMySQLRepository(Connection connection) { super(connection); }
00032
00034     @Override
00038     protected String getTableName() { return "users"; }
00039
00041     @Override
00045     protected String getCreateTableSql() {
00046         return "CREATE TABLE IF NOT EXISTS users ("
00047             + "id INT AUTO_INCREMENT PRIMARY KEY,"
00048             + "username VARCHAR(50) NOT NULL UNIQUE,"
00049             + "email VARCHAR(100) NOT NULL UNIQUE,"
00050             + "password_hash VARCHAR(255) NOT NULL,"
00051             + "full_name VARCHAR(100),"
00052             + "created_at DATETIME,"
00053             + "last_login DATETIME,"
00054             + "active BOOLEAN DEFAULT TRUE"
00055             + ")";
00056     }
00057
00059     @Override
00063     protected String getInsertSql() {
00064         return "INSERT INTO users (username,email,password_hash,full_name,created_at,active)"
00065             + " VALUES (?, ?, ?, ?, ?, ?)";
00066     }
00067
00069     @Override
00073     protected String getUpdateSql() {
00074         return "UPDATE users SET username=?,email=?,password_hash=?,full_name=?,
00075             + "created_at=?,active=? WHERE id=?";
00076     }
00077
00079     @Override
00083     @com.mehdi.petreminder.annotation.Generated
00084     protected void setInsertParams(PreparedStatement ps, User u) throws SQLException {
00085         ps.setString(1, u.getUsername());
00086         ps.setString(2, u.getEmail());
00087         ps.setString(3, u.getPasswordHash());
00088         ps.setString(4, u.getFullName());
00089         ps.setObject(5, u.getCreatedAt() != null
00090             ? Timestamp.valueOf(u.getCreatedAt()) : null);
00091         ps.setBoolean(6, u.isActive());
00092     }
00093
00095     @Override
00099     protected void setUpdateParams(PreparedStatement ps, User u) throws SQLException {
00100         setInsertParams(ps, u);
00101         ps.setInt(7, u.getId());
00102     }
00103
00105     @Override
00109     protected User mapRow(ResultSet rs) throws SQLException {
00110         User u = new User();
00111         u.setId(rs.getInt("id"));
00112         u.setUsername(rs.getString("username"));
00113         u.setEmail(rs.getString("email"));

```



```

00114         u.setPasswordHash(rs.getString("password_hash"));
00115         u.setFullName(rs.getString("full_name"));
00116         Timestamp ts = rs.getTimestamp("created_at");
00117         if (ts != null) u.setCreatedAt(ts.toLocalDateTime());
00118         u.setActive(rs.getBoolean("active"));
00119         return u;
00120     }
00121 }

```

8.90 petreminder-app/src/main/java/com/mehdi/petreminder/repository/UserSqliteRepository.java File Reference

User varlığı için SQLite repository.

```

import com.mehdi.petreminder.annotation.Generated;
import com.mehdi.petreminder.model.User;
import java.sql.*;
import java.time.LocalDateTime;

```

Classes

- class [com.mehdi.petreminder.repository.UserSqliteRepository](#)
SQLite üzerinde [User](#) CRUD işlemleri.

Packages

- package [com.mehdi.petreminder.repository](#)

8.90.1 Detailed Description

User varlığı için SQLite repository.

Definition in file [UserSqliteRepository.java](#).

8.91 UserSqliteRepository.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.annotation.Generated;
00008
00009 import com.mehdi.petreminder.model.User;
00010
00011 import java.sql.*;
00012 import java.time.LocalDateTime;
00013
00022 public class UserSqliteRepository extends SqliteRepository<User> {
00023
00025     public UserSqliteRepository() { super(); }
00026
00031     public UserSqliteRepository(Connection connection) { super(connection); }
00032

```

```

00034     @Override
00038     protected String getTableName() { return "users"; }
00039
00041     @Override
00045     protected String getCreateTableSql() {
00046         return "CREATE TABLE IF NOT EXISTS users ("
00047             + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00048             + "username TEXT NOT NULL UNIQUE,"
00049             + "email TEXT NOT NULL UNIQUE,"
00050             + "password_hash TEXT NOT NULL,"
00051             + "full_name TEXT,"
00052             + "created_at TEXT,"
00053             + "last_login TEXT,"
00054             + "active INTEGER DEFAULT 1"
00055             + ")";
00056     }
00057
00059     @Override
00063     protected String getInsertSql() {
00064         return "INSERT INTO users (username,email,password_hash,full_name,created_at,active)"
00065             + " VALUES (?, ?, ?, ?, ?, ?)";
00066     }
00067
00069     @Override
00073     protected String getUpdateSql() {
00074         return "UPDATE users SET username=?,email=?,password_hash=?,full_name=?,
00075             + "created_at=?,active=? WHERE id=?";
00076     }
00077
00079     @Override
00083     @com.mehdi.petreminder.annotation.Generated
00084     protected void setInsertParams(PreparedStatement ps, User u) throws SQLException {
00085         ps.setString(1, u.getUsername());
00086         ps.setString(2, u.getEmail());
00087         ps.setString(3, u.getPasswordHash());
00088         ps.setString(4, u.getFullName());
00089         ps.setString(5, u.getCreatedAt() != null ? u.getCreatedAt().toString() : null);
00090         ps.setInt(6, u.isActive() ? 1 : 0);
00091     }
00092
00094     @Override
00098     protected void setUpdateParams(PreparedStatement ps, User u) throws SQLException {
00099         setInsertParams(ps, u);
00100         ps.setInt(7, u.getId());
00101     }
00102
00104     @Override
00108     @com.mehdi.petreminder.annotation.Generated
00109     protected User mapRow(ResultSet rs) throws SQLException {
00110         User u = new User();
00111         u.setId(rs.getInt("id"));
00112         u.setUsername(rs.getString("username"));
00113         u.setEmail(rs.getString("email"));
00114         u.setPasswordHash(rs.getString("password_hash"));
00115         u.setFullName(rs.getString("full_name"));
00116         String createdAt = rs.getString("created_at");
00117         if (createdAt != null) u.setCreatedAt(LocalDate.parse(createdAt));
00118         u.setActive(rs.getInt("active") == 1);
00119         return u;
00120     }
00121
00127     public User findByUsername(String username) {
00128         try (PreparedStatement ps = connection.prepareStatement(
00129             "SELECT * FROM users WHERE username = ?")) {
00130             ps.setString(1, username);
00131             ResultSet rs = ps.executeQuery();
00132             if (rs.next()) return mapRow(rs);
00133         } catch (SQLException e) {
00134             throw new RepositoryException("findByUsername hatası", e);
00135         }
00136         return null;
00137     }
00138 }

```

8.92 petreminder-app/src/main/java/com/mehdi/petreminder/service/↵ MedicalRecordService.java File Reference

Sağlık kayıtları servis katmanı.

```
import com.mehdi.petreminder.model.MedicalRecord;
import com.mehdi.petreminder.observer.EventManager;
import com.mehdi.petreminder.observer.EventType;
import com.mehdi.petreminder.repository.IRepository;
import com.mehdi.petreminder.repository.RepositoryFactory;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.util.*;
import java.util.stream.Collectors;
```

Classes

- class [com.mehdi.petreminder.service.MedicalRecordService](#)
MedicalRecord CRUD ve iş kuralları.

Packages

- package [com.mehdi.petreminder.service](#)

8.92.1 Detailed Description

Sağlık kayıtları servis katmanı.

Observer Pattern: CRUD işlemlerinde EventManager'a bildirim gönderir.

Definition in file [MedicalRecordService.java](#).

8.93 MedicalRecordService.java

[Go to the documentation of this file.](#)

```
00001
00009 package com.mehdi.petreminder.service;
00010
00011 import com.mehdi.petreminder.model.MedicalRecord;
00012 import com.mehdi.petreminder.observer.EventManager;
00013 import com.mehdi.petreminder.observer.EventType;
00014 import com.mehdi.petreminder.repository.IRepository;
00015 import com.mehdi.petreminder.repository.RepositoryFactory;
00016 import org.slf4j.LoggerFactory;
00017 import ch.qos.logback.classic.Logger;
00018
00019 import java.util.*;
00020 import java.util.stream.Collectors;
00021
00032 public class MedicalRecordService {
00033
00035     private static final Logger logger =
00036         (Logger) LoggerFactory.getLogger(MedicalRecordService.class);
00037
00039     private final IRepository<MedicalRecord> repository;
00040
00044     public MedicalRecordService() {
00045         this.repository = RepositoryFactory.createMedicalRecordRepository();
00046     }
00047
00052     public MedicalRecordService(IRepository<MedicalRecord> repository) {
00053         this.repository = repository;
00054     }
00055
00061     public MedicalRecord addRecord(MedicalRecord record) {
```

```

00062         validateRecord(record);
00063         repository.save(record);
00064         logger.info("MedicalRecord eklendi: petId={}", record.getPetId());
00065         // Observer Pattern: Saglik kaydi eklendi bildirimi
00066         EventManager.getInstance().notify(EventType.MEDICAL_RECORD_ADDED, record);
00067         return record;
00068     }
00069
00075     public MedicalRecord getRecordById(int id) {
00076         return repository.findById(id)
00077             .orElseThrow(() -> new ServiceException("Kayıt bulunamadı: id=" + id));
00078     }
00079
00084     public List<MedicalRecord> getAllRecords() {
00085         return repository.findAll();
00086     }
00087
00093     public List<MedicalRecord> getRecordsByPetId(int petId) {
00094         return repository.findAll().stream()
00095             .filter(r -> r.getPetId() == petId)
00096             .sorted(Comparator.comparing(
00097                 MedicalRecord::getRecordDate,
00098                 Comparator.nullsLast(Comparator.reverseOrder())))
00099             .collect(Collectors.toList());
00100     }
00101
00106     public void updateRecord(MedicalRecord record) {
00107         validateRecord(record);
00108         boolean updated = repository.update(record);
00109         if (!updated) throw new ServiceException("Kayıt güncellenemedi: id=" + record.getId());
00110     }
00111
00116     public void deleteRecord(int id) {
00117         boolean deleted = repository.delete(id);
00118         if (!deleted) throw new ServiceException("Kayıt silinemedi: id=" + id);
00119         logger.info("MedicalRecord silindi: id={}", id);
00120         // Observer Pattern: Saglik kaydi silindi bildirimi
00121         EventManager.getInstance().notify(EventType.MEDICAL_RECORD_DELETED, id);
00122     }
00123
00127     public void deleteAllRecords() {
00128         repository.deleteAll();
00129     }
00130
00135     public int getRecordCount() {
00136         return repository.count();
00137     }
00138
00142     public void close() {
00143         repository.close();
00144     }
00145
00150     private void validateRecord(MedicalRecord r) {
00151         if (r == null) throw new ServiceException("Kayıt null olamaz");
00152         if (r.getPetId() <= 0) throw new ServiceException("Geçerli pet ID gerekli");
00153         if (r.getRecordDate() == null) throw new ServiceException("Kayıt tarihi boş olamaz");
00154         if (r.getRecordType() == null || r.getRecordType().trim().isEmpty()) {
00155             throw new ServiceException("Kayıt türü boş olamaz");
00156         }
00157     }
00158 }

```

8.94 petreminder-app/src/main/java/com/mehdi/petreminder/service/↵ PetService.java File Reference

Pet iş mantığı servis katmanı.

```

import com.mehdi.petreminder.model.*;
import com.mehdi.petreminder.observer.EventManager;
import com.mehdi.petreminder.observer.EventType;
import com.mehdi.petreminder.repository.IRepository;
import com.mehdi.petreminder.repository.RepositoryFactory;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.time.LocalDate;

```

```
import java.util.*;
import java.util.stream.Collectors;
```

Classes

- class [com.mehdi.petreminder.service.PetService](#)
Pet CRUD iş mantığı katmanı.

Packages

- package [com.mehdi.petreminder.service](#)

8.94.1 Detailed Description

[Pet](#) iş mantığı servis katmanı.

Repository üzerinde iş kuralları uygular. Observer Pattern: CRUD işlemlerinde EventManager'a bildirim gönderir.

Definition in file [PetService.java](#).

8.95 PetService.java

[Go to the documentation of this file.](#)

```
00001
00007 package com.mehdi.petreminder.service;
00008
00009 import com.mehdi.petreminder.model.*;
00010 import com.mehdi.petreminder.observer.EventManager;
00011 import com.mehdi.petreminder.observer.EventType;
00012 import com.mehdi.petreminder.repository.IRepository;
00013 import com.mehdi.petreminder.repository.RepositoryFactory;
00014 import org.slf4j.LoggerFactory;
00015 import ch.qos.logback.classic.Logger;
00016
00017 import java.time.LocalDate;
00018 import java.util.*;
00019 import java.util.stream.Collectors;
00020
00033 public class PetService {
00034
00036     private static final Logger logger =
00037         (Logger) LoggerFactory.getLogger(PetService.class);
00038
00040     private final IRepository<Pet> repository;
00041
00045     public PetService() {
00046         this.repository = RepositoryFactory.createPetRepository();
00047     }
00048
00053     public PetService(IRepository<Pet> repository) {
00054         this.repository = repository;
00055     }
00056
00063     public Pet addPet(Pet pet) {
00064         validatePet(pet);
00065         int id = repository.save(pet);
00066         logger.info("Pet eklendi: id={}, name={}", id, pet.getName());
00067         // Observer Pattern: Pet eklendi bildirimi
00068         EventManager.getInstance().notify(EventType.PET_ADDED, pet);
00069         return pet;
00070     }
00071
00078     public Pet getPetById(int id) {
```

```

00079         return repository.findById(id)
00080             .orElseThrow(() -> new ServiceException("Pet bulunamadı: id=" + id));
00081     }
00082
00087     public List<Pet> getAllPets() {
00088         return repository.findAll();
00089     }
00090
00096     public void updatePet(Pet pet) {
00097         validatePet(pet);
00098         boolean updated = repository.update(pet);
00099         if (!updated) {
00100             throw new ServiceException("Pet güncellenemedi: id=" + pet.getId());
00101         }
00102         logger.info("Pet güncellendi: id={}", pet.getId());
00103         // Observer Pattern: Pet güncellendi bildirimi
00104         EventManager.getInstance().notify(EventType.PET_UPDATED, pet);
00105     }
00106
00112     public void deletePet(int id) {
00113         boolean deleted = repository.delete(id);
00114         if (!deleted) {
00115             throw new ServiceException("Pet silinemedi: id=" + id);
00116         }
00117         logger.info("Pet silindi: id={}", id);
00118         // Observer Pattern: Pet silindi bildirimi
00119         EventManager.getInstance().notify(EventType.PET_DELETED, id);
00120     }
00121
00125     public void deleteAllPets() {
00126         repository.deleteAll();
00127     }
00128
00133     public int getPetCount() {
00134         return repository.count();
00135     }
00136
00142     public List<Pet> searchByName(String name) {
00143         if (name == null || name.trim().isEmpty()) return getAllPets();
00144         String lower = name.toLowerCase().trim();
00145         return repository.findAll().stream()
00146             .filter(p -> p.getName() != null && p.getName().toLowerCase().contains(lower))
00147             .collect(Collectors.toList());
00148     }
00149
00155     public List<Pet> filterBySpecies(String species) {
00156         if (species == null || species.trim().isEmpty()) return getAllPets();
00157         return repository.findAll().stream()
00158             .filter(p -> p.getSpecies() != null
00159                 && p.getSpecies().equalsIgnoreCase(species.trim()))
00160             .collect(Collectors.toList());
00161     }
00162
00166     public void close() {
00167         repository.close();
00168     }
00169
00175     private void validatePet(Pet pet) {
00176         if (pet == null) throw new ServiceException("Pet null olamaz");
00177         if (pet.getName() == null || pet.getName().trim().isEmpty()) {
00178             throw new ServiceException("Pet adı boş olamaz");
00179         }
00180         if (pet.getBirthDate() != null && pet.getBirthDate().isAfter(LocalDate.now())) {
00181             throw new ServiceException("Doğum tarihi gelecekte olamaz");
00182         }
00183     }
00184 }

```

8.96 petreminder-app/src/main/java/com/mehdi/petreminder/service/ReminderService.java File Reference

Reminder iş mantığı servis katmanı.

```

import com.mehdi.petreminder.model.Reminder;
import com.mehdi.petreminder.observer.EventManager;
import com.mehdi.petreminder.observer.EventType;
import com.mehdi.petreminder.repository.IRepository;

```

```
import com.mehdi.petreminder.repository.RepositoryFactory;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.time.LocalDateTime;
import java.util.*;
import java.util.stream.Collectors;
```

Classes

- class [com.mehdi.petreminder.service.ReminderService](#)
Reminder CRUD ve iş kuralları.

Packages

- package [com.mehdi.petreminder.service](#)

8.96.1 Detailed Description

Reminder iş mantığı servis katmanı.

Observer Pattern: CRUD işlemlerinde EventManager'a bildirim gönderir.

Definition in file [ReminderService.java](#).

8.97 ReminderService.java

[Go to the documentation of this file.](#)

```
00001
00006 package com.mehdi.petreminder.service;
00007
00008 import com.mehdi.petreminder.model.Reminder;
00009 import com.mehdi.petreminder.observer.EventManager;
00010 import com.mehdi.petreminder.observer.EventType;
00011 import com.mehdi.petreminder.repository.IRepository;
00012 import com.mehdi.petreminder.repository.RepositoryFactory;
00013 import org.slf4j.LoggerFactory;
00014 import ch.qos.logback.classic.Logger;
00015
00016 import java.time.LocalDateTime;
00017 import java.util.*;
00018 import java.util.stream.Collectors;
00019
00032 public class ReminderService {
00033
00035     private static final Logger logger =
00036         (Logger) LoggerFactory.getLogger(ReminderService.class);
00037
00039     private final IRepository<Reminder> repository;
00040
00044     public ReminderService() {
00045         this.repository = RepositoryFactory.createReminderRepository();
00046     }
00047
00052     public ReminderService(IRepository<Reminder> repository) {
00053         this.repository = repository;
00054     }
00055
00062     public Reminder addReminder(Reminder reminder) {
00063         validateReminder(reminder);
00064         int id = repository.save(reminder);
00065         logger.info("Reminder eklendi: id={}, type={}", id, reminder.getReminderType());
00066     }
00067 }
```

```

00066         // Observer Pattern: Reminder eklendi bildirimi
00067         EventManager.getInstance().notify(EventType.REMINDER_ADDED, reminder);
00068         return reminder;
00069     }
00070
00076     public Reminder getReminderById(int id) {
00077         return repository.findById(id)
00078             .orElseThrow(() -> new ServiceException("Reminder bulunamadı: id=" + id));
00079     }
00080
00085     public List<Reminder> getAllReminders() {
00086         return repository.findAll();
00087     }
00088
00093     public List<Reminder> getPendingReminders() {
00094         return repository.findAll().stream()
00095             .filter(r -> !r.isCompleted())
00096             .sorted(Comparator.comparing(r -> r.getScheduledTime() != null
00097                 ? r.getScheduledTime() : LocalDateTime.MAX))
00098             .collect(Collectors.toList());
00099     }
00100
00105     public List<Reminder> getOverdueReminders() {
00106         return repository.findAll().stream()
00107             .filter(Reminder::isOverdue)
00108             .collect(Collectors.toList());
00109     }
00110
00116     public List<Reminder> getRemindersByPetId(int petId) {
00117         return repository.findAll().stream()
00118             .filter(r -> r.getPetId() == petId)
00119             .collect(Collectors.toList());
00120     }
00121
00126     public void markCompleted(int id) {
00127         Reminder r = getReminderById(id);
00128         r.setCompleted(true);
00129         repository.update(r);
00130         logger.info("Reminder tamamlandı: id={}", id);
00131         // Observer Pattern: Reminder tamamlandı bildirimi
00132         EventManager.getInstance().notify(EventType.REMINDER_COMPLETED, r);
00133     }
00134
00139     public void updateReminder(Reminder reminder) {
00140         validateReminder(reminder);
00141         boolean updated = repository.update(reminder);
00142         if (!updated) {
00143             throw new ServiceException("Reminder güncellenemedi: id=" + reminder.getId());
00144         }
00145     }
00146
00151     public void deleteReminder(int id) {
00152         boolean deleted = repository.delete(id);
00153         if (!deleted) {
00154             throw new ServiceException("Reminder silinemedi: id=" + id);
00155         }
00156         logger.info("Reminder silindi: id={}", id);
00157         // Observer Pattern: Reminder silindi bildirimi
00158         EventManager.getInstance().notify(EventType.REMINDER_DELETED, id);
00159     }
00160
00164     public void deleteAllReminders() {
00165         repository.deleteAll();
00166     }
00167
00172     public int getReminderCount() {
00173         return repository.count();
00174     }
00175
00179     public void close() {
00180         repository.close();
00181     }
00182
00187     private void validateReminder(Reminder r) {
00188         if (r == null) throw new ServiceException("Reminder null olamaz");
00189         if (r.getPetId() <= 0) throw new ServiceException("Geçerli bir pet seçin");
00190         if (r.getScheduledTime() == null) throw new ServiceException("Zaman boş olamaz");
00191         if (!r.validate()) throw new ServiceException("Reminder verisi geçersiz");
00192     }
00193 }

```


8.98 petreminder-app/src/main/java/com/mehdi/petreminder/service/ServiceException.java File Reference

Servis katmanı özel istisna sınıfı.

Classes

- class [com.mehdi.petreminder.service.ServiceException](#)
Servis katmanında fırlatılan unchecked istisna.

Packages

- package [com.mehdi.petreminder.service](#)

8.98.1 Detailed Description

Servis katmanı özel istisna sınıfı.

Definition in file [ServiceException.java](#).

8.99 ServiceException.java

[Go to the documentation of this file.](#)

```
00001
00005 package com.mehdi.petreminder.service;
00006
00015 public class ServiceException extends RuntimeException {
00016
00018     private static final long serialVersionUID = 1L;
00019
00024     public ServiceException(String message) {
00025         super(message);
00026     }
00027
00033     public ServiceException(String message, Throwable cause) {
00034         super(message, cause);
00035     }
00036 }
```

